CAN THO UNIVERSITY
COLLEGE OF INFORMATION AND COMMUNICATION TECHNOLOGY



GRADUATION THESIS
BACHELOR OF ENGINEERING IN
SOFTWARE ENGINEERING
(HIGH-QUALITY PROGRAM)

DESIGN AND IMPLEMENTATION OF BIOFLOW-SCOPE — AN AI-ASSISTED WEB PLATFORM FOR REPRODUCIBLE BIOINFORMATICS PIPELINES

Student: Nguyen Duy Khanh
Student ID: B2205944
Class: 2022-2026

Can Tho, August 2026

CAN THO UNIVERSITY
COLLEGE INFORMATION AND COMMUNICATION TECHNOLOGY
FACULTY OF SOFTWARE ENGINEERING

GRADUATION THESIS
BACHELOR OF ENGINEERING IN
SOFTWARE ENGINEERING
(HIGH-QUALITY PROGRAM)

DESIGN AND IMPLEMENTATION OF BIOFLOW-SCOPE — AN AI ASSISTED WEB PLATFORM FOR REPRODUCIBLE BIOINFORMATICS PIPELINES

Supervisor
Ph.D. Ho Quang Thai

Student:
Nguyen Duy Khanh
Student ID: B2205944
Class: DI2296F2 , K48

Can Tho, August 2026

CAN THO UNIVERSITY
COLLEGE OF INFORMATION AND COMMUNICATION TECHNOLOGY

SOCIALIST REPUBLIC OF VIETNAM
Independence – Freedom – Happiness

THESIS REVISION CONFIRMATION

IN ACCORDANCE WITH THE COMMITTEE'S COMMENTS

Thesis Title: Development of an AI-Integrated Social Network Mobile Application for Can Tho University Students

Student: Nguyen Duy Khanh Student ID: B2205944

Class Code: DI2296F2

The thesis was presented before the Thesis Examination Committee of the Software Engineering Program.

Members of the Examination Committee:

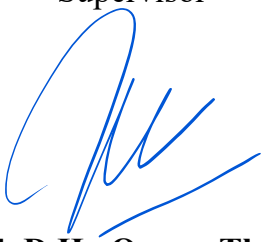
- Ph.D Truong Minh Thai
- Ph.D Nguyen Thanh Khoa
- Ph.D Ho Quang Thai

Reporting date: August 20, 2026

The thesis has been revised in accordance with the Committee's comments.

Can Tho, 16/08/2026

Supervisor



Ph.D Ho Quang Thai

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to the lecturers of the College of Information and Communication Technology, Can Tho University, for their dedicated teaching and for providing me with both foundational and specialized knowledge throughout my studies. This knowledge has been an important foundation that enabled me to complete this thesis.

In particular, I would like to express my deepest gratitude to Ph.D Ho Quang Thai, who directly supervised, guided, and provided valuable feedback throughout the implementation of this project. The supervisor's dedication and insightful comments helped me recognize shortcomings and improve the product to the best possible extent.

Due to my limited knowledge and experience, this project inevitably contains shortcomings. I sincerely welcome comments and suggestions from the lecturers so that the project can be further improved. Thank you very much.

Can Tho, 16/08/ 2026

Student



Nguyen Duy Khanh

SUPERVISOR'S COMMENTS

……
……
……
……
……
……
……
……
……
……
……
……
……
……
……
……
……
……
……
……
……
……
……

Can Tho, /08 / 2026

Supervisor

Ph.D Ho Quang Thai

DECLARATION OF ORIGINALITY

I hereby declare that the thesis “Development of BioFlow-Scope — An AI-Assisted Web Platform for Reproducible Bioinformatics Pipelines” is my own work carried out under the supervision of my supervisor. The contents, data, and results presented in this report are truthful. All references used are fully cited and acknowledged in accordance with the applicable regulations. I take full responsibility for the integrity and truthfulness of the content of this project.

Can Tho, 16 /08/ 2026

Student



Nguyen Duy Khanh

ABSTRACT

Modern bioinformatics research requires scientists to combine many command-line tools (BLAST, MUSCLE, FastQC, SPAdes, etc.) together with several reference databases to analyze genes and proteins. These analysis pipelines are typically hard to reproduce, hard to share, and heavily dependent on programming skills and the local computing environment. Moreover, annotating the biological function of a protein from its raw amino-acid sequence is difficult when relying solely on traditional similarity search, which loses sensitivity for distantly related proteins.

This project builds BioFlow-Scope, an AI-assisted integrated web platform that unifies three functional areas in a single system: reproducible bioinformatics pipeline management through a visual editor with containerized execution; protein semantic search based on protein-language-model embeddings; and an AI research assistant that can operate the platform through tools under human-in-the-loop approval control.

The system is developed as a modular monolith with a Spring Boot 3.3 (Java 21) backend, a Next.js 14 (TypeScript) frontend, and a Python FastAPI sidecar for ESM-2 inference. Bioinformatics tools run as BioContainers in Docker; semantic protein search uses Qdrant for vector storage and retrieval; and the AI assistant is built on LangChain4j with multi-provider model routing. An iterative, milestone-based development process was applied.

The project delivered a working web platform covering authentication and authorization; pipeline authoring, execution and provenance; dataset and artifact management; protein semantic search; an AI assistant combining RAG, tool calling and human-in-the-loop approval; and workspace, sharing and reproducibility functions. The final testing record dated 08 August 2026 reports 47/47 functional test cases passed; backend execution recorded 1,133 passes, one environment-dependent skip and no failures; frontend type checking completed without errors; and 35 Playwright end-to-end scenarios were recorded as passed. JaCoCo measured 51.70% instruction coverage and 57.28% class coverage. These results support the tested local scenarios but are not interpreted as a claim of production readiness or universal scientific accuracy.

Keywords: Bioinformatics, reproducible pipelines, protein semantic search, protein language model, AI assistant, human-in-the-loop approval.

TÓM TẮT

Nghiên cứu tin sinh học hiện đại đòi hỏi nhà khoa học phối hợp nhiều công cụ dòng lệnh (BLAST, MUSCLE, FastQC, SPAdes…) cùng nhiều cơ sở dữ liệu tham chiếu để phân tích gene và protein. Các quy trình phân tích này thường khó tái lập, khó chia sẻ và phụ thuộc nhiều vào kỹ năng lập trình cũng như môi trường máy tính của từng người. Bên cạnh đó, việc chú giải chức năng của một protein từ trình tự axit amin thô là bài toán khó khi chỉ dựa vào so khớp tương đồng truyền thống, vốn mất độ nhạy với những protein có quan hệ họ hàng xa. Thực tế này đặt ra nhu cầu về một nền tảng tập trung, thân thiện với nhà nghiên cứu, giúp quản lý quy trình một cách tái lập và khai thác được sức mạnh của trí tuệ nhân tạo.

Đề tài xây dựng BioFlow-Scope, một nền tảng web tích hợp trí tuệ nhân tạo hợp nhất ba nhóm chức năng trong cùng một hệ thống: quản lý và thực thi quy trình (pipeline) tin sinh học tái lập được thông qua trình soạn thảo trực quan và thực thi trong container; tìm kiếm protein theo ngữ nghĩa dựa trên embedding của mô hình ngôn ngữ protein; và một trợ lý nghiên cứu AI có khả năng thao tác nền tảng thông qua các công cụ, dưới sự kiểm soát phê duyệt của con người.

Hệ thống được phát triển theo kiến trúc modular monolith với backend Spring Boot 3.3 (Java 21), giao diện Next.js 14 (TypeScript) và một sidecar Python FastAPI phục vụ suy luận mô hình ESM-2. Việc thực thi công cụ tin sinh sử dụng các image BioContainers chạy trong Docker; tìm kiếm ngữ nghĩa sử dụng Qdrant để lưu và truy vấn vector protein; trợ lý AI được xây dựng trên LangChain4j với khả năng định tuyến nhiều nhà cung cấp mô hình. Đề tài áp dụng quy trình phát triển lặp và tăng dần theo các cột mốc (milestone).

Đề tài đã xây dựng được một hệ thống web tích hợp các nhóm chức năng gồm xác thực và phân quyền; thiết kế, thực thi và lưu provenance của pipeline; quản lý dataset và artifact; tìm kiếm protein theo ngữ nghĩa; trợ lý AI kết hợp RAG, tool calling và cơ chế phê duyệt Human-In-The-Loop; cùng các chức năng workspace, chia sẻ và tái lập kết quả. Các kết quả này xác nhận hệ thống đáp ứng những kịch bản đã kiểm thử trên môi trường cục bộ, nhưng không được diễn giải như chứng nhận production readiness hoặc độ chính xác khoa học trong mọi điều kiện.

Từ khóa: tin sinh học, quy trình tái lập, tìm kiếm protein theo ngữ nghĩa, mô hình ngôn ngữ protein, trợ lý AI, phê duyệt con người trong vòng lặp.

TABLE OF CONTENTS

THESIS REVISION CONFIRMATION i
ACKNOWLEDGEMENTS ii
SUPERVISOR'S COMMENTS iii
DECLARATION OF ORIGINALITY iv
ABSTRACT v
TÓM TẮT vi
TABLE OF CONTENTS vii
LIST OF FIGURES xii
LIST OF TABLES xiv
ABBREVIATIONS AND TERMS xvii
PART 1. INTRODUCTION 1
1. Problem Statement 1
2. History of Approaches to the Problem 1
2.1. International Context 1
2.2. Domestic Context 3
3. Project Objectives 3
3.1. General Objective 3
3.2. Specific Objectives 3
4. Research Subject and Scope 4
4.1. Research Subject 4
4.2. Research Scope 4
5. Research Process and Technologies Used 4
5.1. Research Process 4
5.2. Technologies Used 5
5.3. Development and Supporting Tools 5
6. Main Contributions of the Project 5
7. Structure of the Report 6
PART 2. CONTENT 8
CHAPTER 1. PROBLEM DESCRIPTION AND SYSTEM REQUIREMENTS 8
1.1. Problem Description 8
1.2. System Scope and Boundaries 8

1.2.1. Functional Scope of the Thesis....8
1.2.2. Content Outside the Current Scope....9
1.2.3. Integration Boundaries....9
1.3. Actors and Roles....9
1.4. Functional Requirements....14
1.4.1. Diagram of 45 Key BioFlow-Scope Use Cases....14
1.4.2. Specifications of Core Business Use Cases....15
1.5. Non-Functional Requirements....57
1.5.1. Performance and Processing Requirements....57
1.5.2. Safety, Reliability, and Data Integrity Requirements....58
1.5.3. Usability and Interface Requirements....58
1.5.4. Security Requirements....59
1.6. Operating Environment....59
1.7. Design Assumptions and Constraints....60
CHAPTER 2. THEORETICAL BACKGROUND AND TECHNOLOGIES....62
2.1. Bioinformatics Analysis Workflows....62
2.1.1. Essential Molecular Biology Background....62
2.1.2. Sequencing Data and the Concept of Reads....64
2.1.3. Sequence Alignment, Assembly, and Biological Inference....65
2.1.4. Analysis Approaches Supported by BioFlow-Scope....66
2.2. Workflow, Pipeline, and DAG....68
2.2.1. Workflow Components....69
2.2.2. Directed Acyclic Graph (DAG)....69
2.2.3. Limitations of the DAG Model....71
2.3. Reproducibility and Data Provenance....71
2.3.1. Distinguishing Repeatability, Reproducibility, and Replicability....71
2.3.2. Provenance and Data Lineage....72
2.3.3. Why Bioinformatics Reproducibility Is Difficult....73
2.3.4. FAIR, PROV-O, RO-Crate, and BioCompute Object....74
2.4. Embeddings and Vector Search....76
2.4.1. Distributed Representations and Latent Space....76
2.4.2. Similarity and Nearest-Neighbor Search....76
2.4.3. Two Types of Embeddings in BioFlow-Scope....77

2.4.4. Vector Search for RAG 79
2.4.5. Scientific and Technical Limitations of Embeddings 80
2.5. Semantic protein search 80
2.5.1. From a DNA Gene to a Protein Query Sequence 80
2.5.2. Processing flow of protein search 80
2.5.3. Differences from BLAST and Alignment-Based Search 81
2.6. AI Agent and Tool Calling 82
2.6.1. Distinguishing Chatbots, RAG, and AI Agents 82
2.6.2. Lifecycle of a Tool Call 83
2.6.3. RAG, Memory, and Evidence 83
2.6.4. Human-in-the-Loop and Action Control 83
CHAPTER 3. SOLUTION DESIGN AND IMPLEMENTATION 85
3.1. System Architecture 85
3.1.1. Overall System Architecture 85
3.1.2. Container Diagram 87
3.2. Data Design 98
3.2.1. Data Classification and Storage Strategy 98
3.2.2. Conceptual Data Model 98
3.2.3. Physical Data Model 100
3.2.4. Vector and Temporary Data 110
3.2.5. Detailed Data Dictionary 110
3.3. Design of AI Functions 121
3.3.1. Overview of AI Functions 121
3.3.2. Semantic Protein Search 123
3.3.3. AI Agent, RAG, and Tool Calling 127
3.3.4. Human-in-the-Loop, Evidence, and AI Policy 131
3.3.5. Other Supporting AI Functions 135
3.4. Design of Business Functions 135
3.4.1. Guest Functions 135
3.4.2. Researcher Functions 137
3.4.3. Extended Functions of the Workspace Owner 154
3.4.4. Administrator Functions 160
3.4.5. Scheduler and System Functions 166
CHAPTER 4. TESTING AND EVALUATION 170

4.1. Testing Objectives....170
4.2. Test plan....170
4.2.1. Functions Under Test....170
4.2.2. Approach....170
4.2.3. Criteria for Test Success / Failure....171
4.2.4. Suspension Criteria and Resumption Requirements....171
4.2.5. Test Deliverables....172
4.3. Test management....172
4.3.1. Test Activities and Process....172
4.3.2. Test Environment....172
4.3.3. Communication Among Relevant Groups....172
4.3.4. Resources and Allocation....173
4.3.5. Training....173
4.3.6. Risks....173
4.4. Test scenarios....173
4.5. Test cases....174
4.5.1. Functional Testing....174
4.6. Test evaluation....179
PART 3. CONCLUSION....181
1. Achieved Results....181
1.1. Knowledge and Skills....181
1.2. Program and Practical Application Potential....181
2. Limitations and Challenges....182
REFERENCES....183
APPENDIX A. BIOFLOW-SCOPE INSTALLATION AND USER GUIDE 186
A.1. Deployment Scope....186
A.2. Environment Requirements....187
A.3. Preparing Source Code and Configuration....187
A.4. Starting the Infrastructure and Application....187
A.5. Preparing the Local AI Model....188
A.6. Initial Usage Flow....189
A.7. Testing and Build....189
A.8. Stopping the System and Troubleshooting....189
APPENDIX B. API, DATABASE, AND STATE MACHINES....191

B.1. API Architecture.....191
B.2. Database Table Groups 192
B.3. Main State Machines 192
B.4. API Design and State Transition Rules.....193

LIST OF FIGURES

Figure 0.1. Galaxy workflow management interface 2
Figure 0.2. nf-core workflow list interface 2
Figure 0.3. UniProt/BLAST similarity search interface 3
Figure 0.4. BioFlow-Scope research and development process 5
Figure 1.1. General Use Case diagram of BioFlow-Scope 10
Figure 1.2. Diagram of 45 BioFlow-Scope Use Cases 14
Figure 2.1. DNA–RNA–protein genetic information flow 62
Figure 2.2. DNA fragmentation, sequencing, and assembly 64
Figure 2.3. Structure of a FASTQ record 65
Figure 2.4. Reference alignment and de novo assembly 65
Figure 2.5. Levels of de novo assembly 66
Figure 2.6. RNA-seq analysis workflow 66
Figure 2.7. Bioinformatics analysis branches in BioFlow-Scope 67
Figure 2.8. Example of a directed acyclic graph 70
Figure 2.9. RNA-seq workflow in Galaxy 70
Figure 2.10. Read-analysis pipeline represented as a DAG 70
Figure 2.11. W3C PROV data model 72
Figure 2.12. Simplified provenance model of BioFlow-Scope 72
Figure 2.13. Illustration of an HNSW index 76
Figure 2.14. Generating protein embeddings using average pooling 79
Figure 2.15. Retrieval-Augmented Generation architecture 79
Figure 2.16. Semantic Protein Search flow in BioFlow-Scope 80
Figure 2.17. Protein search using a protein language model 82
Figure 2.18. Reason Only, Act Only, and ReAct mechanisms 83
Figure 3.1. Overall architecture of BioFlow-Scope 86
Figure 3.2. C4 Container Diagram – BioFlow-Scope 88
Figure 3.3. C4 Component Diagram – Pipeline Execution and Provenance 91
Figure 3.4. C4 Component Diagram – Semantic Protein Search 93
Figure 3.5. C4 Component Diagram – AI Agent Core 95
Figure 3.6. C4 Component Diagram – AI Governance, Approval and Eviden 97
Figure 3.7. Conceptual data model of BioFlow-Scop 99
Figure 3.8. Physical data model – identity & administration 100
Figure 3.9. Physical data model – workspace & collaboration 101
Figure 3.10. Physical data model – pipeline & execution 102
Figure 3.11. Physical data model – data source & trigger 103
Figure 3.13. Physical data model – research project & memory 105
Figure 3.14. Physical data model – AI Agent core 106
Figure 3.15. Physical data model – AI governance & evidence 107

Figure 3.16. Physical data model – publication & reproducibility 108
Figure 3.17. Physical data model – protein search & biodatabase 109
Figure 3.18. Semantic Protein Search interface 123
Figure 3.19. Activity Diagram – Semantic Protein Search 126
Figure 3.20. AI Agent Chat, RAG, and Tool Calling interface 128
Figure 3.21. Activity Diagram – AI Agent, RAG, and Tool Calling 130
Figure 3.22. HITL, Evidence, and AI Policy interface 132
Figure 3.23. Activity Diagram – Human-in-the-Loop, Evidence, and AI Policy 134
Figure 3.24. Publicly shared Run interface 135
Figure 3.25. Activity Diagram – Accessing a publicly shared Run 137
Figure 3.26. Smart Canvas interface – Pipeline design 137
Figure 3.27. Activity Diagram – Designing and editing a Pipeline on Smart Canvas 139
Figure 3.28. Pipeline version-management and Fork interface 140
Figure 3.29. Activity Diagram – Managing Pipeline versions and Fork 141
Figure 3.30. Data-configuration and Run-start interface 142
Figure 3.31. Activity Diagram – Configuring data and starting a Run 143
Figure 3.32. Run monitoring and control interface 144
Figure 3.33. Activity Diagram – Monitoring and controlling a Run 145
Figure 3.34. Resume/Rerun and Run-comparison interface 146
Figure 3.35. Activity Diagram – Resuming, rerunning, and comparing Runs 147
Figure 3.36. Dataset Library and Dataset Collection interface 148
Figure 3.37. Activity Diagram – Managing Datasets, Collections, and Lineage 149
Figure 3.38. Artifact and result viewing interface using the Inline Viewer 150
Figure 3.39. Activity Diagram – Viewing Artifacts and results with the Inline Viewer 151
Figure 3.40. Analyze History and result-publication interface 152
Figure 3.41. Activity Diagram – Analyze History and result publication 154
Figure 3.42. Workspace member invitation and onboarding interface 155
Figure 3.43. Activity Diagram – Inviting and onboarding Workspace members 156
Figure 3.44. Role, member, and permission-policy management interface 157
Figure 3.45. Activity Diagram – Managing roles, members, and permission policies 158
Figure 3.46. Workspace lifecycle-management interface 159
Figure 3.47. Activity Diagram – Managing the Workspace lifecycle 160
Figure 3.48. Workspace creation and Owner-assignment interface 161
Figure 3.49. Activity Diagram – Creating a Workspace and assigning an Owner 162
Figure 3.50. User and quota management interface 163
Figure 3.51. Activity Diagram – Managing users and quotas 164
Figure 3.52. Database and system-resource management interface 165
Figure 3.53. Activity Diagram – Managing reference databases and system resources 166
Figure 3.54. Data Source and Pipeline Trigger interface 167
Figure 3.55. Activity Diagram – Scheduled and data-triggered Pipeline execution 169

LIST OF TABLES

Table 1.1. Use Case Specification — UC05 15
Table 1.2. Use Case Specification — UC11 17
Table 1.3. Use Case Specification — UC13 18
Table 1.4. Use Case Specification — UC14 20
Table 1.5. Use Case Specification — UC17 21
Table 1.6. Use Case Specification — UC19 23
Table 1.7. Use Case Specification — UC20 25
Table 1.8. Use Case Specification — UC27 27
Table 1.9. Use Case Specification — UC29 28
Table 1.10. Use Case Specification — UC30 30
Table 1.11. Use Case Specification — UC47 31
Table 1.12. Use Case Specification — UC48 32
Table 1.13. Use Case Specification — UC79 34
Table 1.14. Use Case Specification — UC97 36
Table 1.15. Use Case Specification — UC38 37
Table 1.16. Use Case Specification — UC53 40
Table 1.17. Use Case Specification — UC61 42
Table 1.18. Use Case Specification — UC68 45
Table 1.19. Use Case Specification — UC58 48
Table 1.20. Use Case Specification — UC62 50
Table 1.21. Use Case Specification — UC87 52
Table 1.22. Use Case Specification — UC100 53
Table 1.23. Use Case Specification — UC34 55
Table 1.24. Use Case Specification — UC37 56
Table 2.1. Fundamental biological terms 62
Table 2.2. Representative bioinformatics analysis approaches in BioFlow-Scope 67
Table 2.3. Workflow and pipeline components 69
Table 2.4. Levels of computational result verification 71
Table 2.5. Causes of reduced reproducibility and control measures 73
Table 2.6. Provenance and reproducibility standards and mechanisms 74
Table 2.7. Embedding and vector-search concepts 77
Table 2.8. Two embedding flows in BioFlow-Scope 78
Table 2.9. Information returned by Semantic Protein Search 80
Table 2.10. Levels of AI support in BioFlow-Scope 82
Table 2.11. AI Agent control layers in the source code 84
Table 3.1. Data classification and storage strategy 98
Table 3.2. Vector stores and temporary data 110

Table 3.3. Structure of the users table 110
Table 3.4. Structure of the workspaces table 112
Table 3.5. Structure of the runs table 113
Table 3.6. Structure of the pipelines table 116
Table 3.7. Structure of the agent_tasks table 117
Table 3.8. Structure of the agent_sessions table 120
Table 3.9. Summary of BioFlow-Scope AI functions 121
Table 3.10. Input/output data – Semantic Protein Search 123
Table 3.11. Interface components – Semantic Protein Search 124
Table 3.12. Input/output data – AI Agent 127
Table 3.13. Interface components – AI Agent, RAG, and Tool Calling 128
Table 3.14. Input/output data – HITL, Evidence, and AI Policy 131
Table 3.15. Interface components – Human-in-the-Loop 132
Table 3.16. Interface components – Accessing a publicly shared Run 136
Table 3.17. Data – Accessing a publicly shared Run 136
Table 3.18. Interface components – Designing and editing a Pipeline on Smart Canvas 137
Table 3.19. Data – Designing and editing a Pipeline on Smart Canvas 138
Table 3.20. Interface components – Managing Pipeline versions and Fork 140
Table 3.21. Data – Managing Pipeline versions and Fork 140
Table 3.22. Interface components – Configuring data and starting a Run 142
Table 3.23. Data – Configuring data and starting a Run 143
Table 3.24. Interface components – Monitoring and controlling a Run 144
Table 3.25. Data – Monitoring and controlling a Run 145
Table 3.26. Interface components – Resuming, rerunning, and comparing Runs 146
Table 3.27. Data – Resuming, rerunning, and comparing Runs 146
Table 3.28. Interface components – Managing Datasets, Collections, and Lineage 148
Table 3.29. Data – Managing Datasets, Collections, and Lineage 149
Table 3.30. Interface components – Viewing Artifacts and results with the Inline Viewer 150
Table 3.31. Data – Viewing Artifacts and results with the Inline Viewer 150
Table 3.32. Interface components – Managing Analyze History and publishing reproducible results 152
Table 3.33. Data – Managing Analyze History and publishing reproducible results 153
Table 3.34. Interface components – Inviting and onboarding Workspace members 155
Table 3.35. Data – Inviting and onboarding Workspace members 155
Table 3.36. Interface components – Managing roles, members, and permission policies 157
Table 3.37. Data – Managing roles, members, and permission policies 157
Table 3.38. Interface components – Managing the Workspace lifecycle 159

Table 3.39. Data – Managing the Workspace lifecycle 160
Table 3.40. Interface components – Creating a Workspace and assigning an Owner ... 161
Table 3.41. Data – Creating a Workspace and assigning an Owner 161
Table 3.42. Interface components – Managing users and quotas 163
Table 3.43. Data – Managing users and quotas 163
Table 3.44. Interface components – Managing reference databases and system resources 165
Table 3.45. Data – Managing reference databases and system resources 165
Table 3.46. Interface components – Executing Pipelines by schedule and data trigger 167
Table 3.47. Data – Executing Pipelines by schedule and data trigger 168
Table 4.1. Functional groups under test 170
Table 4.2. Criteria for determining test pass/failure 171
Table 4.3. Test suspension and resumption criteria 171
Table 4.4. Test deliverables 172
Table 4.5. Testing activities 172
Table 4.6. Test environment 172
Table 4.7. Testing risks 173
Table 4.8. Test scenarios 173
Table 4.9. Functional testing – Authentication and accounts 174
Table 4.10. Functional testing – Pipeline and Run 175
Table 4.11. Functional testing – Protein Semantic Search 176
Table 4.12. Functional testing – Dataset 177
Table 4.13. Functional testing – Viewer 177
Table 4.14. Functional testing – AI Agent 178
Table 4.15. Functional testing – Workspace/Project 178
Table 4.16. Functional testing – Reproducibility 179
Table 4.17. Functional testing – Administration 179
Table 4.18. Summary of test results 179
Table A.1. BioFlow-Scope deployment components 186
Table A.2. Post-startup service checks 188
Table A.3. Common problems and troubleshooting 189
Table J.1. Main API endpoint groups 191
Table J.2. Main database-table groups 192
Table J.3. Main state machines 192

ABBREVIATIONS AND TERMS

No.	Term / Abbreviation	Definition / Explanation
1	AI	Artificial Intelligence
2	API	Application Programming Interface
3	BLAST	Basic Local Alignment Search Tool — sequence similarity search tool
4	DAG	Directed Acyclic Graph
5	DTO	Data Transfer Object
6	ERD	Entity Relationship Diagram
7	ESM-2	Evolutionary Scale Modeling 2 — protein language model
8	FASTA/FASTQ	Biological sequence storage formats (sequence / sequence with quality scores)
9	HITL	Human-in-the-Loop — human participation in the approval loop
10	JWT	JSON Web Token
11	LLM	Large Language Model
12	MSA	Multiple Sequence Alignment
13	ORM	Object-Relational Mapping
14	PDB	Protein Data Bank
15	RAG	Retrieval-Augmented Generation
16	REST	Representational State Transfer
17	SSE	Server-Sent Events — server-pushed event stream
18	UUID	Universally Unique Identifier
19	VCF	Variant Call Format — format for storing genetic variants
20	YAML	YAML Ain’t Markup Language — text-based configuration language

PART 1. INTRODUCTION

1. Problem Statement

Bioinformatics analysis often requires combining multiple tools, data formats, and reference databases. When individual steps are performed separately using command-line tools, notebooks, and personal directories, managing parameters, tool versions, intermediate data, and result provenance becomes difficult, reducing reproducibility and creating barriers for users without strong programming expertise.

BioFlow-Scope was developed to centralize these activities in a web platform: designing and executing containerized pipelines, managing datasets and artifacts, recording provenance, performing semantic protein search using embeddings, and assisting users through an AI Agent. Risky AI actions are controlled through authorization, policy, and Human-in-the-Loop mechanisms.

The project focuses on creating a locally deployable, easy-to-use environment for gene and protein analysis with support for traceability, verification, and reproducibility of results.

2. History of Approaches to the Problem

Existing systems address individual parts of the problem effectively, but differ in visualization, reproducibility, protein search, and AI support. Representative systems used for comparison with BioFlow-Scope include Galaxy, Nextflow/nf-core, BLAST/UniProt, and cloud-based analysis platforms.

2.1. International Context

Galaxy provides an intuitive web interface for building and running bioinformatics workflows, with strong support for accessibility and reproducibility; however, self-hosting is relatively resource-intensive and the platform does not focus on embedding-based protein search or AI Agents [1].

Nextflow and nf-core support code-based workflows, containerization, and deployment across multiple infrastructures; nf-core provides community-developed pipelines built on Nextflow. This approach is well suited to large computational workflows but requires users to work more extensively with configuration and workflow code [2], [3].

BLAST and UniProt are standard tools for sequence-based similarity search and protein annotation [4], [5]. BioFlow-Scope does not replace these tools; instead, it

adds a semantic search layer based on protein embeddings and connects the results to analysis workflows.

Commercial platforms such as DNAnexus and Seven Bridges provide powerful cloud infrastructure, but they target large-scale deployments and incur higher operating costs than the local-deployment objective of this project.

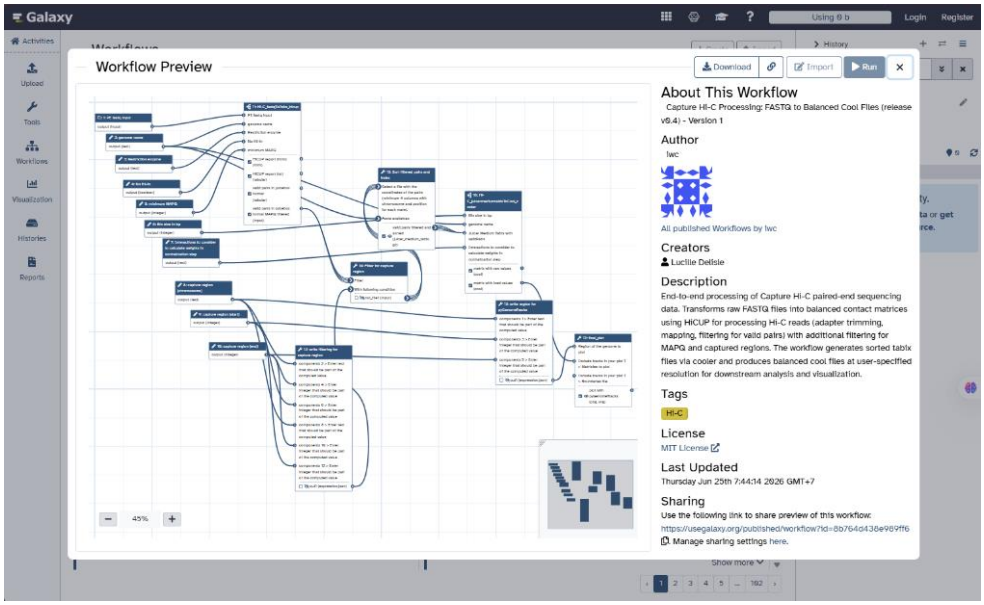


Figure 0.1. *Galaxy workflow management interface*

Source: Galaxy Project [1].

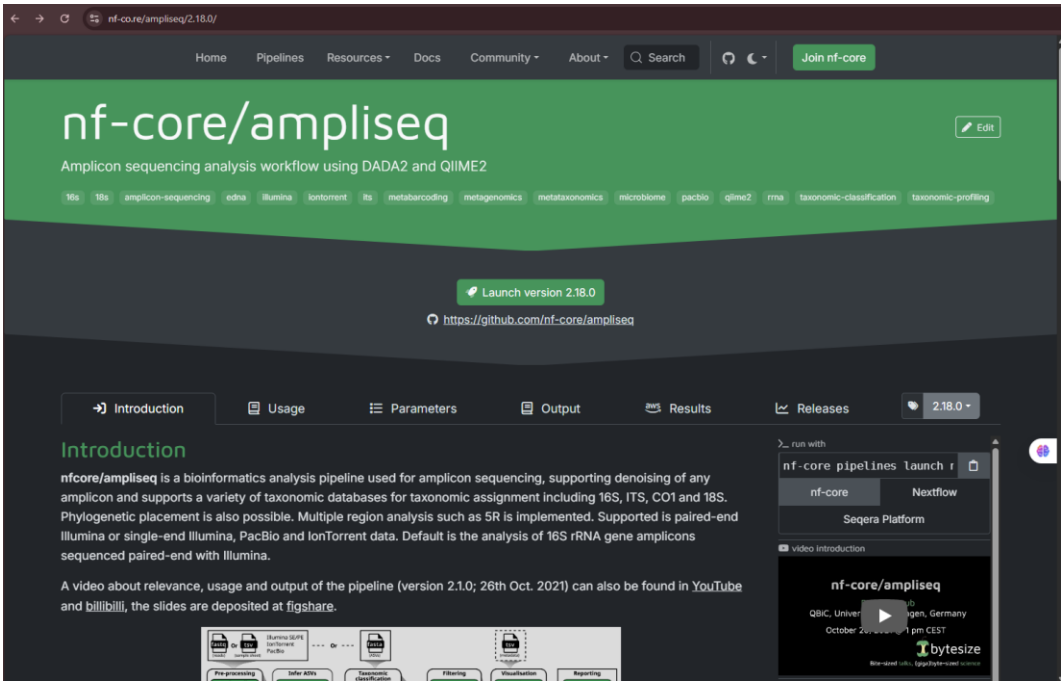


Figure 0.2. *nf-core workflow list interface*

Source: nf-core.

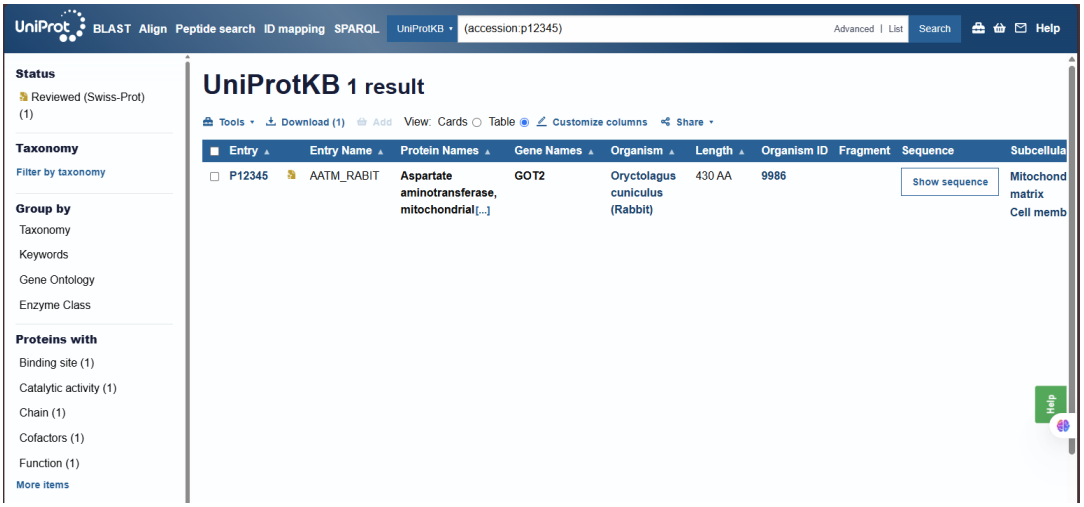


Figure 0.3. *UniProt/BLAST similarity search interface*

Source: UniProt [5] and BLAST [4].

2.2. Domestic Context

In Vietnam, bioinformatics activities still rely primarily on international tools such as BLAST, Galaxy, R/Python, and pipelines installed independently by individual research groups. The integration of reproducible workflows, data management, protein embeddings, and AI assistance within a unified platform remains uncommon.

BioFlow-Scope addresses this gap through a locally deployable web system that combines pipeline management, data management, provenance, semantic protein search, and a controlled AI Agent.

3. Project Objectives

3.1. General Objective

Design and develop BioFlow-Scope, a web platform that supports researchers in creating and executing reproducible bioinformatics pipelines, managing data and results, searching proteins using semantic embeddings, and using a controlled AI Agent.

3.2. Specific Objectives

Develop a client-server web system with a Spring Boot backend, Next.js frontend, and PostgreSQL, Redis, and Qdrant data infrastructure, with support for local deployment using Docker.

Provide Smart Canvas/YAML for pipeline design, DAG validation, containerized tool execution, Run monitoring, and provenance recording, including versions, parameters, input data, logs, and artifacts.

Integrate Semantic Protein Search using ESM-2 and Qdrant, together with UniProt/PDB data, to support protein retrieval.

Develop an AI Agent using RAG and Tool Calling; apply authorization, policy, evidence/citation, and Human-in-the-Loop controls to risky actions.

Ensure that major functions are tested at the backend, integration, and user-interface levels, while clearly documenting limitations outside the completed scope of the thesis.

4. Research Subject and Scope

4.1. Research Subject

The research subject is a software platform supporting bioinformatics analysis workflows, focusing on workflow and provenance management, research data, containerized tool execution, semantic protein search, controlled AI Agents, and the user experience of a web application.

4.2. Research Scope

Functionally, the thesis focuses on authentication and authorization; Pipeline/Smart Canvas; Run and provenance; Dataset, Collection, Analyze History, and Viewer; Semantic Protein Search; AI Agent, RAG, Tool Calling, evidence, and Human-in-the-Loop; Workspace, sharing, and related administrative functions.

For deployment, the system is evaluated as a web application running locally on a server or personal computer. The backend uses a Spring Boot modular monolith; bioinformatics tools run through Docker; protein embeddings are served by a FastAPI sidecar; Qdrant stores vectors; and Redis supports caching and rate limiting.

Out-of-scope items currently include a mobile application, Kubernetes or commercial multi-region deployment, training new foundation models, object storage/message brokers without implementation evidence, and the incomplete BB340 component. Large-scale protein indexing tasks are performed offline and then imported into the system.

5. Research Process and Technologies Used

5.1. Research Process

The implementation process consists of four main stages: surveying the problem and related systems; analyzing requirements and designing the architecture, data, and interface; implementing individual subsystems; and testing and evaluating the results. Requirements and designs were iteratively refined based on the source code, migrations, configuration, and test results. During implementation, subsystems were

developed by module: authentication, pipeline/run, dataset/viewer, protein search, AI Agent, workspace, and publication. Testing was carried out in parallel using unit tests, integration tests, Playwright, and manual testing.

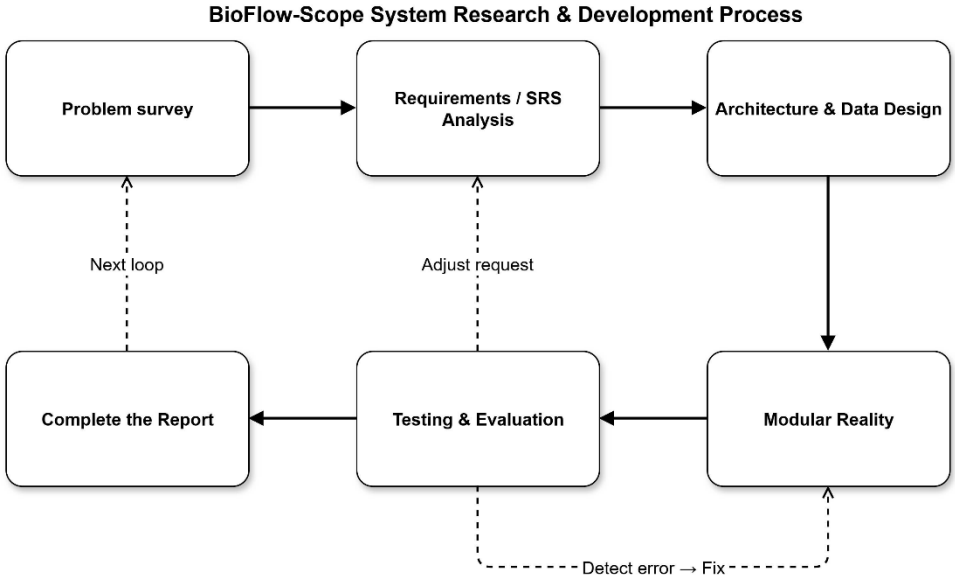


Figure 0.4. *BioFlow-Scope research and development process*

5.2. Technologies Used

The backend uses Java 21, Spring Boot 3.3, Spring Security, PostgreSQL/Flyway, Redis, and Qdrant. The frontend uses Next.js, React, TypeScript, React Flow, and CodeMirror. A Python/FastAPI service runs ESM-2 and text embedding; the AI Agent is integrated through LangChain4j and Ollama or an external provider according to configuration. Docker/BioContainers are used to isolate bioinformatics tool execution environments. [6]

5.3. Development and Supporting Tools

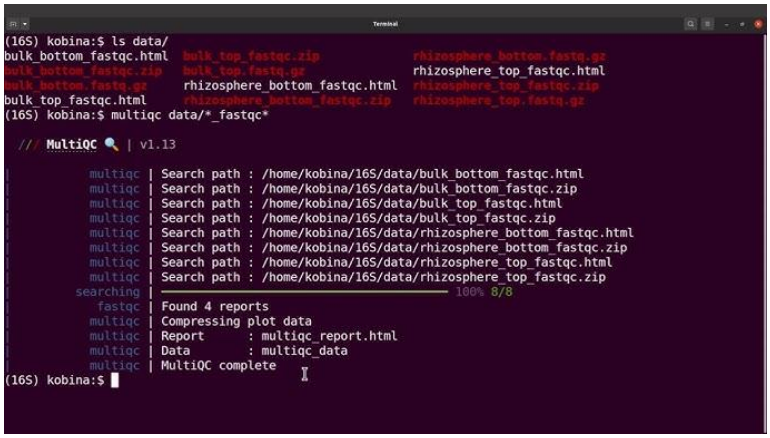
Gradle and npm manage builds and dependencies; Docker Compose starts the local infrastructure. JUnit 5, Mockito, Testcontainers, Playwright, TypeScript strict mode, and JaCoCo are used for testing and quality control; detailed results are presented in Chapter 4.

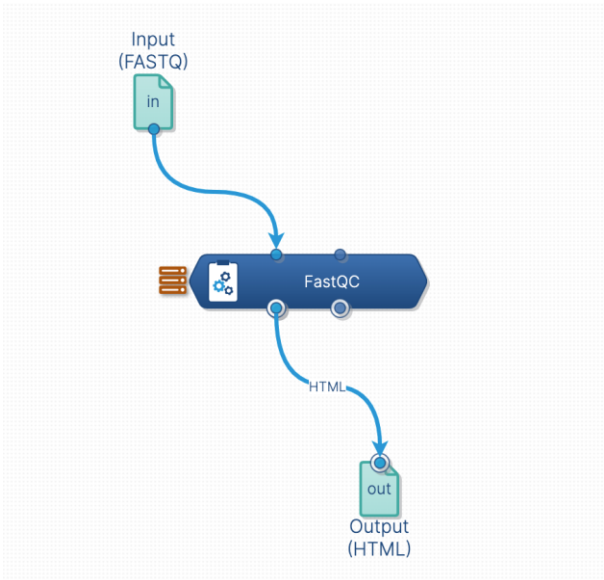
6. Main Contributions of the Project

In terms of the product, the project delivers a web platform that unifies pipeline design, containerized tool execution, data management, provenance, result viewing, semantic protein search, and AI assistance within a single locally deployable environment.

From a scientific and collaborative perspective, each Run is linked to a pipeline version, parameters, data, logs, and artifacts; History, sharing, and provenance export mechanisms support verification and reproducibility. The AI Agent is constrained by authorization, policy, evidence, and Human-in-the-Loop controls rather than being allowed to modify data freely.

From a technical perspective, the system demonstrates how a modular monolith, relational and vector databases, Docker/BioContainers, protein embeddings, RAG, and AI safety mechanisms can be combined in an application for bioinformatics research.





7. Structure of the Report

The thesis consists of two main parts. The Introduction presents the background, objectives, scope, research process, and contributions of the project. The Content section consists of four chapters:

Chapter 1 describes the problem and system requirements;

Chapter 2 presents the theoretical background and technologies;

Chapter 3 presents the design and implementation;

Chapter 4 presents testing and evaluation. The references and appendices provide cited sources and supplementary technical information.

PART 2. CONTENT

CHAPTER 1. PROBLEM DESCRIPTION AND SYSTEM REQUIREMENTS

This chapter defines the problem, scope, actors, and major requirements of BioFlow-Scope based on the existing specifications and source code.

1.1. Problem Description

Bioinformatics workflows typically involve multiple tools, data items, and dependent steps. The difficulty lies not only in running tools, but also in managing parameters, versions, intermediate data, execution environments, and result traceability. Manual operations across multiple programs increase the risk of inconsistencies and make results difficult to reproduce.

BioFlow-Scope integrates three groups of capabilities: designing and executing DAG-based pipelines in containers; Semantic Protein Search using embeddings; and an AI Agent with RAG, tool calling, and Human-in-the-Loop mechanisms. The system also stores provenance, checksums, artifacts, and Run states to support result verification.

1.2. System Scope and Boundaries

1.2.1. Functional Scope of the Thesis

The main scope of the thesis includes the BioFlow-Scope platform with the following functional groups:

- User authentication, profiles, and account configuration.
- Pipeline management, pipeline versioning, and Smart Canvas.
- Launching, executing, monitoring, canceling, resuming, and rerunning analysis sessions.
- Managing datasets, collections, data sources, and Analyze History.
- Sequence-based protein search, enrichment with UniProt/PDB information, and direct artifact viewing.
- AI Agent, agent tasks, evidence, citations, AI policy, and human-in-the-loop approval.
- Workspace, research projects, members, roles, and collaboration.
- Template Gallery, dashboard, notifications, and administrative functions.

- Run sharing, export/import of reproducibility packages, provenance, and ORCID linking.

1.2.2. Content Outside the Current Scope

The following items are not considered completed outcomes of the thesis:

- A dedicated mobile application; the current system is evaluated as a web application.
- Large-scale commercial deployment, multi-region architecture, or a complete Kubernetes infrastructure.
- S3/MinIO object storage and Kafka/RabbitMQ message brokers.
- BB340 and specialized prediction models are not yet complete. BB340 is considered only an example for the future development of a mechanism to integrate self-developed tools through a remote worker.

1.2.3. Integration Boundaries

BioFlow-Scope manages users, pipelines, execution states, business data, access permissions, and provenance. External services such as UniProt/PDB, data sources, LLM providers, Docker Engine, and identity systems provide integration capabilities only; their data or responses are controlled through the backend.

1.3. Actors and Roles

Actors are separated according to their permission scopes; Figure 1.1 presents the overall relationships, while the subsequent diagrams show functions by actor.

Guest: registers, signs in, and views publicly shared Runs/reports through a valid token; cannot access private resources.

Researcher: the primary business actor, managing Pipelines, Runs, Datasets, History, Protein Search, Artifacts, the AI Agent, and sharing/reproducibility functions within the granted scope.

Workspace Owner: a Researcher with additional Workspace administration permissions, including members, roles, policies, quotas, and the lifecycle of shared resources.

Administrator: manages users, quotas, reference databases, system resources, notifications, and system-wide auditing.

AI Assistant (AI Agent): supports retrieval, planning, and tool calling but has no independent authority; side-effecting actions must comply with policy, user permissions, and HITL controls.

Scheduler: evaluates schedules/data triggers and creates background Runs according to configuration, while complying with quotas and duplicate-prevention mechanisms.

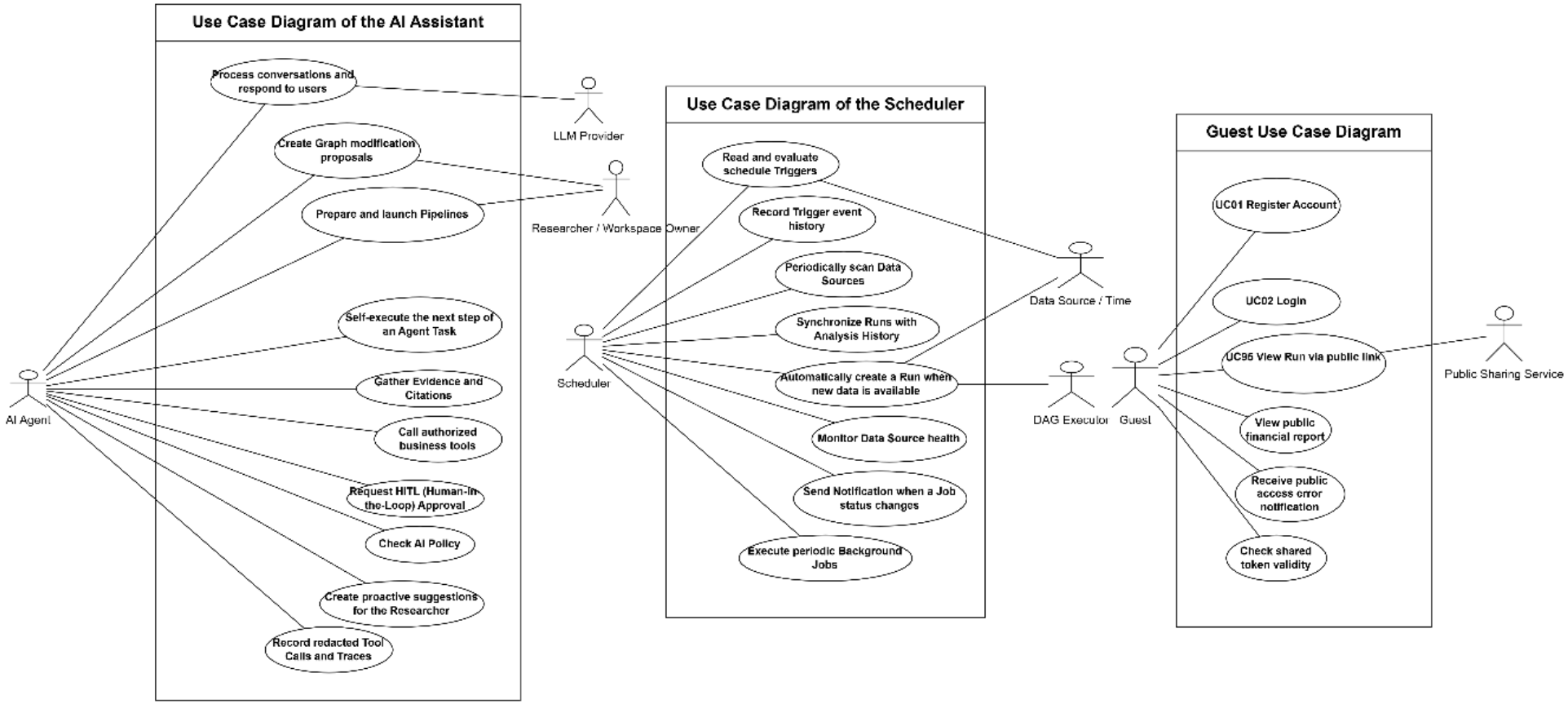
External systems: UniProt, PDB, SRA/Ensembl, ORCID, LLM providers, SMTP, Qdrant, and Docker Engine provide integration services and are not direct users.

General Use Case diagram of BioFlow-Scope:



Figure 1.1. *General Use Case diagram of BioFlow-Scope*

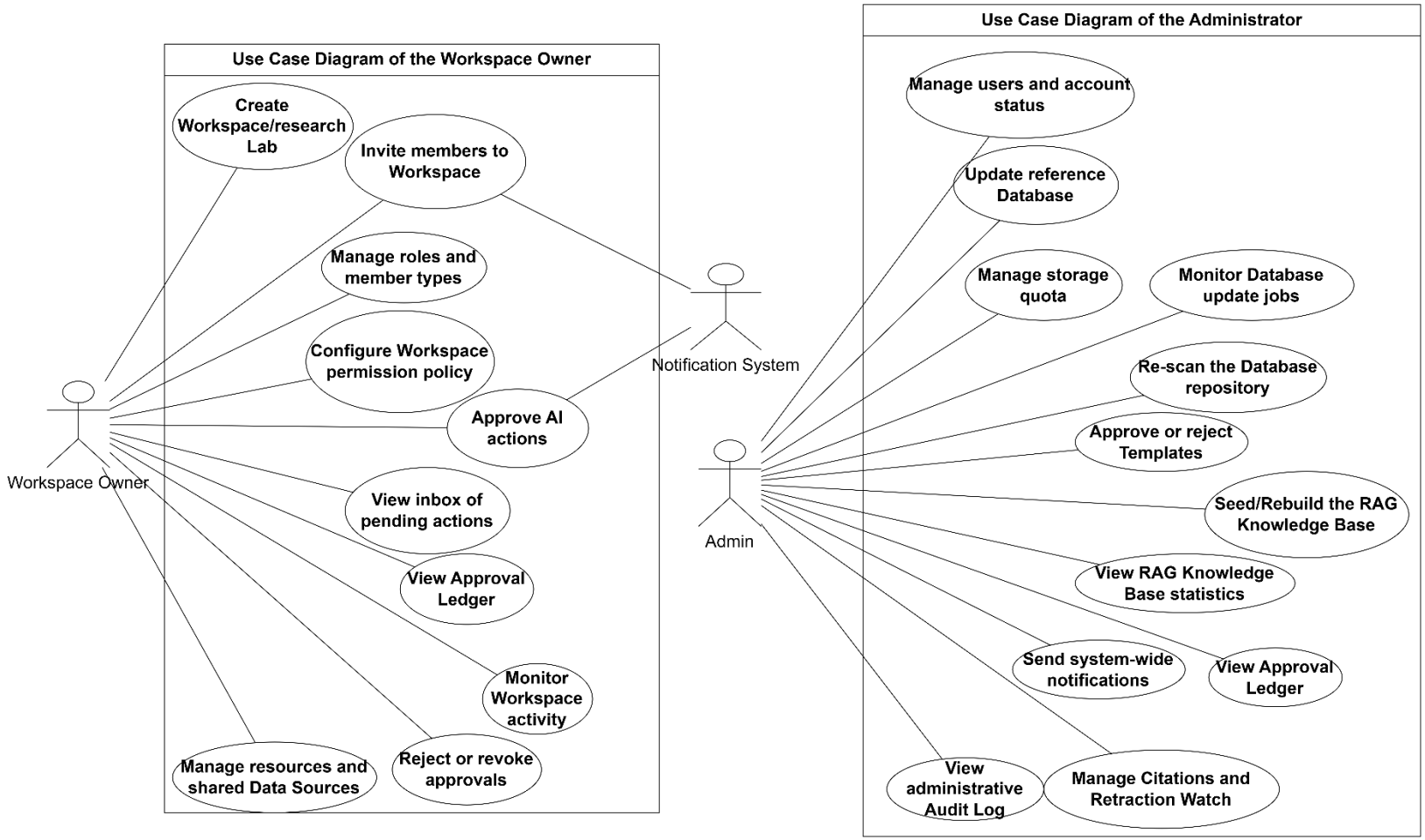
Use Case diagram for Guest, AI Assistant, and Scheduler:



Use Case diagram for the Researcher:



Use Case diagram for the Workspace Owner and Administrator:



1.4. Functional Requirements

1.4.1. Diagram of 45 Key BioFlow-Scope Use Cases

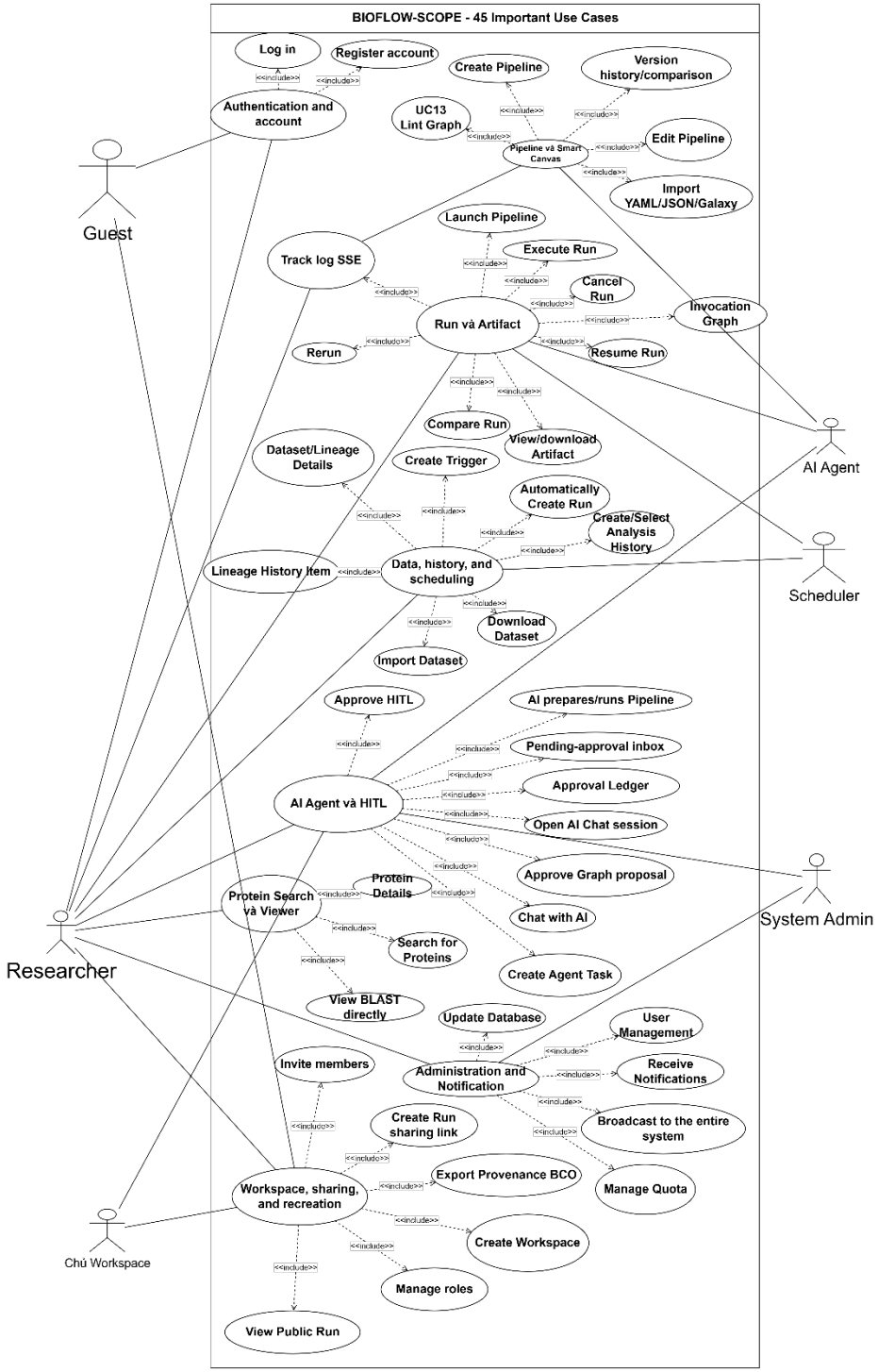


Figure 1.2. *Diagram of 45 BioFlow-Scope Use Cases*

1.4.2. Specifications of Core Business Use Cases

The main content specifies 24 Use Cases with high illustrative value; supporting, CRUD, or navigation Use Cases are summarized in the 45-Use-Case diagram in Section 1.4.1 to keep the report concise while still representing the full scope.

1.4.2.1. Pipeline, Run, Data, and Reproducibility

1.4.2.1.1. UC05: Create a New Pipeline (Create Pipeline)

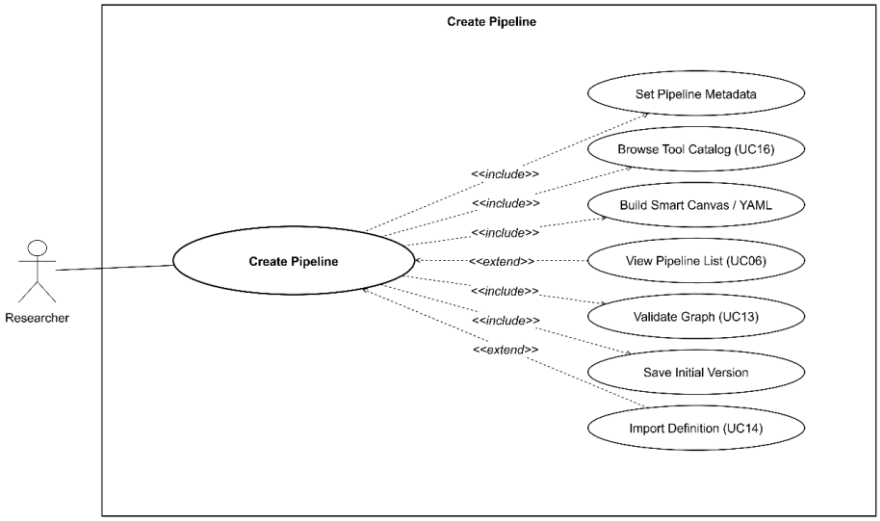


***Table 1.**1. Use Case Specification — UC05*

Attribute	Content
ID and name:	UC05: Create a New Pipeline (Create Pipeline)
Primary actor:	Researcher
Description:	Allows the Researcher to create a new analysis workflow (Pipeline) by dragging and dropping bioinformatics tools onto the visual canvas (Smart Canvas) and connecting them into a processing sequence, or by writing the definition directly in the declarative YAML language. Before saving, the system always validates whether the processing sequence is valid (i.e., no processing step has a cyclic dependency).
Preconditions:	The Researcher is signed in and has designed at least one valid processing sequence on Smart Canvas or entered YAML content.

Attribute	Content
Postconditions:	A new Pipeline is saved to the Researcher's Pipeline library as the first version and is private by default (visible only to the owner).
Main flow:	1. The Researcher opens the Create Pipeline screen and enters a name and description. 2. The Researcher drags and drops tools from the Tool catalog (UC16) onto the canvas and connects the steps according to the correct input/output data types (or switches to YAML editing mode). 3. The Researcher clicks "Save Pipeline". 4. The system validates the entire processing sequence (required information is present and there are no cyclic dependencies between steps). 5. The system saves the new Pipeline and records it as the version
Alternative / exception flows:	4a. The processing sequence contains a cyclic dependency (for example, step A requires the result of step B, while B also requires the result of A) → the system rejects the save operation and clearly highlights the error location on the canvas. 4b. Required information is missing (the Pipeline has no name or the YAML syntax is invalid) → the system rejects the save operation and displays a specific error message.
Business rules:	The visual representation (Smart Canvas) and the YAML representation must always remain consistent — the Researcher cannot save a Pipeline with two inconsistent representations, and the system maintains only one authoritative definition at any point in time. Cyclic dependencies between steps are always treated as errors and may not be saved merely as warnings.

1.4.2.1.2. UC11: View History and Compare Pipeline Versions

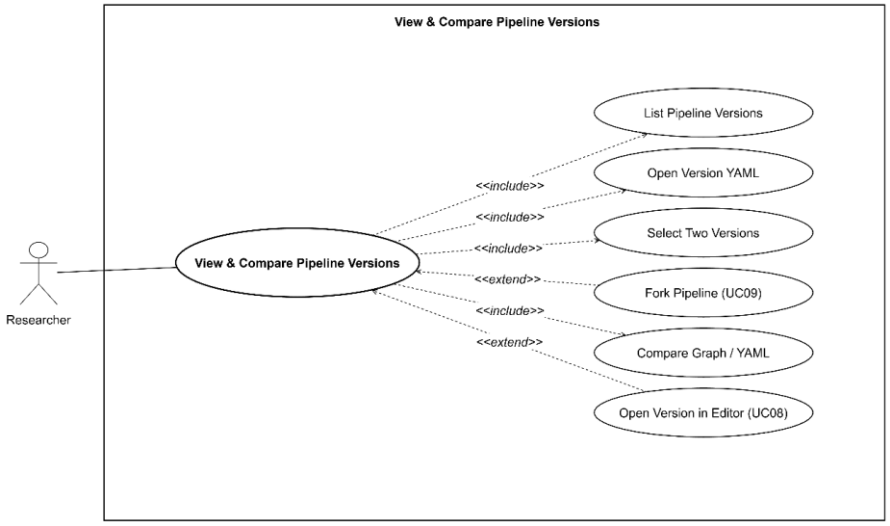


***Table 1.**2. Use Case Specification — UC11*

Attribute	Content
ID and name:	UC11: View History and Compare Pipeline Versions
Primary actor:	Researcher
Description:	Allows the Researcher to view saved versions of a Pipeline, inspect the YAML of each version, and compare two versions to identify changes in the workflow definition.
Preconditions:	The Pipeline exists and the Researcher has permission to access it.
Postconditions:	The Pipeline is not modified; the system returns the version history or comparison result as requested.
Main flow:	1. The Researcher opens the Pipeline to be inspected. 2. Selects Version History. 3. The system checks access permissions and loads the list of versions in chronological order. 4. The Researcher selects a version to view its saved YAML. 5. When comparison is needed, the Researcher selects two versions.

Attribute	Content
Alternative / exception flows:	3a. The Researcher does not have access permission → the system rejects the request. 4a. The version does not exist or does not belong to the Pipeline → the system reports that it was not found. 5a. The two selected versions are invalid → the system asks the Researcher to select them again.
Business rules:	A created version is an immutable snapshot of the Pipeline definition. A Run that has already been initialized must continue to reference the exact version fixed at execution time.

1.4.2.1.3. UC13: Lint and Validate the Pipeline Graph

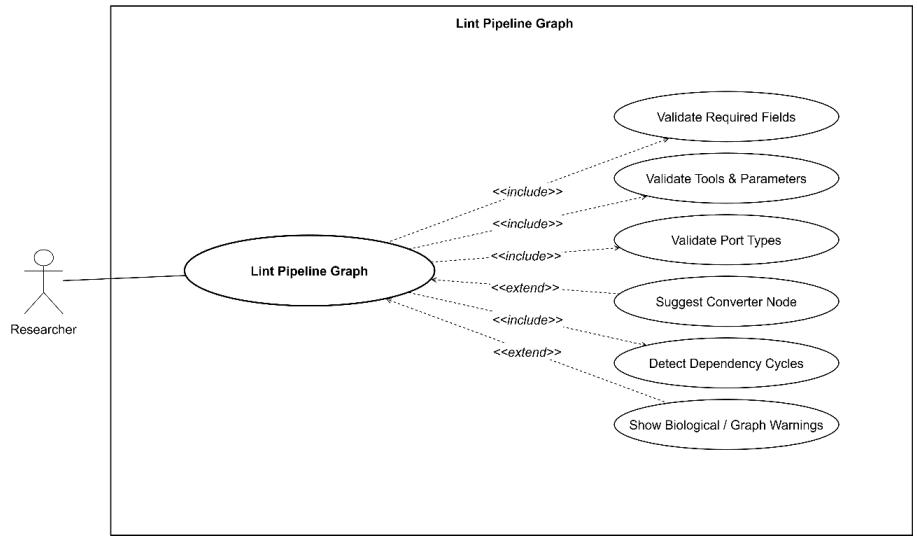


***Table 1.**3. Use Case Specification — UC13*

Attribute	Content
ID and name:	UC13: Lint and Validate the Pipeline Graph
Primary actor:	Researcher
Description:	Validates the Pipeline Graph before saving or running it in order to detect structural errors, invalid tools, missing inputs, incorrect data-type connections, and unsupported components.

Attribute	Content
Preconditions:	The Pipeline Graph contains node, edge, or corresponding YAML data.
Postconditions:	The system returns either a valid result or a list of errors/warnings; the Graph is not saved merely by performing the lint operation.
Main flow:	1. The Researcher requests Graph validation. 2. The interface sends the name, description, and Graph definition to the backend. 3. The backend analyzes the node and edge structure. 4. The system checks the DAG, node identifiers, edge references, Tool Registry, required inputs, and data-type compatibility. 5. The system classifies blocking errors and warnings.
Alternative / exception flows:	3a. The Graph definition cannot be parsed → a syntax error is returned. 4a. A dependency cycle, missing input, nonexistent tool, or unsupported placeholder node is detected → the Graph is marked invalid. 4b. A non-blocking issue is detected → a warning is returned for the Researcher to consider.
Business rules:	The backend lint/compiler is the source of truth for whether a Graph can be saved and executed. The frontend must not bypass blocking errors.

1.4.2.1.4. UC14: Import a Pipeline from YAML, JSON, or Galaxy

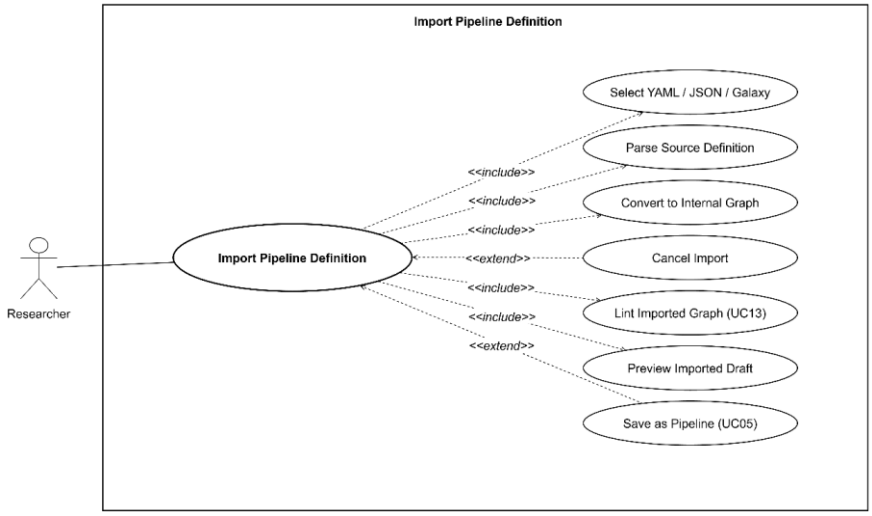


Table 1.*4. Use Case Specification — UC14*

Attribute	Value
ID and name:	UC14: Import a Pipeline from YAML, JSON, or Galaxy
Primary actor:	Researcher
Description:	Allows the Researcher to import an existing workflow definition and convert it into a draft BioFlow-Scope Graph for further validation, editing, and saving.
Preconditions:	The imported content is not empty and is in one of the formats supported by the system.
Postconditions:	A draft Graph and a list of warnings are returned; the Pipeline is saved only after the Researcher confirms and performs UC05 or UC08.
Main flow:	1. The Researcher selects the import format. 2. Pastes the content or selects the corresponding file. 3. The system parses the format and converts the steps, connections, and parameters into a BioFlow-Scope Graph. 4. The system identifies tools through the Tool Registry. 5. The system returns the draft Graph and conversion warnings.

Attribute	Value
Alternative / exception flows:	3a. The content has invalid syntax → import fails and the error location is displayed. 4a. A Galaxy tool or subworkflow is not supported → a placeholder/warning is created; the Graph cannot run until the issue is resolved. 5a. Safe conversion is not possible → the system does not automatically save the Pipeline.
Business rules:	Import creates a draft and does not imply that the Pipeline is executable. Every imported Graph must still pass through the backend lint/compiler.

1.4.2.1.5. UC17: Launch a Pipeline (Start Run)

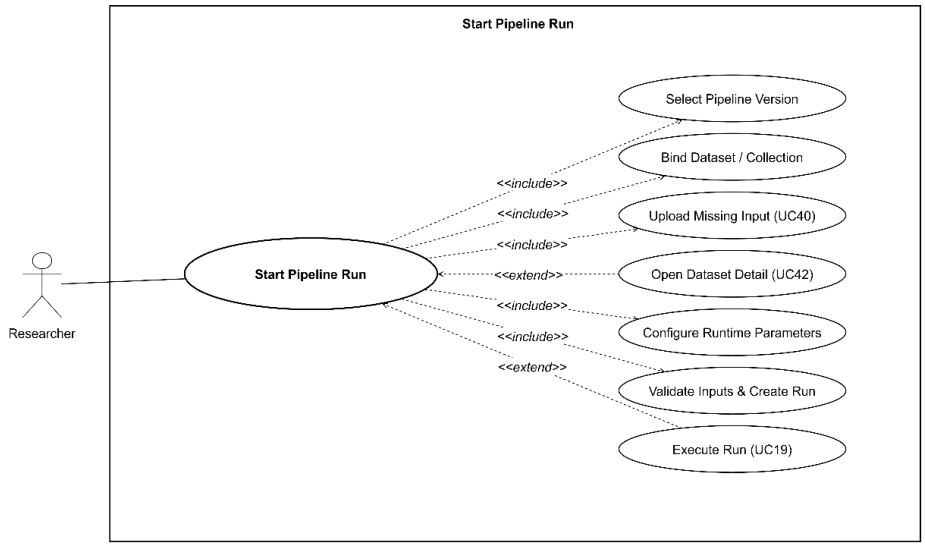


Table 1.*5. Use Case Specification — UC17*

Attribute	Value
ID and name:	UC17: Launch a Pipeline (Start Run)
Primary actor:	Researcher
Description:	The Researcher selects input data (an existing dataset or a newly uploaded file) for a Pipeline and initializes a new Run. The system "freezes" the exact Pipeline version to be used at launch time so that, even if the original Pipeline is modified later, this Run can still reproduce the original behavior and results.

Attribute	Value
Preconditions:	The Pipeline exists; the Researcher has permission to run it (as the owner, a member of a research group with which it is shared, or because the Pipeline is public).
Postconditions:	A new Run is created in the "Pending" state (not yet executed); if the Researcher selected an existing dataset, that data is already attached to the Run.
Main flow:	1. The Researcher opens the Pipeline detail screen and clicks "Run". 2. The system displays the Run setup screen, allowing selection of an existing dataset from the data library or leaving it empty for a later upload (UC18). 3. The Researcher confirms the setup and clicks "Start". 4. The system rechecks the Researcher's permission to run the Pipeline. 5. The system creates a new Run in the "Pending" state and freezes the current Pipeline version for use by this Run.
Alternative / exception flows:	1a. The Pipeline no longer exists → the system displays "Pipeline does not exist". 4a. The Researcher does not have permission to run the Pipeline (for example, another user's private Pipeline or a group Pipeline where the Researcher is not a member) → the system rejects the request and displays "You do not have permission to run this Pipeline".
Business rules:	Each Run is permanently associated with exactly one Pipeline version at initialization time — subsequent editing of the Pipeline (UC08) does not alter the behavior of existing Runs. This is a core principle for ensuring reproducibility of research results and supporting provenance and BCO report export (UC97).

1.4.2.1.6. UC19: Execute a Run (Execute)

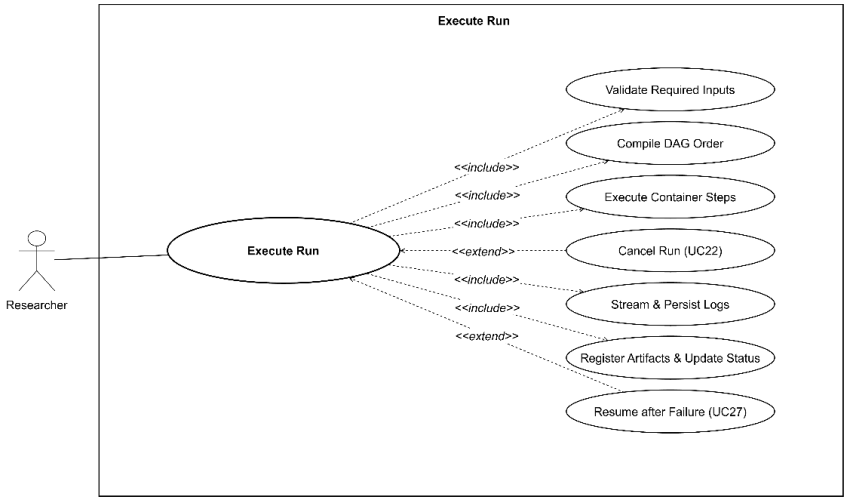


Table 1.*6. Use Case Specification — UC19*

Attribute	Content
ID and name:	UC19: Execute a Run (Execute)
Primary actor:	Researcher
Secondary actor:	AI Agent (may execute a Run on behalf of the Researcher while always complying with the same permission constraints)
Description:	After a Run has been initialized (UC17) and its input data is ready, the Researcher confirms actual execution. The system executes each processing step sequentially according to the dependency order defined in the Pipeline until all steps complete or an error occurs.
Triggering event:	The Researcher clicks the "Execute" button on the Run detail screen while the Run is in the "Pending" state.
Preconditions:	The Run is in the "Pending" state; all required Pipeline inputs (as declared) are available — either selected by the Researcher when launching the Run (UC17) or uploaded later (UC18).

Attribute	Content
Postconditions:	The Run transitions to the "Running" state and then terminates in one of the following states: "Completed" (all steps succeeded), "Failed" (a step failed), or "Cancelled" (if the Researcher cancels the Run during execution — UC22). The result of each completed step is stored as an Artifact (UC26).
Main flow:	1. The Researcher opens the Run detail screen and rechecks the input data. 2. The Researcher clicks "Execute". 3. The system checks whether all required input data are available. 4. The system changes the Run state to "Running" and begins executing each processing step sequentially according to the dependency order in the Pipeline. 5. For each step, the system marks the step as running, executes the corresponding tool, records the output (Artifact) and process log (UC20), and then marks the step as completed before moving to the next step. 6. After all steps are completed, the system marks the Run as "Completed".
Alternative / exception flows:	3a. Required input data are missing → the system refuses execution, clearly displays the missing input(s), and the Run remains in the "Pending" state. 5a. A processing step fails during execution → the system stops immediately at that step, marks the step as failed and the entire Run as "Failed"; subsequent steps are not executed. 5b. The Researcher requests cancellation while the Run is executing (UC22) → the system stops after the current step and marks the Run as "Cancelled".
Priority:	Cao

Attribute	Content
Business rules:	Processing steps are always executed sequentially according to the dependency order defined during Pipeline design — no two steps run in parallel within the same Run, ensuring controlled use of computing resources. As soon as one step fails, the entire Run stops and is marked Failed; the system does not automatically skip the failed step and continue with subsequent steps.
Assumptions:	"Launching" (UC17) and "Executing" (UC19) are separated into two distinct steps so that the Researcher has an opportunity to review and adjust input data before the system actually consumes computing resources.
Additional information:	None.

1.4.2.1.7. UC20: Monitor Run Logs in Real Time

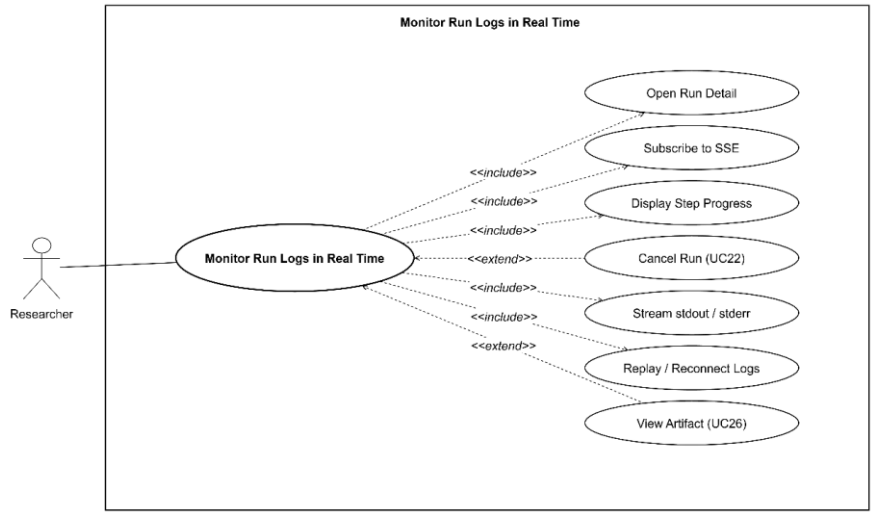


Table 1.*7. Use Case Specification — UC20*

Attribute	Content
ID and name:	UC20: Monitor Run Logs in Real Time
Primary actor:	Researcher

Attribute	Content
Description:	While a Run is executing (UC19), the Researcher can open the Run detail screen to view each log line and the status of each processing step as events occur, without manually refreshing the page.
Preconditions:	The Run is in the "Running" state (or has just transitioned to running).
Postconditions:	The Researcher can observe logs and progress in real time until the Run ends (Completed/Failed/Cancelled); the screen automatically stops updating when the Run finishes.
Main flow:	1. The Researcher opens the detail screen of the running Run. 2. The system immediately displays the most recent log lines generated earlier (even if the Researcher opens the screen late, recent logs are not missed). 3. The system continues streaming each new log line and updating the state of each step on the screen as events occur, without requiring further action from the Researcher. 4. When the Run ends, the system sends an end-of-stream signal; the screen displays the final Run state.
Alternative / exception flows:	4a. The log-monitoring connection is interrupted (network loss) → the screen displays a connection-loss warning; the Researcher can reload the page to view stored logs (UC21), and the logs are not lost because they are persistently stored in parallel.
Business rules:	If the Researcher opens the monitoring screen after the Run has already been executing for some time, the system still displays a number of the most recent log lines generated earlier (rather than starting from an empty stream), allowing the Researcher to understand the context immediately.

1.4.2.1.8. UC27: Resume a Failed or Cancelled Run (Resume Run)

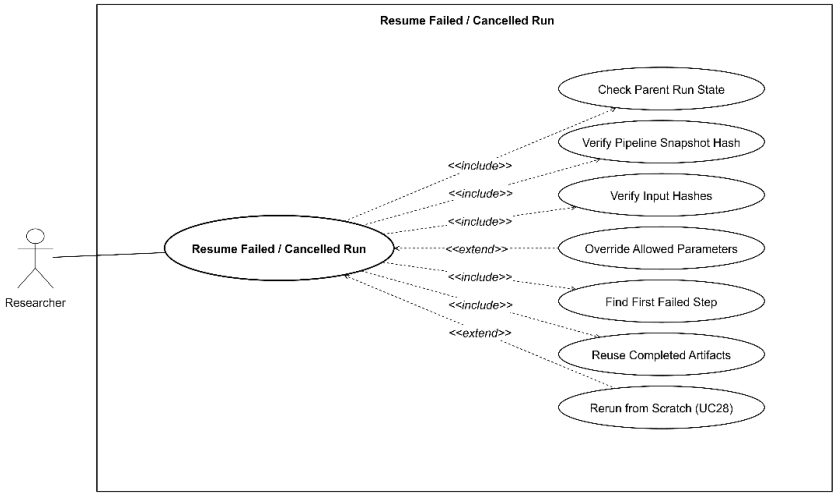


Table 1.*8. Use Case Specification — UC27*

Attribute	Content
ID and name:	UC27: Resume a Failed or Cancelled Run (Resume Run)
Primary actor:	Researcher
Description:	Creates a child Run that resumes from the first failed step and reuses results from previously completed steps, provided that the Pipeline and input files have not changed.
Preconditions:	The Researcher owns the Run; the Run is in FAILED/CANCELLED state; the Pipeline YAML and input data still match the stored hashes.
Postconditions:	A child Run is created, linked to the parent Run, records the restart step, and is submitted to the executor.

Attribute	Content
Main flow:	1. The Researcher selects Resume and may provide valid overrides. 2. The system checks the state of the parent Run. 3. The system compares the current YAML hash with the snapshot of the parent Run. 4. The system re-hashes the input directory and verifies that the data have not changed. 5. The system finds the first FAILED step in execution order.
Alternative / exception flows:	2a. The Run is not in an allowed state → Resume is rejected. 3a. The Pipeline has changed → a new Run is required. 4a. The input has changed → a new Run is required.
Business rules:	Resume may be used only when the previously executed portion can be proven to remain valid. The child Run must store its relationship to the parent Run to preserve provenance.

1.4.2.1.9. UC29: Compare Multiple Runs of the Same Pipeline

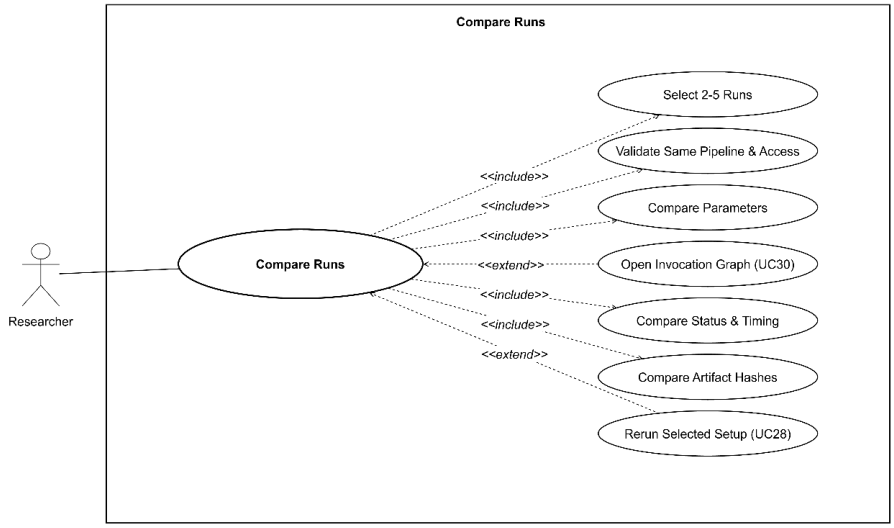


***Table 1.**9. Use Case Specification — UC29*

Attribute	Content
ID and name:	UC29: Compare Multiple Runs of the Same Pipeline

Attribute	Content
Primary actor:	Researcher
Description:	Compares the states, timing, parameters, Pipeline hashes, and artifacts of multiple Runs side by side to analyze differences between experiments.
Preconditions:	Select 2 to 5 Runs; all belong to the Researcher and use the same Pipeline.
Postconditions:	A comparison table is generated without modifying Run data.
Main flow:	1. The Researcher selects 2–5 Runs. 2. The system checks ownership, count, duplicate IDs, and the source Pipeline. 3. The system merges the set of parameter keys across the Runs. 4. The system merges artifact names. 5. For each Run, the system retrieves the state, start/end times, YAML hash, parameters, artifact hashes, and artifact sizes.
Alternative / exception flows:	2a. Fewer than 2 or more than 5 Runs are selected → the system requests a new selection. 2b. Duplicate IDs are present → the request is rejected. 2c. The Runs belong to different Pipelines or do not belong to the user → the request is rejected.
Business rules:	Only Runs from the same Pipeline may be compared so that parameter and artifact fields have comparable meanings. Large YAML snapshots are not included verbatim in the diff table.

1.4.2.1.10. UC30: View the Invocation Graph of a Run

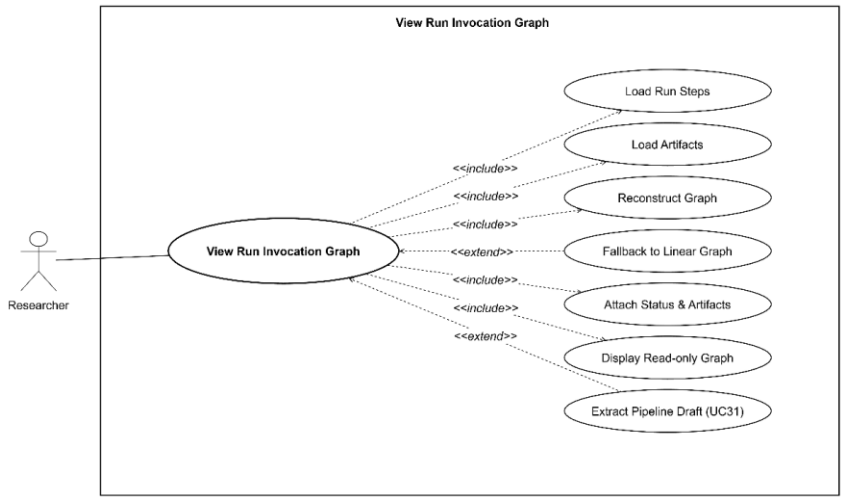


Table 1.*10. Use Case Specification — UC30*

Attribute	Content
ID and name:	UC30: View the Invocation Graph of a Run
Primary actor:	Researcher
Description:	Displays the actual execution graph of a Run, linking nodes to step states and generated artifacts to support traceability.
Preconditions:	The Run exists and the Researcher has permission to view it.
Postconditions:	A read-only graph is returned; Run data are not changed.
Main flow:	1. The Researcher requests the Invocation Graph. 2. The system loads the Run, Run Steps, and Artifacts. 3. If the Run has a Graph snapshot, the system builds nodes/edges from the snapshot. 4. The system attaches step states and artifact lists to the corresponding nodes. 5. If no Graph snapshot exists, the system builds a linear graph from the step order.
Alternative / exception flows:	2a. The Run does not exist or does not belong to the Researcher → the request is rejected. 3a. The Graph snapshot cannot be parsed → a linear model is used as a fallback.

Attribute	Content
Business rules:	The Invocation Graph must reflect the snapshot at Run time and must not use the current Pipeline Graph if the Pipeline has since changed.

1.4.2.1.11. UC47: Import a Dataset from an External Source

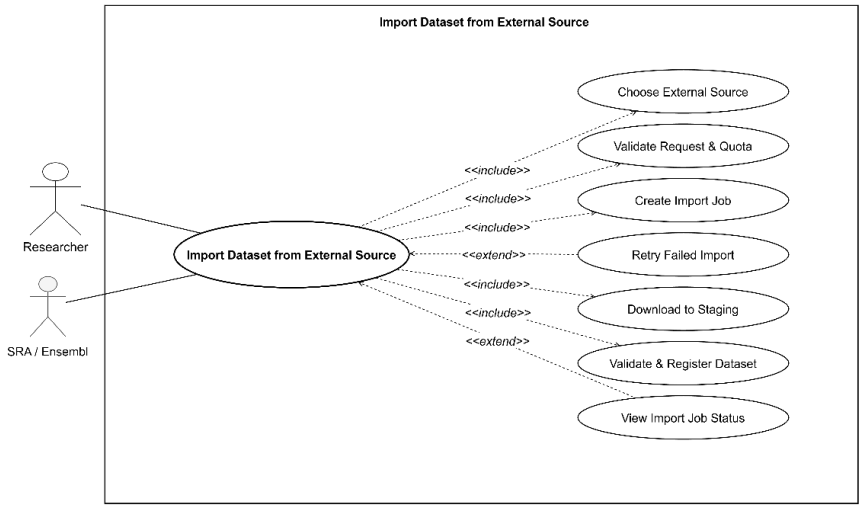


Table 1.*11. Use Case Specification — UC47*

Attribute	Content
ID and name:	UC47: Import a Dataset from an External Source
Primary actor:	Researcher
Description:	Creates a background job to bring scientific data from an external source into the staging area, validate it, and register it as a BioFlow-Scope Dataset.
Preconditions:	The Researcher is signed in; the source and request are valid; sufficient storage quota remains; and the external service is available.
Postconditions:	A Dataset Import Job is created and transitions through its states; on success the Dataset/file is registered, and on failure the reason is stored.

Attribute	Content
Main flow:	1. The Researcher selects an import source and enters the required information. 2. The system validates the request, Workspace permissions, and estimated quota. 3. An Import Job is created in the pending state. 4. A Worker receives the job and downloads the data into staging. 5. The file, metadata, and actual size are validated.
Alternative / exception flows:	2a. The source/mode is invalid → job creation is rejected. 4a. The external source does not respond or the accession does not exist → the job fails. 5a. The quota is exceeded or the file is invalid → the Dataset is not registered.
Business rules:	An import job must have a clear state, and a Dataset must not be recorded as successfully imported before the file is verified. Stub/mock modes must not be presented as complete production flows.

1.4.2.1.12. UC48: View BLAST Results Directly as a Table

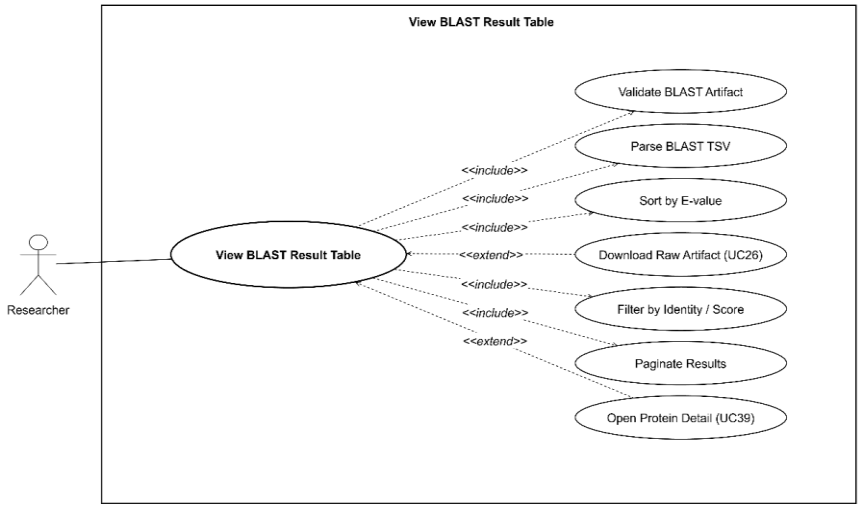


***Table 1.**12. Use Case Specification — UC48*

Attribute	Content
ID and name:	UC48: View BLAST Results Directly as a Table

Attribute	Content
Primary actor:	Researcher
Description:	Allows the Researcher to immediately view biological sequence matching results (BLAST) as a sortable and filterable table in the browser, instead of downloading the file and opening it with another tool.
Preconditions:	The selected Artifact is a BLAST result in a tabular format supported by the system.
Postconditions:	The Researcher can view the BLAST result table (each row is a matched sequence with metrics such as similarity and e-value), sort by column, filter by a minimum similarity threshold, and browse pages when the result contains many rows.
Main flow:	1. The Researcher opens the BLAST result Artifact and selects "View Inline". 2. The system displays the result table, sorted by ascending e-value by default (best matches first). 3. The Researcher can change the sort column, enter a minimum similarity threshold to filter results, or navigate between pages when the table contains many rows. 4. The system updates the displayed table according to the Researcher's selections.
Alternative / exception flows:	1a. The selected Artifact is not a valid BLAST result format → the system refuses inline viewing and suggests downloading the file for viewing with an external tool (UC26). 1b. The result file is too large for the browser's inline-viewing limit → the system refuses inline viewing and suggests downloading the file instead.

Attribute	Content
Business rules:	Results are paginated when the number of rows is large so that the browser remains responsive even when the result file contains tens of thousands of matching rows. Filtering by a similarity threshold affects only the display and does not modify the stored original result file.

1.4.2.1.13. UC79: View the Lineage of an Analyze History Item

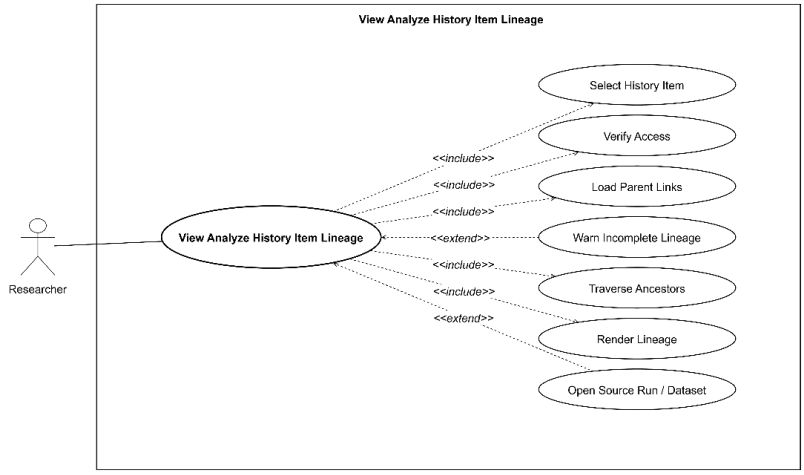


***Table 1.**13. Use Case Specification — UC79*

Attribute	Content
ID and name:	UC79: View the Lineage of an Analyze History Item
Primary actor:	Researcher
Description:	Displays the chain of ancestors that produced a History Item, allowing the Researcher to trace an output back to its source data and processing step.
Preconditions:	The Item exists and belongs to a History that the Researcher is permitted to access.
Postconditions:	A read-only Lineage tree/chain is returned.

Attribute	Content
Main flow:	1. The Researcher opens the list of Items in the History. 2. Selects a Dataset or output and requests its lineage. 3. The system checks ownership. 4. The current node and its source/parent relationship are loaded. 5. The process is repeated until the root node is reached to construct the ancestry chain.
Alternative / exception flows:	3a. The Item does not belong to the user → the request is rejected. 4a. A parent relationship is missing → the available portion of the lineage is displayed together with a warning that the data are incomplete. 4b. The Item has been purged → the complete lineage cannot be reconstructed.
Business rules:	Lineage must use stored snapshots/relationships and must not be inferred from file names. The chain must prevent invalid data cycles.

1.4.2.1.14. UC97: Export BCO-Compliant Provenance (BioCompute Object)

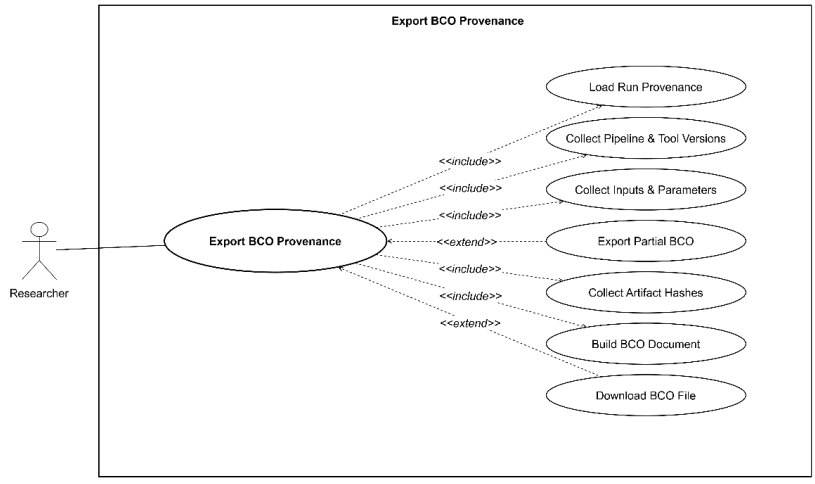


Table 1.*14. Use Case Specification — UC97*

Attribute	Content
ID and name:	UC97: Export BCO-Compliant Provenance (BioCompute Object)
Primary actor:	Researcher
Description:	Allows the Researcher to export all provenance information from a completed Pipeline Run as a document conforming to the international BioCompute Object (BCO) standard — including the sequence of processing steps, software/tools used, parameters, and output results with integrity-check hashes — for scientific publication or independent verification.
Preconditions:	The Run exists and is accessible to the Researcher; the Run should have ended (Completed/Failed) so that provenance information is complete.
Postconditions:	The Researcher receives a document fully describing the Run process in a BCO-compliant structure, which can be downloaded or attached to a scientific paper/report.
Main flow:	1. The Researcher opens the detail screen of a completed Run. 2. Clicks "Export BCO Report". 3. The system compiles all related information: the name and version of the Pipeline used, the ordered list of processing steps, the tools/software used at each step, supplied parameters, and the list of result files with integrity-check hashes. 4. The system presents this information according to the international BCO structure. 5. The Researcher downloads the document or copies it for inclusion in a scientific publication.

Attribute	Content
Alternative / exception flows:	1a. The Researcher does not have access to this Run → the system refuses to export the report. 3a. The Run is still executing and no step has completed → the exported report reflects only the information available at export time (and may be incomplete); the system recommends waiting for the Run to complete before exporting the most complete report.
Business rules:	The exported document must never contain internal system storage paths (only logical identifiers/file names) to prevent infrastructure information from leaking when the document is shared publicly. Each result file in the document includes an integrity hash, allowing an external reader to verify that the file received is exactly the file generated by that Run and has not been modified.

1.4.2.2. Semantic Protein Search and Controlled AI Agent

1.4.2.2.1. UC38: Search for Similar Proteins by Sequence

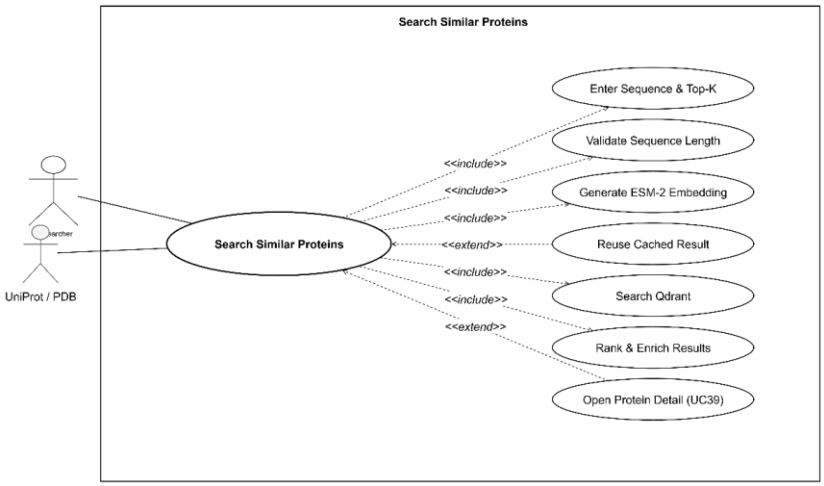


***Table 1.**15. Use Case Specification — UC38*

Attribute	Content
ID and name:	UC38: Search for Similar Proteins by Sequence
Primary actor:	Researcher
Secondary actor:	No

Attribute	Content
Description:	Allows the Researcher to enter an amino-acid sequence and find known proteins with the most similar functions/structures based on biological semantic similarity (rather than only raw character matching as in traditional BLAST). Results are ranked in descending order of similarity.
Triggering event:	The Researcher pastes a protein sequence into the search field and clicks "Search".
Preconditions:	The Researcher is signed in; the input sequence is between 10 and 5,000 characters long.
Postconditions:	The system returns up to the number of similar proteins requested by the Researcher (default 20, maximum 100), with each result including a similarity score and basic information; the query is recorded in the Researcher's search history.
Main flow:	1. The Researcher opens the Protein Search screen. 2. Pastes an amino-acid sequence into the input field and optionally selects the desired number of results (Top-K). 3. Clicks "Search". 4. The system validates the sequence length. 5. The system computes and finds the most similar proteins in the reference repository and ranks them by similarity score. 6. The system displays the result list: identifier, protein name, organism, length, and similarity score. 7. The system stores the query in the Researcher's history (the raw sequence is not stored; only its length and hash are retained for privacy).

Attribute	Content
Alternative / exception flows:	4a. The input sequence is too short (fewer than 10 characters) or too long (more than 5,000 characters) → the system rejects the search and displays the permitted length limits. 5a. The similarity-computation service is temporarily unavailable → the system reports an error and asks the Researcher to try again later.
Priority:	Cao
Business rules:	The same sequence (ignoring whitespace and letter case), searched with the same requested number of results within 24 hours, returns the previously computed result immediately instead of recomputing it, providing faster responses for frequently queried sequences. The search history does not store the Researcher's raw protein sequence; only its length and hash are retained to protect research data privacy.
Assumptions:	The Researcher understands that the results are similarity-based predictions rather than definitive statements of biological function and must be cross-checked against scientific literature before conclusions are drawn.
Additional information:	None.

1.4.2.2.2. UC53: Chat with the AI Agent

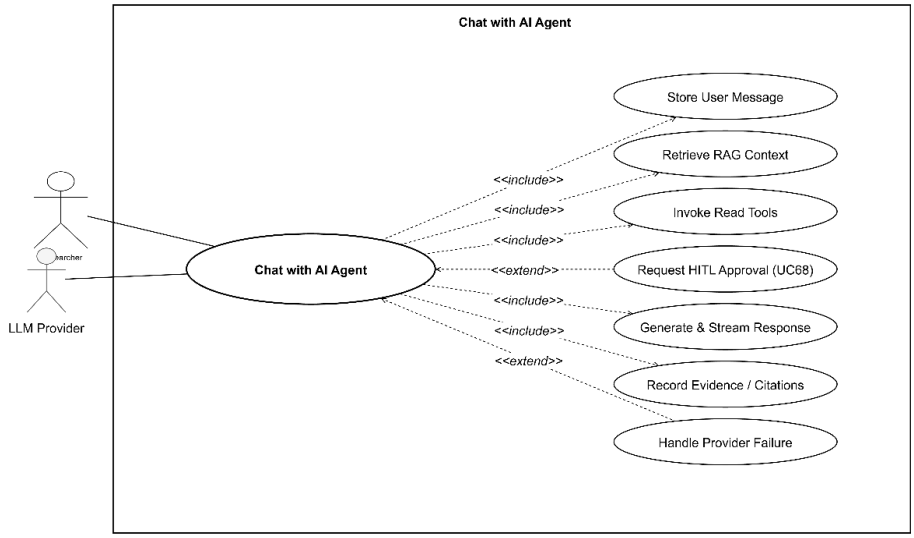


Table 1.*16. Use Case Specification — UC53*

Attribute	Content
ID and name:	UC53: Chat with the AI Agent
Primary actor:	Researcher
Secondary actor:	AI Agent (processes and responds to questions and can retrieve real system data instead of providing only generic answers)
Description:	The Researcher submits a natural-language question or request to the AI Assistant within an existing conversation session. The AI Assistant can retrieve actual information from the system (for example, the state of a Run, Protein Search results, or data in the library) to answer accurately instead of relying only on general knowledge. The response is displayed progressively as it is generated, creating the experience of a live typed response.
Triggering event:	The Researcher enters a question/request in the chat box within an open session and clicks Send.
Preconditions:	The conversation session already exists (UC52) and belongs to the Researcher.

Attribute	Content
Postconditions:	The question and answer are saved in the session history; if the AI Assistant retrieved additional real data to answer, those retrieval steps are also transparently recorded in the session.
Main flow:	1. The Researcher enters a question/request in the chat box and clicks Send. 2. The system saves the Researcher's message to the session. 3. The AI Assistant analyzes the request; if real data are needed (for example, Run status or Protein results), the AI Assistant automatically retrieves them from the system before answering. 4. The AI Assistant generates a response, and the system progressively displays each new portion on screen until the answer is complete. 5. The complete response (and retrieval steps, if any) is stored in the session history.
Alternative / exception flows:	3a. The AI Assistant needs to perform a high-risk action (for example, automatically running a Pipeline) → it does not execute the action immediately but creates a request that must first be approved by the Researcher/an authorized person (UC68), while clearly notifying the Researcher in the chat. 4a. The question appears to contain sensitive information that requires confirmation before being sent outside the system → the system asks the Researcher to confirm before the AI Assistant continues processing.
Priority:	Cao
Business rules:	The AI Assistant is always required to retrieve real system data when a question concerns specific data (Run, Protein, Dataset, etc.) instead of guessing or fabricating information, ensuring that answers have a verifiable basis. Any action that can have a real effect on the system (for example, running a Pipeline) must pass human approval before the AI Assistant is allowed to execute it, without exception.

Attribute	Content
Assumptions:	The Researcher understands that the AI Assistant may need several additional seconds to retrieve data before returning a complete answer, unlike an immediate text-only response.
Additional information:	None.

1.4.2.2.3. UC61: AI Agent Automatically Prepares & Launches a Pipeline

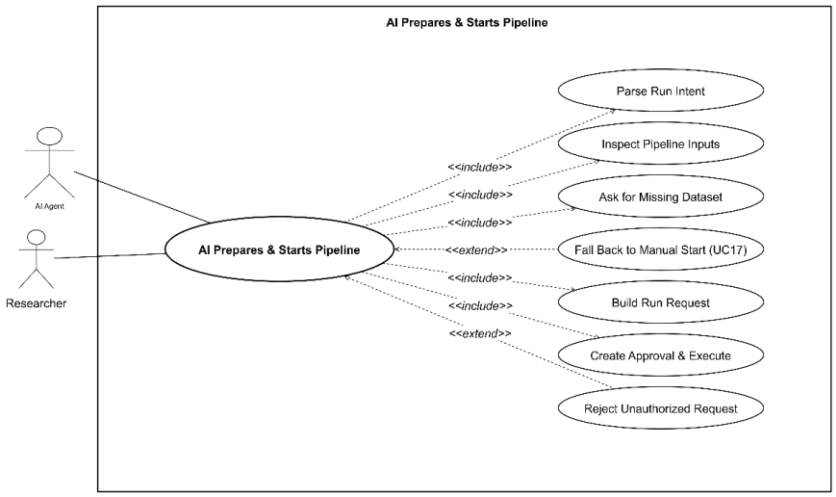


Table 1.*17. Use Case Specification — UC61*

Attribute	Content
ID and name:	UC61: AI Agent Automatically Prepares & Launches a Pipeline
Primary actor:	AI Agent
Secondary actor:	Researcher (the final approver before the Pipeline is actually run)

Attribute	Content
Description:	During a conversation (UC53), if the Researcher asks the AI Assistant to run a Pipeline, the AI Assistant automatically gathers all required information (selecting suitable input data for each required Pipeline parameter) and prepares a complete execution request. However, the AI Assistant is not allowed to run it immediately — the request must always be explicitly approved by the Researcher (or another authorized person) before the system actually launches the Pipeline.
Triggering event:	The Researcher asks the AI Assistant, in natural language through the chat interface, to run a specific Pipeline.
Preconditions:	The Researcher has permission to run the requested Pipeline; the AI Assistant has identified all necessary input data (from the conversation context or by asking the Researcher for clarification).
Postconditions:	A pending approval request (including complete information about which Pipeline and which input data) is created and displayed to the Researcher as a confirmation card directly in the chat; the Pipeline **has not yet been run** until an approval decision is made.

Attribute	Content
Main flow:	1. The Researcher asks the AI Assistant in chat to run a Pipeline. 2. The AI Assistant identifies any missing required Pipeline input parameters and, when necessary, asks the Researcher to select the appropriate data (an existing dataset) for each parameter. 3. Once all required information is available, the AI Assistant creates a complete execution request and sends it to the approval queue (UC68) instead of running it immediately. 4. The system displays an Accept/Reject confirmation card directly in the chat so the Researcher can review all information before deciding. 5. If the Researcher approves (UC68), the system automatically launches and executes the Pipeline (combining UC17 and UC19 into one operation), and no additional click is required.
Alternative / exception flows:	2a. The AI Assistant cannot identify suitable input data even after asking for clarification → the AI Assistant informs the Researcher that the inputs must be selected manually through the standard Pipeline launch screen (UC17). 1a. The Researcher does not have permission to run the requested Pipeline → the AI Assistant refuses to create the request and explains the reason in chat.
Priority:	Cao
Business rules:	The AI Assistant is never allowed to launch a Pipeline on its own without human approval — this is a hard Human-in-the-Loop safety constraint with no exceptions, even when the Researcher has explicitly asked the AI Assistant to do so. Once approved, the system combines launch and execution into one operation, unlike the standard manual flow (where the Researcher performs UC17 and UC19 separately), because all input data have already been fully confirmed at approval time and no additional manual check is necessary.

Attribute	Content
Assumptions:	The Researcher understands that this operation only reduces setup effort and does not bypass safety controls — every Pipeline proposed for execution by AI still requires explicit confirmation.
Additional information:	None.

1.4.2.2.4. UC68: Approve an AI Agent Action (Human-in-the-Loop Approval)

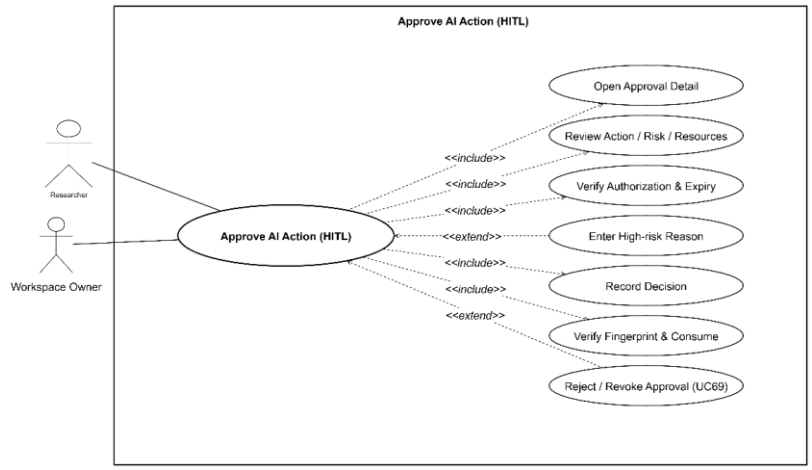


Table 1.*18. Use Case Specification — UC68*

Attribute	Content
ID and name:	UC68: Approve an AI Agent Action (Human-in-the-Loop Approval)
Primary actor:	Researcher / Workspace Owner
Secondary actor:	AI Agent (the party proposing the action requiring approval)
Description:	Every action proposed by the AI Assistant on behalf of the Researcher that can have a real effect on the system (for example, running a Pipeline) must stop and wait for an authorized person to review it and choose Approve or Reject before the action is actually executed.

Attribute	Content
Triggering event:	The Researcher opens the "Approval Inbox" (UC67) or sees a confirmation card displayed directly in the chat (from UC61) and reviews the request content.
Preconditions:	There is a request in the "Pending Approval" state that has not expired; the reviewing Researcher has decision authority (the owner of the relevant content, or the Workspace Owner if the action is within a group scope).
Postconditions:	The request transitions to "Approved" (after which the system executes the action — UC71) or "Rejected" (the action is not performed); the decision is fully recorded in the approval ledger (UC70) for later review.
Main flow:	1. The Researcher opens the pending request and reviews all details (what action will be performed and which object will be affected). 2. The Researcher selects "Approve" or "Reject"; for a high-risk action, the system requires the Researcher to provide a reason for the decision and confirm that the risk has been read and understood before continuing. 3. The system verifies that the request is still valid (not expired and not already handled). 4. The system records the decision and updates the request state. 5. If the decision is "Approve", the system automatically executes the approved action immediately afterward (UC71).

Attribute	Content
Alternative / exception flows:	3a. The request expires before the Researcher decides → the system rejects the operation and displays "Request expired"; the AI Assistant must create a new request if the action is still desired. 3b. The request was already decided previously (by the same Researcher in another session or because it expired) → the system refuses to process it again and reports the current state. 2a. The action is high-risk but the Researcher does not provide a reason or does not confirm the risk → the system refuses to record the decision and requires the missing information.
Priority:	Cao
Business rules:	Each approval request is valid only for a limited period — if no decision is made before expiration, it automatically transitions to "Expired" and can no longer be executed. The executed action must exactly match the approved content; if anything changes between approval and execution, the system refuses execution. For high or critical risk actions, the Researcher must provide a reason for the decision and confirm understanding of the risk before approval is allowed; one-click approval without consideration is not permitted. Each request can be executed only once; the same approval cannot be reused for multiple executions.
Assumptions:	None.
Additional information:	None.

1.4.2.2.5. UC58: Approve or Reject an AI Graph Editing Proposal

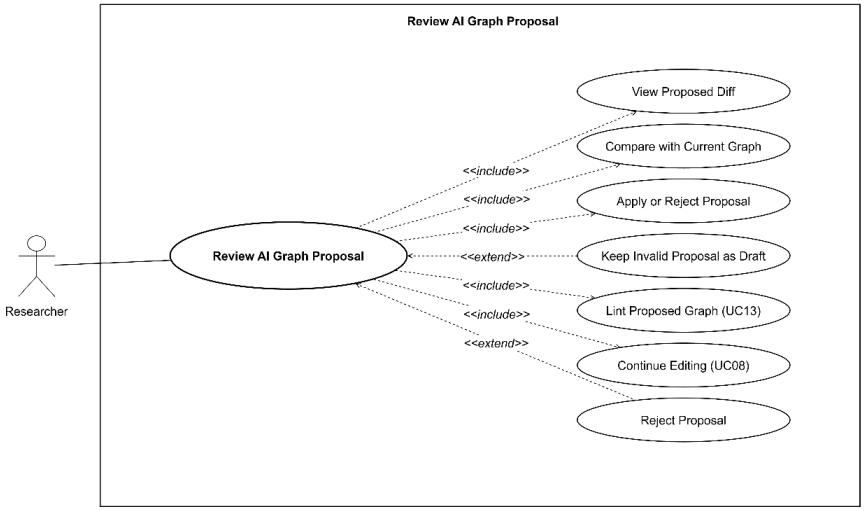


Table 1.*19. Use Case Specification — UC58*

Attribute	Content
ID and name:	UC58: Approve or Reject an AI Graph Editing Proposal
Primary actor:	Researcher
Secondary actor:	AI Agent and Pipeline Graph Service
Description:	Allows the Researcher to preview a Graph proposed by AI, inspect the changes, and decide whether to apply or reject them instead of allowing AI to modify the Pipeline automatically.
Triggering event:	The AI Agent creates a Graph Proposal within a chat session or Assist operation.
Preconditions:	The Researcher has permission to edit the Pipeline/Graph; the proposal remains pending.
Postconditions:	If approved, the proposed content is applied to the draft Graph for further linting/saving; if rejected, the current Pipeline remains unchanged.

Attribute	Content
Main flow:	1. The system displays the proposal together with a description of changes, nodes/edges, and lint warnings. 2. The Researcher compares the proposal with the current Graph. 3. The Researcher selects Apply or Reject. 4. If applied, the interface replaces/updates the draft Graph with the proposal. 5. The system runs backend linting. 6. The Researcher continues editing and saves using UC08. 7. If rejected, the proposal state is updated and the Graph remains unchanged.
Alternative / exception flows:	1a. The Proposal does not exist or has expired → it cannot be applied. 4a. The Researcher loses editing permission → the request is rejected. 5a. The proposed Graph is invalid → it remains a draft and errors are displayed; it is not saved automatically.
Priority:	Cao
Business rules:	AI is allowed only to propose changes; applying and saving a Pipeline requires the Researcher's decision. A Proposal cannot bypass linting or permission checks.
Assumptions:	The chat/Agent session is linked to the target Pipeline or Graph.
Additional information:	Source-code evidence: `AgentGraphProposal`, proposal APIs/logic in the `agent` module, and `PipelineGraphAssistService`.

1.4.2.2.6. UC62: Create a Multi-Step Agent Task

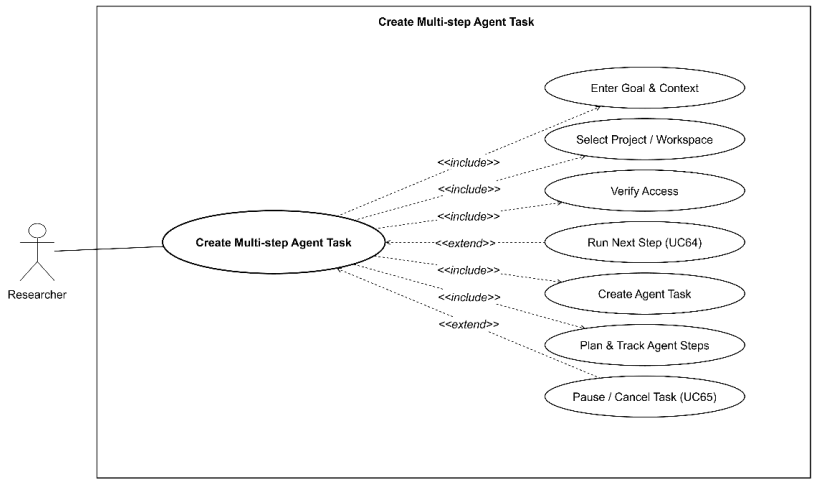


Table 1.*20. Use Case Specification — UC62*

Attribute	Content
ID and name:	UC62: Create a Multi-Step Agent Task
Primary actor:	Researcher
Secondary actor:	AI Agent Orchestrator
Description:	Creates a multi-step research task for the AI Agent that may be linked to a Project/Workspace, with state tracking and controlled step-by-step execution.
Triggering event:	The Researcher opens Agent Tasks and enters the task objective.
Preconditions:	The Researcher is signed in; the referenced Project/Workspace (if any) is within the accessible scope.
Postconditions:	The Agent Task and its initial plan/steps are stored and can continue through Run Next.

Attribute	Content
Main flow:	1. The Researcher enters the task title, objective, and context. 2. Selects a Project/Workspace if needed. 3. The system checks permissions and creates the Agent Task. 4. The Orchestrator initializes the required Agent Steps/scratchpad entries. 5. The system returns Task details and the current state. 6. The Researcher can select Run Next, monitor SSE events, pause, or cancel the task.
Alternative / exception flows:	2a. The Project/Workspace is invalid or inaccessible → the request is rejected. 3a. Required request data are missing → a validation error is returned. 4a. The AI provider is unavailable → the Task is retained with an appropriate error state for the user to handle.
Priority:	Cao
Business rules:	Each Task belongs to its creator and selected context. Any step containing a risky action must create an Approval Request instead of executing automatically.
Assumptions:	The AI provider and related tools are configured when the Task needs to run.
Additional information:	Source-code evidence: `AgentTaskController`, `AgentTaskService`, and the `agent_tasks`, `agent_steps`, and `agent_scratchpad_entries` tables.

1.4.2.3. Workspace Owner Use Cases

1.4.2.3.1. UC87: Manage Roles and Remove Members from a Workspace

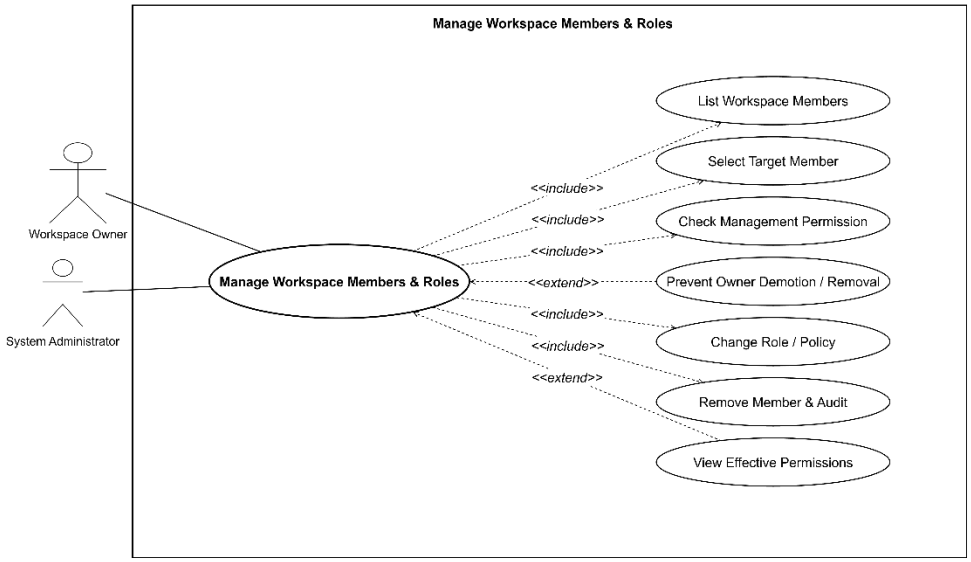


Table 1.*21. Use Case Specification — UC87*

Attribute	Content
ID and name:	UC-BS01: Manage Users and Account Status
Primary actor:	Admin
Description:	Manages the account catalog: search/filter accounts, create accounts, grant or revoke Admin privileges, lock/unlock accounts, and support controlled impersonation.
Preconditions:	The operator has the ADMIN role.
Postconditions:	The account/role/status is updated or a short-lived impersonation token is created; all sensitive actions are audited.
Main flow:	1. The Admin searches or filters by email, role, and status. 2. Opens user details or creates a new user. 3. Selects a role change, lock/unlock, or impersonation action. 4. The system checks safety constraints. 5. The change is saved or a time-limited impersonation token is generated.

Attribute	Content
Alternative / exception flows:	2a. The email for the new account already exists → the request is rejected. 4a. An Admin attempts to lock or impersonate their own account → the request is rejected. 4b. The last active administrator cannot be locked or have administrative privileges revoked.
Business rules:	Administrative operations must be audited. The system must protect the last Admin account to prevent loss of system-wide administrative access.

1.4.2.4. Administrator Use Cases

1.4.2.4.1. UC100: Update the Reference Database Version

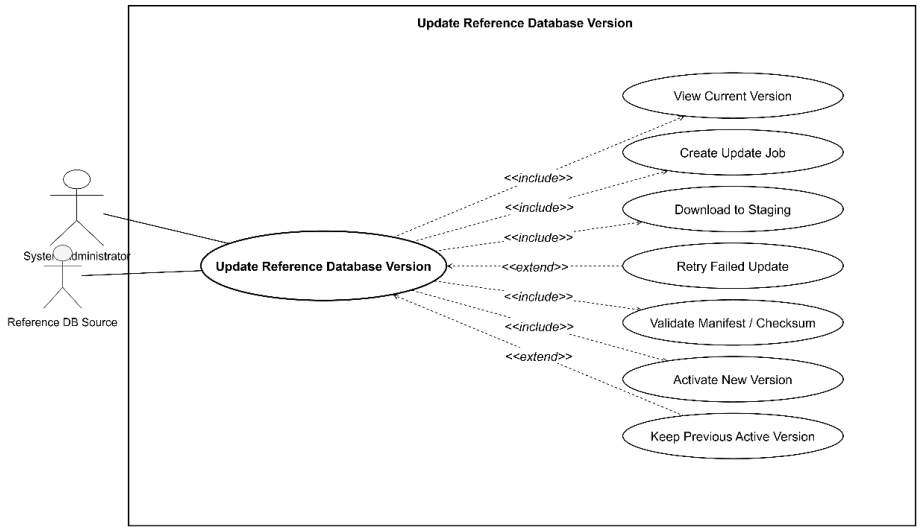


Table 1.*22. Use Case Specification — UC100*

Attribute	Content
ID and name:	UC-BS02: Manage Storage Quotas (Quota)
Primary actor:	Admin
Description:	Sets the default quota, individual user quotas, or Workspace quotas and applies these limits when data are uploaded/imported.
Preconditions:	The Admin is signed in; scope is DEFAULT, USER, or WORKSPACE; scopeId is valid when required.

Attribute	Content
Postconditions:	The quota is upserted and audited; subsequent storage requests use the new limit.
Main flow:	1. The Admin selects the quota scope. 2. The system displays the current quota list. 3. The Admin enters the target and storage limit. 4. The system parses/validates the value and scope. 5. The quota is saved.
Alternative / exception flows:	4a. The scope is invalid or DEFAULT includes a scopeId → the request is rejected. 4b. The storage value has an invalid format → the system asks for a new value. 7a. The new data size exceeds the quota or the safe disk-capacity limit → the request is rejected with QUOTA_EXCEEDED/PAYLOAD_TOO_LARGE.
Business rules:	Effective quota resolution prioritizes USER/WORKSPACE records; if none exists, DEFAULT or legacy configuration is used. Validation must occur before large data are written.

1.4.2.5. Scheduler and System Use Cases

1.4.2.5.1. UC34: Create a Schedule Trigger or Data Source Watch Trigger

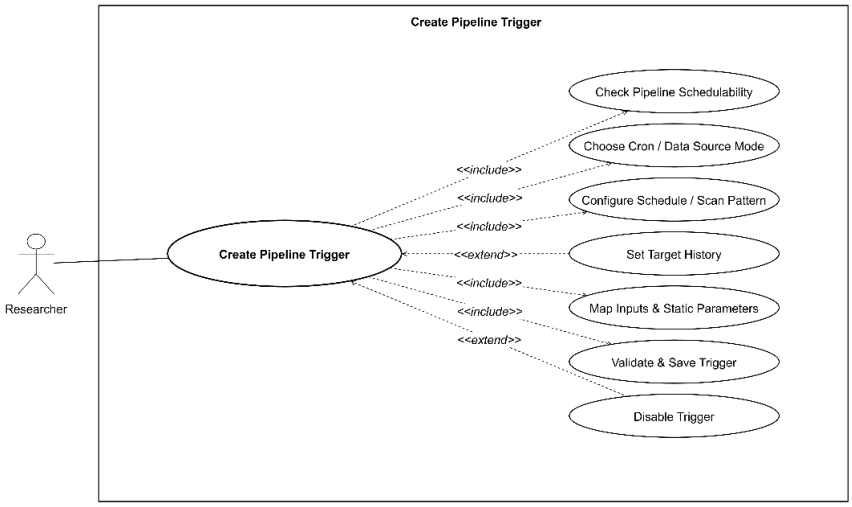


***Table 1.**23. Use Case Specification — UC34*

Attribute	Content
ID and name:	UC34: Create a Schedule Trigger or Data Source Watch Trigger
Primary actor:	Researcher
Description:	Configures a Pipeline to run automatically according to a cron expression or when matching data appear in a Data Source.
Preconditions:	The Researcher has permission to access the Pipeline; the Pipeline has passed schedulability checks; and the Data Source exists if file-watching mode is used.
Postconditions:	A Pipeline Trigger is saved with its mode, schedule, input bindings, static parameters, and processing options.
Main flow:	1. The Researcher opens Trigger configuration. 2. The system evaluates whether the Pipeline can run with static inputs or requires input files. 3. The Researcher selects cron or Data Source mode. 4. Enters the cron expression, scan pattern, input mapping, static parameters, and target History if needed. 5. The system checks permissions, configuration, and input compatibility.
Alternative / exception flows:	2a. The Pipeline is not eligible for scheduling → the reason is displayed. 3a. The Data Source does not exist or cannot be accessed → the request is rejected. 4a. The cron expression or input binding is invalid → the Trigger is not saved.
Business rules:	A Trigger may create Runs only within the Pipeline and data scopes accessible to its creator. The number of Runs generated per scan must be limited to prevent uncontrolled creation.

1.4.2.5.2. UC37: Automatically Create a Run When New Data Are Detected

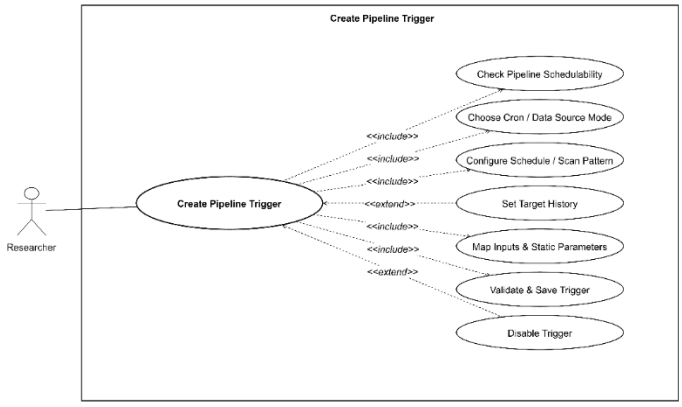


Table 1.*24. Use Case Specification — UC37*

Attribute	Content
ID and name:	UC37: Automatically Create a Run When New Data Are Detected
Primary actor:	Scheduler (System)
Description:	Scans Data Sources according to enabled Triggers, identifies completed files/file sets, and automatically creates the corresponding Runs.
Preconditions:	The Trigger is enabled; the Pipeline and Data Source remain valid; and the new file satisfies the finalization conditions and pattern.
Postconditions:	A Trigger Event is recorded; a Run is created/started, or the event is marked failed, pending retry, or ignored.
Main flow:	1. The Scheduler loads enabled Triggers. 2. For a Data Source Trigger, the system scans permitted paths and applies the scan pattern. 3. It checks whether the file is stable or has a completion marker. 4. Previously processed data are excluded. 5. A Trigger Event is created and the file is mapped to Pipeline inputs.

Attribute	Content
Alternative / exception flows:	2a. The Data Source is unavailable → the error is recorded and backoff is applied. 3a. The file is not yet complete → it is retained for the next scan. 5a. The binding is incomplete or the data are corrupted → the event is marked failed/paused.
Business rules:	The same input data must not create duplicate Runs for the same Trigger. Every processing attempt must have a Trigger Event for auditing and to support retry/ignore operations.

1.5. Non-Functional Requirements

1.5.1. Performance and Processing Requirements

- The system must operate stably in a local development environment, preferably on a personal computer with Docker Desktop and sufficient memory to run the backend, frontend, PostgreSQL, Redis, Qdrant, Ollama, and bioinformatics tool containers.
- Common web operations such as signing in, viewing the pipeline list, viewing Run history, viewing datasets, opening the dashboard, and viewing resource details should respond quickly when supporting infrastructure is operating normally.
- Heavy tasks such as running bioinformatics pipelines, importing large datasets, performing embedding-based protein search, generating AI reports, or updating reference databases are not required to return results immediately. These tasks should be processed through explicit progress states, with logs or notifications so users can monitor them.
- The system must support real-time log monitoring through Server-Sent Events (SSE) for Pipeline Runs and AI Assistant responses, allowing users to see the current step instead of waiting until the task finishes.
- Uploaded data should be written directly to temporary storage/working directories rather than keeping entire large files in memory. The current configuration supports large dataset files but must still comply with system storage quotas.
- Bioinformatics tools should run in containers with explicitly defined versions to reduce errors caused by differences in installation environments between machines.

1.5.2. Safety, Reliability, and Data Integrity Requirements

- The system must store complete information about Pipelines, Runs, execution steps, input parameters, output Artifacts, and verification hashes to support traceability and reproducibility of research results.
- Result files must be associated with the correct user, Pipeline/Run, and access permissions. Users may not view or download Artifacts belonging to resources they are not authorized to access.
- When a user cancels a running Run, the system should record the cancelled state and stop the executing container when possible, preventing background processes from continuing to consume resources.
- Data-import tasks, reference-database updates, or Pipeline creation/execution through the AI Assistant must have clearly controlled states: pending, running, completed, failed, or cancelled.
- Important data such as the database schema must be managed using versioned migrations. The application must not modify database structures in an uncontrolled manner during startup.
- For publicly shared results, the system displays only data explicitly permitted for publication and must not expose internal paths, account information, or private data from other Runs.

1.5.3. Usability and Interface Requirements

- The interface should be clear, consistent, and appropriate for researchers, students, or lecturers working in bioinformatics.
- Major screens such as Dashboard, Pipeline, Workflow Editor, Run Detail, Protein Search, Dataset, Viewer, and AI Chat should have easy-to-read layouts, understandable function names, and directly visible processing states.
- The visual Pipeline editor should allow users to understand relationships between analysis steps through nodes and edges. When data types are configured incorrectly or connections are invalid, the interface should report errors early before the Pipeline is submitted for execution.
- Common reports and Artifacts such as BLAST tabular output, VCF, HTML reports, logs, and raw files should support inline viewing or downloading depending on the format.

- The interface should respond well at common personal-computer screen sizes. For advanced operations such as drag-and-drop workflow editing or viewing large data tables, the experience is optimized for desktop/laptop screens.
- Colors, icons, fonts, success/failure/running states, and action buttons should be used consistently according to the project's design system.

1.5.4. Security Requirements

- The system must require authentication for major business APIs. Public APIs must be limited to functions explicitly designed for anonymous access, such as viewing a Run through a public link.
- User passwords must be hashed before being stored in the database. In the current source code, the backend uses BCrypt with an appropriate cost factor for account authentication.
- Login sessions use time-limited JWTs; the secret must be sufficiently long and configured through environment variables in real deployments rather than using default values in production.
- Authorization must clearly separate Guest, Researcher, and Admin roles. Administrative paths such as reference-database management or the RAG knowledge base are restricted to accounts with the Admin role.
- Risky actions proposed by the AI Assistant, such as running a Pipeline, importing a Dataset, editing a workflow, or publishing results, must pass through the Human-in-the-Loop mechanism and Approval Ledger before execution.
- The system should rate-limit calls to sensitive endpoints such as sign-in, registration, protein search, public links, and AI message submission to reduce the risk of resource abuse.
- CORS must be configured using an explicit allowlist of origins. Outside the development environment, HTTPS should be enabled at the deployment layer to protect data in transit.

1.6. Operating Environment

- The backend runs on Java 21 with Spring Boot 3.3 and uses Gradle for building and testing.
- The frontend runs on Next.js 14, React 18, and TypeScript; the browser must support modern JavaScript to provide all interactive functions.
- The embedding sidecar runs on Python/FastAPI and is dedicated to generating protein vectors using ESM-2 or a development stub mode.

- The local infrastructure includes PostgreSQL 16, Qdrant, Redis, Ollama, and Keycloak, configured through Docker Compose. The backend, frontend, and sidecar run on the host during development.
- The operating machine requires Docker Engine/Docker Desktop to run containerized bioinformatics tools. Some Pipelines may need to download large images or access reference databases and therefore require adequate disk capacity.
- Functions requiring external data such as UniProt, PDB, SRA, ORCID, or external LLM providers require a stable Internet connection and the corresponding API keys/configuration where required by the service.
- The system is designed to operate effectively on a personal computer for research, experimentation, and demonstrations. Deployment for many users or large datasets requires additional reverse proxy, HTTPS, backup, monitoring, and resource-allocation configuration.

1.7. Design Assumptions and Constraints

- The backend follows a modular-monolith architecture: one main application is deployed, but modules are separated by business domain and cross-module repository access is avoided.
- Backend APIs should return a consistent response format with clear error codes so the frontend can display appropriate messages to users.
- Database changes must be introduced through new Flyway migrations; previously used migrations must not be edited directly.
- Pipelines must be modeled as DAGs; dependency loops between steps are not allowed because they prevent safe execution scheduling.
- Available bioinformatics tools must be declared in the Tool Registry together with a specific YAML name and container image. A Pipeline may not execute a tool outside the allowlist.
- The AI Assistant may act only through services/APIs that enforce permissions and approval mechanisms; it may not bypass data ownership or independently perform risky actions that the user has not confirmed.
- The interface should prioritize ease of use and observability of analysis state over decorative effects. Business screens must help users understand where data are located, which steps have run, and which results can be verified.

- The remote-worker mechanism for self-developed tools, including BB340, is outside the current implementation and evaluation scope. Its integration architecture is proposed only in the future-development section.

CHAPTER 2. THEORETICAL BACKGROUND AND TECHNOLOGIES

2.1. Bioinformatics Analysis Workflows

Bioinformatics combines biology, computer science, mathematics, and statistics to process biological data. Data from DNA/RNA sequencing are transformed through computational steps for tasks such as variant detection, gene expression analysis, genome assembly, finding similar proteins, and evolutionary inference.

2.1.1. Essential Molecular Biology Background

DNA stores genetic information using four bases A, C, G, and T; RNA uses U instead of T and participates in transmitting/regulating information; proteins are amino-acid chains that fold into structures carrying out cellular functions. These data layers form the foundation of the workflows supported by BioFlow-Scope.

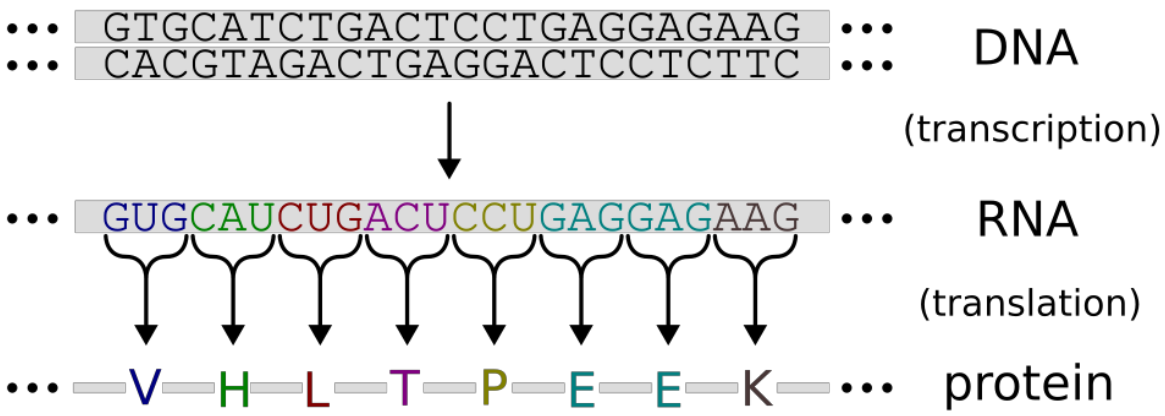


Figure 2.1. *DNA–RNA–protein genetic information flow*

Source: Madprime, Wikimedia Commons, CC0.

Table 2.*1. Fundamental biological terms*

Term	Explanation in the project context
Genome	The complete genetic material of an organism or cell. A "reference genome" is a representative sequence used as a coordinate system for aligning reads and describing gene/variant positions.
Chromosome	A structure that organizes DNA. Chromosome/contig names commonly become reference fields in SAM/BAM, VCF, BED, and GFF.

Term	Explanation in the project context
Gene	A region of DNA whose functional product is RNA or protein. A gene may include coding and regulatory regions; not all DNA encodes proteins.
Transcript / transcriptome	A transcript is an RNA molecule transcribed from DNA; the transcriptome is the complete set of RNA present in a sample under a particular condition.
Protein / proteome	A protein is a functional product formed from an amino-acid sequence; the proteome is the set of proteins expressed in a cell, tissue, or organism.
Nucleotide / base	The basic unit of DNA/RNA. Nucleotide sequences are commonly represented by strings of A, C, G, T/U; the character N represents an undetermined base.
Amino acid	The basic unit of proteins. Protein data commonly use a one-letter alphabet representing the 20 standard amino acids.
Variant	A difference between an observed sequence and a reference sequence or between individuals. Common forms include SNPs, insertions, deletions, and complex variants.
Allele / genotype / haplotype	An allele is a version of a locus; a genotype is the combination of an individual's alleles at a locus; a haplotype is a group of alleles located on the same chromosome copy that tend to be inherited together.
Homology	A relationship based on shared evolutionary ancestry. Orthologs are genes in different species separated by speciation, whereas paralogs arise through duplication within a genome.
Annotation	Assigning meaning to a sequence or genomic region, for example gene locations, CDS, rRNA, tRNA, protein function, or taxonomic classification.

Term	Explanation in the project context
Phylogeny	A model of evolutionary relationships, commonly represented as a tree; leaves represent taxa/sequences, edges represent relationships, and branch lengths may reflect the amount of change.

2.1.2. Sequencing Data and the Concept of Reads

DNA sequencing is the process of determining the order of the nucleotides adenine (A), cytosine (C), guanine (G), and thymine (T) in a DNA molecule. However, most next-generation sequencing instruments do not directly read an entire chromosome or genome in a single pass.

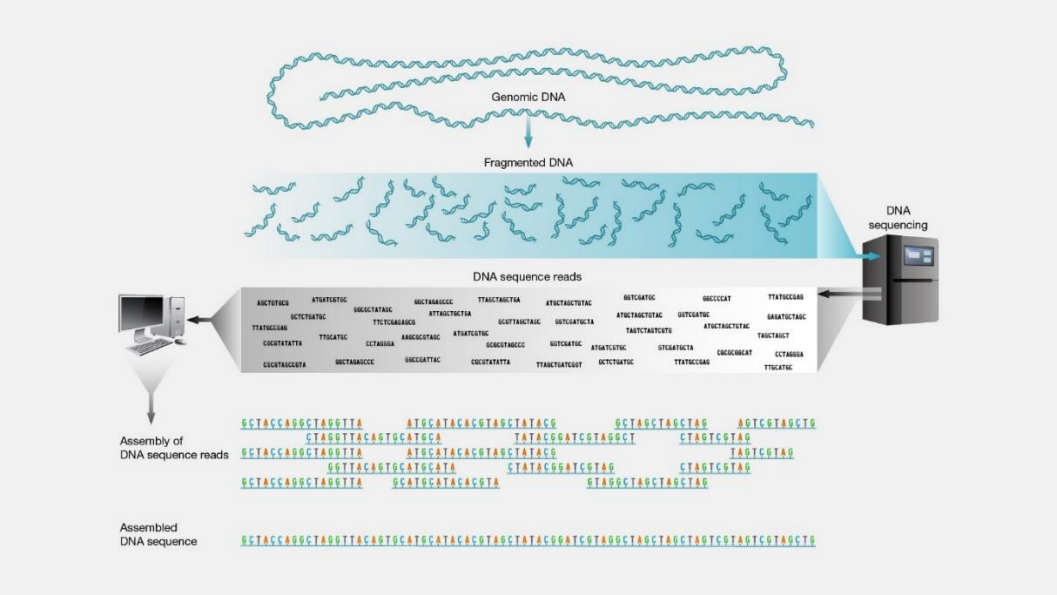


Figure 2.2. *DNA fragmentation, sequencing, and assembly*

Source: National Human Genome Research Institute.

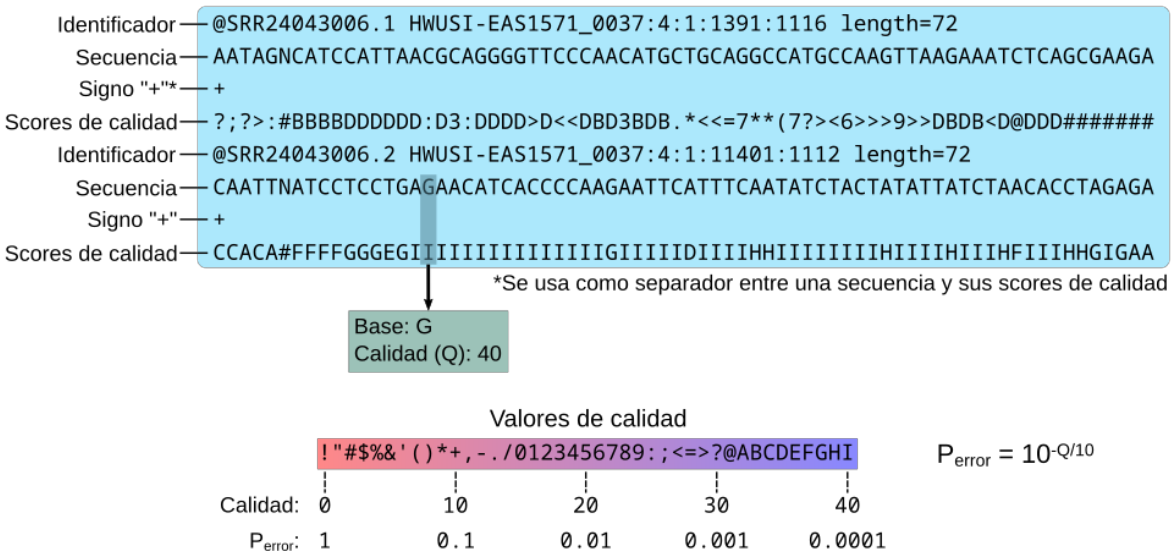


Figure 2.3. *Structure of a FASTQ record*

Source: Nesper94, Wikimedia Commons, CC BY 4.0.

BioFlow-Scope works with multiple biological formats at different stages of analysis. The specific format is determined by the corresponding tool and input/output port; runtime rules are presented in Appendix H.

2.1.3. Sequence Alignment, Assembly, and Biological Inference

After DNA data have been sequenced and quality-controlled, sequencing reads must be organized and analyzed to obtain biological information. Depending on the research objective and the availability of a reference genome, the data can be processed in two main ways: aligning reads to a reference sequence or performing de novo assembly directly from the reads.

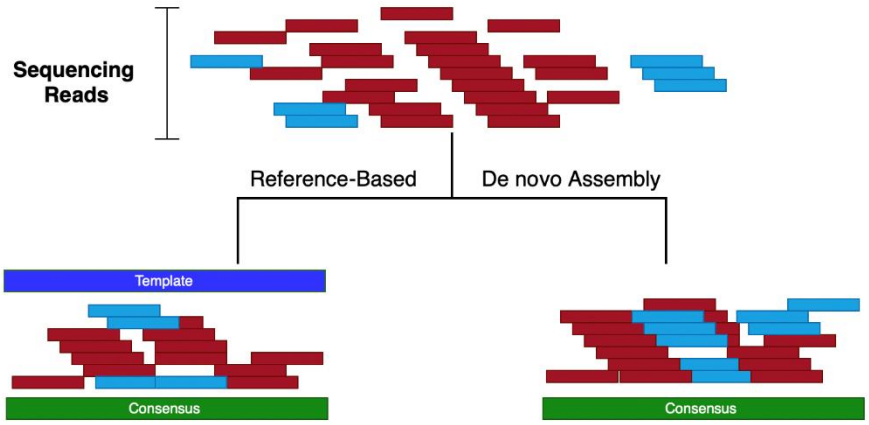


Figure 2.4. *Reference alignment and de novo assembly*

Source: Erekevan, Wikimedia Commons, CC BY-SA 4.0.

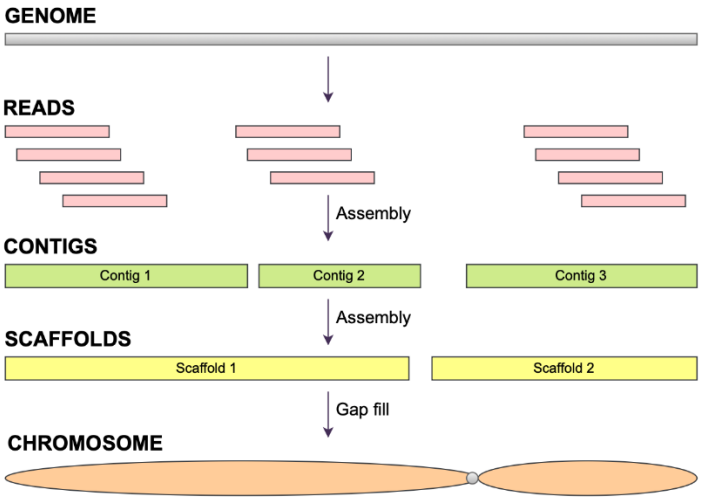


Figure 2.5. *Levels of de novo assembly*

Source: RiyaSaju, Wikimedia Commons, CC BY-SA 4.0.

2.1.4. Analysis Approaches Supported by BioFlow-Scope

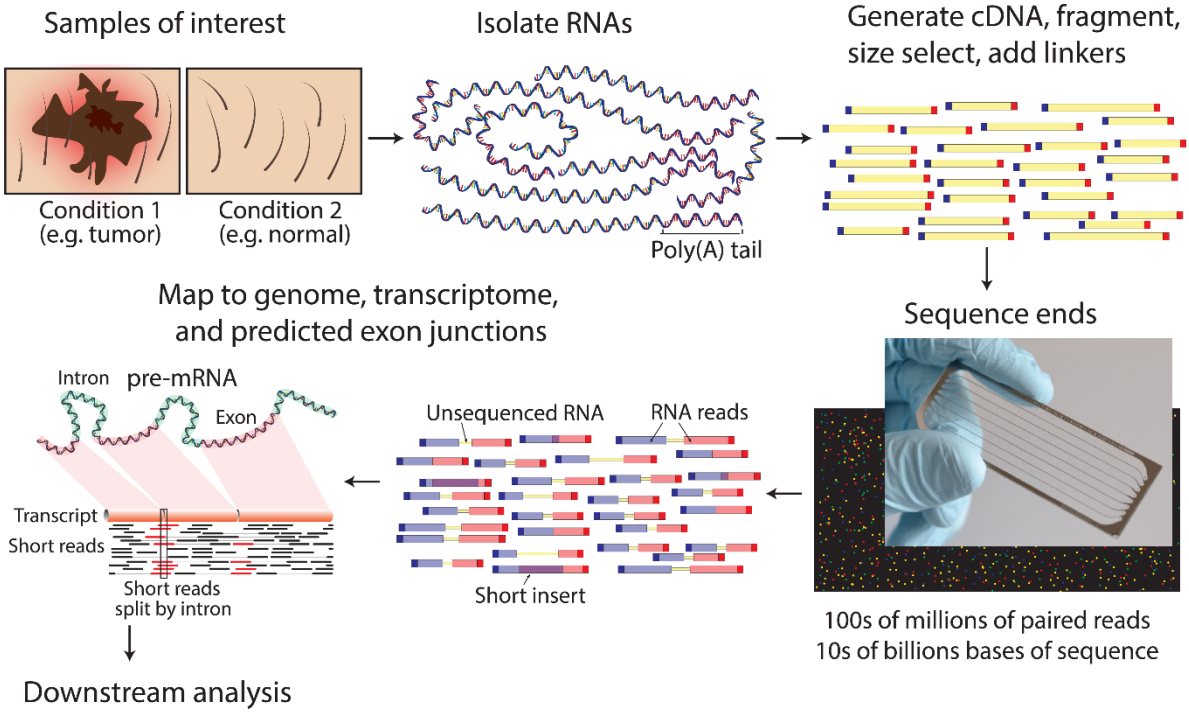


Figure 2.6. *RNA-seq analysis workflow*

Source: Griffith et al., PLOS Computational Biology, 2015, CC BY [7].

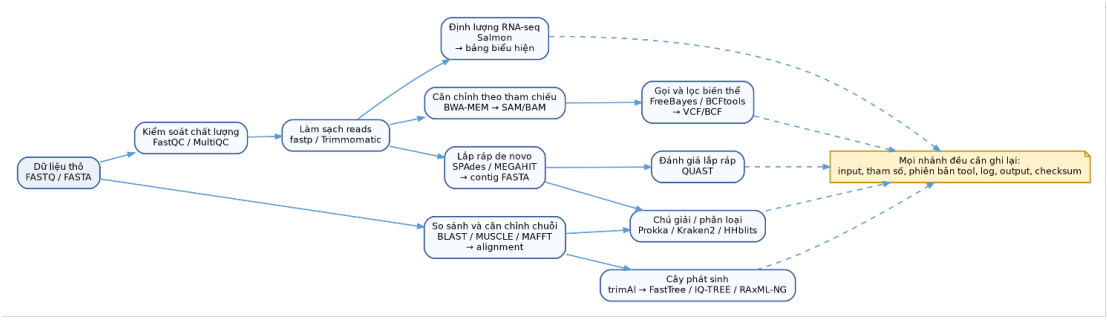


Figure 2.7. *Bioinformatics analysis branches in BioFlow-Scope*

Source: compiled by the author; representative tools are cited in Table 2.2.

Table 2.*2. Representative bioinformatics analysis approaches in BioFlow-Scope*

Analysis type	Typical sequence of steps	Scientific significance and notes
Read quality control	FASTQ → FastQC → report; multiple reports can be aggregated by MultiQC. [8], [9]	Detects cycle-specific quality degradation, adapters, abnormal GC content, duplicates, or contamination; it does not automatically determine the biological cause.
Data cleaning	FASTQ → fastp → trimmed/filtered FASTQ. [10]	Reduces the effects of adapters and low-quality bases; parameters must be recorded because overly aggressive trimming may remove signal.
Variant analysis	FASTQ → QC/trimming → BWA-MEM → SAM/BAM → BCFtools → VCF. [10], [11]	Identifies SNPs/indels relative to a reference; filtering and confidence assessment should follow the study design.

Analysis type	Typical sequence of steps	Scientific significance and notes
Genome/metagenome assembly	FASTQ → QC/trimming → SPAdes → contig FASTA → QUAST. [12], [13]	Reconstructs sequences when no suitable reference is available; assembly quality should be assessed before annotation.
RNA-seq	FASTQ → QC/trimming → Salmon → abundance table. [14]	Estimates transcript abundance; differential expression analysis requires a subsequent statistical step.
Protein/DNA similarity analysis	FASTA → BLAST or HHblits → match/alignment list. [4], [15]	Finds similar sequences or protein families; results should be evaluated using coverage, identity, E-value, and functional evidence.
Phylogenetics	FASTA → MAFFT → IQ-TREE 2 → Newick. [16], [17]	Infers evolutionary relationships from an alignment; model selection and support assessment are required.
Annotation and classification	Contig FASTA → Prokka; reads/contigs → Kraken2. [18], [19]	Assigns genes/functions to prokaryotic genomes or classifies taxa in metagenomic data.

The catalog of bioinformatics tools and runtime versions is presented in Appendix H.2; Chapter 2 retains only the principles needed to explain workflows and analysis results.

2.2. Workflow, Pipeline, and DAG

A workflow is a logical model of steps and dependencies; a pipeline is a workflow specified in sufficient detail to be validated and executed; a run is a specific

execution with defined inputs, parameters, and version. BioFlow-Scope represents dependencies as a DAG.

2.2.1. Workflow Components

***Table 2.**3. Workflow and pipeline components*

Concept	Meaning in BioFlow-Scope
Tool	A program or service that performs a scientific transformation, such as fastp, BWA-MEM, or Salmon.
Task/Step	A configured instance of a tool within a workflow, including its name, command, parameters, inputs, and expected outputs.
Port	A typed input/output point. Ports prevent, for example, connecting FASTQ data to a location that requires VCF, or connecting a parameter to a data port.
Dependency	A relationship in which one step can run only after another step has completed or produced the required data.
Artifact	A file or result object stored after a step/run, with metadata such as name, type, size, and checksum.
Run	One execution of a pipeline. Two runs of the same pipeline may differ in inputs, parameters, tool versions, or environment.
Scheduler/Trigger	A mechanism that initiates a run on a schedule or when a data source contains new data.
Provenance	A record explaining which inputs, activities, tools, and actors produced an output.

2.2.2. Directed Acyclic Graph (DAG)

A DAG (Directed Acyclic Graph) consists of directed vertices and edges without cycles. In a workflow, an edge u → v means that v depends on the output of u; if a dependency cycle occurs, the graph cannot determine a valid execution order and is considered invalid.

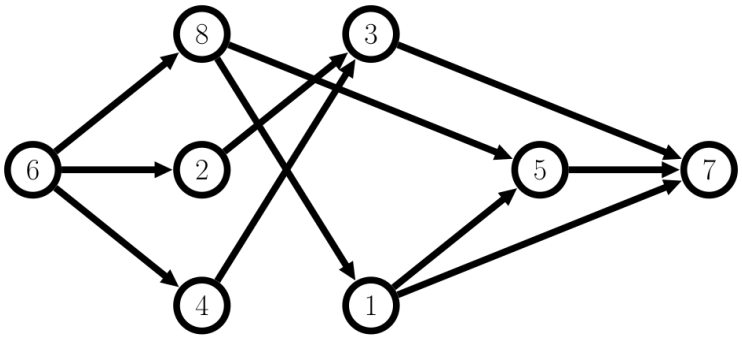


Figure 2.8. *Example of a directed acyclic graph*

Source: Dimitris131, Wikimedia Commons, Public Domain.

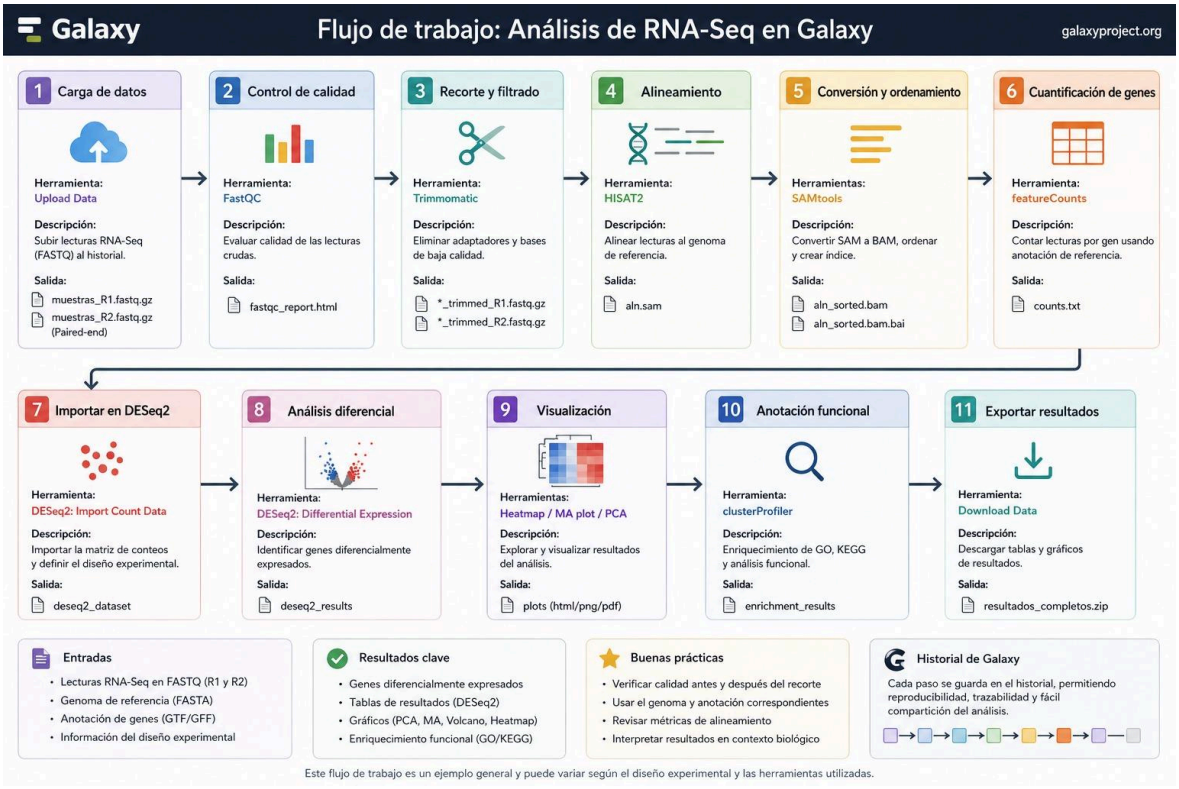


Figure 2.9. *RNA-seq workflow in Galaxy*

Source: Mariana Amaro, Wikimedia Commons, CC0.

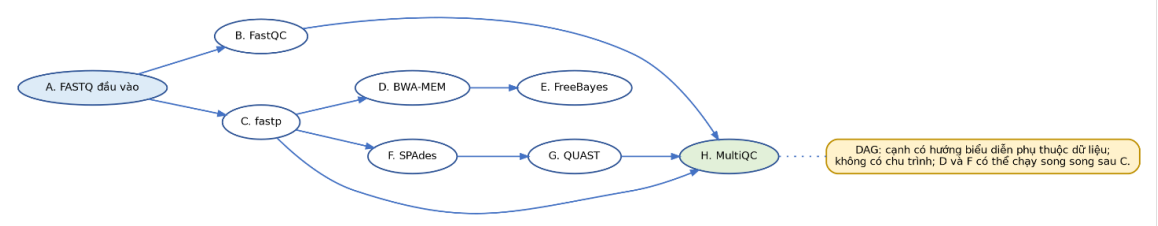


Figure 2.10. *Read-analysis pipeline represented as a DAG*

Source: developed by the author.

2.2.3. Limitations of the DAG Model

DAGs are suitable for most data-flow pipelines, but they do not directly represent loops with an unknown number of iterations, mid-process user interaction, or complex dynamic conditions. Workflow systems commonly address these limitations through scatter/gather, conditionals, subworkflows, or dynamically generated DAGs.

2.3. Reproducibility and Data Provenance

A computational result can be verified only when the inputs, transformation steps, tools/versions, parameters, outputs, and execution environment are known. BioFlow-Scope's snapshot, checksum, artifact, and export/import mechanisms are presented in Chapter 3 and the technical appendices.

2.3.1. Distinguishing Repeatability, Reproducibility, and Replicability

Table 2.*4. Levels of computational result verification*

Concept	Interpretation used in this report
Repeatability (under the same conditions)	The same team/operator reruns the same data, code, environment, and parameters within a short period. The goal is to determine whether the procedure is stable under equivalent conditions.
Computational reproducibility	Another party has sufficient data, code, workflow, versions, and environment information to recreate the published computational results. In bioinformatics, containers and workflow managers strongly support this objective. [20].
Replicability / independent validation	An independent study uses new data or an independent method to test the same scientific conclusion. This is a stronger level of evidence and cannot be achieved merely by exporting a pipeline ZIP file.
Traceability	The ability to trace a result back to the step, command, tool, input, operator, and creation time.
Integrity verification	Verification by checksum that the file's bytes have not changed. Integrity does not imply that the scientific content is correct.

2.3.2. Provenance and Data Lineage

Provenance describes the origin of data and the process by which it was created; lineage emphasizes the derivation chain between inputs and outputs. W3C PROV-O models three basic components—Entity, Activity, and Agent—allowing an artifact to be described in terms of the activity that generated it, the data from which it was derived, and the actor responsible. [21].

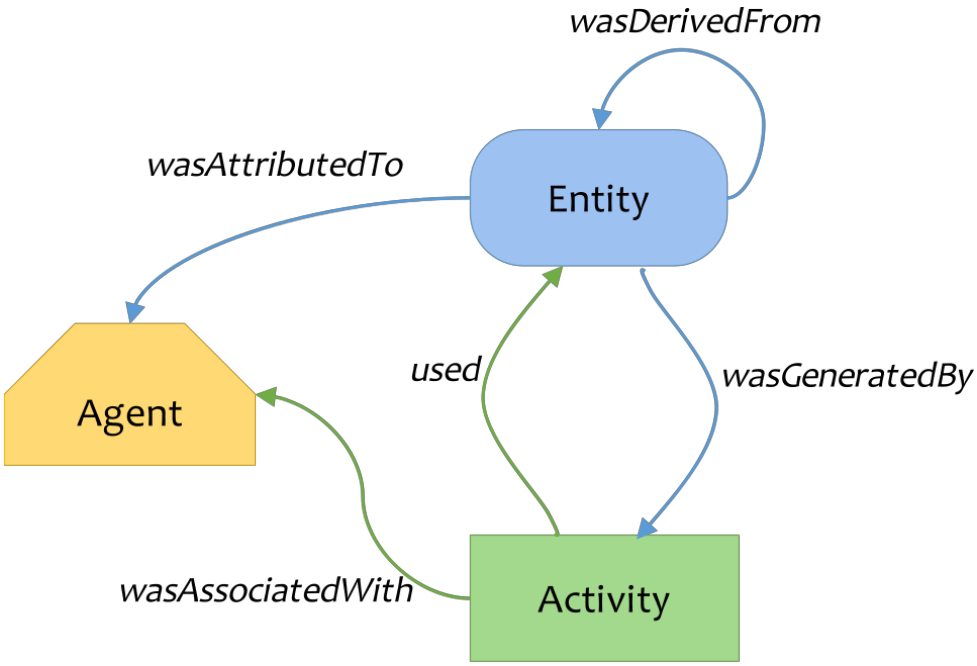


Figure 2.11. *W3C PROV data model*

Source: W3C PROV.

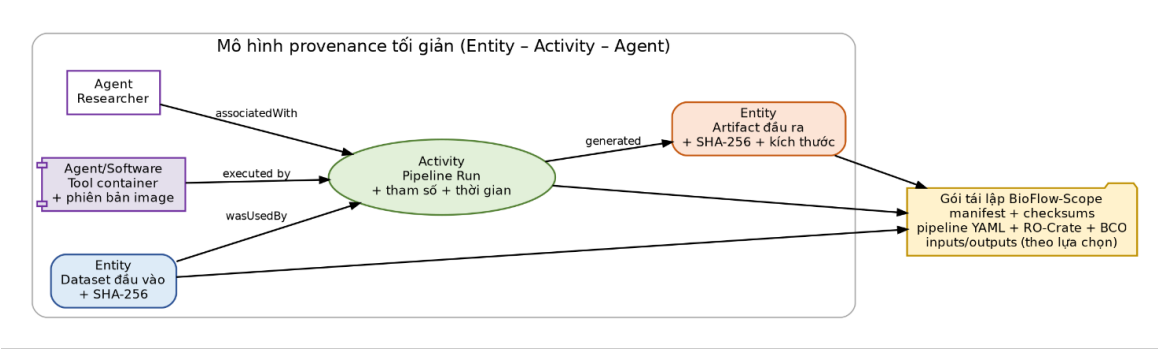


Figure 2.12. *Simplified provenance model of BioFlow-Scope*

Source: compiled by the author from [21], [22]–[23] and the project source code.

2.3.3. Why Bioinformatics Reproducibility Is Difficult

Table 2.*5. Causes of reduced reproducibility and control measures*

Source of variation	Example	Evidence/control information to retain
Input data	Files may be renamed, their bytes may change, sample metadata may be missing, or the files may become inaccessible.	Persistent IDs, checksums, manifests, access permissions, and input snapshots.
Reference database	Genome/annotation/BLAST/Kraken databases may be updated, changing results even when the command remains the same.	Record accession, version, download date, checksum, or package a snapshot when licensing permits.
Tools and dependencies	The same tool name with a different version/library may change the algorithm, defaults, or output format.	Use versioned container images; immutable digests are preferable.
Parameters and commands	Hidden defaults or small parameter changes can alter trimming, mapping, filtering, and tree-search results.	Store the fully expanded command, parameter snapshot, and configuration.
Nondeterminism	Random seeds, multithreading, processing order, or hardware can produce differences.	Record seeds/threads/hardware; use appropriate comparison criteria instead of always requiring byte-identical output.

Source of variation	Example	Evidence/control information to retain
Manual steps	Editing alignments, renaming columns, or selecting files manually is often not recorded.	Convert the step into a workflow operation or explicitly record the manual action and the person who performed it.
Execution environment	The OS, CPU instruction set, locale, filesystem, and resource limits may affect results.	Containers, resource records, logs, and environment configuration.
Biological interpretation	Identical technical outputs do not guarantee that the hypothesis or scientific conclusion is correct.	Expert review, statistical testing, controls, and independent validation.

2.3.4. FAIR, PROV-O, RO-Crate, and BioCompute Object

FAIR comprises the four principles Findable, Accessible, Interoperable, and Reusable. FAIR does not mean that data must be public; metadata and access mechanisms should be clear, while access itself may remain controlled. For workflows, FAIR requires not only result files but also the workflow, software, metadata, and provenance to be findable, understandable, and reusable. [24].

Table 2.*6. Provenance and reproducibility standards and mechanisms*

Standard/mechanism	Purpose	Application/limitations in BioFlow-Scope
W3C PROV-O	An ontology that standardizes the representation of Entities, Activities, Agents, and provenance relationships.	Provides a conceptual model for explaining lineage; BioFlow does not necessarily export complete RDF PROV-O. [21]

Standard/mechanism	Purpose	Application/limitations in BioFlow-Scope
RO-Crate 1.1	Packages research objects and metadata using JSON-LD; describes datasets, files, software, people, and relationships.	BioFlow creates ro-crate-metadata.json, a root Dataset, workflow YAML, a CreateAction for the run, inputs/outputs, and SoftwareApplication entries. [22]
Workflow Run RO-Crate	A profile extending RO-Crate to record the provenance of a workflow run and package inputs, outputs, and code.	The structure of `RoCrateBuilder` is modeled after this profile; compatibility is claimed only after validation. [25]
IEEE 2791 BioCompute Object	A communication standard for HTS computations and analyses that organizes information into provenance, usability, description, execution, parametric, I/O, and error domains.	BcoExportService maps run/step/command/artifact/lineage information to the corresponding domains. [23]
Manifest + SHA-256	Not an ontology; an integrity mechanism for listing files and checksums.	The BioFlow bundle includes `manifest.json` and `checksums.sha256`, supporting detection of missing or modified files.

2.4. Embeddings and Vector Search

Computers can directly compare two character sequences through exact matching or alignment, but machine-learning models generally require numerical inputs. An embedding is a transformation that maps an object such as a word, sentence, protein sequence, or document to a fixed-dimensional numeric vector.

2.4.1. Distributed Representations and Latent Space

An embedding vector can be viewed as a coordinate in a high-dimensional space. Each dimension does not necessarily correspond directly to a clearly named attribute such as “protein length” or “number of motifs”; instead, information is usually distributed across many dimensions.

2.4.2. Similarity and Nearest-Neighbor Search

After an object has been represented as a vector, the system requires a function for measuring proximity. Three common measures are Euclidean distance, dot product, and cosine similarity. BioFlow-Scope configures text collections in Qdrant to use cosine distance. Cosine similarity emphasizes the angle between two vectors rather than their absolute magnitudes, making it commonly suitable when vectors are normalized.

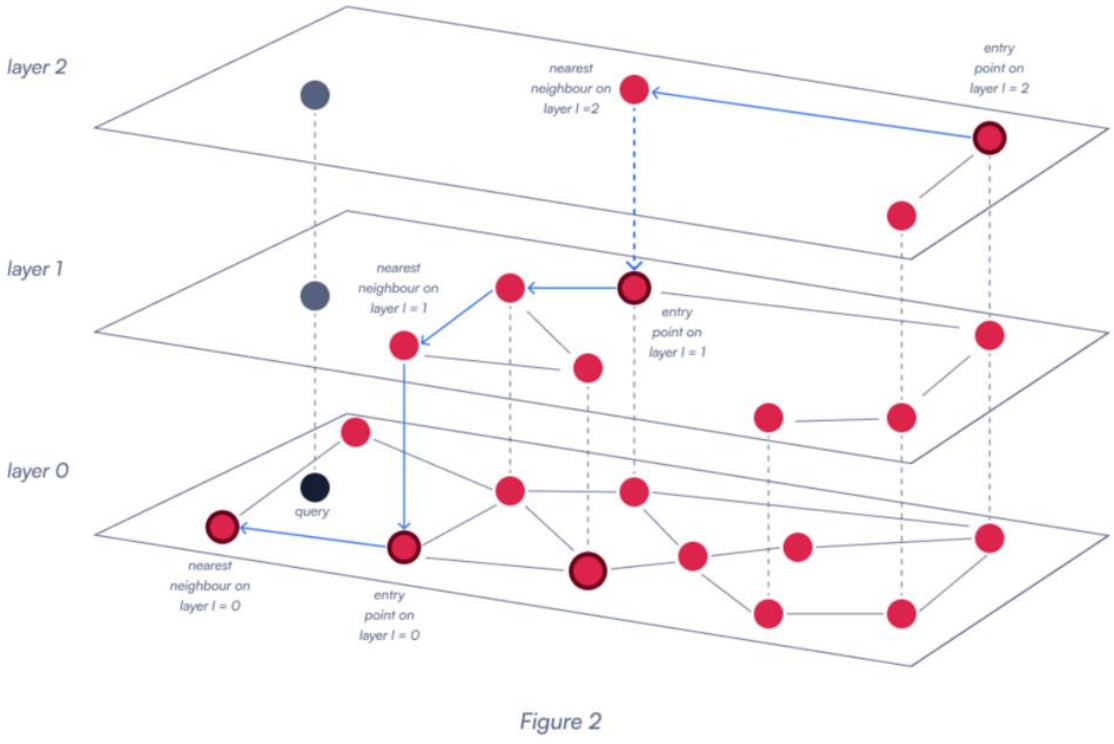


Figure 2.13. *Illustration of an HNSW index*

Source: Qdrant.

Table 2.*7. Embedding and vector-search concepts*

Concept	Brief explanation	Meaning in BioFlow-Scope
Embedding	A numeric vector representing an object after representation learning.	Represents a protein sequence or a text passage.
Residue/token embedding	A vector for each amino acid or token, dependent on its surrounding context.	An intermediate output before aggregation into a whole-sequence vector.
Pooling	Combines multiple local vectors into a fixed-size vector, for example by averaging.	The sidecar uses mean pooling for ESM-2 protein embeddings.
Vector dimension	The number of elements in a vector.	Protein: 480 dimensions; text: 384 dimensions in the current configuration.
Cosine similarity	Measures the angle between two vectors.	Used for RAG collections; higher scores are generally ranked as closer.
Top-K	The number of nearest neighbors returned.	Protein search defaults to 20; RAG defaults to 5 and is configurable.
Payload	Metadata associated with a vector.	Stores accession, protein description or source, URL, and chunk content.
ANN index	An approximate-nearest-neighbor index used to accelerate search.	Qdrant manages indexes for vector collections.

2.4.3. Two Types of Embeddings in BioFlow-Scope

BioFlow-Scope uses a separate FastAPI service as an embedding sidecar. The sidecar exposes two endpoints: `/embed` for protein sequences and `/embed/text` for text. These endpoints produce vectors with different dimensions and meanings; protein vectors cannot be directly compared with text vectors.

Table 2.*8. Two embedding flows in BioFlow-Scope*

Characteristic	Embedding protein	Text embedding
Endpoint	POST /embed	POST /embed/text
Input	An amino-acid sequence of 10 to 5,000 characters at the protein API.	A question, guidance document, or knowledge passage.
Default production model	facebook/esm2_t12_35M_UR50D	sentence-transformers/all-MiniLM-L6-v2
Vector dimension	480	384
Aggregation/normalization	Mean pooling over the hidden states of retained tokens.	SentenceTransformer encode with normalize_embeddings =true.
Purpose	Finds nearby proteins in the swiss_prot_v2025 collection.	Finds relevant chunks in tool_manuals and pubmed_v2025.
Cache	Protein-search results are cached in Redis for 24 hours.	Query vectors are cached in Caffeine for approximately one hour.
Notable limitations	The tokenizer is limited by the default ESM_MAX_LEN of 1,024 tokens.	Long content is truncated to the default TEXT_EMBEDDING_ MAX_LEN of 512 tokens.

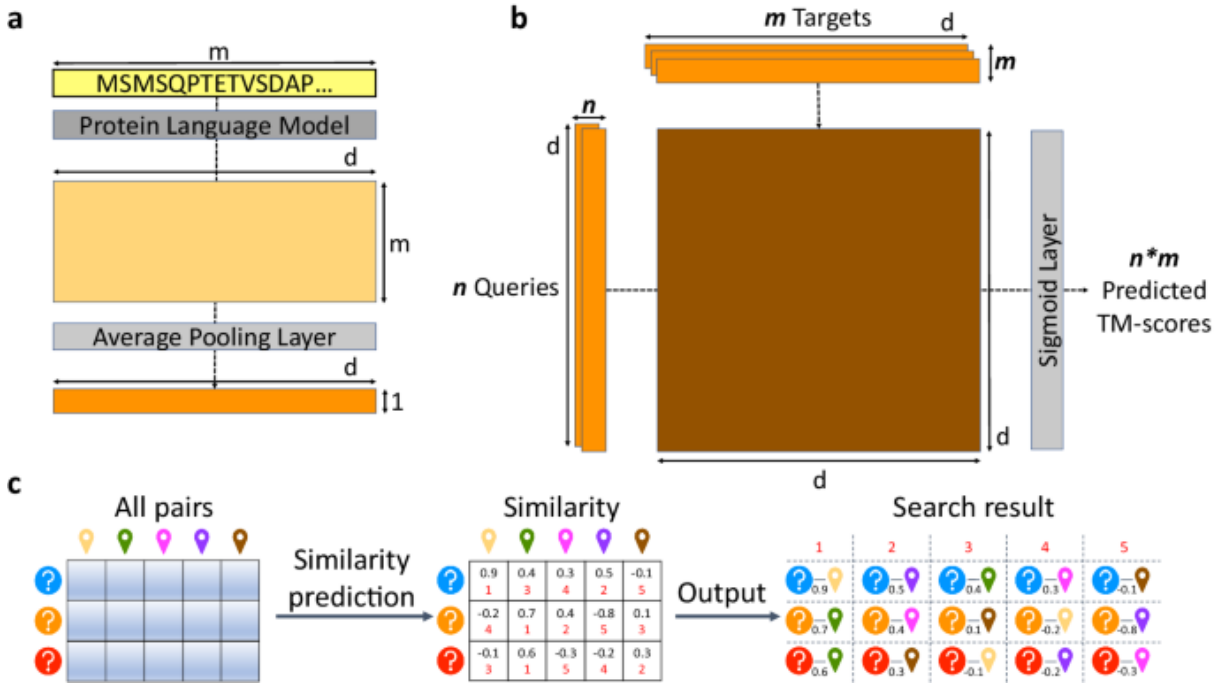


Figure 2.14. *Generating protein embeddings using average pooling*

Source: Liu et al., PLMSearch [26].

2.4.4. Vector Search for RAG

Retrieval-Augmented Generation (RAG) combines a generative language model with an external retrievable memory. Instead of requiring the model to rely only on knowledge encoded in its parameters, the system retrieves relevant document passages and places them in context before generating an answer. [27].

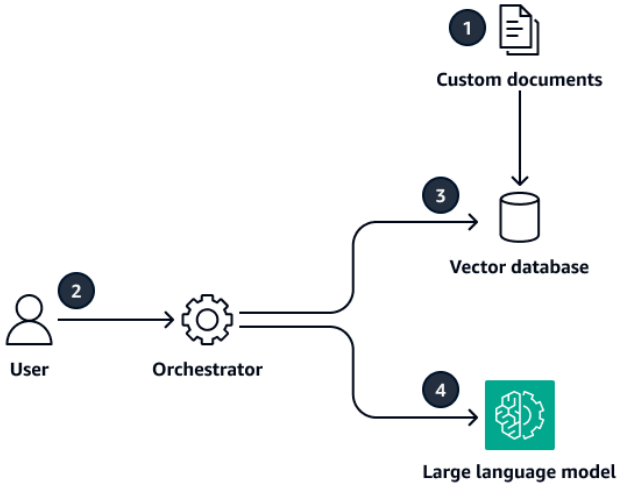


Figure 2.15. *Retrieval-Augmented Generation architecture*

Source: AWS Prescriptive Guidance.

2.4.5. Scientific and Technical Limitations of Embeddings

A vector is a compressed representation, not an explanation of a biological mechanism. Two proteins that are close in embedding space may still differ in functional domains or important active sites.

2.5. Semantic protein search

Semantic protein search is the process of finding proteins whose representations are close to a query sequence in embedding space. Here, "semantic" refers to relationships learned by the model from sequence data; it does not imply that the system understands protein function in the way a domain expert does.

2.5.1. From a DNA Gene to a Protein Query Sequence

In DNA-gene analysis, Semantic Protein Search is used at the interpretation stage after annotation: a predicted CDS can be translated into a protein sequence and then used to retrieve embedding-similar candidates that support functional hypotheses.

2.5.2. Processing flow of protein search

The protein-search flow is implemented through the following main steps:

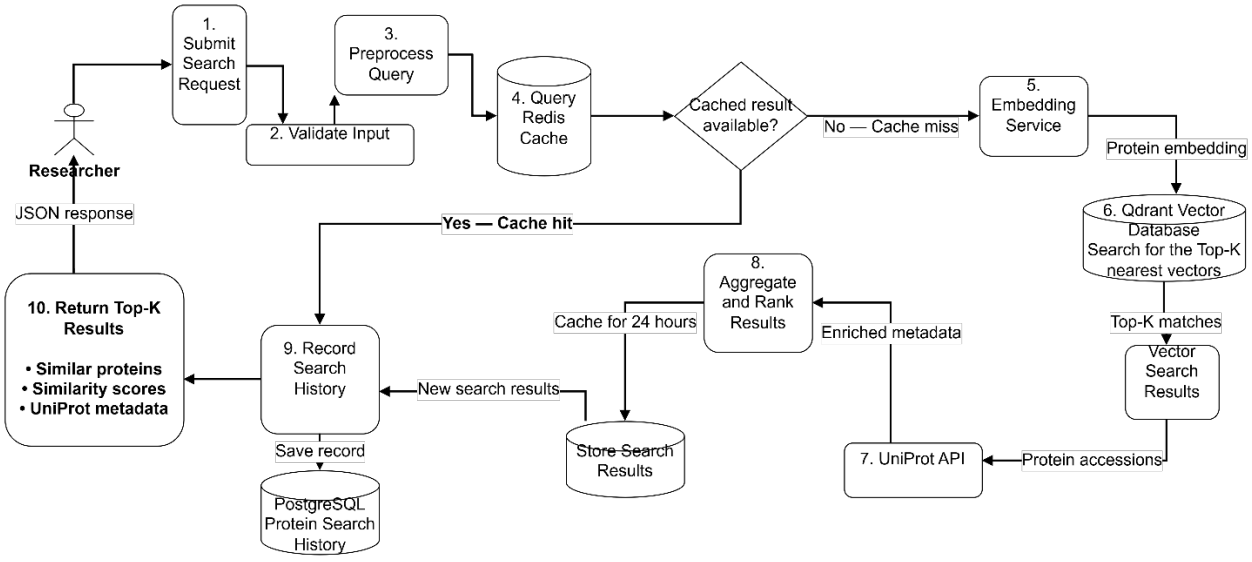


Figure 2.16. *Semantic Protein Search flow in BioFlow-Scope*

Table 2.*9. Information returned by Semantic Protein Search*

Field	Meaning	Interpretation
Accession	A stable protein identifier in UniProt.	Used to retrieve the detailed record and cross-check references.

Field	Meaning	Interpretation
Score	The vector-similarity score returned by Qdrant.	Used for ranking within the same collection; it is not a probability or an E-value.
Protein name	Protein name obtained from UniProt.	A database annotation whose evidence and reviewed/unreviewed status should be examined.
Organism	The organism containing the protein.	Helps assess evolutionary context and the plausibility of transferring functional annotations across species.
Length	The number of amino acids in the protein.	A large length difference may indicate different domains or a protein fusion.
Description	Description from the vector-index payload.	Depends on the data used when the collection was built and may be older than the current UniProt record.
PDB IDs	Linked structures, if available in UniProt.	Supports structural inspection, but not every protein has an experimentally determined structure.

2.5.3. Differences from BLAST and Alignment-Based Search

BLAST identifies locally similar regions and returns alignments that can be inspected residue by residue, together with metrics such as bit score, identity, coverage, and E-value [4]. Semantic protein search does not generate an alignment during retrieval; it compares fixed-dimensional vectors. Therefore, the two methods answer different questions.

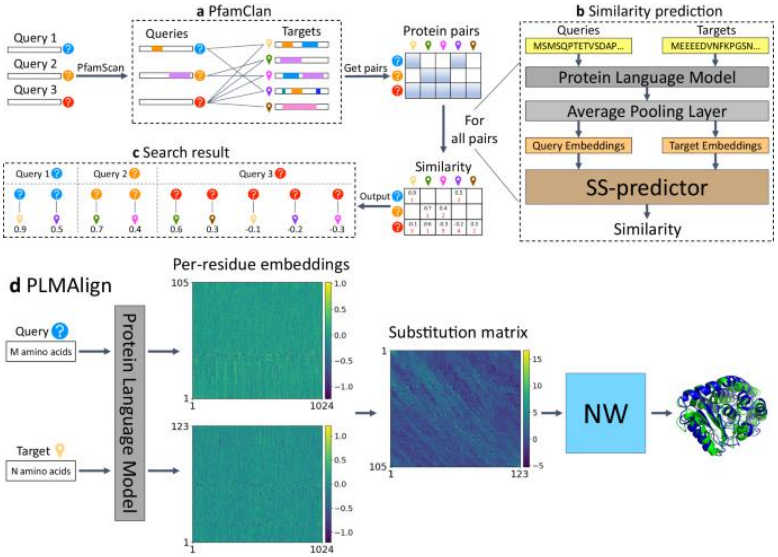


Figure 2.17. *Protein search using a protein language model*

Source: Liu et al., PLMSearch [26].

2.6. AI Agent and Tool Calling

An LLM generates and interprets text; an AI Agent adds memory, RAG, a tool registry, policies, and an execution loop to interact with data and system functions. The detailed tool catalog is provided in Appendix H.

2.6.1. Distinguishing Chatbots, RAG, and AI Agents

Table 2.*10. Levels of AI support in BioFlow-Scope*

Level	Primary capability	Limitations
Chatbot LLM	Receives a question and generates an answer from the prompt and model parameters.	May rely on outdated knowledge or generate unsupported information.
RAG assistant	Retrieves relevant chunks and places them in context before answering.	Quality depends on the documents, embeddings, and retrieval; it does not yet perform actions.
Tool-using Agent	Can select a business function, pass parameters, receive results, and continue responding.	May select the wrong tool or parameters; requires permissions, logging, and HITL.

Level	Primary capability	Limitations
Orchestrated Agent Task	Breaks a goal into multiple steps while tracking state and trace information.	More complex; requires budgets, retries, and clearly defined stopping points.

RAG addresses the question “what information should be retrieved to answer?”, whereas tool calling addresses “what action or data source should be invoked?”. An Agent can use both: first retrieve methodological documentation, then call a tool to obtain a run status or search for a protein, and finally combine the evidence to produce an answer.

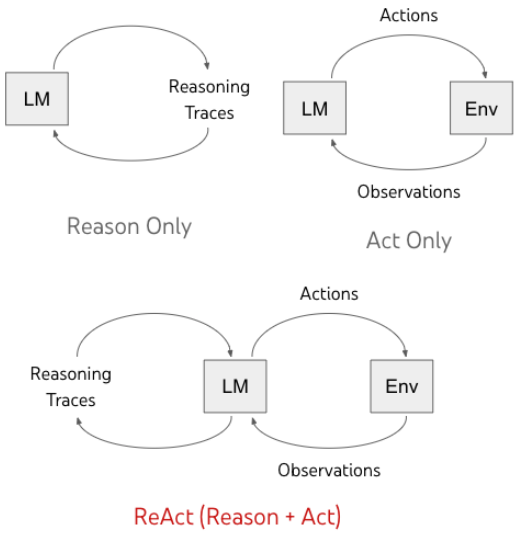


Figure 2.18. *Reason Only, Act Only, and ReAct mechanisms*

Source: Google Research [28].

2.6.2. Lifecycle of a Tool Call

1. The user sends a natural-language request within a chat session.

2.6.3. RAG, Memory, and Evidence

The Agent in BioFlow-Scope maintains a chat session and loads up to the 40 most recent messages. Messages remain stored in PostgreSQL, so when memory is reinitialized the system can reload the session history. A session may also be associated with a research project; project-memory and summarizer components support longer-term context at the project level.

2.6.4. Human-in-the-Loop and Action Control

Tools with side effects must pass permission checks and Human-in-the-Loop control. BioFlow-Scope does not allow the Agent to launch pipelines or import data

solely from a chat request; sensitive operations create a configuration or approval step before execution. [29]

Table 2.*11. AI Agent control layers in the source code*

Mechanism	Objective	Example
Authentication/ownership	Prevents the Agent from operating on resources that do not belong to the user.	Tools receive userId from the system context; services validate access to pipelines, runs, sessions, and projects.
Validation	Prevents missing or malformed parameters.	UUIDs, accessions, import source/mode, and input slots are validated before execution.
HITL approval	Requires the user to confirm actions that have an impact.	Dataset imports, pipeline execution, and other guarded actions.
Sensitive data egress guard	Warns when a prompt contains long sequences, local paths, tokens, or private metadata and an external provider is being used.	Requires consent or recommends redaction before continuing.
AI policy	Applies rules at the system/project/workspace scope.	Blocks prohibited activities or restricts external providers.
Tool-call audit	Stores arguments, result/error, duration, and status.	STARTED → RESULT/ERROR trong agent_tool_calls.
Evidence validation	Links a claim to a tool result, artifact, or knowledge chunk.	Claims without supporting sources may be marked as unverified.

CHAPTER 3. SOLUTION DESIGN AND IMPLEMENTATION

3.1. System Architecture

3.1.1. Overall System Architecture

BioFlow-Scope uses a client–server web architecture, with Next.js/React on the frontend and a Spring Boot modular monolith on the backend. Business domains are separated into modules but deployed within a unified backend, which is appropriate for the scope of the thesis.

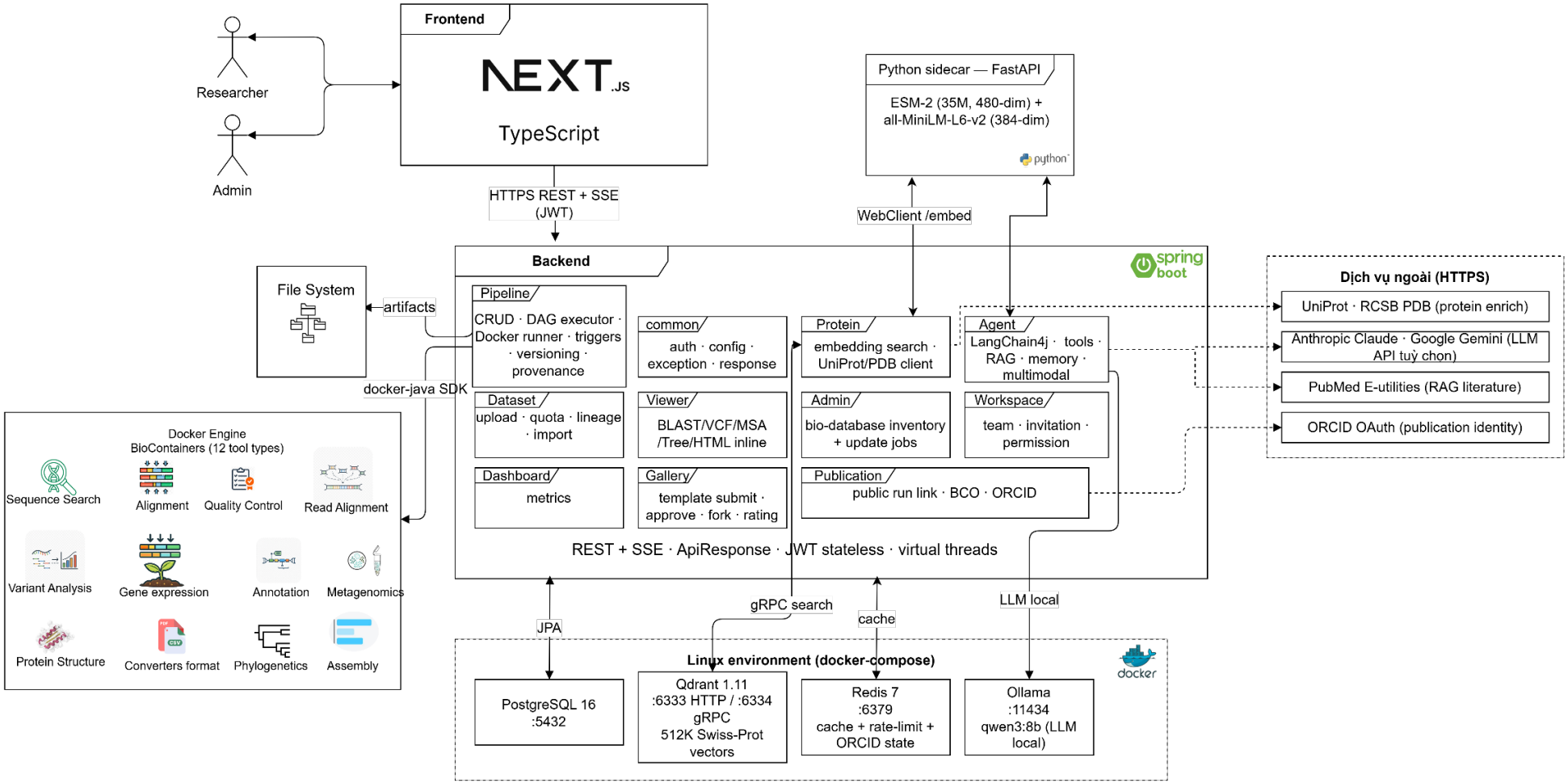


Figure 3.1. *Overall architecture of BioFlow-Scope*

3.1.1.1. Presentation layer

The presentation layer consists of the Web Application and the API interfaces used by the browser. The Web Application provides Smart Canvas for pipeline design, dataset and Analyze History management screens, run-monitoring interfaces, Semantic Protein Search, AI Chat, workspace functions, and administration pages.

3.1.1.2. Application layer

The Spring Boot Backend is the central business layer, responsible for authentication, authorization, APIs, and orchestration of the Pipeline, Dataset, History, Protein Search, Workspace, Publication, and AI Agent modules.

3.1.1.3. Execution and intelligence layer

The execution and intelligence layer includes the scheduler, background workers, DAG Executor, Docker Task Runner, Embedding Service, and Model/Agent Runtime. The scheduler and workers are responsible for triggering tasks that should not be processed synchronously within a short request.

3.1.1.4. Data and storage layer

BioFlow-Scope uses multiple storage mechanisms because each data type has different characteristics. PostgreSQL stores transactional data and data requiring long-term traceability, including users, pipelines, runs, steps, datasets, artifact metadata, workspaces, approvals, evidence, and Agent history.

3.1.1.5. External systems and constraints

External integrations include Keycloak/OIDC, a container registry, UniProt/PDB/PubMed/KEGG, data sources, and LLM providers. The system currently uses local artifact storage; Kafka, Kubernetes, and object storage are not part of the primary deployment architecture.

3.1.2. Container Diagram

The C4 Container Diagram clarifies the major applications and data stores inside the BioFlow-Scope boundary and shows the external systems that connect directly to it. In C4, a “container” does not refer only to a Docker container; it means an application or data store that can be run, deployed, or used to store data.

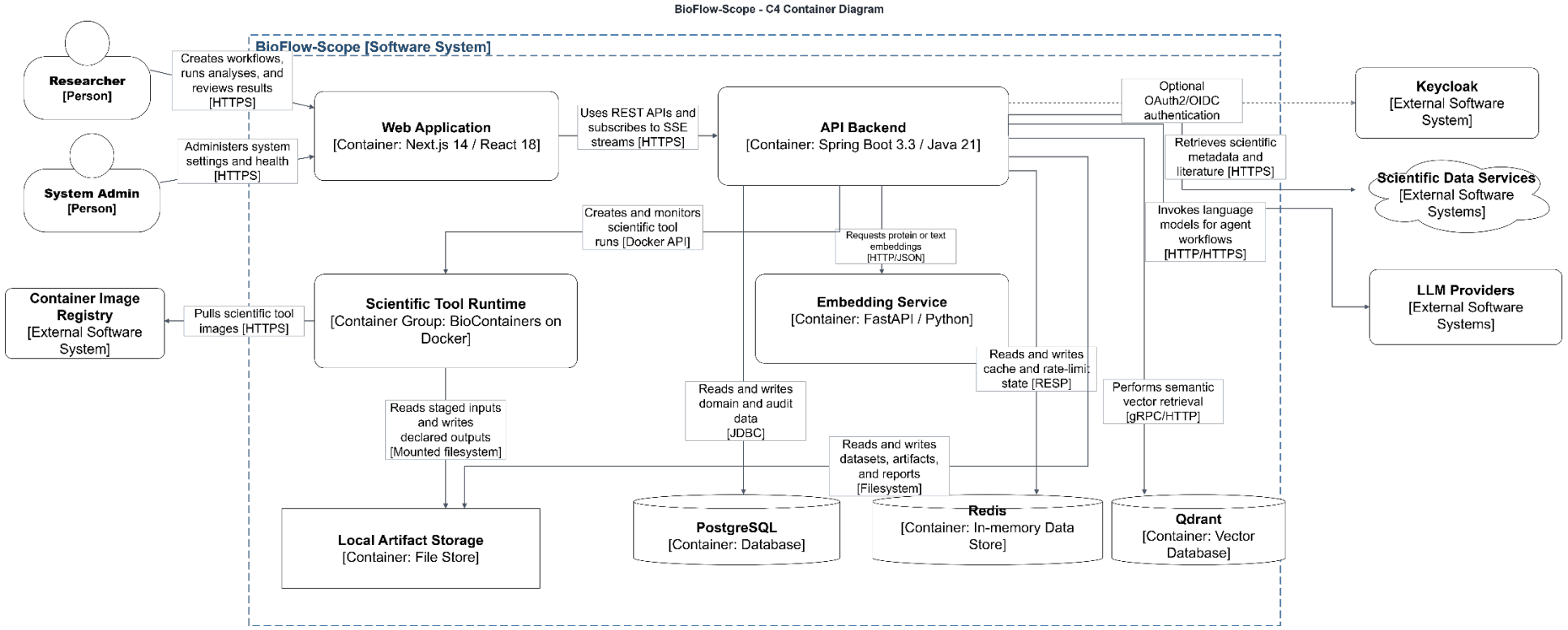


Figure 3.2. *C4 Container Diagram – BioFlow-Scope*

3.1.2.1. Web Application

The Web Application is the container that directly interacts with the Researcher and System Administrator. It is responsible for displaying data and organizing interactions with Smart Canvas, datasets, runs, history, protein search, the AI Agent, and administration. The Web Application does not directly access PostgreSQL, Qdrant, or Docker Engine; all business operations are sent to the API Backend. This principle keeps security and business logic on the server side and prevents the frontend from depending directly on data infrastructure.

3.1.2.2. API Backend

The API Backend receives requests, applies security/policies, and coordinates PostgreSQL, Redis, Qdrant, the Embedding Service, Docker Engine, and external services. All integrations pass through the backend to maintain logging and access control.

3.1.2.3. Scientific Tool Runtime

The Scientific Tool Runtime consists of short-lived containers created for individual Pipeline steps. They read inputs from the working directory, write artifacts, and return stdout/stderr and an exit code to the backend.

3.1.2.4. Embedding Service

The Embedding Service is a Python/FastAPI sidecar that generates protein and text embeddings. It is separated from the Java backend to isolate machine-learning dependencies and make model replacement easier.

3.1.2.5. Data containers

PostgreSQL stores persistent relational data; Redis holds cache and short-lived state; Qdrant provides vector search; Local Artifact Storage stores large files. Artifact metadata, checksums, and lineage are recorded in PostgreSQL.

3.1.2.6. External systems

Keycloak, the Container Image Registry, Scientific Data Services, and LLM Providers lie outside the system boundary because BioFlow-Scope does not own their internal lifecycle. Relationships in the diagram should state the purpose of communication, such as OAuth2/OIDC authentication, downloading tool images, retrieving scientific metadata, or invoking model inference.

3.1.3. Main component diagrams

3.1.3.1. Pipeline Execution and Provenance

Purpose: This subsystem converts a designed pipeline into a traceable and reproducible execution. It not only launches tools but also prepares inputs, validates the DAG, manages the state of each step, records logs, registers artifacts, and synchronizes results into Analyze History.

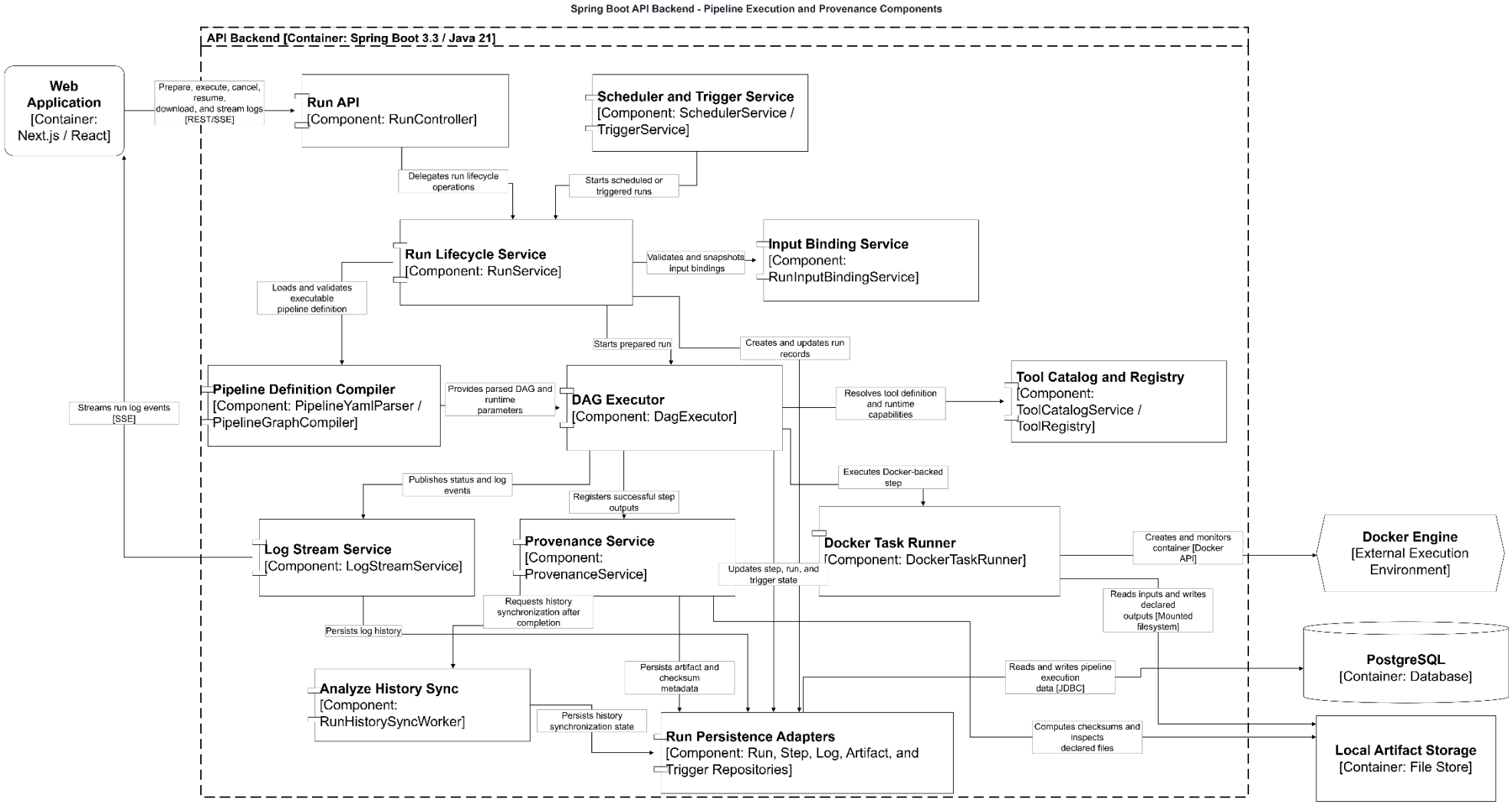


Figure 3.3. *C4 Component Diagram – Pipeline Execution and Provenance*

3.1.3.2. Semantic Protein Search

Purpose: Semantic Protein Search supports the retrieval of candidate proteins based on distance in embedding space and then enriches them with metadata so that researchers have a basis for further verification. This function is not treated as an automatic gene-function determination tool; it is a candidate-retrieval layer that combines a protein model, a vector database, and scientific databases.

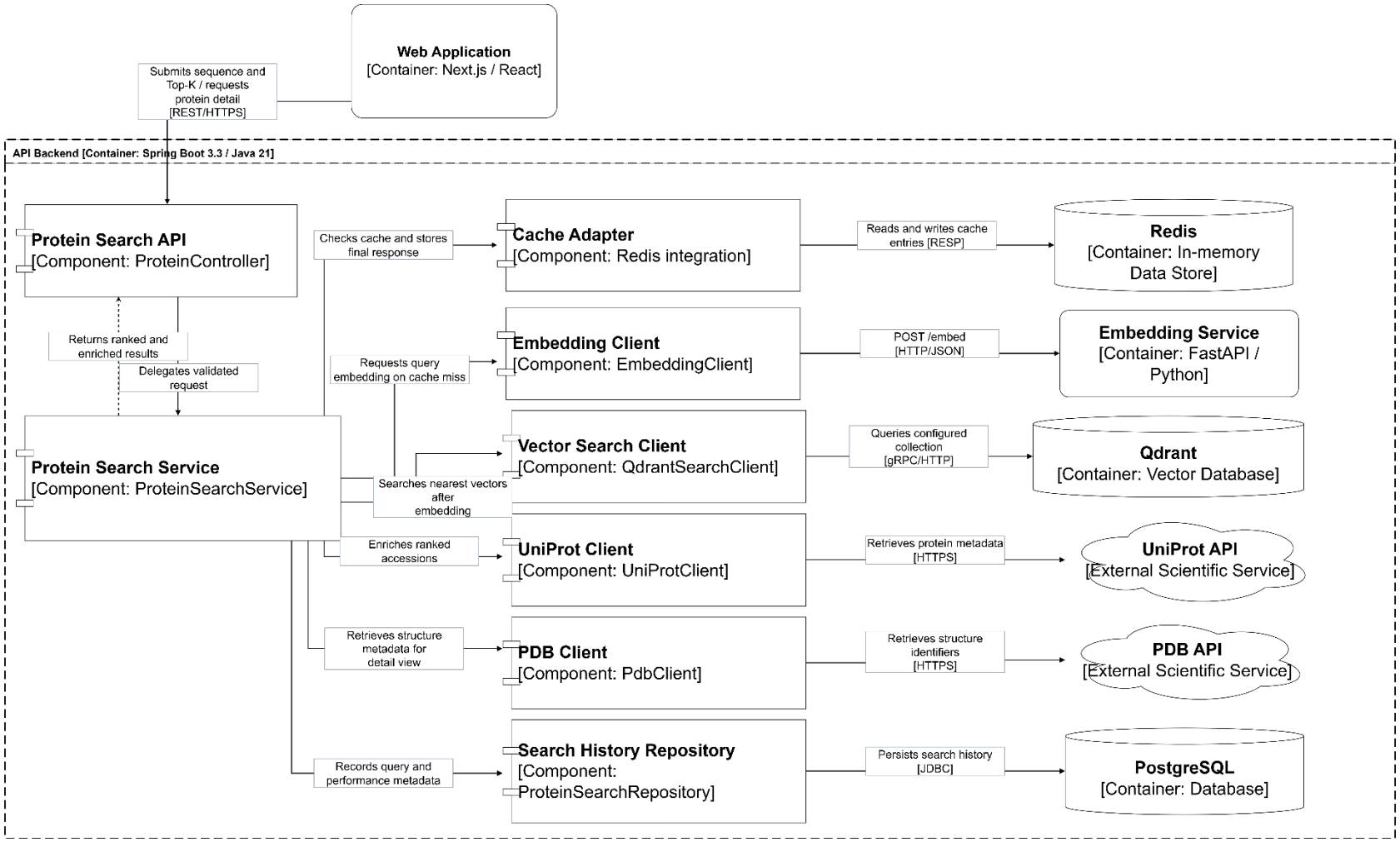


Figure 3.4. *C4 Component Diagram – Semantic Protein Search*

Source: compiled by the author from the BioFlow-Scope source code and deployment configuration.

3.1.3.3. AI Agent Core

Purpose: AI Agent Core provides a natural-language interface through which users can retrieve knowledge and use backend-controlled functions. The subsystem supports both interactive chat and multi-step Agent Tasks. Unlike a conventional chatbot, the model does not directly access data or execute commands on its own; every action must pass through the RAG retriever, tool gateway, domain services, and application control layers.

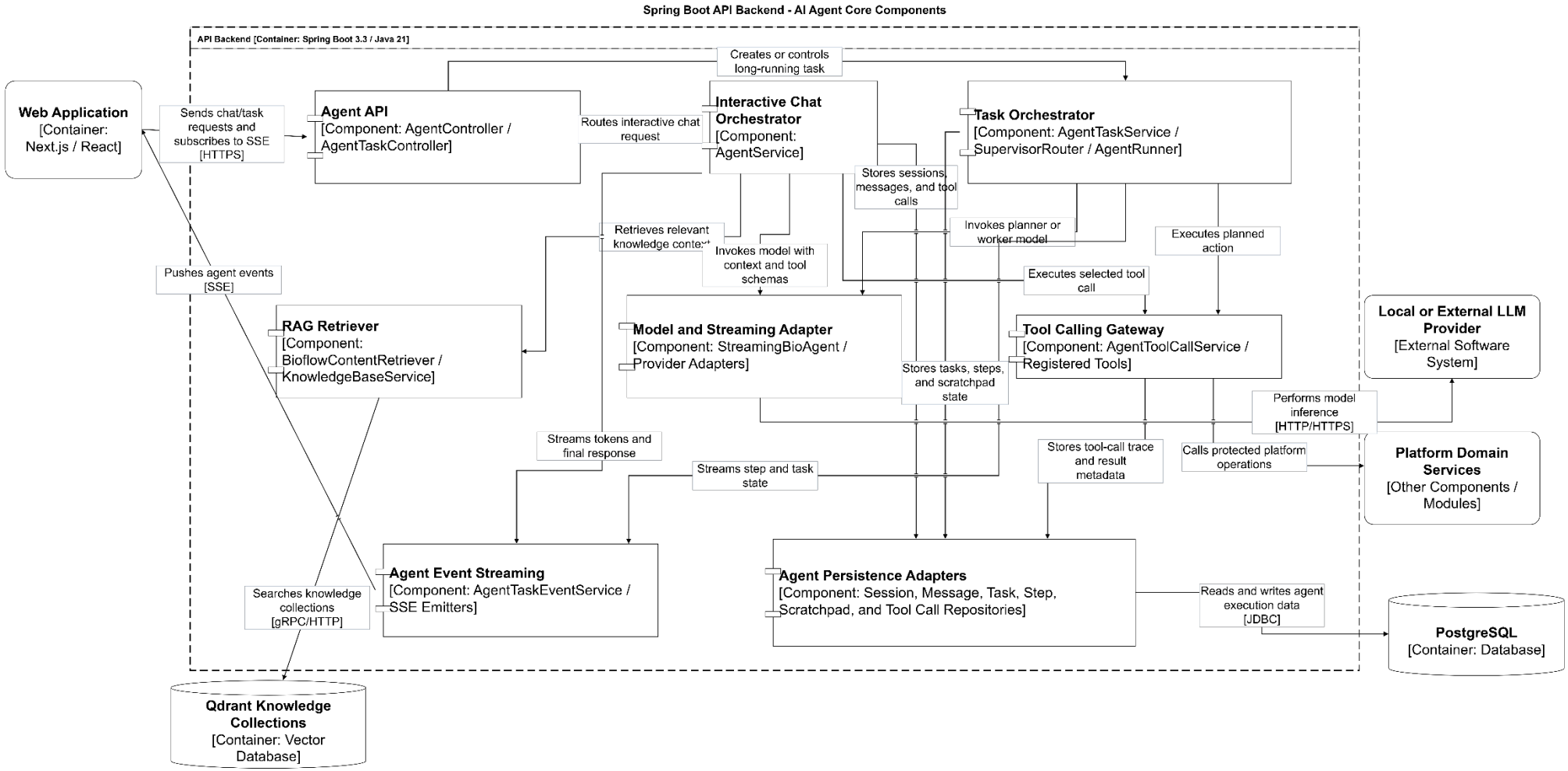


Figure 3.5. *C4 Component Diagram – AI Agent Core*

3.1.3.4. AI Governance, Approval and Evidence

Purpose: The AI governance subsystem restricts Agent actions and helps users verify the sources behind generated content. It addresses three categories of risk: the model may propose actions outside the permitted scope, sensitive data may be sent to an external provider, and responses may contain unverified claims or citations.

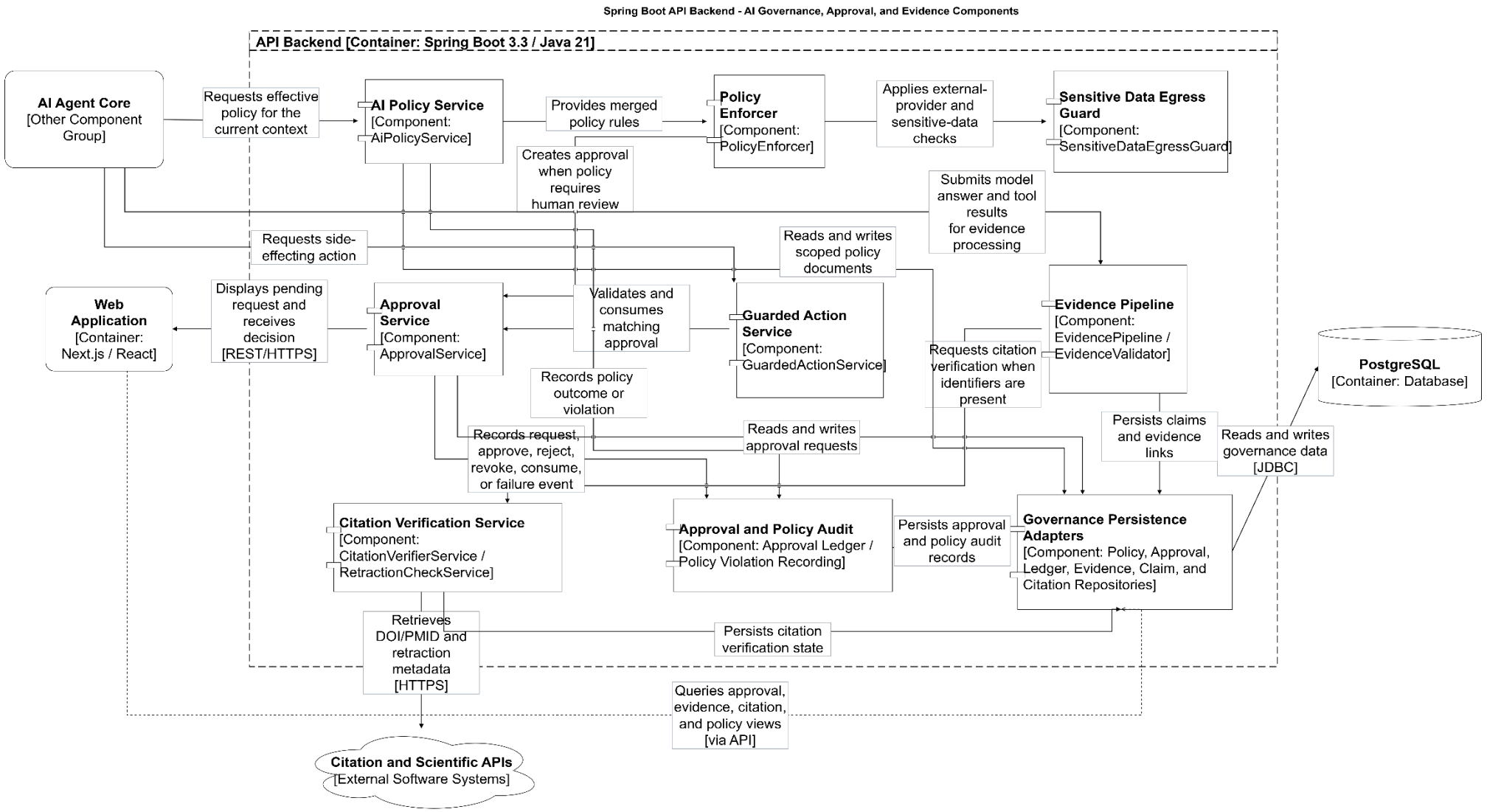


Figure 3.6. *C4 Component Diagram – AI Governance, Approval and Eviden*

3.2. Data Design

3.2.1. Data Classification and Storage Strategy

BioFlow-Scope does not use a single data store for every type of data. PostgreSQL manages transactional data and structured metadata; the filesystem stores input files, output files, and archives; Qdrant stores embedding vectors; and Redis stores caches, counters, and short-lived data.

Table 3.*1. Data classification and storage strategy*

Data category	Example	Storage location	Role
Transactional data	User, workspace, pipeline, run, dataset, approval	PostgreSQL	Ensures relational querying, consistency, and preservation of business history.
File data	FASTQ, BAM, VCF, HTML report, reproduction bundle	Filesystem	Stores large file contents; PostgreSQL stores only metadata and paths.
Vector data	Protein embeddings and text embeddings	Qdrant	Nearest-neighbor search and semantic retrieval.
Temporary data	Result cache, rate-limit counters, short-lived state	Redis	Reduces repeated computation and supports traffic control.

3.2.2. Conceptual Data Model

The conceptual data model presents all BioFlow-Scope data objects in a single diagram. Objects are grouped by domain so readers can identify responsibility boundaries while still seeing cross-domain relationships with business significance, such as a User owning a Workspace, a Pipeline being executed as a Run, a Dataset being bound to a Run, a Run creating Artifacts and History Items, a Project providing context for an Agent, or an Approval being linked to an Agent Task.

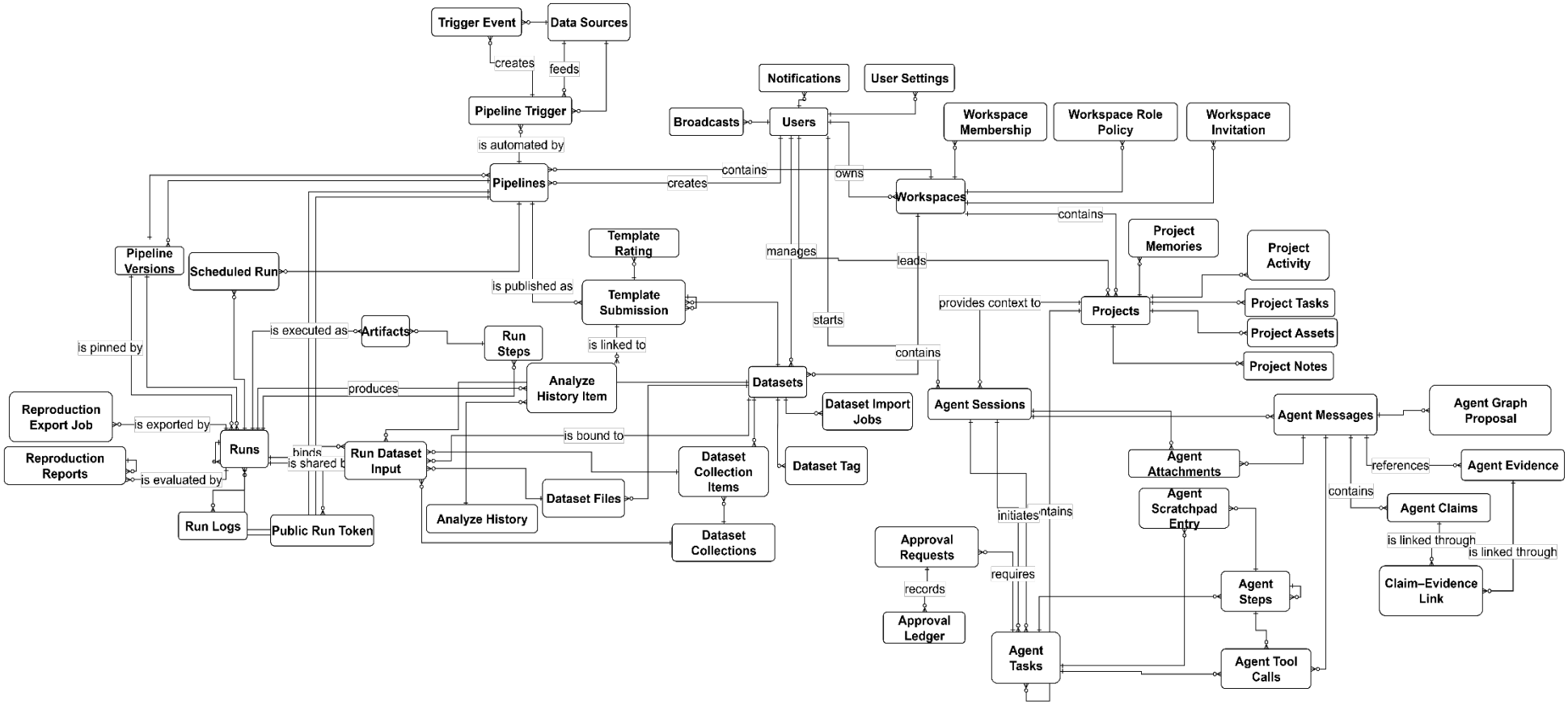


Figure 3.7. *Conceptual data model of BioFlow-Scop*

3.2.3. Physical Data Model

3.2.3.1. PDM for identity & administration

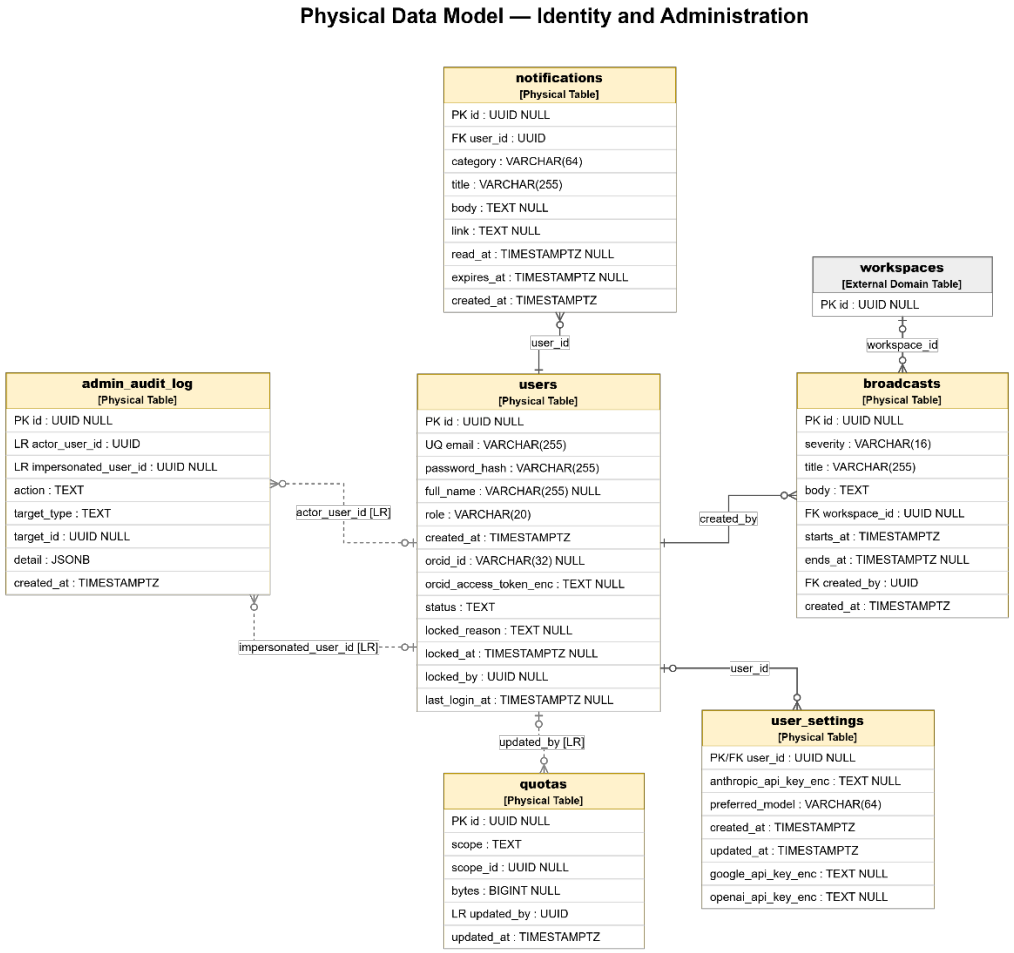


Figure 3.8. *Physical data model – identity & administration*

3.2.3.2. PDM for workspace & collaboration

This domain describes the Lab/workspace, members, invitations, and role-based policies. workspace_members is the junction table that resolves the many-to-many relationship between users and workspaces.

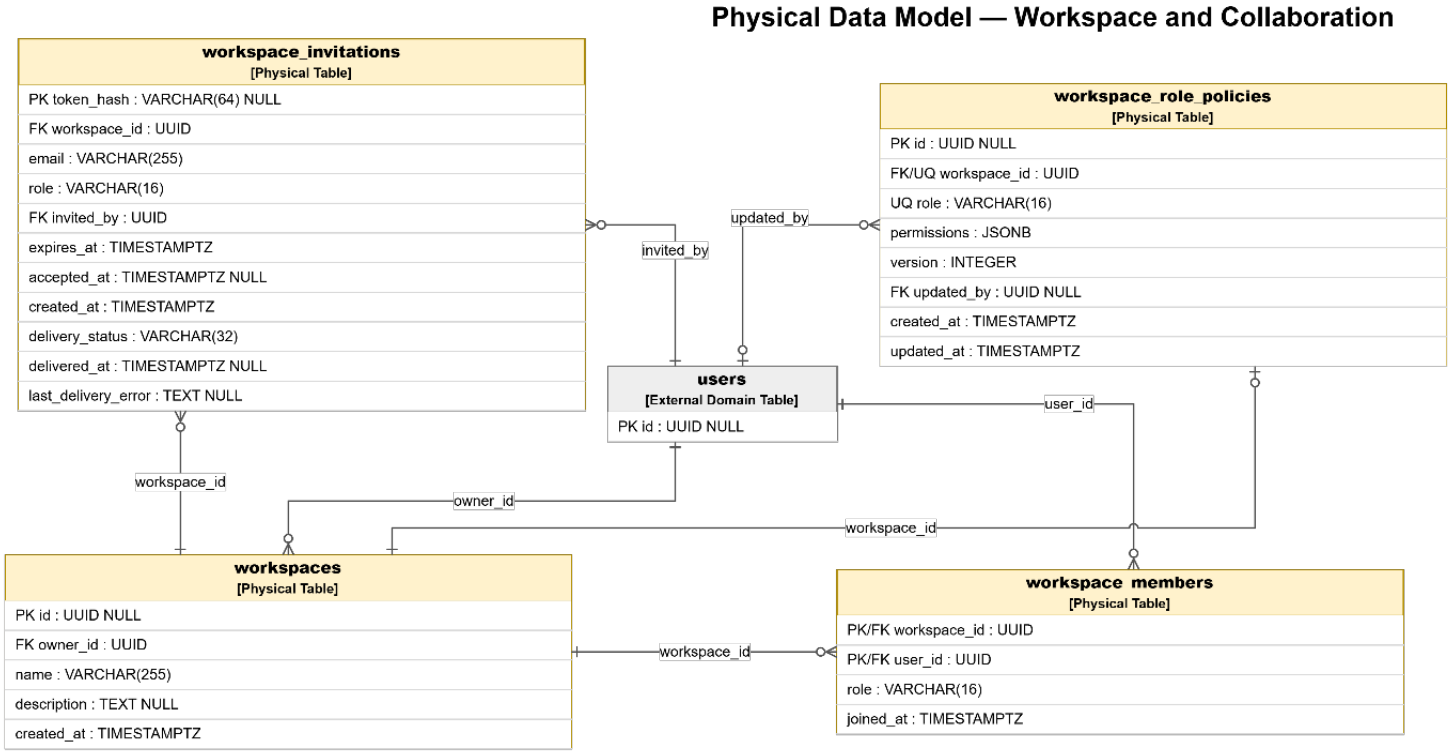


Figure 3.9. *Physical data model – workspace & collaboration*

3.2.3.3. PDM for pipeline & execution

This domain manages the pipeline lifecycle from definition, versioning, and scheduling through runs, steps, logs, and artifacts. The relationships among pipeline_version, run, and artifact form the foundation for reproducibility and traceability.

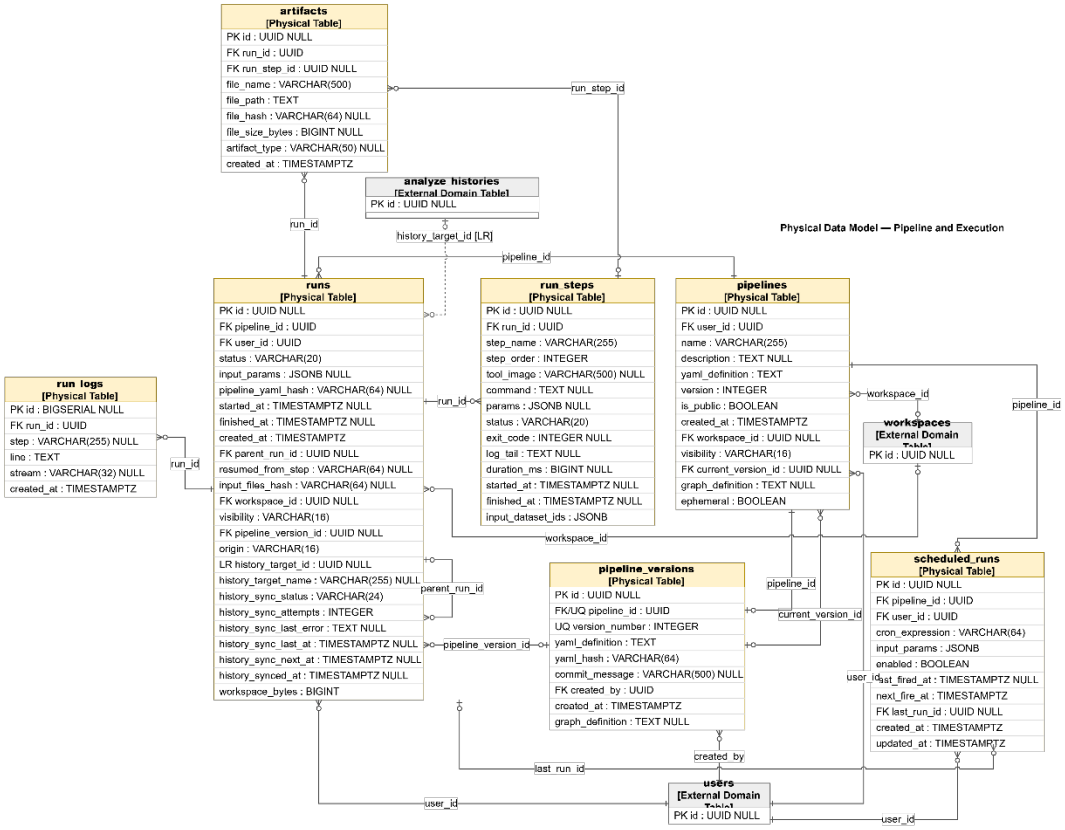


Figure 3.10. *Physical data model – pipeline & execution*

3.2.3.4. PDM for data source & trigger

This domain manages local or workspace data sources and the mechanism for automatically creating runs when matching data appears. trigger_events stores detection status, retries, and the run that was created.

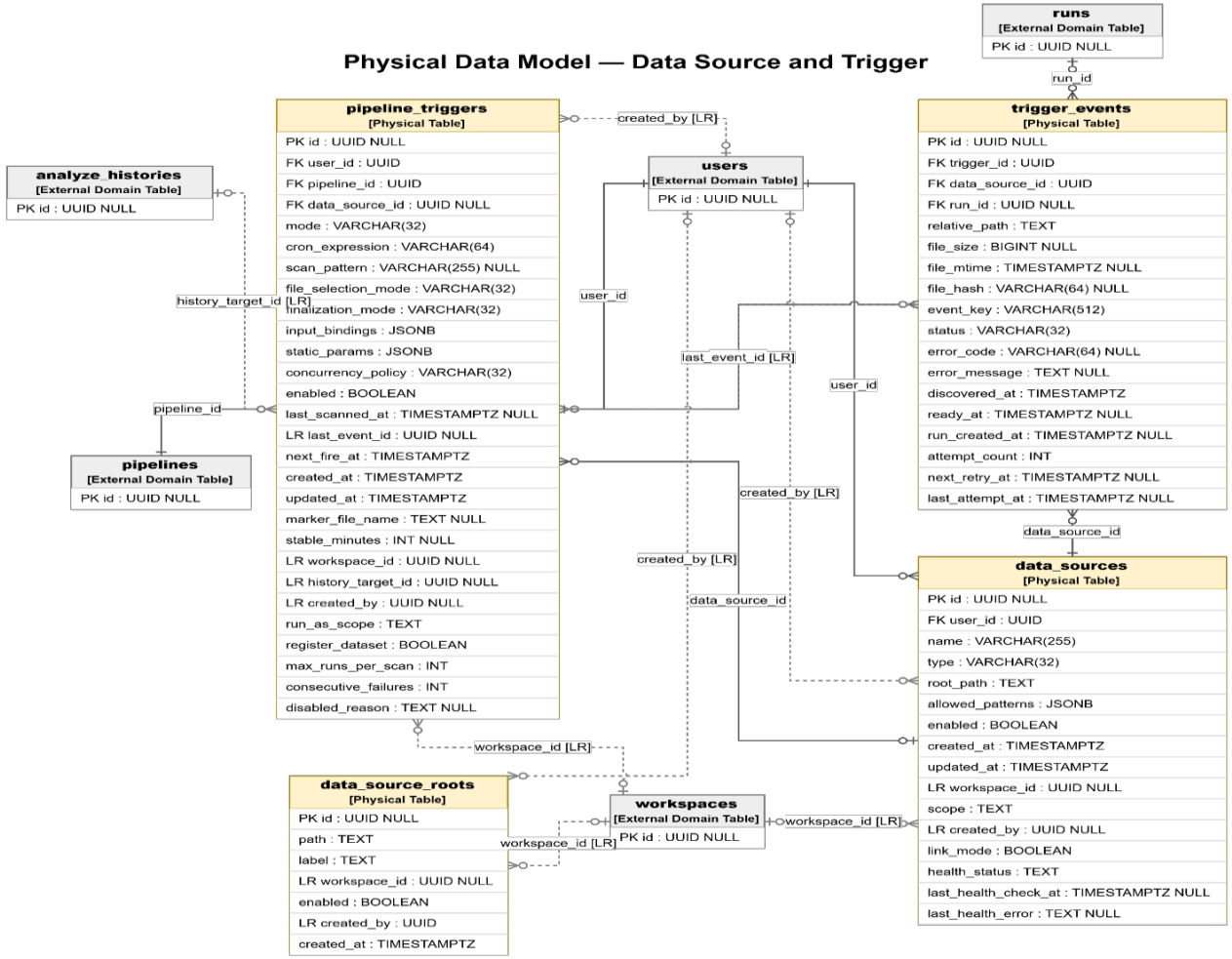
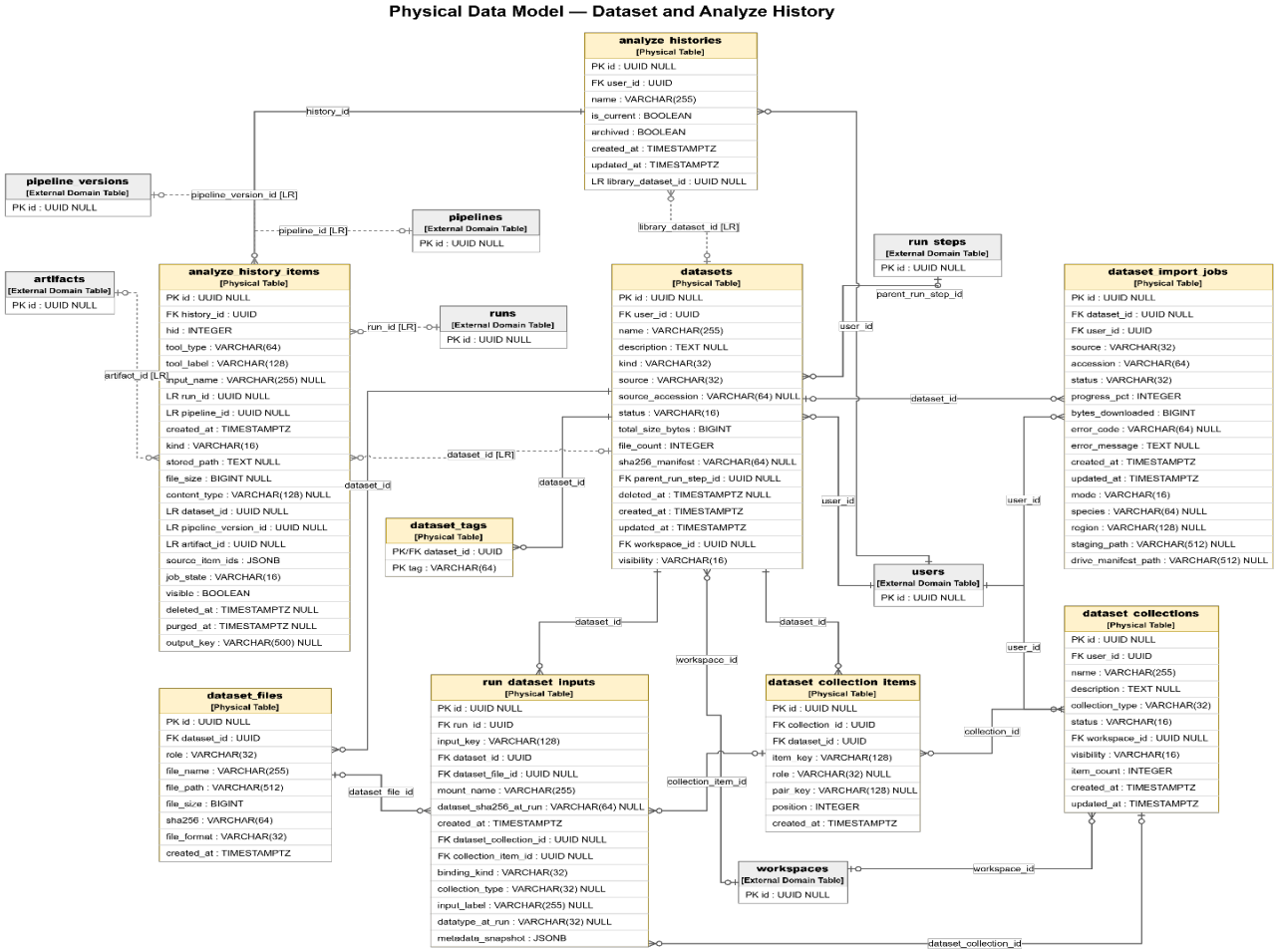


Figure 3.11. *Physical data model – data source & trigger*

3.2.3.5. PDM for dataset & analyze history

This domain covers the dataset library, physical files, collections, import jobs, input bindings, and Analyze History. run_dataset_inputs stores a snapshot of data at execution time, while analyze_history_items links results to runs, pipelines, datasets, and artifacts.



3.2.3.6. PDM for research project & memory

This domain organizes long-term research context, including projects, memory, notes, assets, tasks, and activities. Some asset/target relationships are polymorphic and are therefore stored using a type and UUID rather than a fixed foreign key.

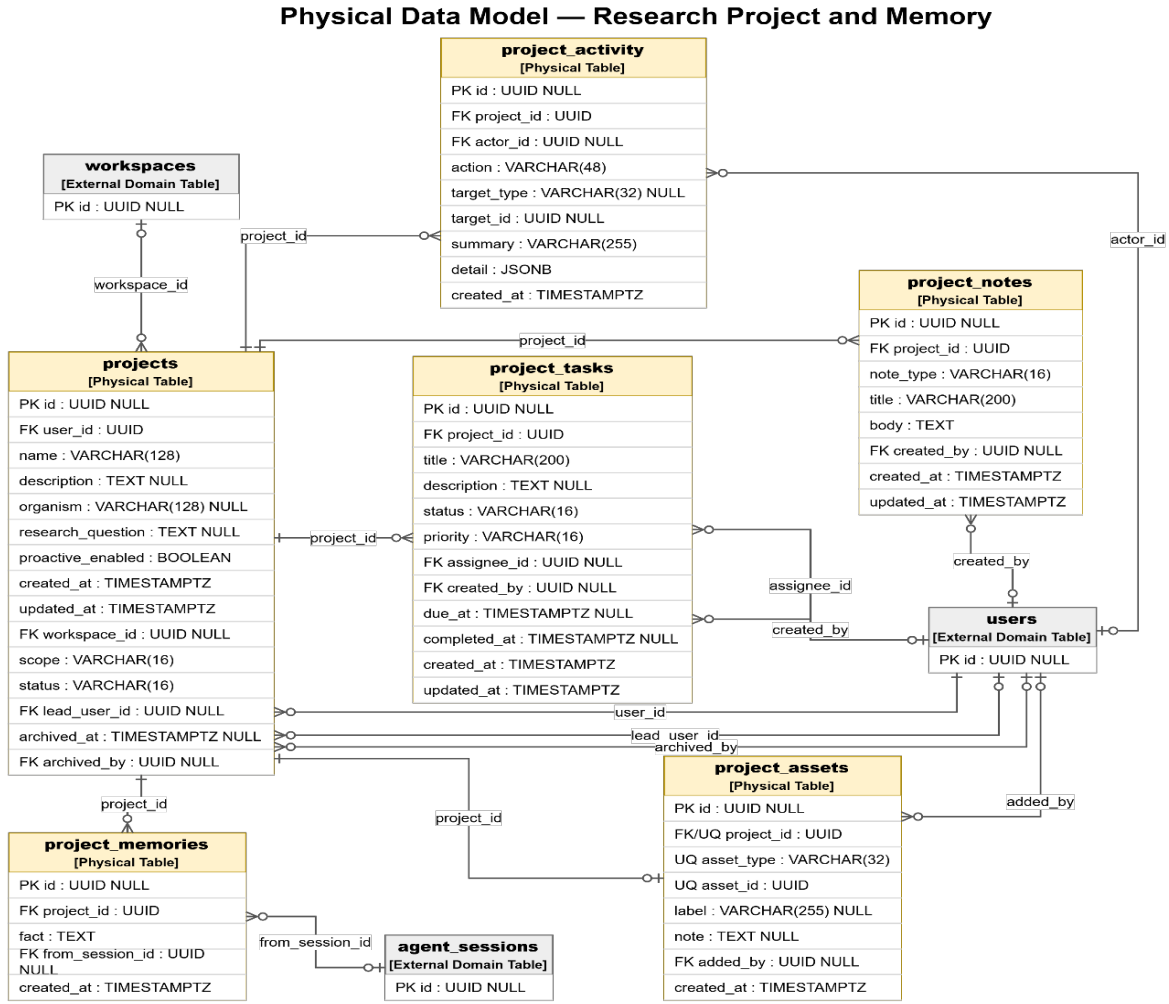


Figure 3.13. *Physical data model – research project & memory*

3.2.3.7. PDM for AI Agent core

This domain stores chat sessions, messages, attachments, multi-step tasks, steps, tool calls, scratchpad entries, and graph proposals. The task–step structure supports stateful and traceable Agent orchestration.

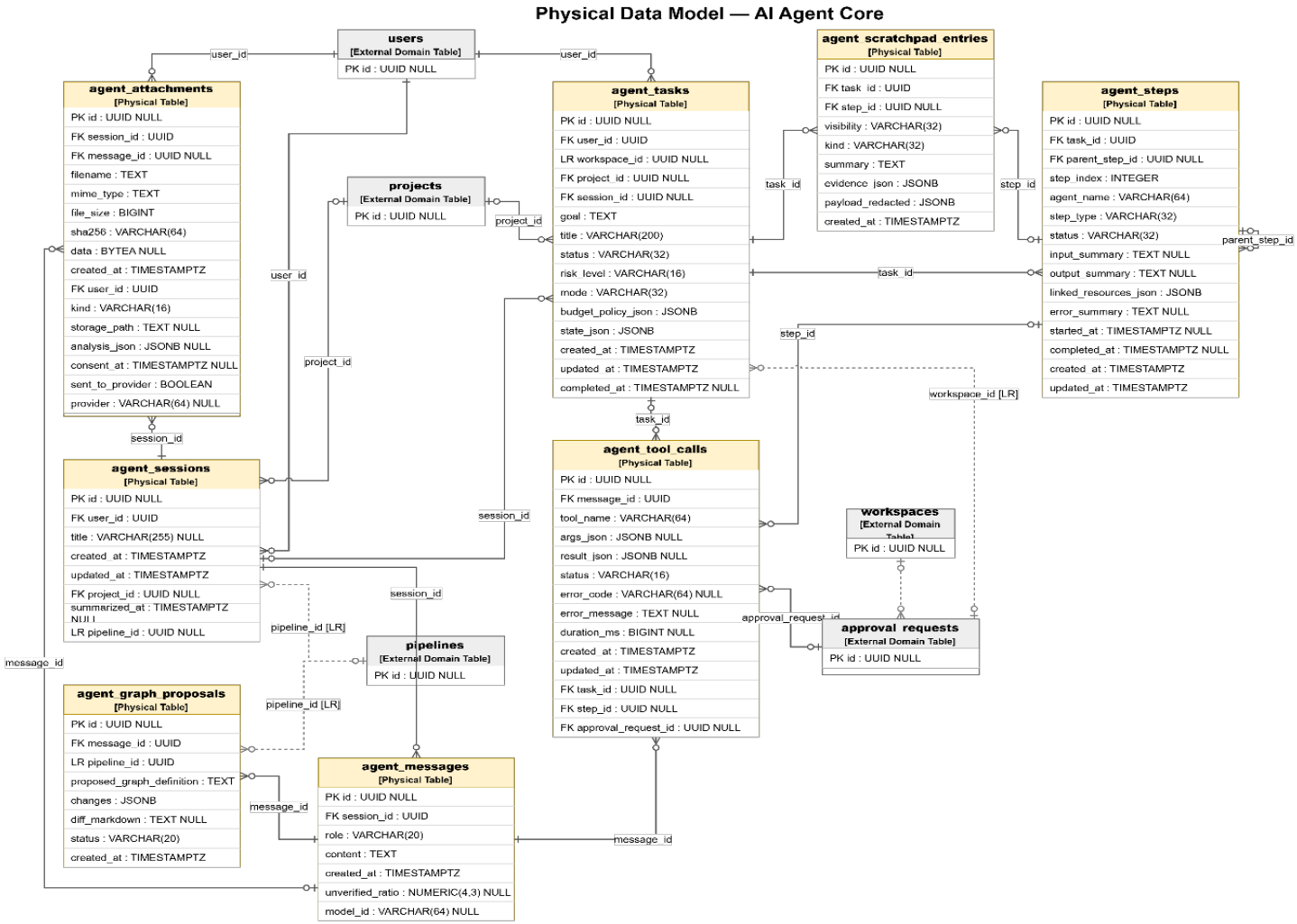


Figure 3.14. *Physical data model – AI Agent core*

3.2.3.8. PDM for AI governance & evidence

This domain stores approvals, ledgers, evidence, claims, policies, and citation verification. agent_claim_evidence is a many-to-many junction table; approval_ledger preserves an immutable history of human decisions.

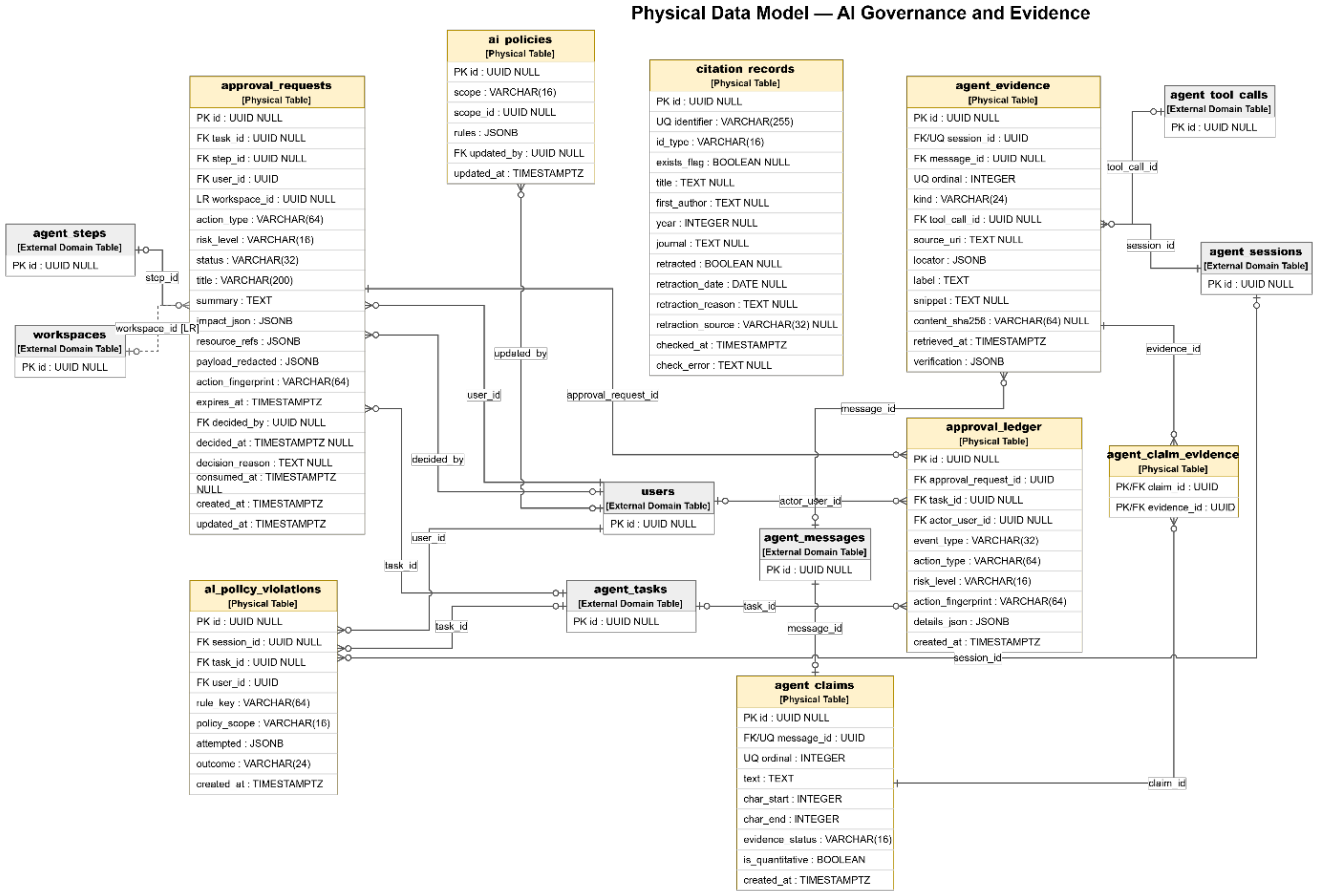


Figure 3.15. *Physical data model – AI governance & evidence*

3.2.3.9. PDM for publication & reproducibility

This domain manages the template gallery, run sharing, reproduction-bundle export/import, reproduction reports, and audit information. Some run/user references are retained as logical references to avoid hard coupling in long-term archive operations.

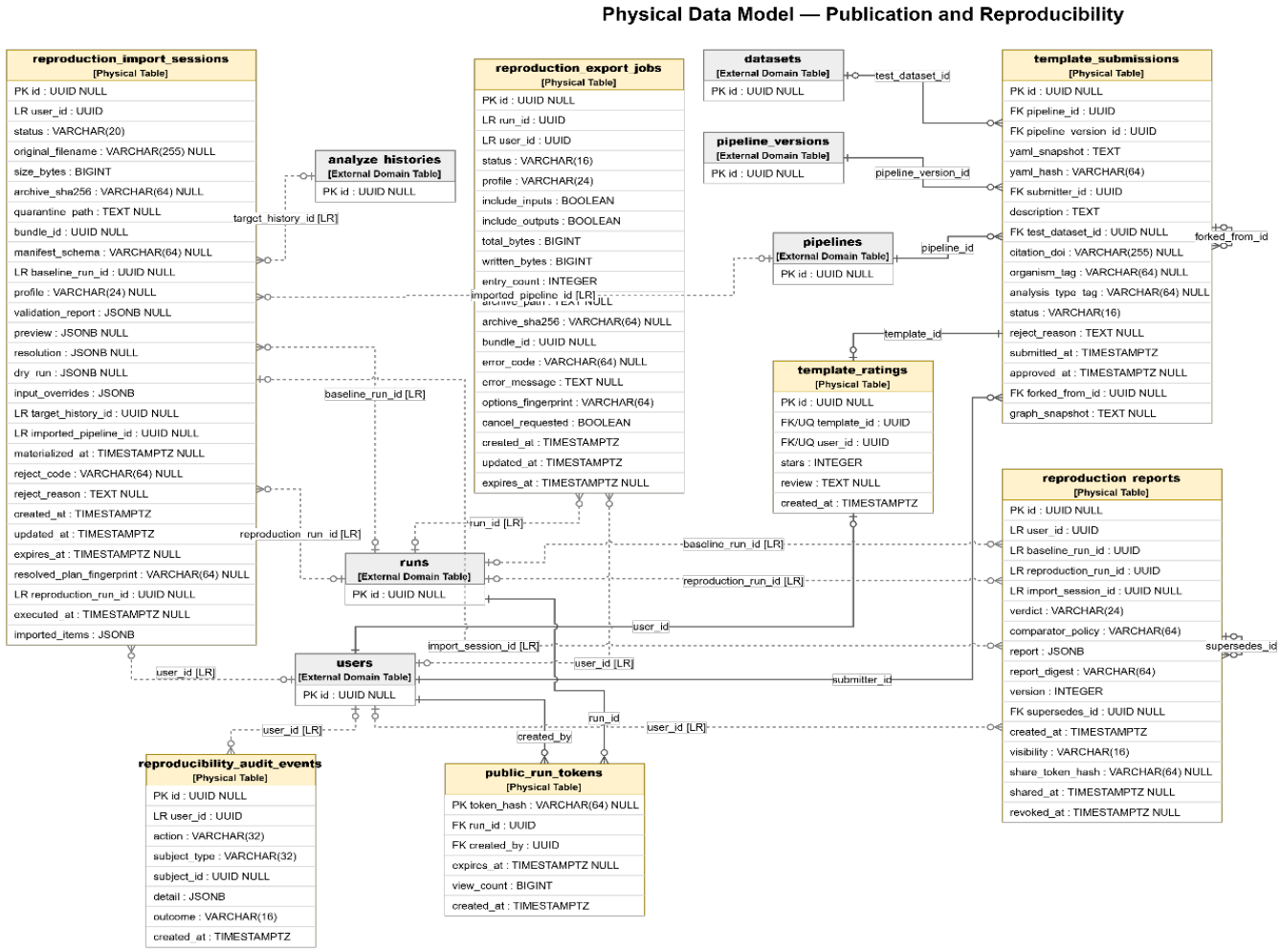


Figure 3.16. *Physical data model – publication & reproducibility*

3.2.3.10. PDM for protein search & biodatabase

This domain stores semantic protein-search history and manages local biological databases, versions, checksums, and update jobs.

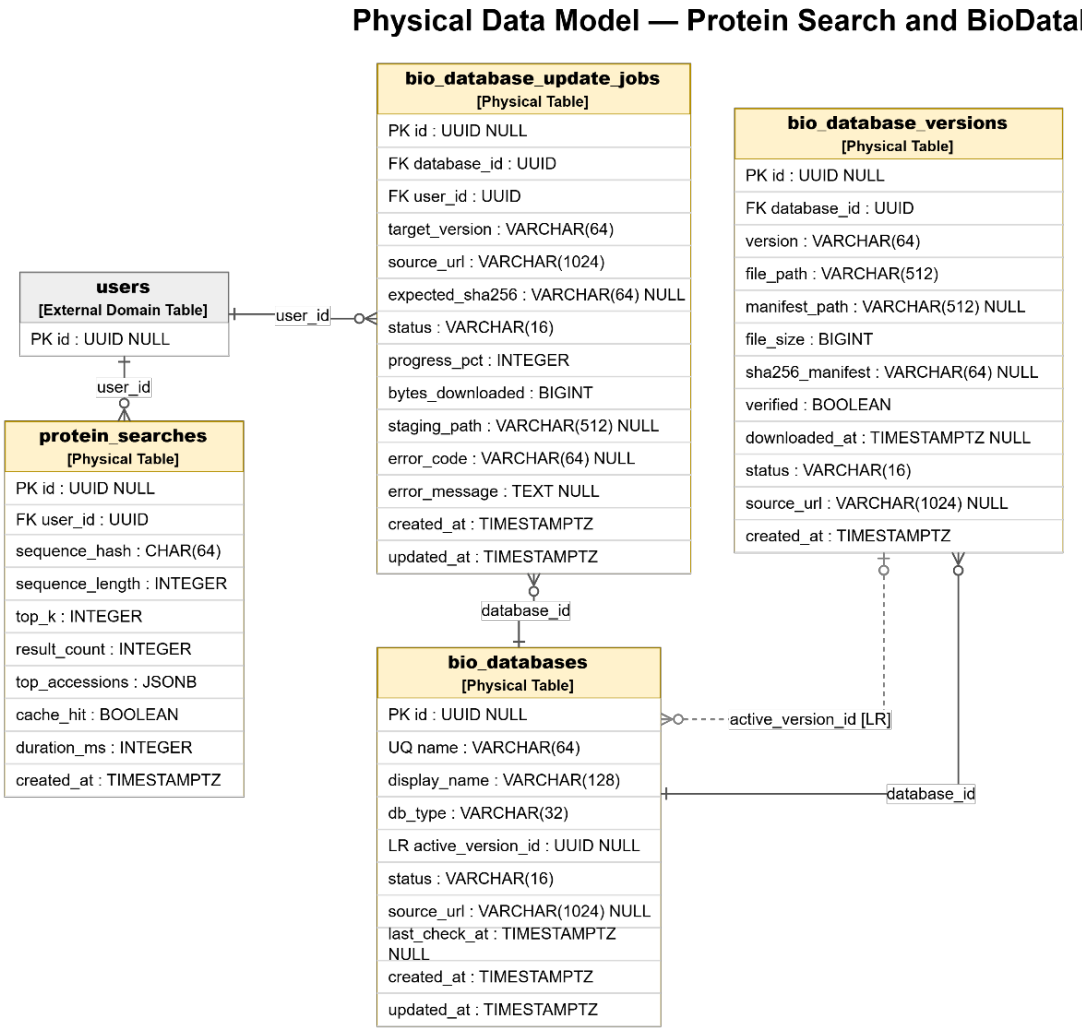


Figure 3.17. *Physical data model – protein search & biodatabase*

3.2.4. Vector and Temporary Data

Qdrant and Redis do not appear as PostgreSQL tables in the PDM. Qdrant organizes data into collections containing vectors and payloads, while Redis uses TTL-based keys for caches or counters. Because these stores do not have a fixed relational model like PostgreSQL, they are described at the logical-structure level rather than included in physical ERD pages.

***Table 3.**2. Vector stores and temporary data*

Data store	Logical object	Main content	Purpose
Qdrant	swiss_prot_v2025	480-dimensional protein vectors and accession/description payloads	Semantic protein search.
Qdrant	tool_manuals, pubmed_v2025	Text embeddings and source metadata	RAG cho AI Agent.
Redis	Protein search cache	Top-K results keyed by query hash and TTL	Reduces repeated embedding/vector searches.
Redis	Rate-limit counters	Counters by user/IP and time window	Protects endpoints and resources.

3.2.5. Detailed Data Dictionary

This section presents six representative data tables to illustrate schema organization. The physical data models in Section 3.2.3 remain the overall reference for relationships among tables.

3.2.5.1. users table

Stores user accounts, roles, lock status, login information, and ORCID links.

***Table 3.**3. Structure of the users table*

No.	Field name	Data type	Nullable	PK/FK	Constraint	Description
1	id	UUID	No	PK	PRIMARY KEY	Unique identifier of the record.
2	email	VARCHAR (255)	No		UNIQUE	Email address used to identify or contact the user.

No.	Field name	Data type	Nullable	PK/FK	Constraint	Description
3	password_hash	VARCHAR(255)	No		—	Hashed password value; plaintext passwords are not stored.
4	full_name	VARCHAR(255)	Yes		—	User's display name.
5	role	VARCHAR(20)	No		—	Role of the user or member within the corresponding scope.
6	created_at	TIMESTAMPTZ	No		—	Time at which the record was created.
7	orcid_id	VARCHAR(32)	Yes		—	Identifier associated with ORCID.
8	orcid_access_token_enc	TEXT	Yes		—	The orcid_access_token_enc attribute supports the business logic of the users table.
9	status	TEXT	No		—	Current status of the record in its business lifecycle.
10	locked_reason	TEXT	Yes		—	Reason recorded for the lock.
11	locked_at	TIMESTAMPTZ	Yes		—	Time at which the record was locked.
12	locked_by	UUID	Yes		—	Identifier of the user or actor who performed the lock operation.

No.	Field name	Data type	Nullable	PK/FK	Constraint	Description
13	last_login_at	TIMESTAMPTZ	Yes		—	Most recent login time.

3.2.5.2. workspaces table

Represents a collaboration space or Lab owned by a user.

Table 3.*4. Structure of the workspaces table*

No.	Field name	Data type	Nullable	PK/FK	Constraint	Description
1	id	UUID	No	PK	PRIMARY KEY	Unique identifier of the record.
2	owner_id	UUID	No	FK	FOREIGN KEY → users	Identifier referencing a record in the users table.
3	name	VARCHAR(255)	No		—	Name of the business object.
4	description	TEXT	Yes		—	Detailed description of the object.
5	created_at	TIMESTAMPTZ	No		—	Time at which the record was created.

3.2.5.3. runs table

Stores a pipeline execution, its status, origin, pinned version, and Analyze History association.

***Table 3.**5. Structure of the runs table*

No.	Field name	Data type	Nullable	PK/FK	Constraint	Description
1	id	UUID	No	PK	PRIMARY KEY	Unique identifier of the record.
2	pipeline_id	UUID	No	FK	FOREIGN KEY → pipelines	Identifier referencing a record in the pipelines table.
3	user_id	UUID	No	FK	FOREIGN KEY → users	Identifier referencing a record in the users table.
4	status	VARCHAR (20)	No		—	Current status of the record in its business lifecycle.
5	input_params	JSONB	Yes		—	Input parameters of the run stored in JSON format.
6	pipeline_yaml_hash	VARCHAR (64)	Yes		—	Hash of the pipeline YAML pinned at execution time.
7	started_at	TIMESTAMPTZ	Yes		—	Time at which the process started.
8	finished_at	TIMESTAMPTZ	Yes		—	Time at which the process finished.
9	created_at	TIMESTAMPTZ	No		—	Time at which the record was created.
10	parent_run_id	UUID	Yes	FK	FOREIGN KEY → runs	Identifier referencing a record in the runs table.

No.	Field name	Data type	Nullable	PK/FK	Constraint	Description
11	resumed_from_step	VARCHAR(64)	Yes		—	The resumed_from_step attribute supports the business logic of the runs table.
12	input_files_hash	VARCHAR(64)	Yes		—	Hash value of the input files used for integrity verification or comparison.
13	workspace_id	UUID	Yes	FK	FOREIGN KEY → workspaces	Identifier referencing a record in the workspaces table.
14	visibility	VARCHAR(16)	No		—	Type or category of visibility.
15	pipeline_version_id	UUID	Yes	FK	FOREIGN KEY → pipeline_versions	Identifier referencing a record in the pipeline_versions table.
16	origin	VARCHAR(16)	No		—	Type or category of origin.
17	history_target_id	UUID	Yes	LR	Logical reference → analyze_histories	Logical identifier or value referencing analyze_histories; no physical foreign key is defined.

No.	Field name	Data type	Nullable	PK/FK	Constraint	Description
18	history_target_name	VARCHAR(255)	Yes		—	Name or label of the target history.
19	history_sync_status	VARCHAR(24)	No		—	Processing status of history synchronization.
20	history_sync_attempts	INTEGER	No		—	Number of history synchronization attempts recorded.
21	history_sync_last_error	TEXT	Yes		—	The history_sync_last_error attribute supports the business logic of the runs table.
22	history_sync_last_at	TIMESTAMPTZ	Yes		—	Time of the most recent history synchronization attempt.
23	history_sync_next_at	TIMESTAMPTZ	Yes		—	Scheduled time of the next history synchronization attempt.
24	history_synced_at	TIMESTAMPTZ	Yes		—	Time at which history synchronization completed.
25	workspace_bytes	BIGINT	No		—	Size or capacity represented by workspace_bytes.

3.2.5.4. pipelines table

Stores the current pipeline definition, ownership, workspace, and visibility.

Table 3.*6. Structure of the pipelines table*

No.	Field name	Data type	Nullable	PK/FK	Constraint	Description
1	id	UUID	No	PK	PRIMARY KEY	Unique identifier of the record.
2	user_id	UUID	No	FK	FOREIGN KEY → users	Identifier referencing a record in the users table.
3	name	VARCHAR(255)	No		—	Name of the business object.
4	description	TEXT	Yes		—	Detailed description of the object.
5	yaml_definition	TEXT	No		—	Pipeline definition content in YAML format.
6	version	INTEGER	No		—	Version value of the object or data.
7	is_public	BOOLEAN	No		—	Indicates whether the resource is public.
8	created_at	TIMESTAMPTZ	No		—	Time at which the record was created.

No.	Field name	Data type	Nullable	PK/FK	Constraint	Description
9	workspace_id	UUID	Yes	FK	FOREIGN KEY → workspaces	Identifier referencing a record in the workspaces table.
10	visibility	VARCHAR(16)	No		—	Type or category of visibility.
11	current_version_id	UUID	Yes	FK	FOREIGN KEY → pipeline_versions	Identifier referencing a record in the pipeline_versions table.
12	graph_definition	TEXT	Yes		—	Pipeline graph-definition content.
13	ephemeral	BOOLEAN	No		—	Flag indicating ephemeral status.

3.2.5.5. agent_tasks table

Stores multi-step Agent goals, risk level, budget, and orchestration status.

Table 3.*7. Structure of the agent_tasks table*

No.	Field name	Data type	Nullable	PK/FK	Constraint	Description
1	id	UUID	No	PK	PRIMARY KEY	Unique identifier of the record.
2	user_id	UUID	No	FK	FOREIGN KEY → users	Identifier referencing a record in the users table.

No.	Field name	Data type	Nullable	PK/FK	Constraint	Description
3	workspace_id	UUID	Yes	LR	Logical reference → workspaces	Logical identifier or value referencing workspaces; no physical foreign key is defined.
4	project_id	UUID	Yes	FK	FOREIGN KEY → projects	Identifier referencing a record in the projects table.
5	session_id	UUID	Yes	FK	FOREIGN KEY → agent_sessions	Identifier referencing a record in the agent_sessions table.
6	goal	TEXT	No		—	Goal content of the record.
7	title	VARCHAR(200)	No		—	Display title of the record.
8	status	VARCHAR(32)	No		—	Current status of the record in its business lifecycle.

No.	Field name	Data type	Nullable	PK/FK	Constraint	Description
9	risk_level	VARCHAR(16)	No		—	The risk_level attribute supports the business logic of the agent_tasks table.
10	mode	VARCHAR(32)	No		—	Type or category of mode.
11	budget_policy_json	JSONB	No		—	Structured budget-policy data stored as JSON.
12	state_json	JSONB	No		—	Structured state data stored as JSON.
13	created_at	TIMESTAMPTZ	No		—	Time at which the record was created.
14	updated_at	TIMESTAMPTZ	No		—	Time at which the record was most recently updated.

No.	Field name	Data type	Nullable	PK/FK	Constraint	Description
15	completed_at	TIMESTAMPTZ	Yes		—	Time at which the process was completed.

3.2.5.6. agent_sessions table

Stores AI conversation sessions by user, project, and optional pipeline.

***Table 3.**8. Structure of the agent_sessions table*

No.	Field name	Data type	Nullable	PK/FK	Constraint	Description
1	id	UUID	No	PK	PRIMARY KEY	Unique identifier of the record.
2	user_id	UUID	No	FK	FOREIGN KEY → users	Identifier referencing a record in the users table.
3	title	VARCHAR(255)	Yes		—	Display title of the record.
4	created_at	TIMESTAMPTZ	No		—	Time at which the record was created.
5	updated_at	TIMESTAMPTZ	No		—	Time at which the record was most recently updated.
6	project_id	UUID	Yes	FK	FOREIGN KEY → projects	Identifier referencing a record in the projects table.

No.	Field name	Data type	Nullable	PK/FK	Constraint	Description
7	summarized_at	TIMESTAMPTZ	Yes		—	Time at which summarization was performed.
8	pipeline_id	UUID	Yes	LR	Logical reference → pipelines	Logical identifier or value referencing pipelines; no physical foreign key is defined.

3.3. Design of AI Functions

Section 3.3 focuses on three prominent AI capabilities: Semantic Protein Search, the AI Agent combining RAG/tool calling, and Human-in-the-Loop with Evidence/AI Policy; supporting functions are summarized only.

3.3.1. Overview of AI Functions

Table 3.9 summarizes AI functions observed in the source code and configuration; BB340 is not counted as a completed thesis function.

Table 3.*9. Summary of BioFlow-Scope AI functions*

No.	Function	Primary purpose	Depth of analysis
1	Semantic Protein Search	Finds proteins close to the query sequence in embedding space and then enriches them with UniProt/PDB metadata.	In-depth analysis
2	AI Chat and RAG	Supports natural-language interaction, retrieves manuals or related documents, and generates context-aware answers.	In-depth analysis
3	Tool Calling	Allows the Agent to invoke lookup, analysis, pipeline, dataset, and publication functions through controlled schemas.	In-depth analysis together with AI Chat

No.	Function	Primary purpose	Depth of analysis
4	Human-in-the-loop Approval	Pauses actions with side effects so users can review, approve, reject, or revoke them.	In-depth analysis
5	Evidence and Citation Verification	Links claims to evidence, verifies DOI/PMID identifiers, and warns about retracted or unverified citations.	In-depth analysis together with HITL
6	AI Policy and Sensitive Data Guard	Applies rules at system/workspace/project scope and checks sensitive data before invoking an external provider.	In-depth analysis together with HITL
7	Pipeline Recommendation	Suggests sequences of tools and analysis steps from a research question.	Brief description
8	Pipeline Editor Copilot	Creates pipeline drafts or proposes adding tool steps to a graph for the user to accept or reject.	Brief description
9	Run Diagnosis and Result Interpretation	Retrieves error logs, interprets artifacts or run results, and supports diagnosis of likely causes.	Brief description
10	Statistical, Comparative and Methods Assistance	Generates summary statistics, compares multiple runs, and writes Materials and Methods from provenance.	Brief description
11	Literature and Pathway Search	Queries PubMed and KEGG to support interpretation of methods, genes, or pathways.	Brief description
12	Multimodal Attachments and Project Memory	Accepts images or FASTA/FASTQ/VCF files, creates safe summaries, and maintains research context by project.	Brief description

The interface images in Sections 3.3–3.4 were captured from the local frontend using a mock API because the backend returned HTTP 503 at the time of capture; the images illustrate the UI only and are not runtime evidence.

3.3.2. Semantic Protein Search

3.3.2.1. Purpose

Semantic Protein Search ranks proteins by proximity in embedding space to broaden the candidate set; the results do not replace BLAST/alignment and still require examination of domains, structure, and experimental evidence.

In a DNA-analysis flow, a protein sequence predicted from a CDS can be submitted to Semantic Protein Search during post-annotation interpretation; the current version still requires the user to submit the sequence explicitly.

3.3.2.2. Input and output data

Table 3.*10. Input/output data – Semantic Protein Search*

Category	Content
Input data	An amino-acid sequence of 10 to 5,000 characters and a Top-K value from 1 to 100; the default Top-K is 20.
Intermediate data	Normalized sequence, sequence SHA-256, a 480-dimensional protein embedding, vector hits, and retrieved metadata.
Output data	A ranked protein list including accession, similarity score, protein name, organism, length, description, PDB IDs, and sequence when provided by the source.
Trace data	Sequence hash, sequence length, Top-K, result count, leading accessions, cache hit status, and processing time.

3.3.2.3. Functional interface

The `/search` interface contains an amino-acid/FASTA input area, an IsPETase example button, a Top-K field, a Search button, a ranked protein list, a detail area, and cache/processing-time status when provided by the response.

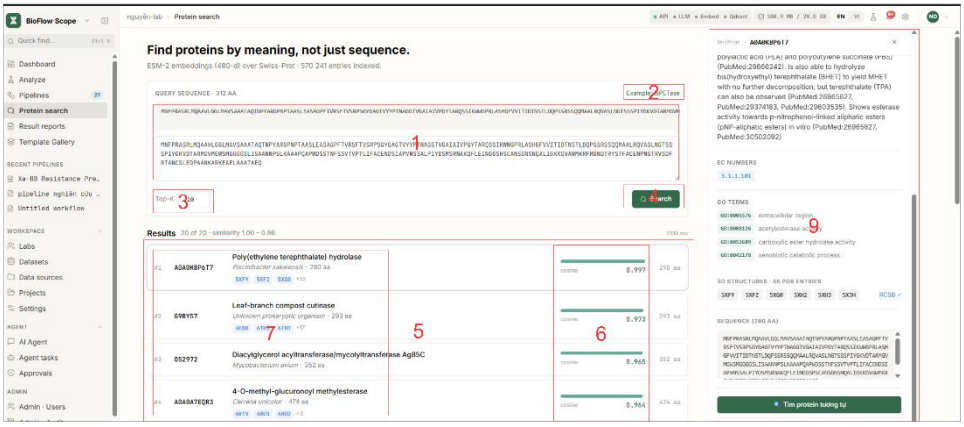


Figure 3.18. *Semantic Protein Search interface*

Note: illustrative interface data. Note: interface at `/search`.

***Table 3.**11. Interface components – Semantic Protein Search*

No.	Control type	Content	Notes
1	Textarea / Sequence Input	Amino-acid sequence or FASTA	Allows a protein sequence to be entered as the Semantic Protein Search query.
2	Button	Example: IsPETase	Quickly loads a sample protein sequence for demonstration and functional testing.
3	Number Input	Top-K	Specifies the maximum number of similar proteins to return.
4	Button	Search	Submits the query sequence to generate a protein embedding and perform vector search.
5	Result List / Result Cards	List of similar proteins	Displays results ranked by similarity score.
6	Similarity Score	Similarity score	Indicates proximity between the query-sequence vector and a protein vector in the database.
7	Protein Metadata	Accession, protein name, organism, and sequence length	Provides the main identifying information for each protein result.
8	Detail Control / Result Item	Open protein details	Allows the user to view the sequence, description, and additional scientific-source data when available.
9	Detail Panel	Detailed protein information	Displays additional metadata such as sequence, PDB, GO, or EC information when provided by the API.
10	Status / Processing Information	Cache and processing time	Displays cache status and query duration when these values are provided in the response.

3.3.2.4. Processing procedure

When an amino-acid sequence and Top-K are received, the backend normalizes the input, computes its SHA-256, and checks the Redis cache. A cache hit returns the stored result while still recording query history.

On a cache miss, the backend generates a 480-dimensional vector through the Embedding Service, queries Qdrant, enriches accessions from UniProt/PDB, caches the result for 24 hours, and records history in PostgreSQL before returning the Top-K results.

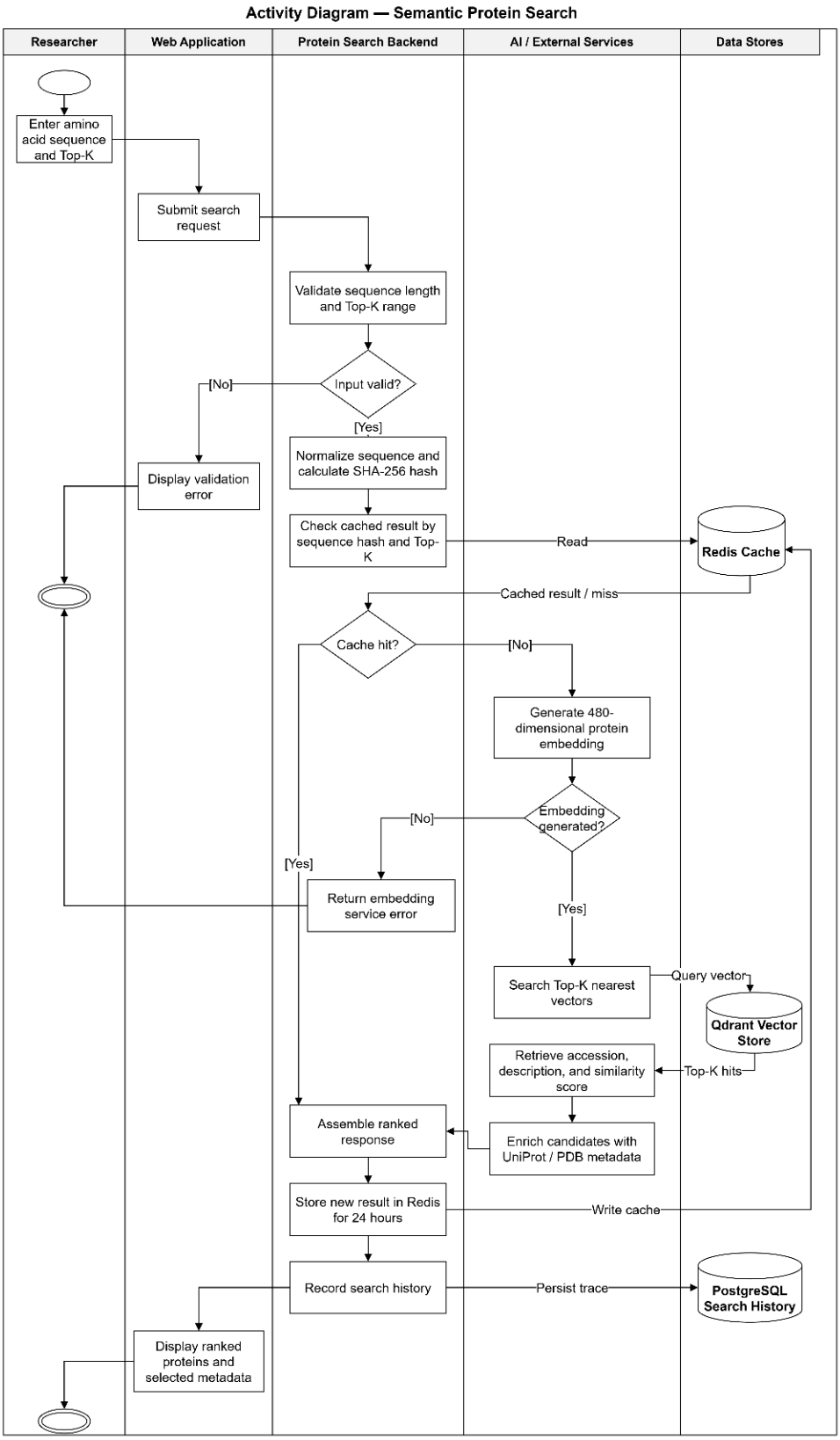


Figure 3.19. *Activity Diagram – Semantic Protein Search*

Source: compiled by the author from the system source code.

3.3.3. AI Agent, RAG, and Tool Calling

3.3.3.1. Purpose

The AI Agent allows users to query and interact with proteins, pipelines, runs, datasets, and documents through natural language; execution permissions remain controlled by the backend.

RAG supplements context from the knowledge base; tool calling allows the model to propose schema-defined operations. The backend validates permissions, inputs, and policy before execution.

3.3.3.2. Input and output data

Table 3.*12. Input/output data – AI Agent*

Category	Content
Input data	Conversation content, session, model preference, consent, project context, and optional attachments such as images, FASTA, FASTQ, or VCF files.
Additional context	Chat memory, project memory, RAG chunks, tool schemas, AI policy, and authenticated-user information.
Intermediate data	Tool-call arguments, tool results, agent trace, attachment summaries, evidence markers, and graph proposals when present.
Output data	Response tokens streamed through SSE, tool events, evidence events, approval-required notifications, graph proposals, or a sanitized final answer.
Trace data	Agent session, user/assistant messages, tool calls, attachments, claims, evidence, proposals, and task/step records when multi-step orchestration is used.

3.3.3.3. Functional interface

The `/chat` interface includes a session list, session search, conversation area, suggested prompts, attachment controls, a request composer, and an area for displaying tool/evidence events while the backend streams state.

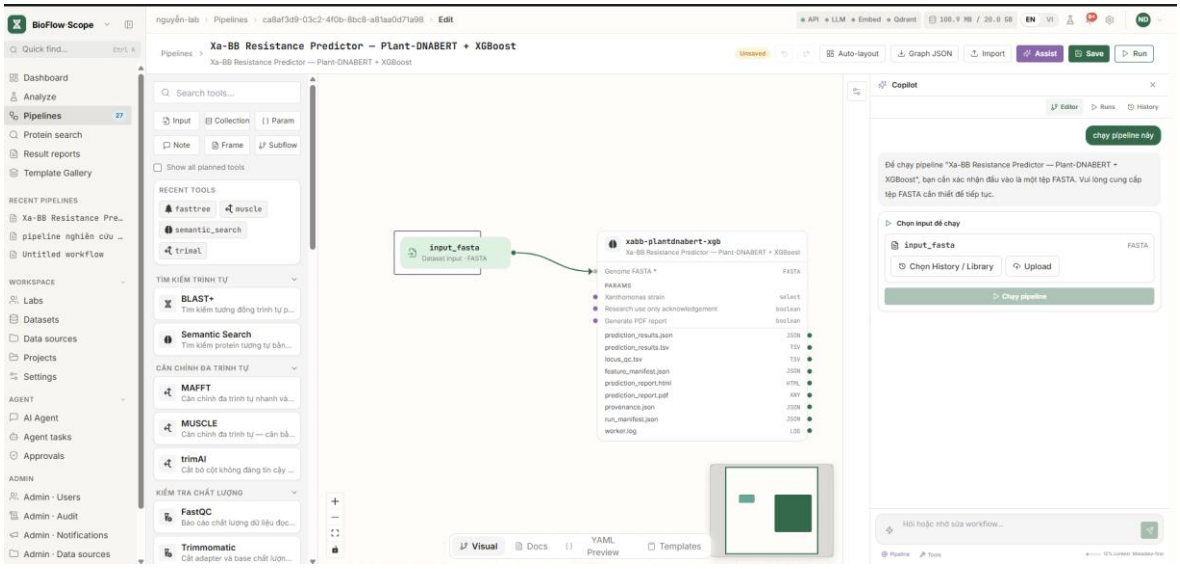


Figure 3.20. *AI Agent Chat, RAG, and Tool Calling interface*

Note: illustrative interface data. Note: interface at `/chat`.

Table 3.*13. Interface components – AI Agent, RAG, and Tool Calling*

No.	Control type	Content	Notes
1	Assistant Panel	Copilot	The AI Agent panel is located on the right side of Smart Canvas and supports direct interaction with the current Pipeline.
2	Tab Selector	Editor / Runs / History	Allows the Agent to work in different contexts: Workflow editing, Run monitoring, or History retrieval.
3	Conversation Area	Conversation with the AI Agent	Displays Copilot responses as a short conversation tied to the context of the currently opened Pipeline.
4	Agent Response Block	Pipeline execution guidance	In the image, the Agent explains that the input must be confirmed as a FASTA file before the Pipeline can continue.
5	Suggested Action Card	Select an input to run	An Agent-suggested action block grouping controls used to prepare input data.
6	Input Item	input_fasta	Displays the exact input slot that the Agent asks the user to configure.
7	Data Type Badge	FASTA	Indicates the data type required by the current input.

No.	Control type	Content	Notes
8	Button	Select History / Library	Allows input data to be selected from an existing History or Dataset Library.
9	Button	Upload	Allows a new local file to be uploaded directly and assigned to the input.
10	Action Button	Run pipeline	The primary action button proposed by the Agent once the input is ready.
11	Composer / Prompt Input	Ask or request workflow changes...	Natural-language request field for continuing interaction with the Agent.
12	Send Button	Send request	Sends the user's prompt to the AI Agent for processing in the context of the current Workflow.

3.3.3.4. Processing procedure

After receiving a request, the backend stores the message, determines the effective AI policy, and checks consent/sensitive data before calling the model.

The backend combines memory, RAG, attachment metadata, and tool schemas into the context. The model may answer directly or generate a tool call; arguments are validated and status is streamed to the interface.

Read-only tools can run after permission checks; tools with side effects create an approval or graph proposal. The Evidence Pipeline links the final answer to recorded evidence/tool results.

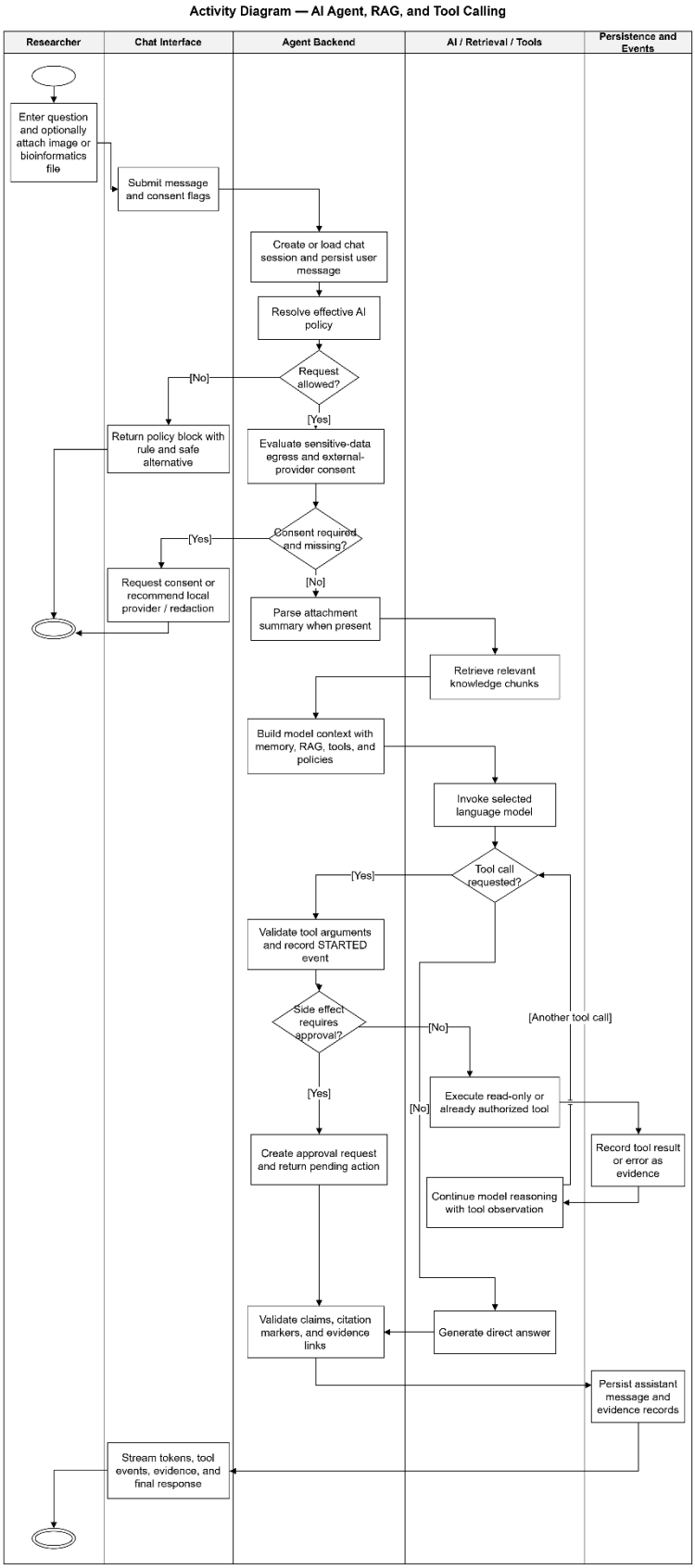


Figure 3.21. *Activity Diagram – AI Agent, RAG, and Tool Calling*

Source: compiled by the author from the system source code.

3.3.3.5. Tool scope and limitations

The Agent supports protein search, database lookup, PubMed/KEGG, pipeline editing/recommendation, run diagnosis, dataset import, statistics, comparison, and Methods writing; correctness still depends on the data, permissions, and related services.

The Agent does not replace scientific decisions such as reference selection, ploidy determination, assembly assessment, or gene-function assignment; its primary role is orchestration and preparation of verifiable information.

3.3.4. Human-in-the-Loop, Evidence, and AI Policy

3.3.4.1. Purpose

HITL controls actions with side effects, Evidence links claims to sources/results, and AI Policy defines permitted scope, consent, and data requirements.

Policy determines whether an action is allowed; Approval determines whether an authorized person must approve it; Evidence identifies the data on which a conclusion is based.

3.3.4.2. Input and output data

Table 3.*14. Input/output data – HITL, Evidence, and AI Policy*

Category	Content
Input data	User request, action type, risk level, safe payload, resource references, effective policy, consent, and tool result.
Approval data	Title, summary, impact, expiration, action fingerprint, approver decision, reason, and consumed timestamp.
Evidence data	Evidence kind, source URI, snippet, content hash, claim text, claim status, citation identifier, and retraction status.
Output data	Policy-blocked response, PENDING/APPROVED/REJECTED/CONSUMED approval, guarded-action result, supported or unverified claim.
Audit data	Approval ledger, AI-policy violations, evidence links, citation-verification records, and generated-resource references.

3.3.4.3. Functional interface

The interface group includes an Approval Inbox with requests, status/risk badges, redacted payloads, Approve/Reject/Revoke/Execute actions, and an approval

ledger; the Project AI Policy page allows AI policies to be configured by project and displays the effective policy currently in force.

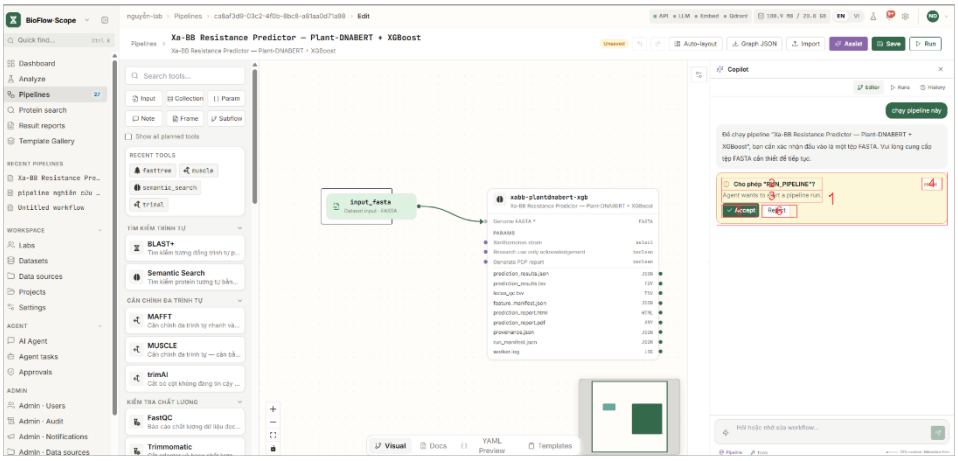


Figure 3.22. *HITL, Evidence, and AI Policy interface*

Note: illustrative interface data.

Table 3.*15. Interface components – Human-in-the-Loop*

No.	Control type	Content	Notes
1	Approval Card	Action approval request	A prominent block containing information about the action the Agent intends to perform and awaiting the user's decision.
2	Approval Title	Allow "RUN_PIPELINE"?	Clearly identifies the action that requires Human-in-the-Loop review.
3	Evidence / Reason Text	Agent wants to start a pipeline run.	A brief description of the Agent's reason or intended action, helping the user understand what is being submitted for approval.
4	Risk Badge	HIGH	Indicates the risk level of the action and helps the user assess it before approval.
5	Decision Button	Accept	Approves the request so the Agent may proceed with the RUN_PIPELINE action.
6	Decision Button	Reject	Rejects the request; the Agent is not permitted to continue the proposed action.

3.3.4.4. Processing procedure

For a risky action, the backend applies policy, classifies the risk, and creates an approval request containing a safe payload, resource references, and an action fingerprint when approval is required.

Before execution, the system rechecks permissions, expiry, status, and fingerprint; an approval can be consumed only once, and every decision is recorded in the ledger.

The Evidence Pipeline links claims to tool results, artifacts, or documentary sources; DOI/PMID identifiers can be verified and retracted publications are flagged according to policy. Missing evidence must be represented explicitly rather than replaced with fabricated sources.

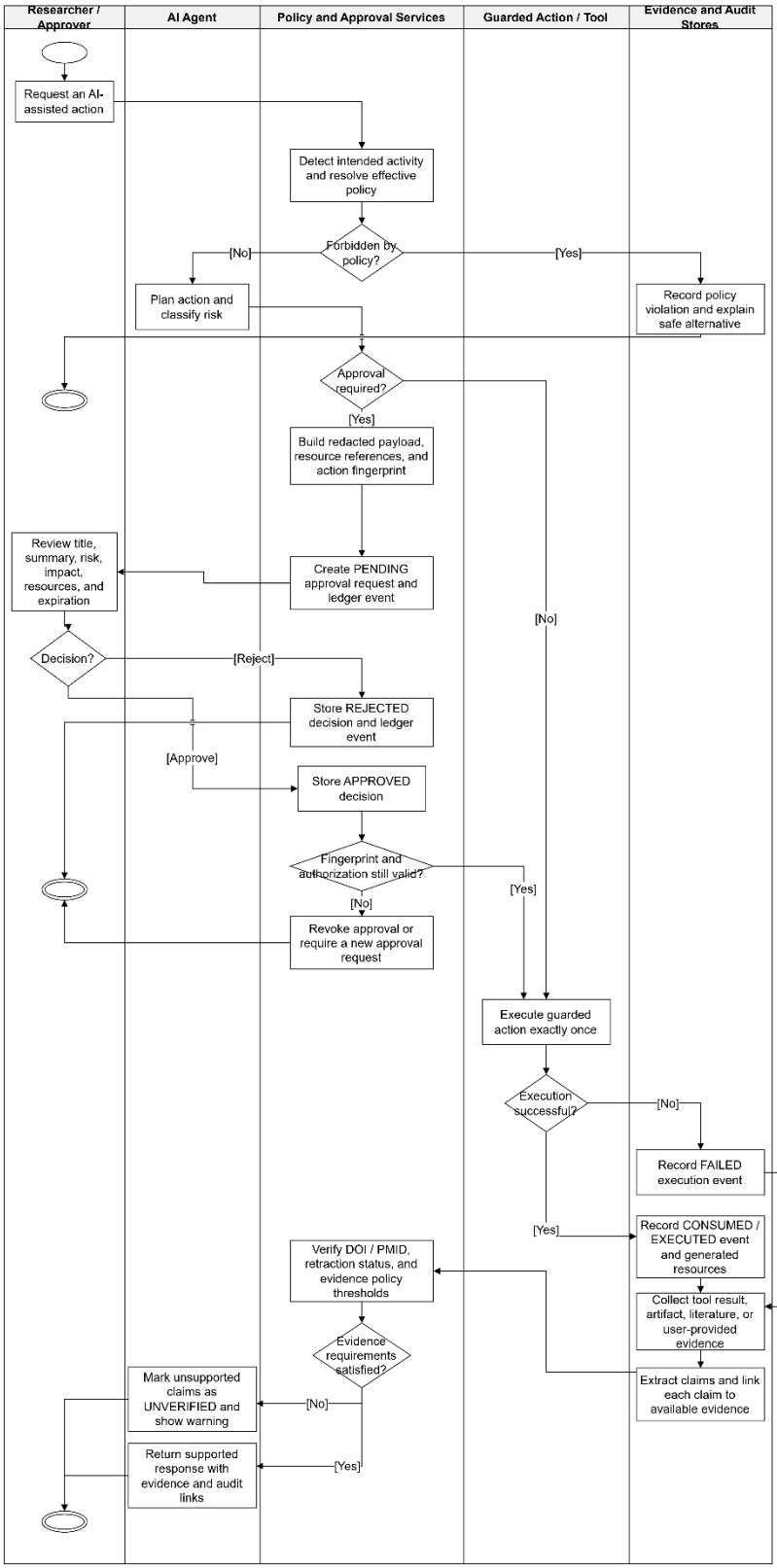


Figure 3.23. *Activity Diagram – Human-in-the-Loop, Evidence, and AI Policy*

3.3.4.5. Control limitations

HITL, Evidence, and Policy reduce risk and improve traceability only; their effectiveness still depends on the information presented, user expertise, and whether every execution path passes through the policy gate.

3.3.5. Other Supporting AI Functions

Other supporting AI functions such as pipeline recommendation, graph proposal, run diagnosis, statistics, PubMed/KEGG, and Methods writing share the same Agent, policy, and evidence foundation, so their architecture is not repeated.

Recommendation functions create drafts only; actions with side effects still pass through approval when required by policy. Attachments and project memory provide only the necessary context to the model and must comply with consent requirements for external providers.

3.4. Design of Business Functions

3.4.1. Guest Functions

3.4.1.1. Accessing a publicly shared Run

Related Use Case: UC95.

A Guest opens a valid public link to view a Run snapshot, reduced provenance, and Artifacts within the token scope; data outside that scope is not displayed.

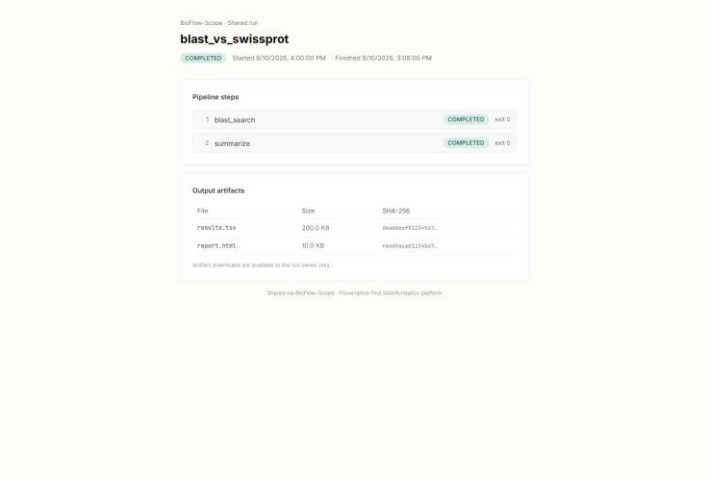


Figure 3.24. *Publicly shared Run interface*

Table 3.*16. Interface components – Accessing a publicly shared Run*

No.	Control type	Content	Notes
1	Header / Status Badge	Shared Run information	Displays the Pipeline name, Run status, and start/end times.
2	Step List	Pipeline step list	Each step displays its ordinal number, step name, COMPLETED status, and exit code.
3	Artifact Table	Output artifacts	Lists output filenames, sizes, and SHA-256 values.
4	Access Notice	Artifact download-permission notice	The image indicates that Artifact downloads are available only to the Run owner.

Table 3.*17. Data – Accessing a publicly shared Run*

No.	Table / data structure name	Create	Update	Delete	Query
1	public_run_tokens		X		X
2	runs				X
3	pipelines				X
4	pipeline_versions				X
5	artifacts				X

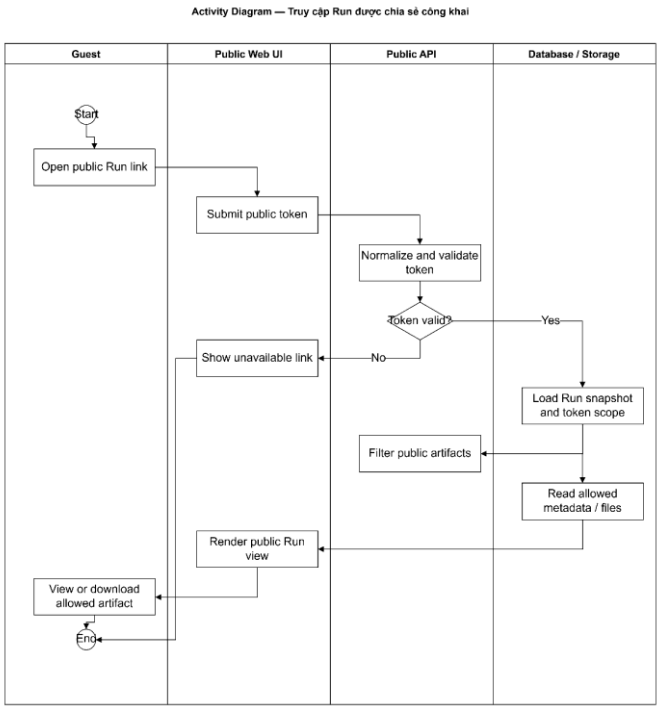


Figure 3.25. *Activity Diagram – Accessing a publicly shared Run*

3.4.2. Researcher Functions

3.4.2.1. Designing and editing a Pipeline on Smart Canvas

Related Use Case: UC05, UC08, UC13, and UC14.

The Researcher builds the graph by dragging and dropping tools, configuring nodes, connecting ports, and linting the Pipeline before saving; the Canvas remains synchronized with the Pipeline definition.

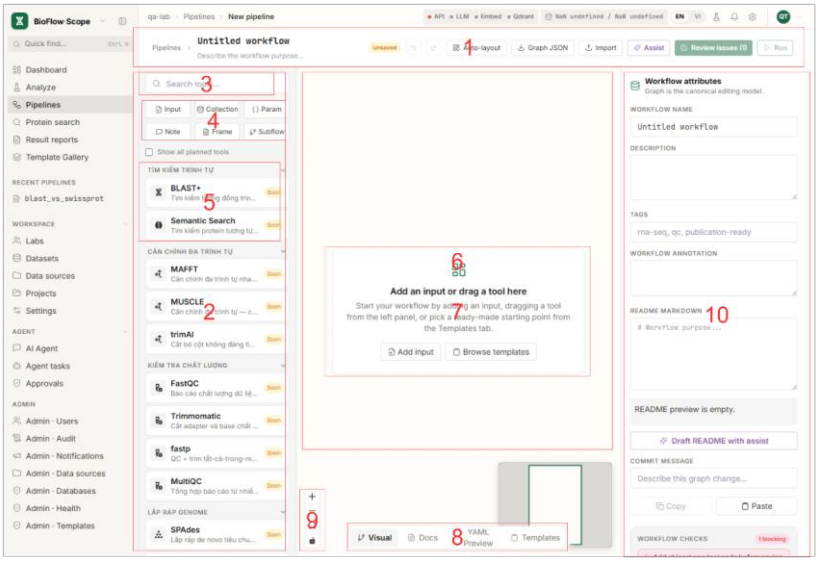


Figure 3.26. *Smart Canvas interface – Pipeline design*

Table 3.*18. Interface components – Designing and editing a Pipeline on Smart Canvas*

No.	Control type	Content	Notes
1	Workflow Header	Workflow name, description, and save status	Displays the current Workflow name, a brief description, and an Unsaved badge.
2	Editor Toolbar	Undo/Redo, Auto-layout, Graph JSON, Import, Assist, Review issues, and Run	A group of direct actions for editing, validating, and running a Pipeline.
3	Search Input	Search tools	Filters the Tool Palette by tool name.
4	Node Controls	Input, Collection, Param, Note, Frame, and Subflow	Quickly creates nodes/structures used to model the Workflow.

No.	Control type	Content	Notes
5	Tool Palette	Tool catalog by category	Displays BLAST+, Semantic Search, MAFFT, MUSCLE, trimAl, FastQC, Trimmomatic, fastp, MultiQC, SPAdes, etc.; planned tools carry a Soon badge.
6	Canvas	Graph-design area	Supports dragging and dropping nodes, connecting data ports, and arranging the Workflow.
7	Canvas Shortcuts	Add input and Browse templates	Quick-start actions when the Canvas is empty.
8	View Tabs	Visual, Docs, YAML Preview, and Templates	Switches among visual representation, documentation, YAML, and templates.
9	Zoom / Minimap	Canvas viewport controls	Zooms in/out, fits the view, and tracks graph position using the minimap.
10	Workflow Attributes	Workflow name, description, tags, and annotation	Configures Pipeline metadata directly.
11	README Panel	README Markdown, preview, and Draft README with assist	Authors the Workflow description and can request AI assistance to draft the README.

Table 3.*19. Data – Designing and editing a Pipeline on Smart Canvas*

No.	Table / data structure name	Create	Update	Delete	Query
1	pipelines	X	X		X
2	pipeline_versions	X			X

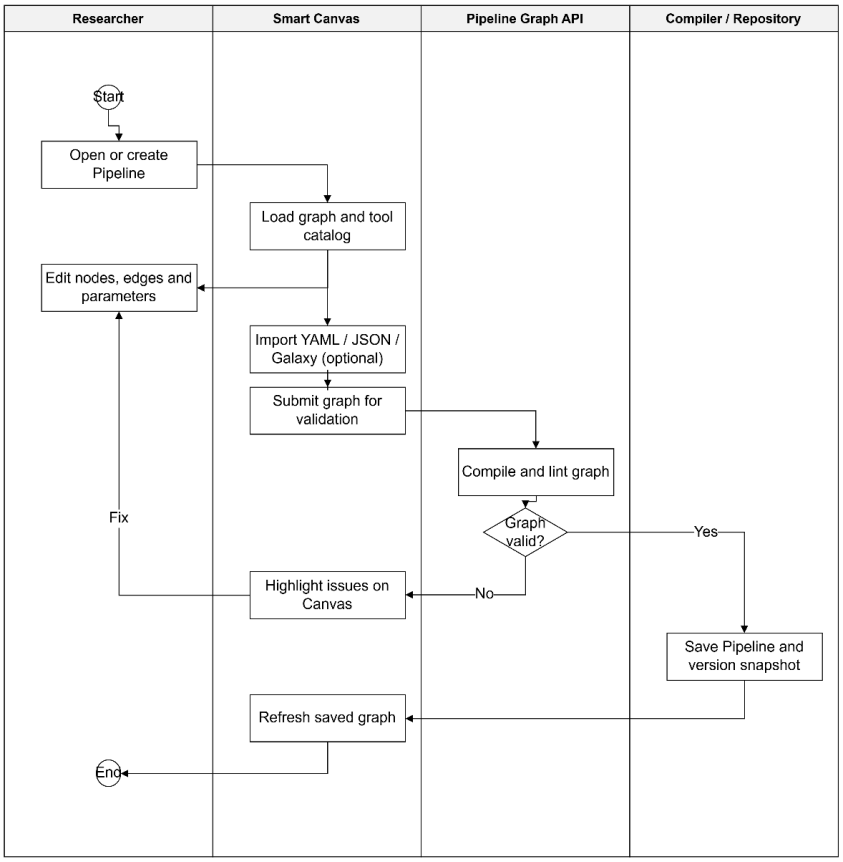


Figure 3.27. *Activity Diagram – Designing and editing a Pipeline on Smart Canvas*

3.4.2.2. Managing Pipeline versions and Fork

Related Use Case: UC09, UC11, and UC12.

Preserves the Pipeline change history, supports version comparison, and creates an independent Pipeline from an accessible version.

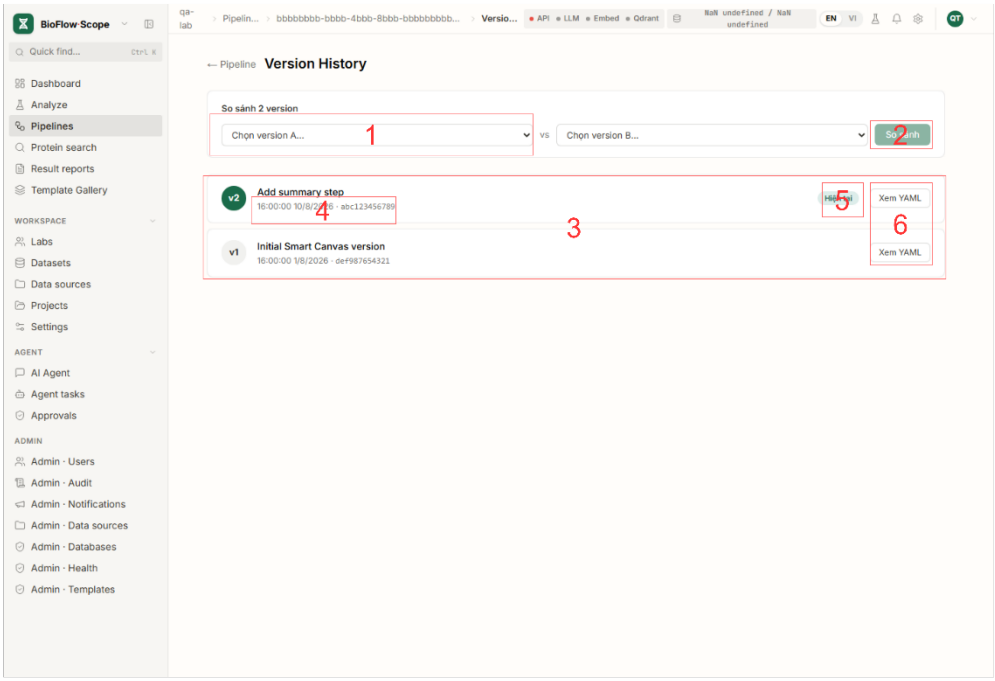


Figure 3.28. *Pipeline version-management and Fork interface*

Note: illustrative interface data.

Table 3.*20. Interface components – Managing Pipeline versions and Fork*

No.	Control type	Content	Notes
1	Version Selector	Select Version A and Version B	The two selectors identify the pair of versions to compare.
2	Button	Compare	Compares the two selected versions.
3	Version List	Version-history list	Each item displays a version number and commit message.
4	Metadata	Save time and hash/version identifier	Allows the stored snapshot to be identified precisely.
5	Badge	Current	Marks the version currently in use.
6	Button	Xem YAML	Opens the YAML definition of each version; the current image does not show the Fork interface.

Table 3.*21. Data – Managing Pipeline versions and Fork*

No.	Table / data structure name	Create	Update	Delete	Query
1	pipelines	X			X

No.	Table / data structure name	Create	Update	Delete	Query
2	pipeline_versions	X			X

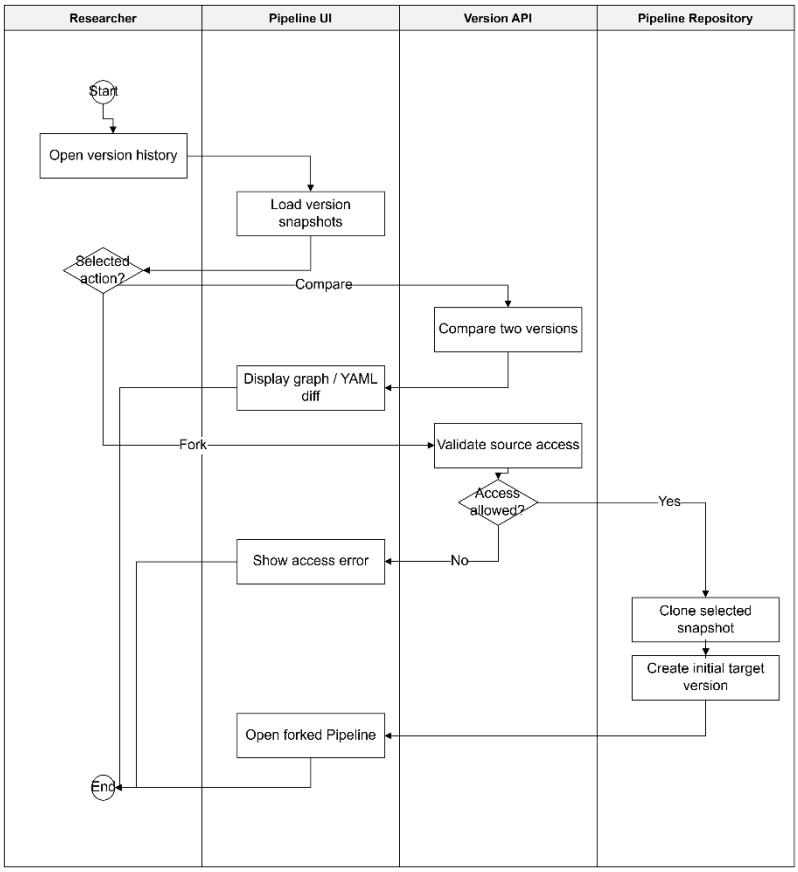


Figure 3.29. *Activity Diagram – Managing Pipeline versions and Fork*

3.4.2.3. Configuring data and starting a Run

Related Use Case: UC17 and UC18.

The Run form links a Pipeline version to a Dataset/Collection and runtime parameters, then creates a snapshot for execution and traceability.

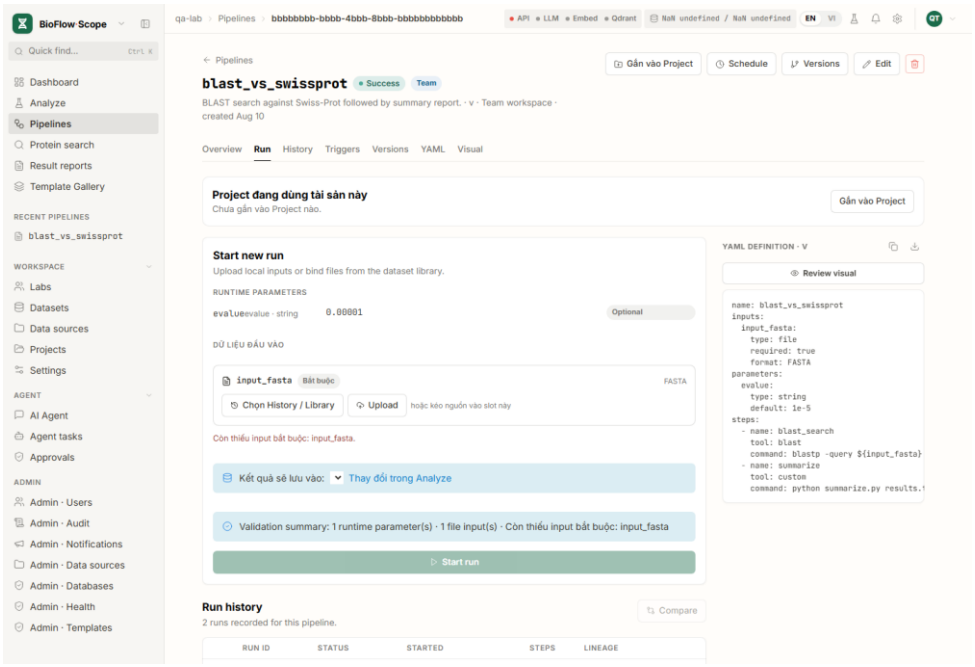


Figure 3.30. *Data-configuration and Run-start interface*

Note: illustrative interface data.

Table 3.*22. Interface components – Configuring data and starting a Run*

No.	Control type	Content	Notes
1	Project Panel	Assign Pipeline/Run to Project	Displays the Project using the asset and an Assign to Project button.
2	Parameter Form	Runtime parameters	Displays runtime parameters, data types, values, and Optional/Required status.
3	Input Slot	input_fasta and input metadata	Displays the input name, a Required badge, and FASTA format.
4	Button	Select History / Library	Links an existing Dataset/History Item to the input slot.
5	Upload Control	Upload input file	Uploads a local file when the data is not already in the library.
6	Validation Message	Missing required-input warning	Displays configuration errors directly before a Run is created.
7	History Destination	Results will be saved to History	Selects/changes the result-synchronization destination through the Change link in Analyze.
8	Validation Summary	Parameter and input summary	Shows the number of runtime parameters, input files, and remaining errors.
9	Button	Start run	Available only when the configuration is valid; creates a new Run.

No.	Control type	Content	Notes
10	YAML Definition Panel	YAML definition and visual review	Allows the Run configuration to be compared with the Pipeline definition and provides copy/download actions.

Table 3.*23. Data – Configuring data and starting a Run*

No.	Table / data structure name	Create	Update	Delete	Query
1	pipelines				X
2	pipeline_versions				X
3	datasets				X
4	dataset_collections				X
5	run_dataset_inputs	X			X
6	runs	X			X

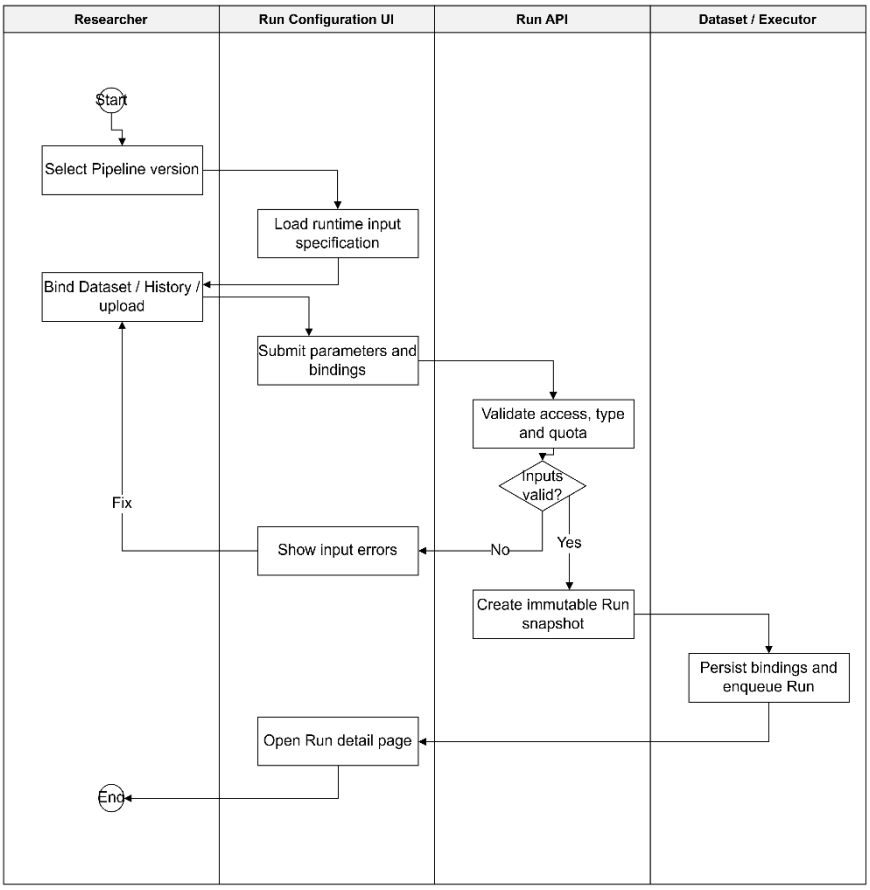


Figure 3.31. *Activity Diagram – Configuring data and starting a Run*

3.4.2.4. Monitoring and controlling a Run

Related Use Case: UC20, UC21, UC22, and UC23.

The Run Detail page displays per-step status, SSE logs, and Artifacts, and allows an active Run to be canceled.

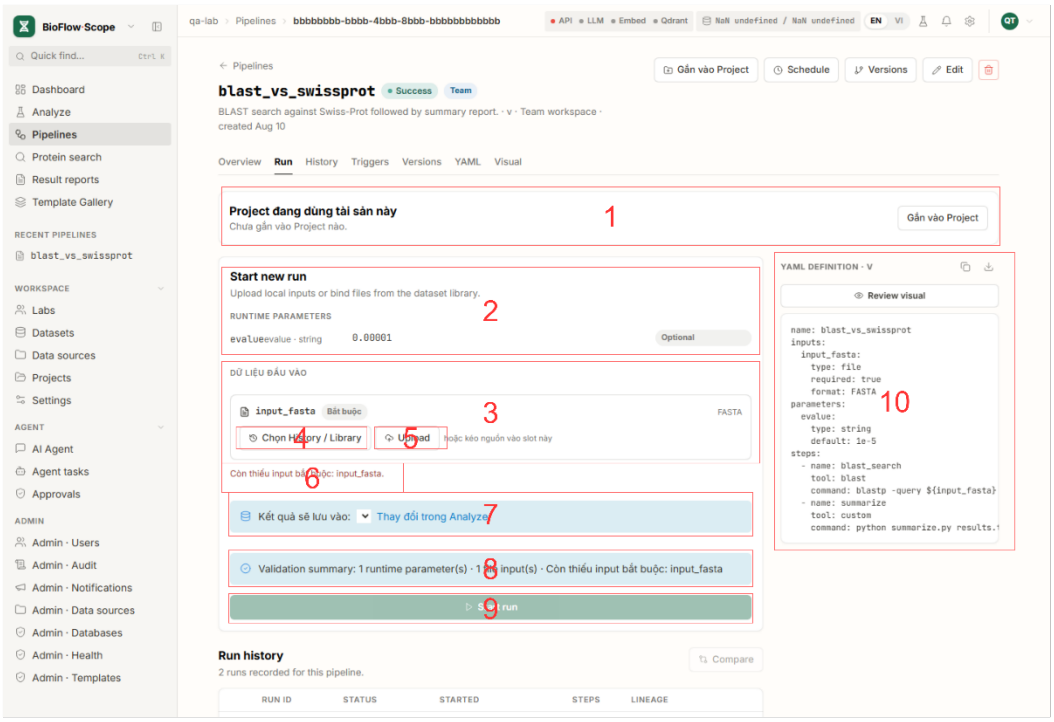


Figure 3.32. *Run monitoring and control interface*

Table 3.*24. Interface components – Monitoring and controlling a Run*

No.	Control type	Content	Notes
1	Run Header	Run ID, status, and time	Displays a Success badge, start/elapsed time, and number of completed steps.
2	Action Toolbar	Assign to Project, Extract workflow, Rerun, and Export run	Direct actions for the Run currently being viewed.
3	Project Panel	Project using this asset	Indicates whether the Run has been associated with a Project.
4	History Status	Target History and output-synchronization status	Displays where results are sent in Analyze History and their synchronization status.
5	Step Progress	Step-based progress bar	Displays individual steps such as blast_search and summarize with Done status.
6	Tab Navigation	Results, Summary, Logs, Steps, Artifacts, Invocation, Provenance, Reproducibility	Switches among Run monitoring and traceability views.
7	Summary Cards	Duration, Steps, Artifacts, and YAML hash	Provides a quick summary of execution state and configuration trace information.

No.	Control type	Content	Notes
8	Snapshot Table	Input binding snapshot	Displays inputs, source, role, mounted-as value, datatype, and SHA-256.
9	Snapshot Table	Runtime parameter snapshot	Displays the actual parameters used in the Run.
10	Snapshot Panel	Collection execution snapshot	Stores/displays a snapshot related to collection execution configuration in the following section of the page.

Table 3.*25. Data – Monitoring and controlling a Run*

No.	Table / data structure name	Create	Update	Delete	Query
1	runs		X		X
2	run_steps	X	X		X
3	run_logs	X			X
4	artifacts	X			X

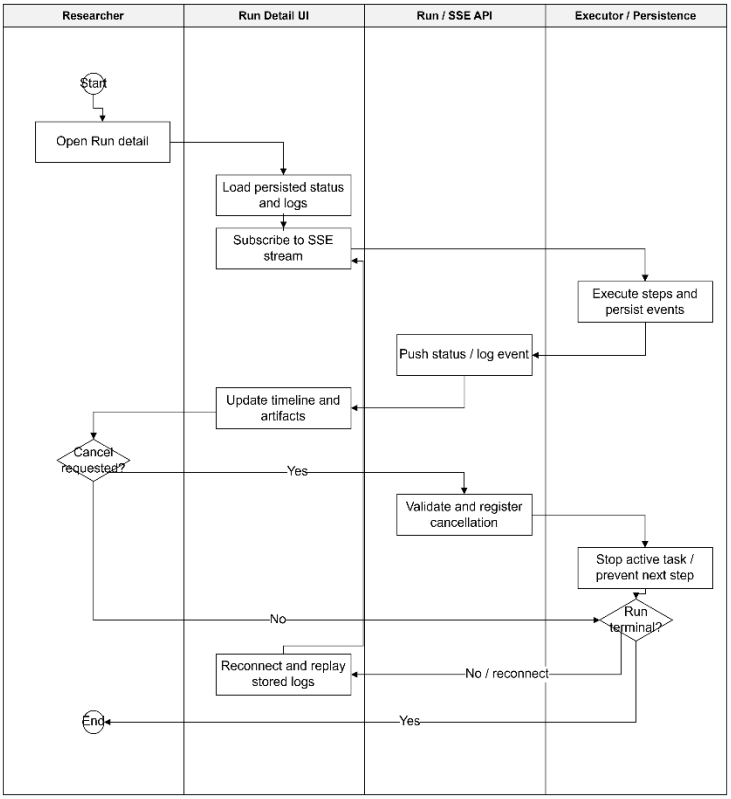


Figure 3.33. *Activity Diagram – Monitoring and controlling a Run*

3.4.2.5. Resuming, rerunning, and comparing Runs

Related Use Case: UC27, UC28, UC29, UC30, and UC31.

The Resume/Rerun/Compare function group reuses valid provenance and artifacts to recover a Run, rerun a previous configuration, or identify differences among executions.

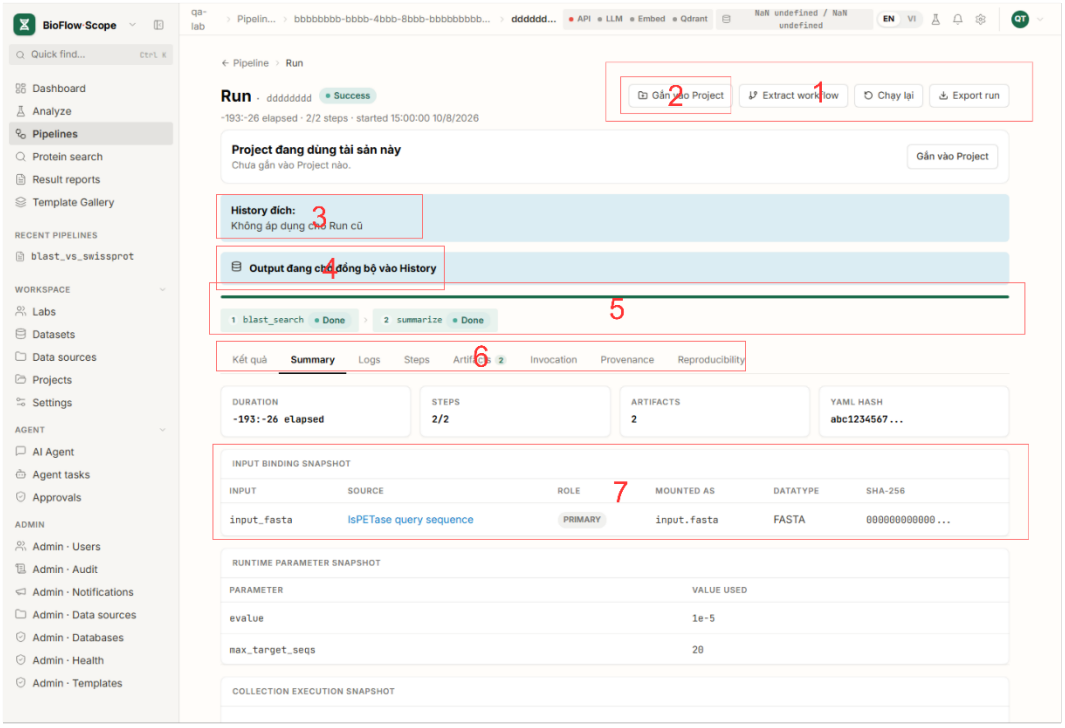


Figure 3.34. *Resume/Rerun and Run-comparison interface*

Note: illustrative interface data.

Table 3.*26. Interface components – Resuming, rerunning, and comparing Runs*

No.	Control type	Content	Notes
1	Action Toolbar	Extract workflow, Rerun, and Export run	The captured Run has Success status, so these actions are visible; Resume/Compare are not shown in this state.
2	Project Panel	Assign Run to Project	Links Run results to a research Project.
3	History Status	Target History and synchronization status	Indicates whether outputs are pending or synchronized into Analyze History.
4	Step Progress	Status of executed steps	Identifies completed steps before reuse or Workflow extraction.
5	Tab Navigation	Artifacts and related traceability tabs	The image shows the Artifacts tab; Invocation/Provenance/Reproducibility tabs appear on the same navigation bar.
6	Artifact Table	Run Artifact list	Displays file, type, size, and SHA-256.

Table 3.*27. Data – Resuming, rerunning, and comparing Runs*

No.	Table / data structure name	Create	Update	Delete	Query
1	runs	X			X

No.	Table / data structure name	Create	Update	Delete	Query
2	run_steps				X
3	run_logs				X
4	artifacts				X
5	pipeline_versions	X			X

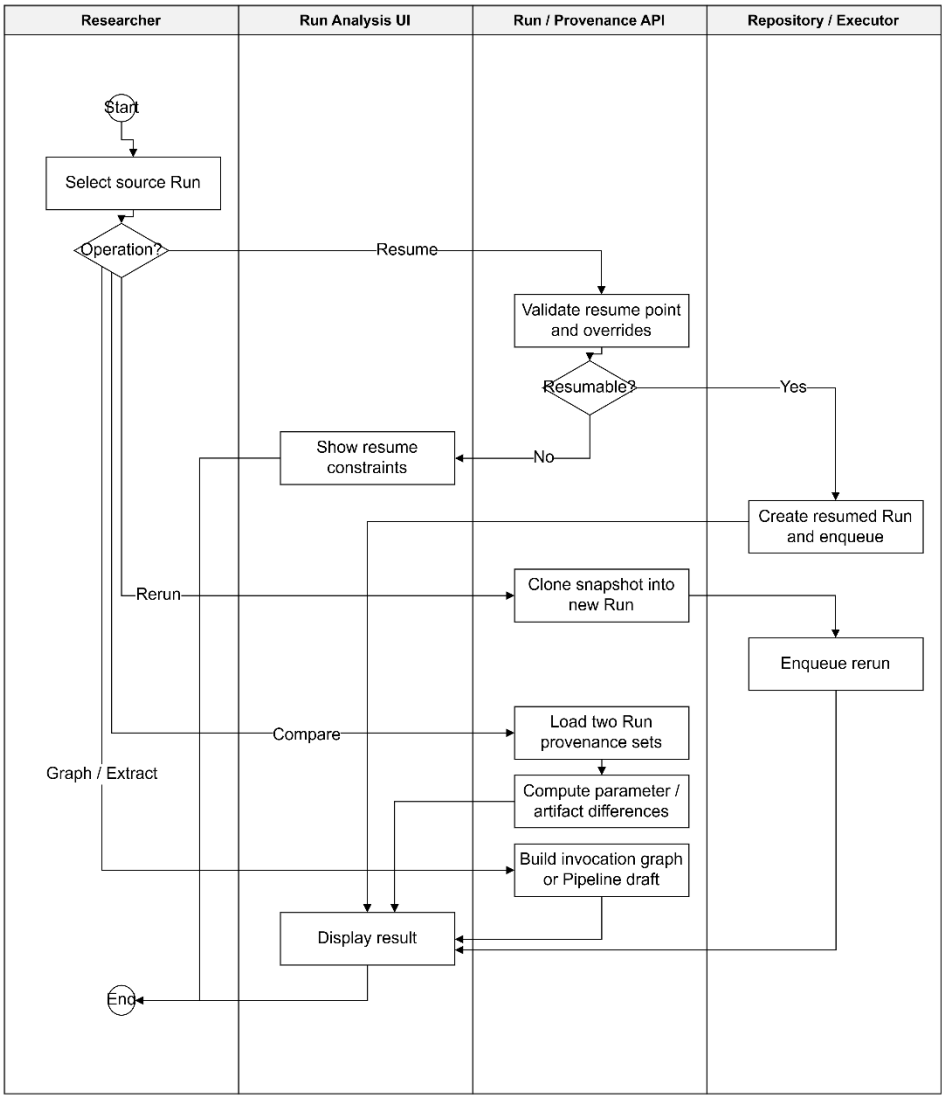


Figure 3.35. *Activity Diagram – Resuming, rerunning, and comparing Runs*

3.4.2.6. Managing Datasets, Collections, and Lineage

Related Use Case: UC40 through UC47.

The Dataset Library organizes biological files into Datasets/Collections with metadata, checksums, quota information, and lineage for reuse in Runs.

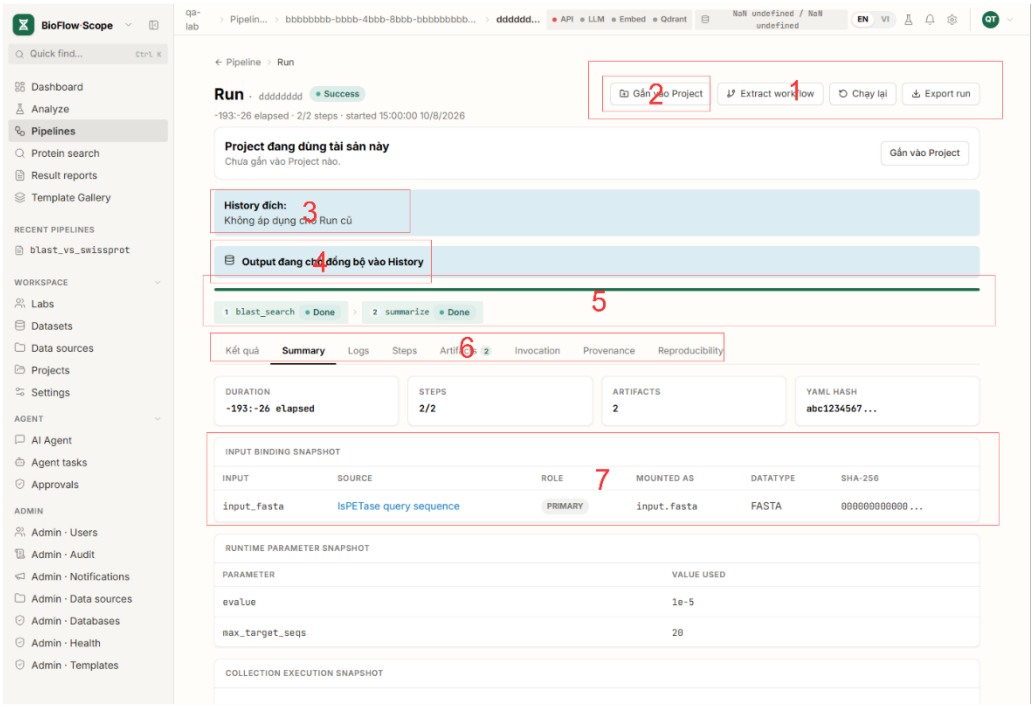


Figure 3.36. *Dataset Library and Dataset Collection interface*

Note: illustrative interface data.

Table 3.*28. Interface components – Managing Datasets, Collections, and Lineage*

No.	Control type	Content	Notes
1	Button	Upload dataset	Creates a new Dataset from uploaded data.
2	Tabs	Mine / Lab Shared Library	Switches between personal Datasets and Datasets shared in the Workspace.
3	Quota Indicator	Personal quota	Displays used/total quota together with Dataset Library and Run-artifact storage usage.
4	Filter Chips	All, demo, fastq, protein, qc	Filters Datasets by tag/display group.
5	Dataset Card	Dataset list	Displays the Dataset name and a brief description.
6	Metadata / Badges	Format, number of files, size, and tags	For example FASTA/FASTQ, file count, storage size, and protein/demo/fastq/qc labels.
7	Button	Assign to Project	Links the Dataset to a research Project.
8	Section	Import jobs	Tracks requests to import Datasets from external sources.
9	Import Table	Source, Accession/Region, Mode, Status, Bytes, Created	Displays import progress and results, for example an SRA import with COMPLETED status.

***Table 3.**29. Data – Managing Datasets, Collections, and Lineage*

No.	Table / data structure name	Create	Update	Delete	Query
1	datasets	X	X	X	X
2	dataset_files	X	X	X	X
3	dataset_collections	X	X	X	X
4	dataset_collection_items	X	X	X	X
5	dataset_import_jobs	X	X		X
6	dataset_tags	X	X	X	X
7	run_dataset_inputs				X
8	analyze_history_items				X

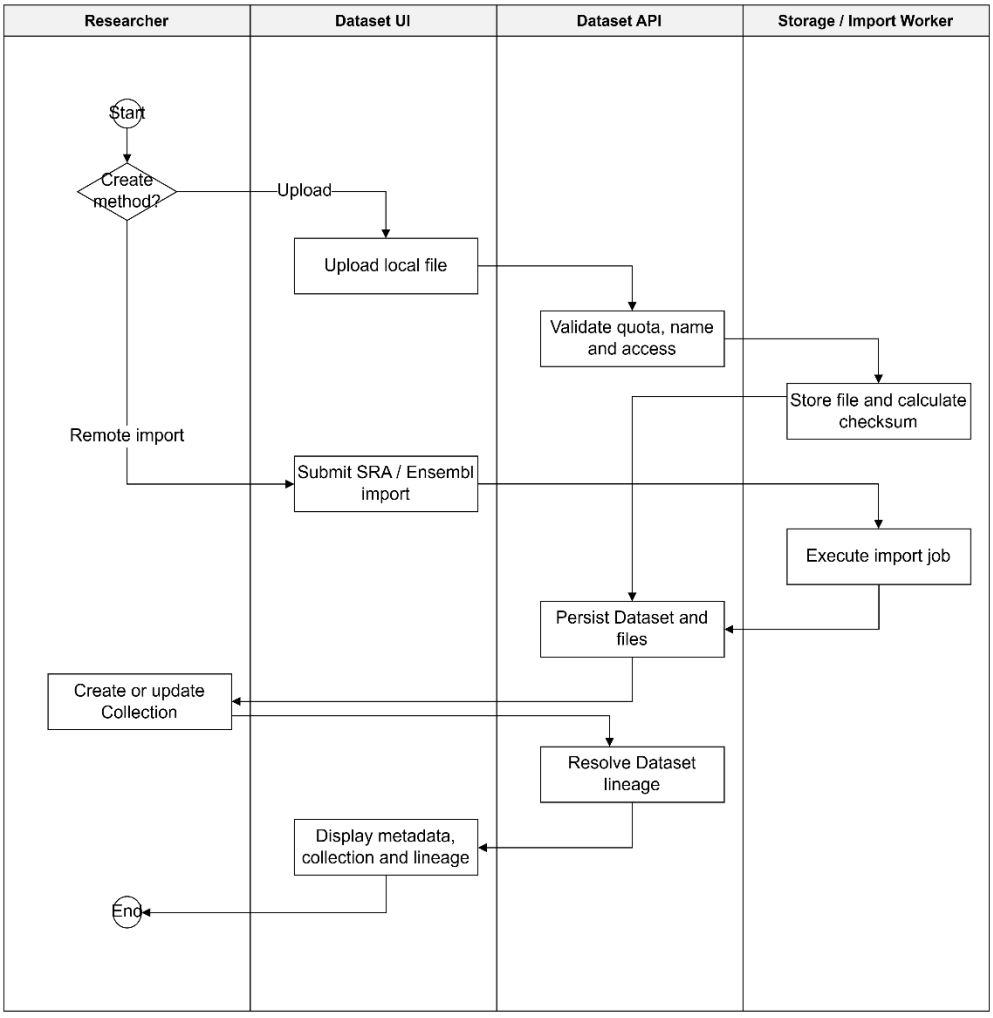


***Figure 3.37.** Activity Diagram – Managing Datasets, Collections, and Lineage*

3.4.2.7. Viewing Artifacts and results with the Inline Viewer

Related Use Case: UC26 and UC48 through UC51.

The system selects a viewer according to Artifact type for preview, filtering/pagination when supported, and raw download for large files or formats without a parser.

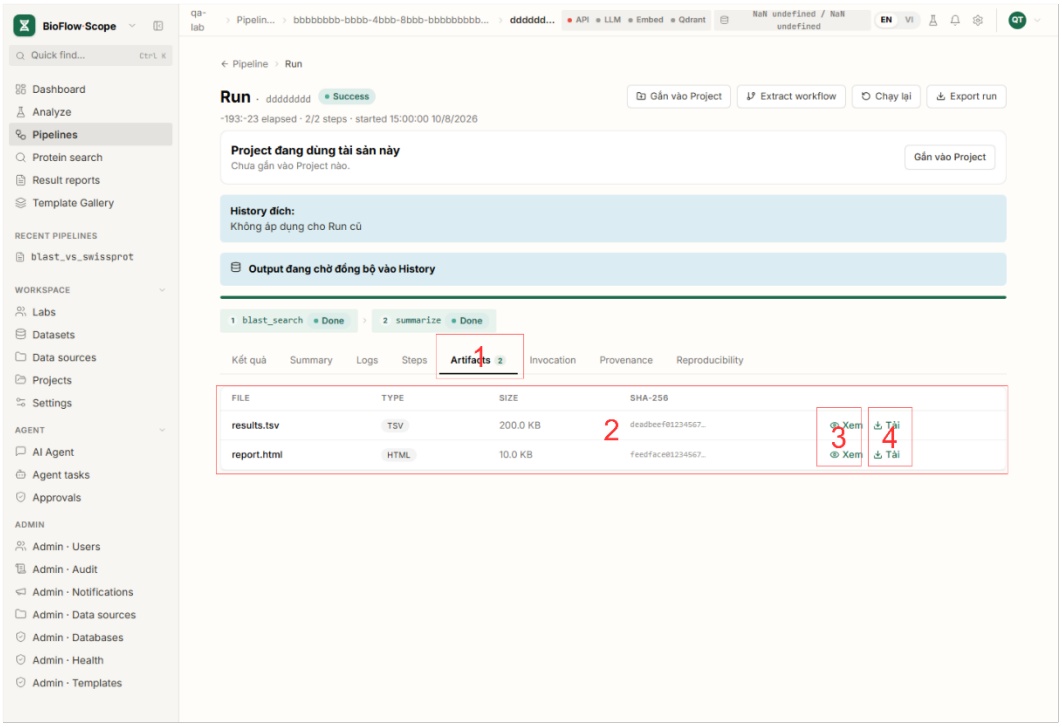


Figure 3.38. *Artifact and result viewing interface using the Inline Viewer*

Note: illustrative interface data.

Table 3.*30. Interface components – Viewing Artifacts and results with the Inline Viewer*

No.	Control type	Content	Notes
1	Tab	Artifacts and Artifact count	The screenshot is on the Run's Artifacts tab.
2	Artifact Table	Result-file list	Displays File, Type, Size, and SHA-256.
3	Action	Xem	Opens the Artifact with an appropriate viewer when its format is supported.
4	Action	Download	Downloads the Artifact when the user has permission.

Table 3.*31. Data – Viewing Artifacts and results with the Inline Viewer*

No.	Table / data structure name	Create	Update	Delete	Query
1	artifacts				X
2	datasets				X
3	dataset_files				X
4	analyze_history_items				X

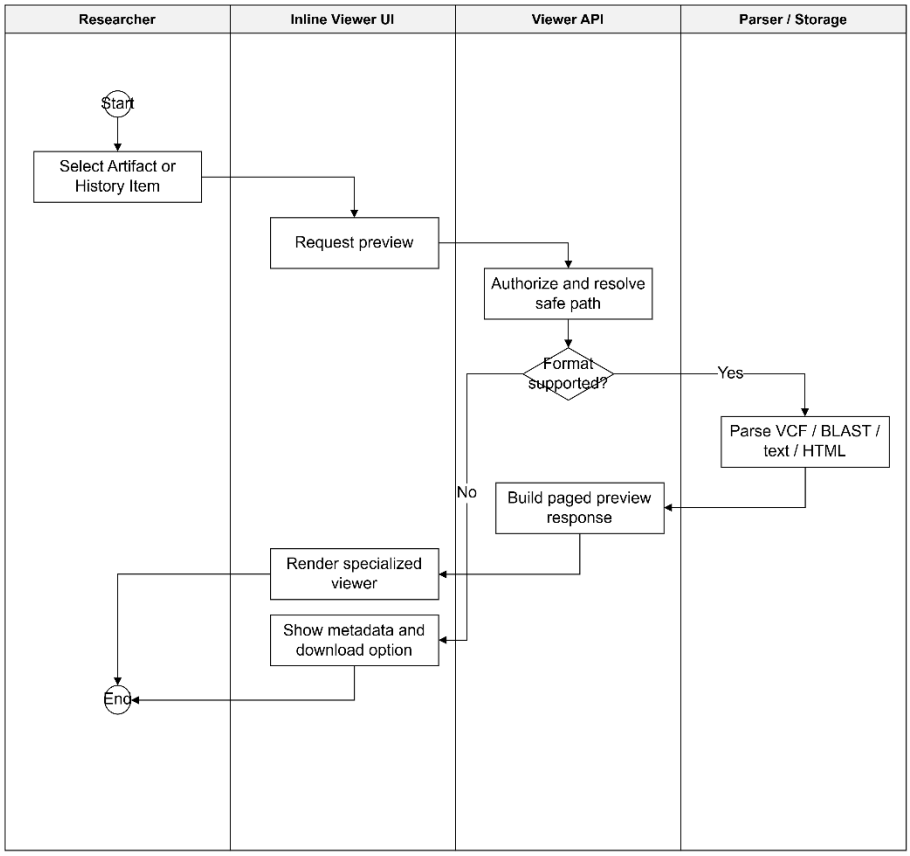


Figure 3.39. *Activity Diagram – Viewing Artifacts and results with the Inline Viewer*

3.4.2.8. Managing Run History and publishing reproducible results

Related Use Case: UC75 through UC80, UC94, and UC96 through UC98.

The History/Reproducibility area brings together results, provenance, and checksums, while supporting public links, BCO export, and reproduction bundles when a Run meets the required conditions.

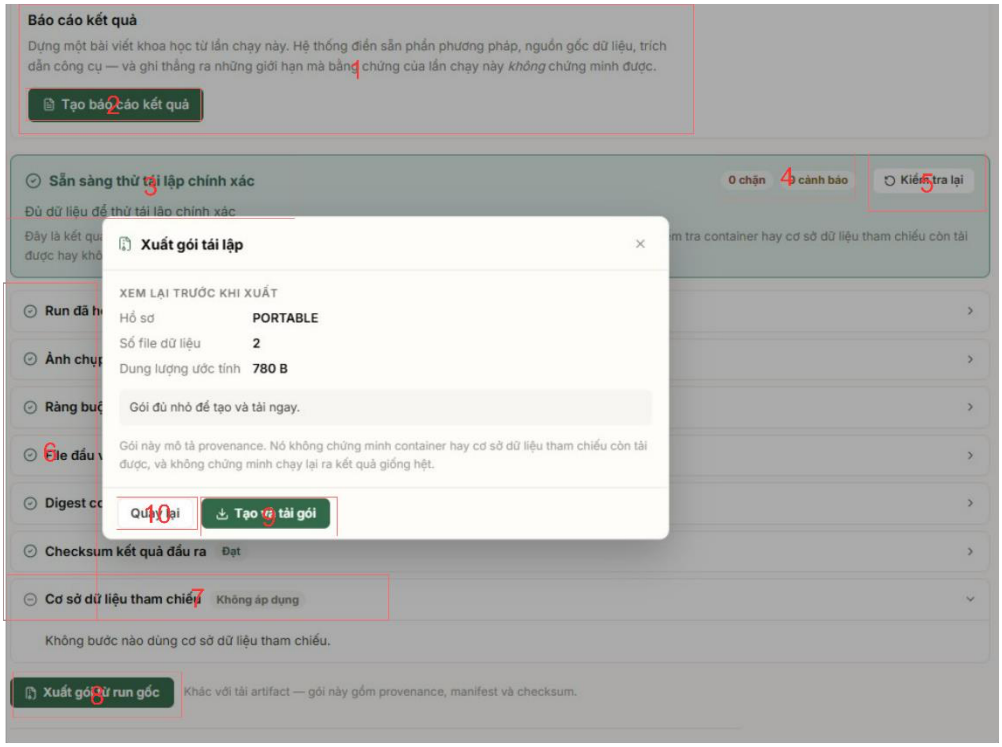


Figure 3.40. *Analyze History and result-publication interface*

Note: illustrative interface data.

Table 3.*32. Interface components – Managing Analyze History and publishing reproducible results*

No.	Control type	Content	Notes
1	Result Report Panel	Result report	An area introducing the function for generating a scientific report from the current Run, including methods, data provenance, tool citations, and information demonstrably supported by the execution.
2	Button	Generate result report	Starts construction of a result report based on the current Run's data and provenance.
3	Reproducibility Readiness Panel	Reproducibility-readiness status	Displays the overall Run assessment, for example “Ready to attempt exact reproduction,” together with counts of blocking conditions and warnings.
4	Status Badges	Blocking and warning counts	Quickly displays issues that may prevent reproduction or require attention before export.
5	Button	Recheck	Requests the system to reevaluate reproducibility conditions based on the current Run state and data.

No.	Control type	Content	Notes
6	Readiness Checklist	Reproducibility checklist	Checks required components such as a completed Run, Pipeline snapshot, execution constraints, input files, container digest, output checksums, and reference databases.
7	Expandable Check Item	Details for each condition	Allows each item to be expanded to view status and explanatory information; for example, a reference database may be marked “Not applicable.”
8	Button	Export bundle from original Run	Starts packaging provenance, manifests, checksums, and other required data from the current Run into a reproduction bundle.
9	Button	Create and download bundle	Confirms creation of the reproduction bundle and downloads it after packaging completes.
10	Button	Back	Cancels the export-confirmation step and returns to the reproducibility-check screen without creating a bundle.

Table 3.*33. Data – Managing Analyze History and publishing reproducible results*

No.	Table / data structure name	Create	Update	Delete	Query
1	analyze_histories	X	X	X	X
2	analyze_history_items	X	X	X	X
3	public_run_tokens	X	X	X	X
4	reproduction_export_jobs	X	X		X
5	reproduction_reports	X			X
6	reproducibility_audit_events	X			X

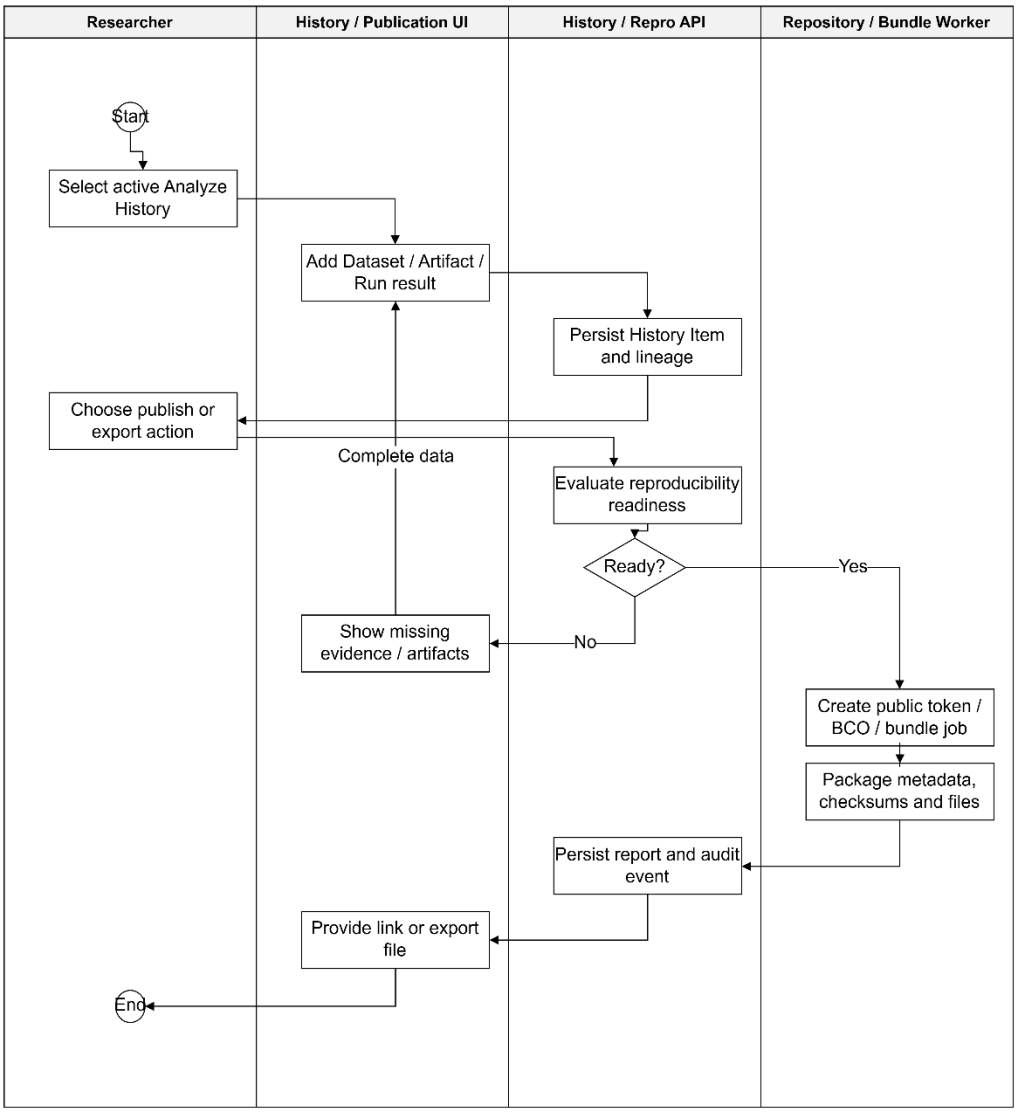


Figure 3.41. *Activity Diagram – Analyze History and result publication*

3.4.3. Extended Functions of the Workspace Owner

Workspace Owner is an extended Researcher role within a specific Workspace. This role is determined by Workspace ownership and is not a separate account type at the system level.

3.4.3.1. Inviting and onboarding Workspace members

Related Use Case: UC85 and UC86.

The Workspace Owner sends invitations with a role and expiration time; a recipient becomes a member only after accepting a valid invitation.

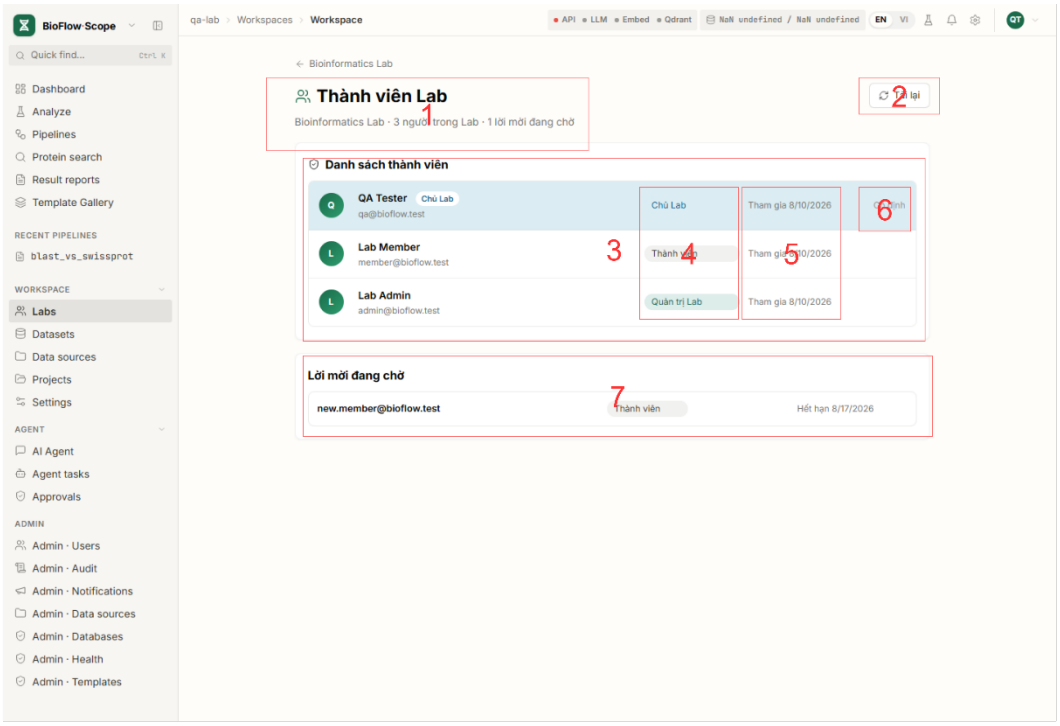


Figure 3.42. *Workspace member invitation and onboarding interface*

Note: illustrative interface data.

Table 3.*34. Interface components – Inviting and onboarding Workspace members*

No.	Control type	Content	Notes
1	Summary	Number of members and pending invitations	Displays an overview of the current Workspace.
2	Button	Reload	Refreshes the member and invitation lists.
3	Member List	Member list	Displays each member's avatar, full name, and email address.
4	Role Badge	Lab Owner / Lab Administrator / Member	Distinguishes the role of each account in the Workspace.
5	Metadata	Join date	Displays when the member joined the Lab.
6	Indicator	Fixed	Indicates that the Owner/Lab Owner cannot be removed like a regular member.
7	Pending Invitation List	Pending invitations	Displays email, invited role, and expiration date; the invitation form is not open in this image.

Table 3.*35. Data – Inviting and onboarding Workspace members*

No.	Table / data structure name	Create	Update	Delete	Query
1	workspace_invitations	X	X		X

No.	Table / data structure name	Create	Update	Delete	Query
2	workspace_members	X			X
3	workspaces				X
4	users				X
5	notifications	X			X

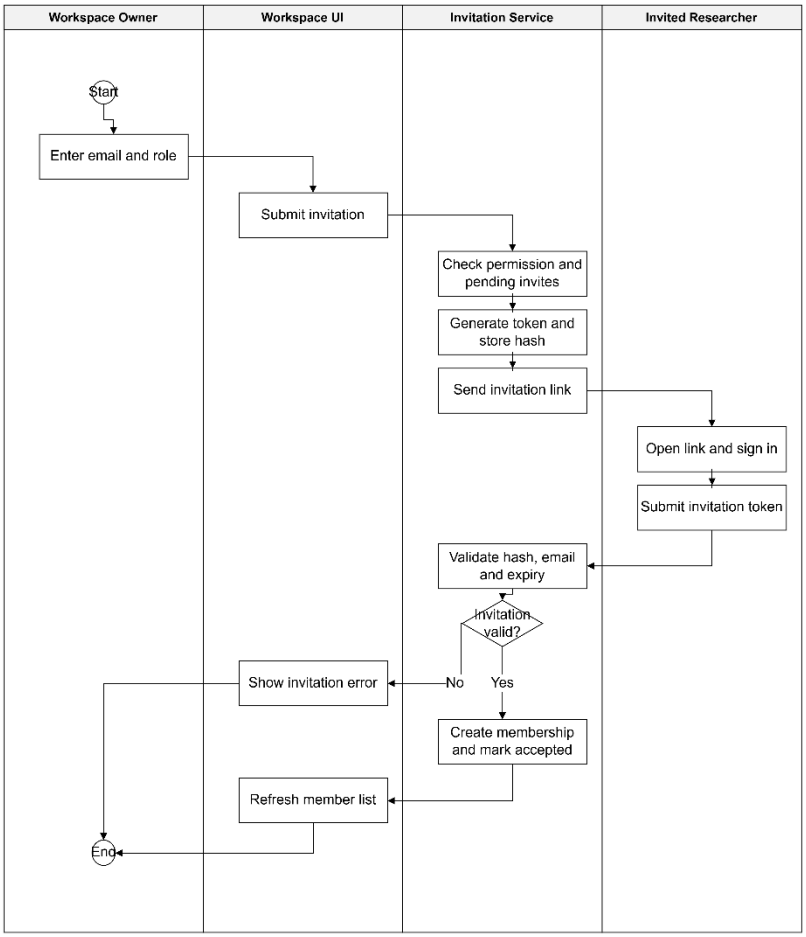


Figure 3.43. *Activity Diagram – Inviting and onboarding Workspace members*

3.4.3.2. Managing roles, members, and permission policies

Related Use Case: UC87.

The Workspace Owner manages membership, roles, and permission policies within the Workspace, with guards that protect ownership rights.

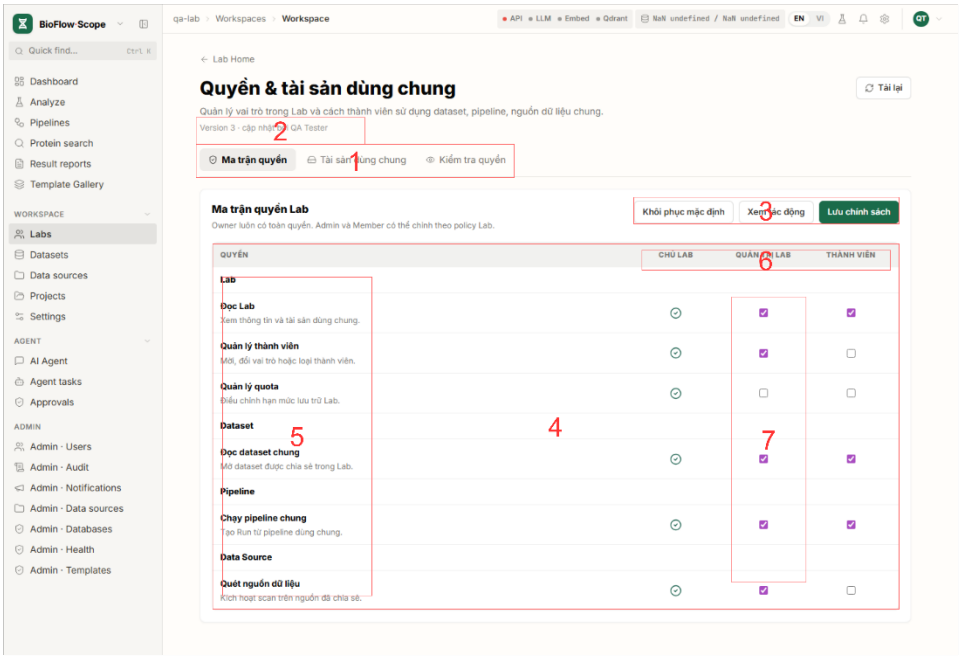


Figure 3.44. *Role, member, and permission-policy management interface*

Note: illustrative interface data.

Table 3.*36. Interface components – Managing roles, members, and permission policies*

No.	Control type	Content	Notes
1	Tabs	Permission matrix / Shared assets / Permission check	Switches among policy configuration, assets, and effective-permission checks.
2	Policy Metadata	Version and updated by	Displays the policy version and the account that most recently applied it.
3	Action Buttons	Restore defaults, Preview impact, Save policy	Restores defaults, previews impact, and saves permission changes.
4	Permission Matrix	Lab permission matrix	A table of permissions grouped by resource and action.
5	Resource Sections	Lab, Dataset, Pipeline, Data Source	Groups permissions by resource type.
6	Role Columns	Lab Owner, Lab Administrator, Member	Compares permissions across roles.
7	Checkbox / Indicator	Enable/disable permission	The Owner's permissions are shown as fixed; Administrator/Member permissions have policy-controlled checkboxes.

Table 3.*37. Data – Managing roles, members, and permission policies*

No.	Table / data structure name	Create	Update	Delete	Query
1	workspace_members		X	X	X
2	workspace_role_policies	X	X	X	X

No.	Table / data structure name	Create	Update	Delete	Query
3	workspaces				X
4	users				X
5	admin_audit_log	X			X

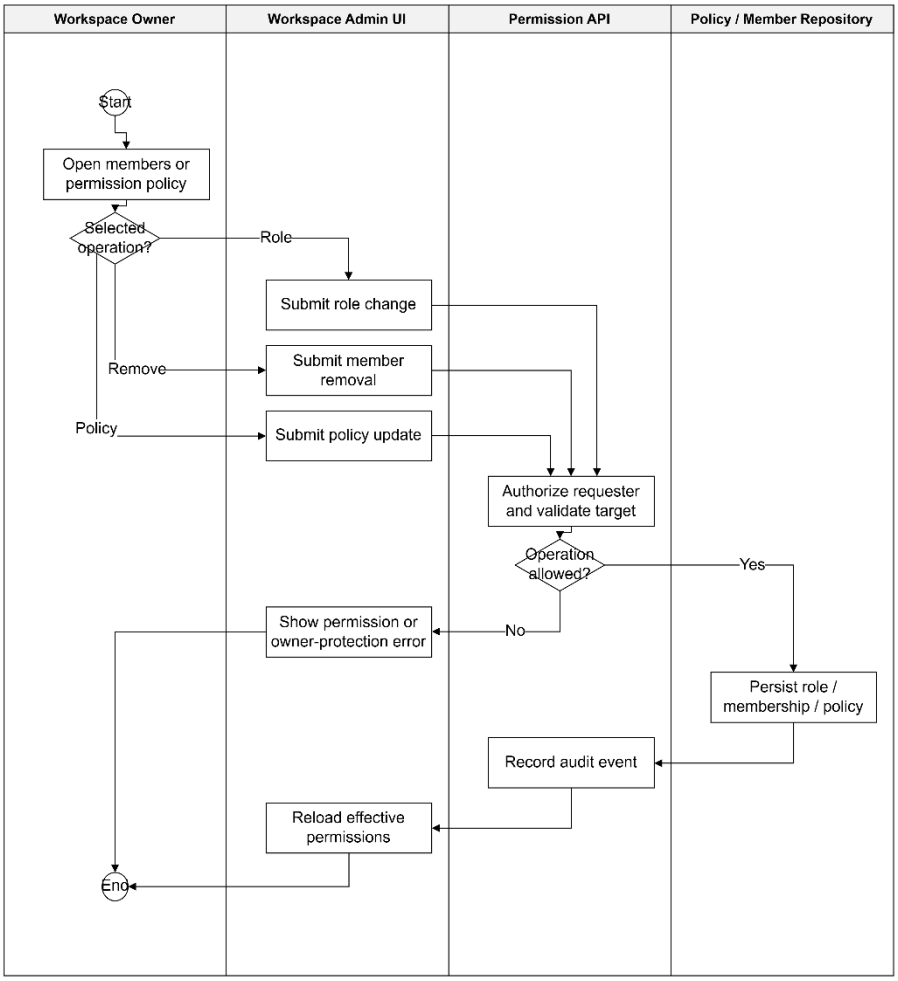


Figure 3.45. *Activity Diagram – Managing roles, members, and permission policies*

3.4.3.3. Managing the Workspace lifecycle

Related Use Case: UC84 and UC88.

Workspace Settings summarizes quota, members, and assets; deletion is permitted only when business constraints are satisfied.

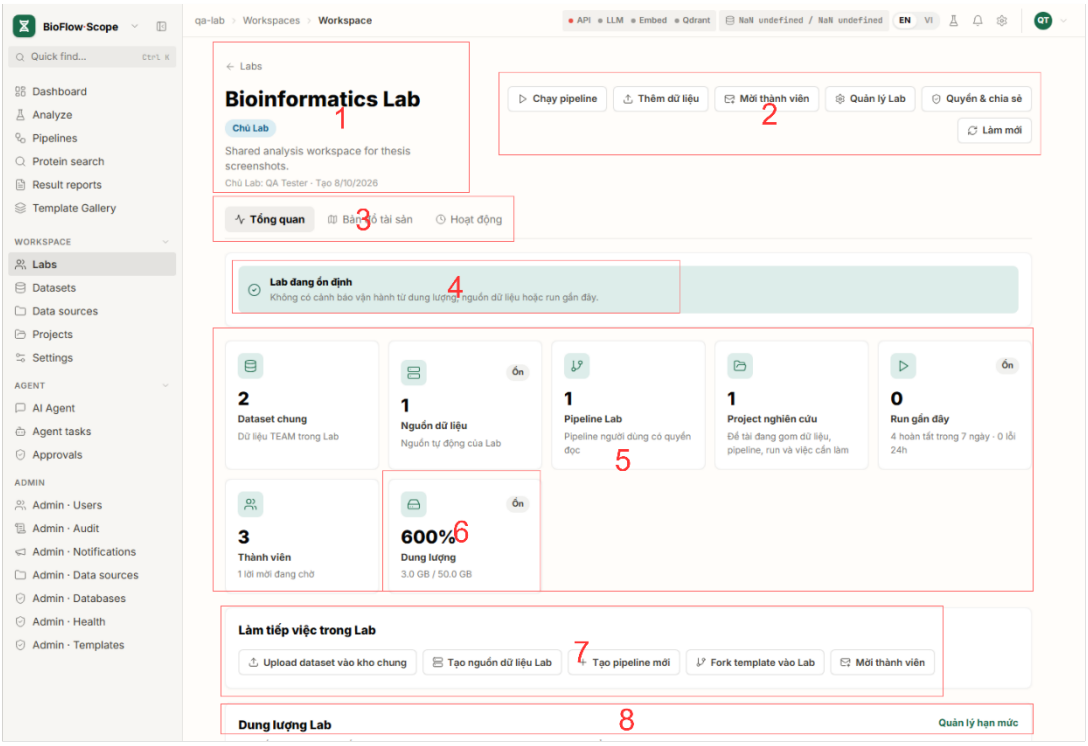


Figure 3.46. *Workspace lifecycle-management interface*

Note: illustrative interface data.

Table 3.*38. Interface components – Managing the Workspace lifecycle*

No.	Control type	Content	Notes
1	Workspace Header	Lab name, role, description, and Owner	Displays identifying information for the Workspace.
2	Action Toolbar	Run pipeline, Add data, Invite member, Manage Lab, Permissions & sharing, Refresh	Administrative/operational actions directly available from Lab Home.
3	Tabs	Overview, Asset map, Activity	Switches among status, assets, and activity logs.
4	Health Banner	Lab operational status	Indicates whether the Lab is healthy or has warnings.
5	Statistic Cards	Shared Datasets, Data Sources, Lab Pipelines, Research Projects, Recent Runs, Members	Summarizes the number of assets and activities in the Workspace.
6	Storage Card	Lab storage	Displays storage used relative to quota.
7	Quick Actions	Upload dataset, Create data source, Create pipeline, Fork template, Invite member	Quick actions for continuing work in the Lab.
8	Storage Section	Lab storage / Manage quota	Entry point for monitoring and managing quota in the lower section of the page.

***Table 3.**39. Data – Managing the Workspace lifecycle*

No.	Table / data structure name	Create	Update	Delete	Query
1	workspaces		X	X	X
2	workspace_members			X	X
3	workspace_role_policies			X	X
4	workspace_invitations			X	X
5	quotas			X	X

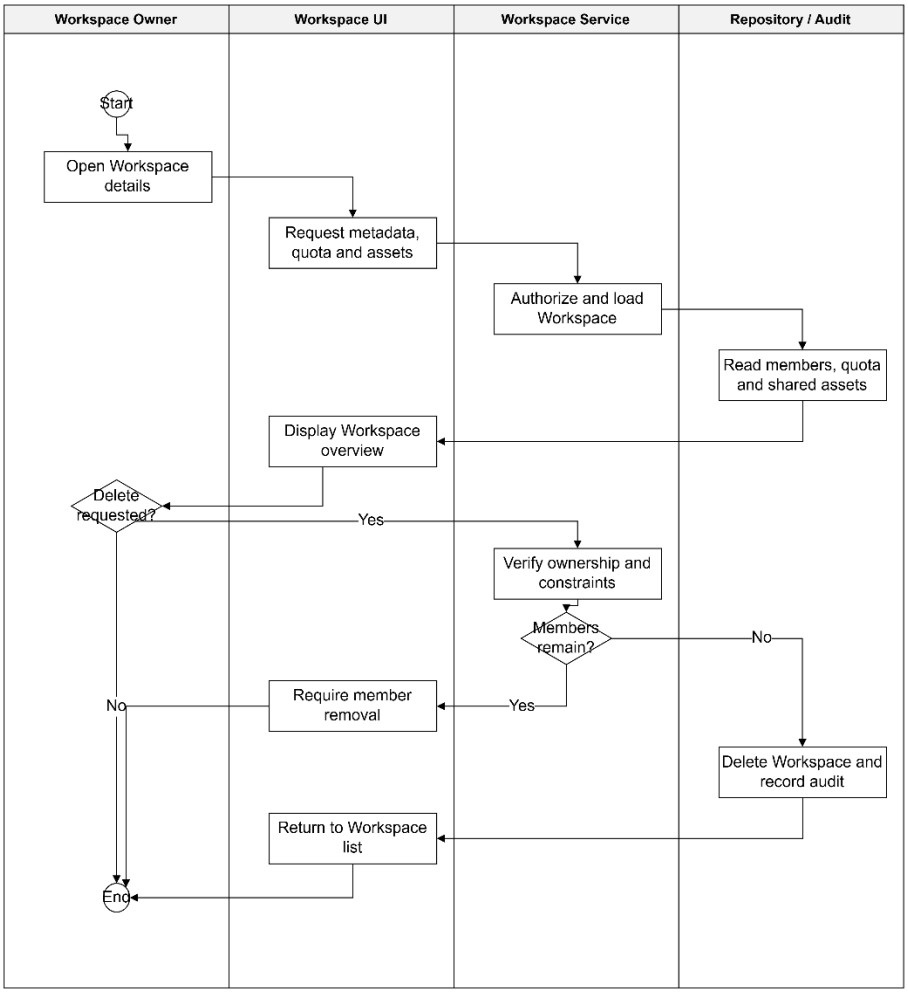


***Figure 3.47.** Activity Diagram – Managing the Workspace lifecycle*

3.4.4. Administrator Functions

3.4.4.1. Creating a Workspace and assigning an Owner

Related Use Case: UC83.

The Administrator creates a Workspace, selects a Researcher as Owner, and sets the initial quota.

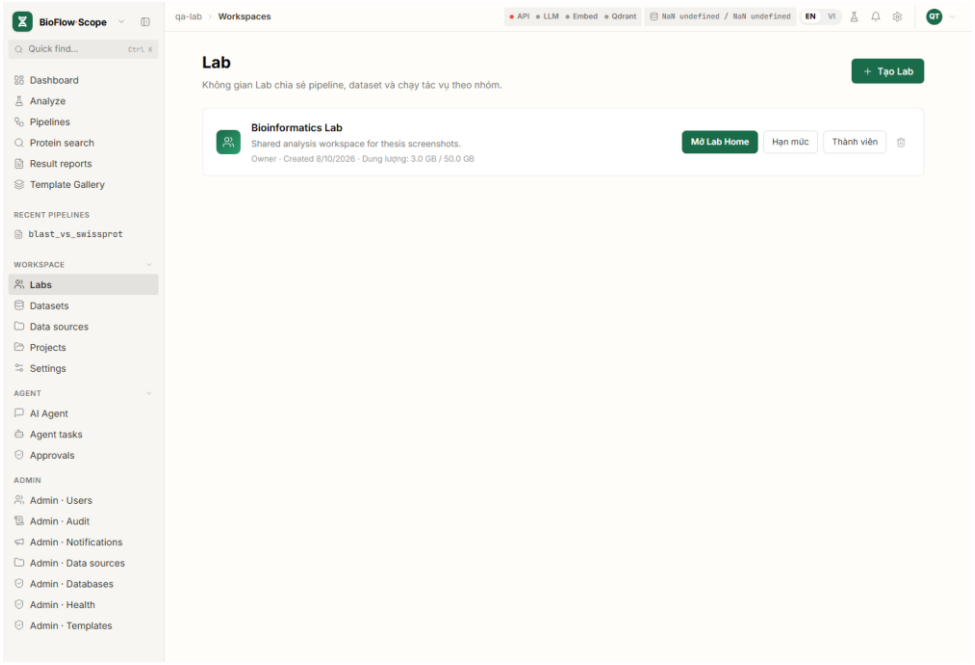


Figure 3.48. *Workspace creation and Owner-assignment interface*

Note: illustrative interface data.

Table 3.*40. Interface components – Creating a Workspace and assigning an Owner*

No.	Control type	Content	Notes
1	Button	Create Lab	Opens the flow for creating a new Workspace; the creation form is not open in the image.
2	Workspace Card	Existing Lab list	Displays a Workspace as a card.
3	Workspace Metadata	Name, description, Owner, creation date, and storage	Summary information used to identify the Workspace before opening it.
4	Button	Open Lab Home	Navigates to the Workspace overview page.
5	Button	Quota	Opens Workspace quota configuration.
6	Button	Members	Opens the member/invitation management page.
7	Icon Button	Delete Workspace	Trash-can icon for deletion; the Owner selector appears only when the Create Lab form is opened and therefore is not numbered in this image.

Table 3.*41. Data – Creating a Workspace and assigning an Owner*

No.	Table / data structure name	Create	Update	Delete	Query
1	workspaces	X			X
2	users				X

No.	Table / data structure name	Create	Update	Delete	Query
3	quotas	X			X
4	admin_audit_log	X			X

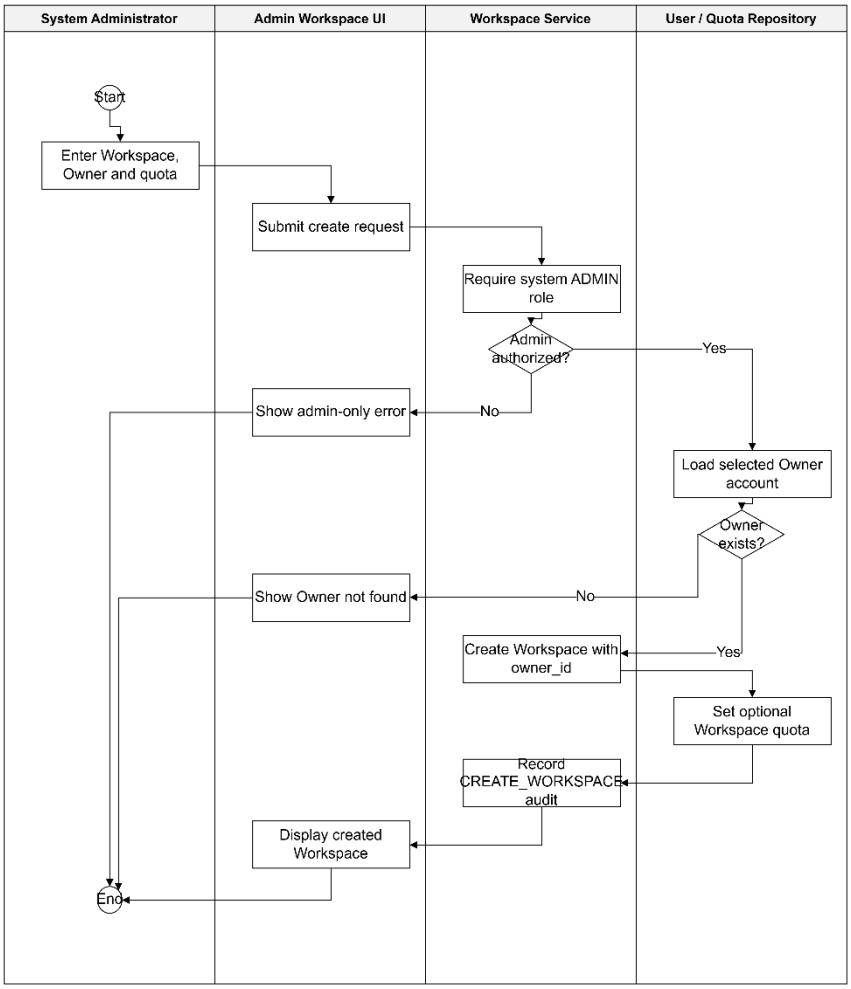


Figure 3.49. *Activity Diagram – Creating a Workspace and assigning an Owner*

3.4.4.2. Managing users and quotas

Related Use Case: UC-BS01 and UC-BS02.

The administration page allows accounts to be searched, statuses/roles to be changed, and quotas to be configured for a user or Workspace.

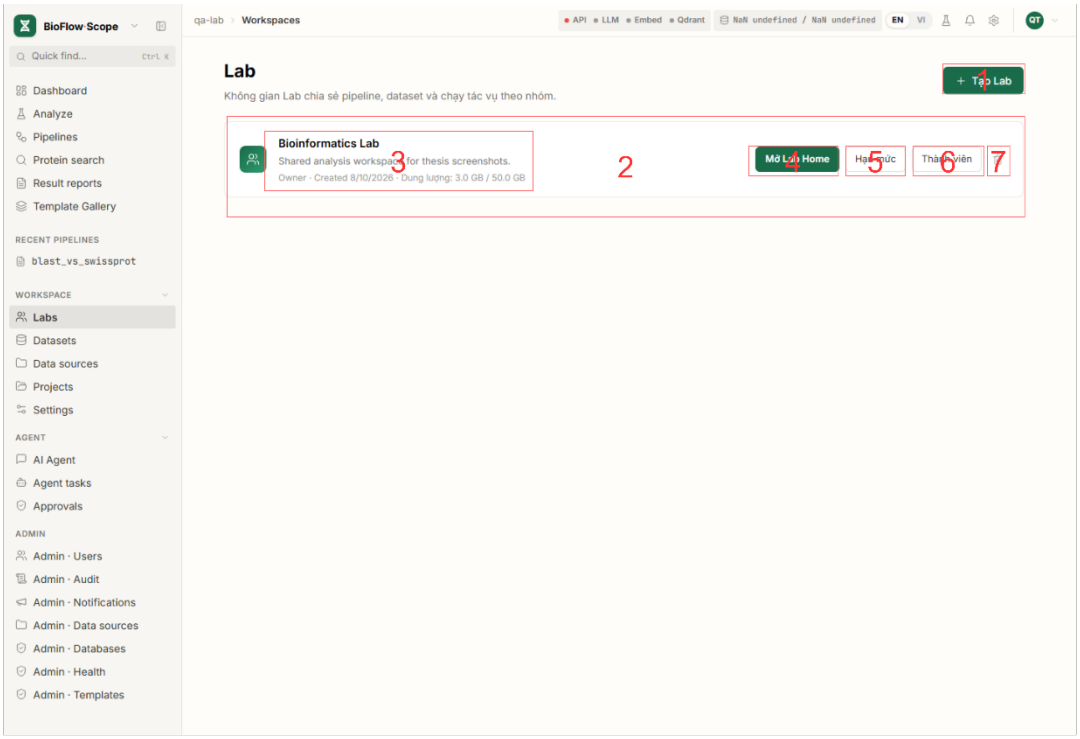


Figure 3.50. *User and quota management interface*

Note: illustrative interface data.

Table 3.*42. Interface components – Managing users and quotas*

No.	Control type	Content	Notes
1	Search Input	Search by name or email	Filters the administrative account list.
2	Dropdown	All roles	Filters users by system role.
3	Dropdown	All statuses	Filters by account status.
4	Button	Default quota	Opens default-quota configuration.
5	Button	Create account	Opens the new-account creation flow.
6	User Table	Full name, Email, Lab, and Last login	Displays account identity and activity information.
7	Badges	Role and Status	Displays values such as Administrator and Active.
8	Quota Display	Storage	Displays the user's used/total quota.
9	Row Action Menu	Per-account action menu	The three-dot button opens administrative actions such as role/status updates according to permissions.

Table 3.*43. Data – Managing users and quotas*

No.	Table / data structure name	Create	Update	Delete	Query
1	users	X	X		X

No.	Table / data structure name	Create	Update	Delete	Query
2	quotas	X	X		X
3	workspaces				X
4	admin_audit_log	X			X

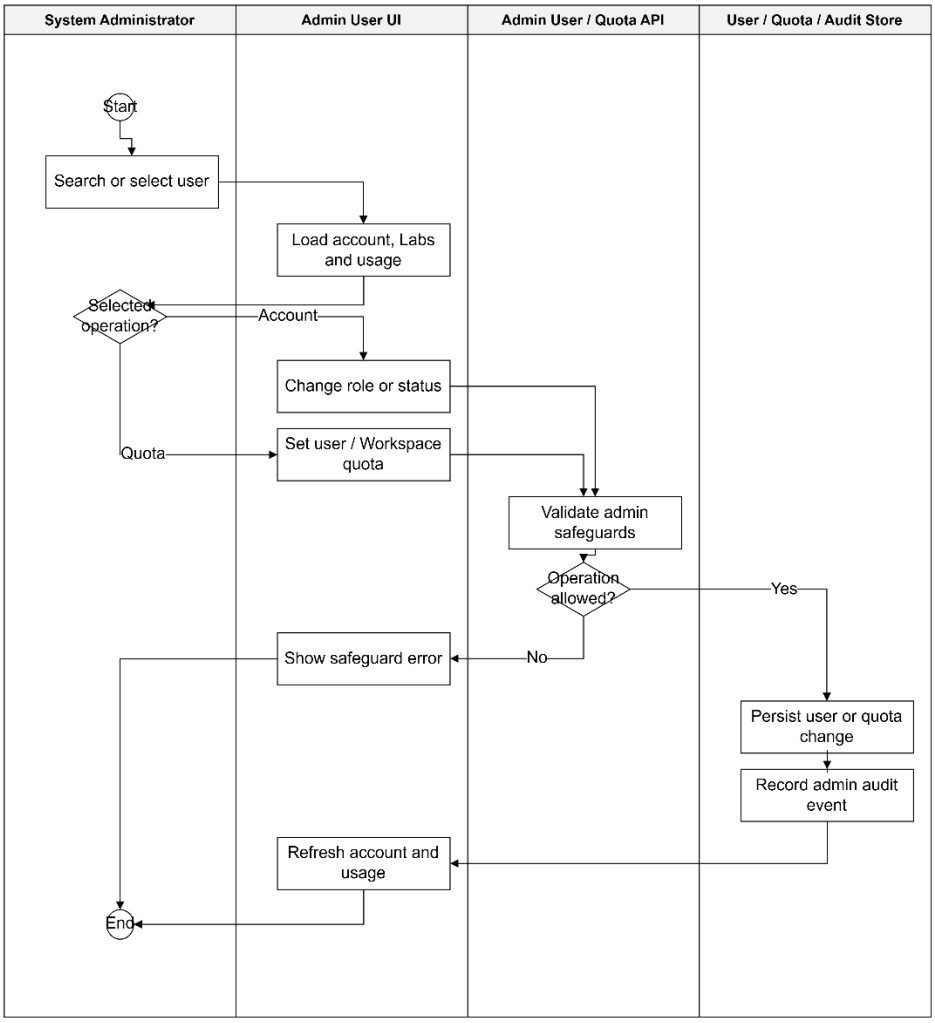


Figure 3.51. *Activity Diagram – Managing users and quotas*

3.4.4.3. Managing reference databases and system resources

Related Use Case: UC99 through UC104.

The Administrator monitors database versions, update jobs, and inventory/knowledge collections used by analysis modules.

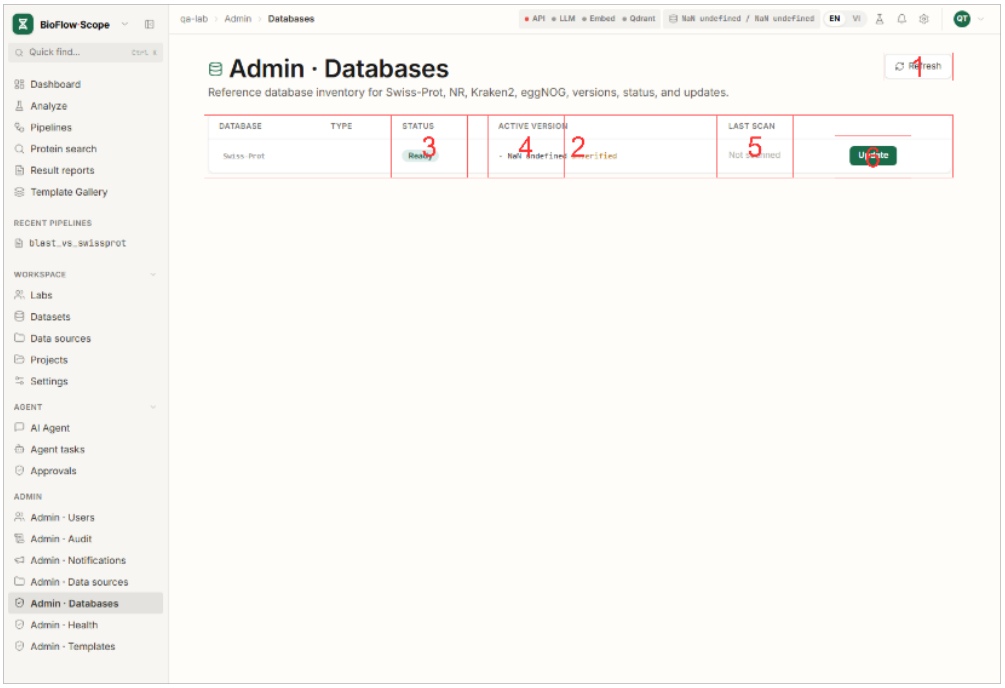


Figure 3.52. *Database and system-resource management interface*

Note: illustrative interface data.

Table 3.*44. Interface components – Managing reference databases and system resources*

No.	Control type	Content	Notes
1	Button	Refresh	Reloads the reference-database inventory.
2	Database Table	Database list	Displays database, type, status, active version, and last scan.
3	Status Badge	Database status	For example, Ready.
4	Version Field	Active version	Displays the active version and verification status when available.
5	Scan Field	Last scan	Shows the most recent inventory reconciliation.
6	Button	Update	Initiates an update for the selected database; the progress panel appears only after an update begins and therefore is not numbered in this image.

Table 3.*45. Data – Managing reference databases and system resources*

No.	Table / data structure name	Create	Update	Delete	Query
1	bio_databases	X	X		X
2	bio_database_versions	X	X		X
3	bio_database_update_jobs	X	X		X

No.	Table / data structure name	Create	Update	Delete	Query
4	admin_audit_log	X			X
5	Qdrant knowledge collections	X	X	X	X

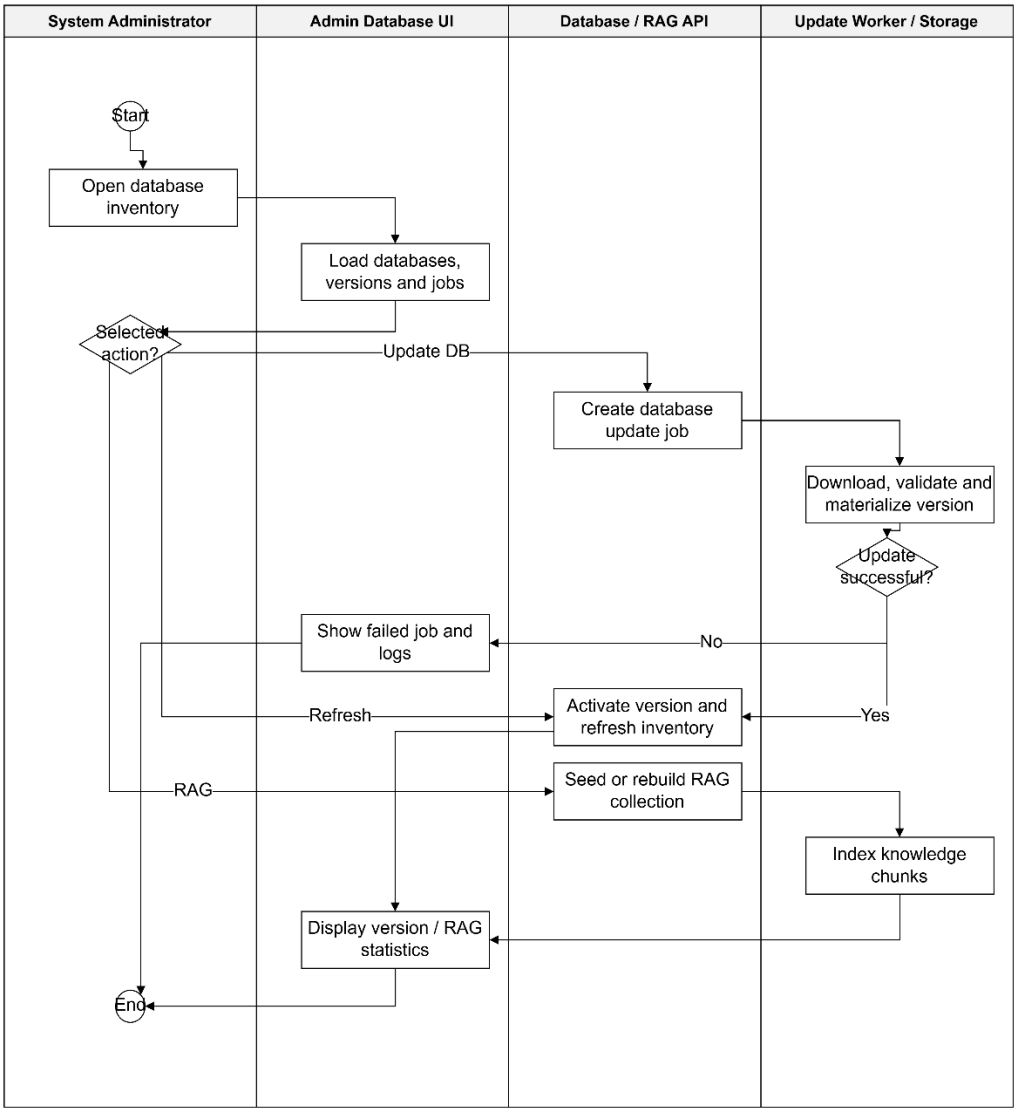


Figure 3.53. *Activity Diagram – Managing reference databases and system resources*

3.4.5. Scheduler and System Functions

3.4.5.1. Executing Pipelines by schedule and data trigger

Related Use Case: UC32 through UC37.

Automatically creates and executes Runs according to a Cron schedule or when new data is detected, while preventing duplicates and applying quota back-pressure.

The Researcher configures a Data Source/Cron Trigger; the Scheduler scans data or schedules, prevents duplication, and automatically creates a Run when conditions are valid.

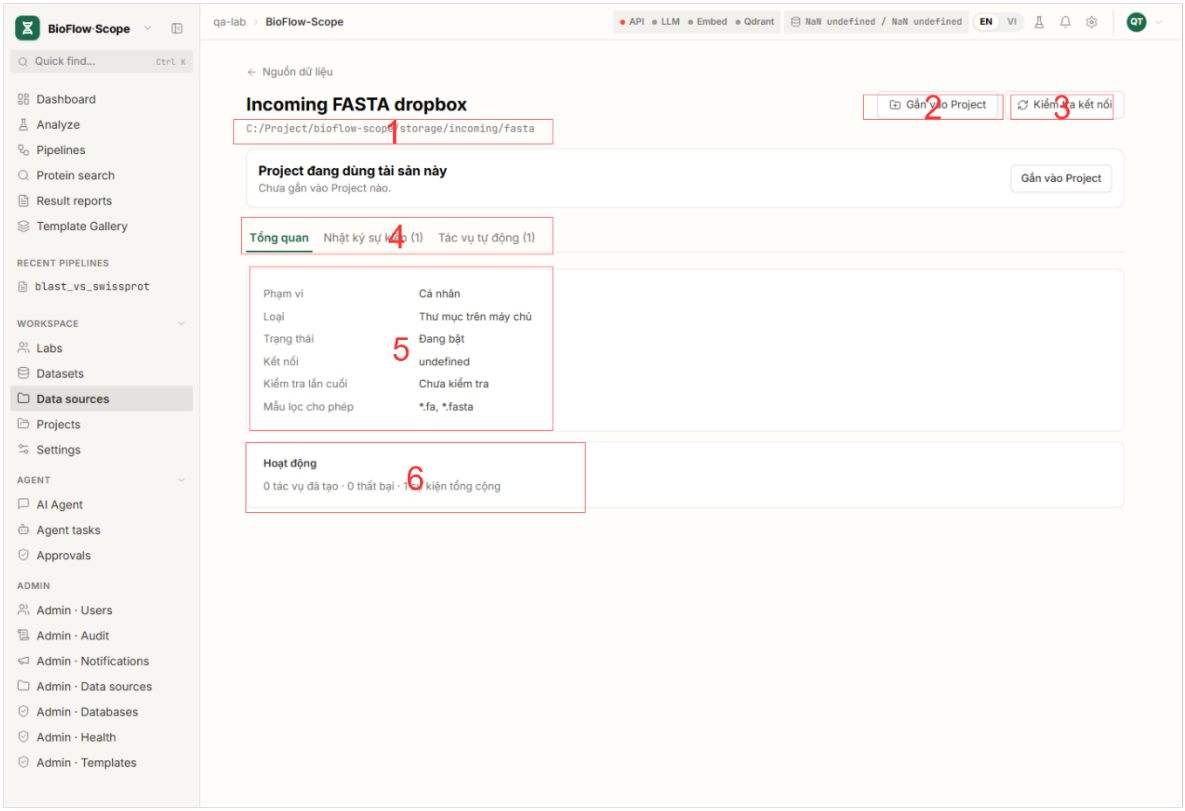


Figure 3.54. *Data Source and Pipeline Trigger interface*

Note: illustrative interface data.

Table 3.*46. Interface components – Executing Pipelines by schedule and data trigger*

No.	Control type	Content	Notes
1	Header / Path	Data Source name and source path	Displays the Incoming FASTA dropbox together with its filesystem path.
2	Project Controls	Assign to Project	A button is available in the toolbar and in the Project panel showing which Project uses this asset.
3	Button	Test connection	Tests whether the Data Source can be accessed.
4	Tabs	Overview, Event log, Automated tasks	Displays event/rule counts directly on the tabs.
5	Overview Panel	Scope, Type, Status, Connection, Last checked, Allowed filter pattern	Summarizes the actual configuration of the current Data Source.

No.	Control type	Content	Notes
6	Activity Summary	Activity	Summarizes running tasks, failed tasks, and successful events; the Cron/Trigger form is not open in this image state.

Table 3.*47. Data – Executing Pipelines by schedule and data trigger*

No.	Table / data structure name	Create	Update	Delete	Query
1	data_sources	X	X		X
2	data_source_roots	X	X		X
3	pipeline_triggers	X	X		X
4	trigger_events	X	X		X
5	scheduled_runs	X	X		X
6	runs	X	X		X
7	run_dataset_inputs	X			X
8	notifications	X			X

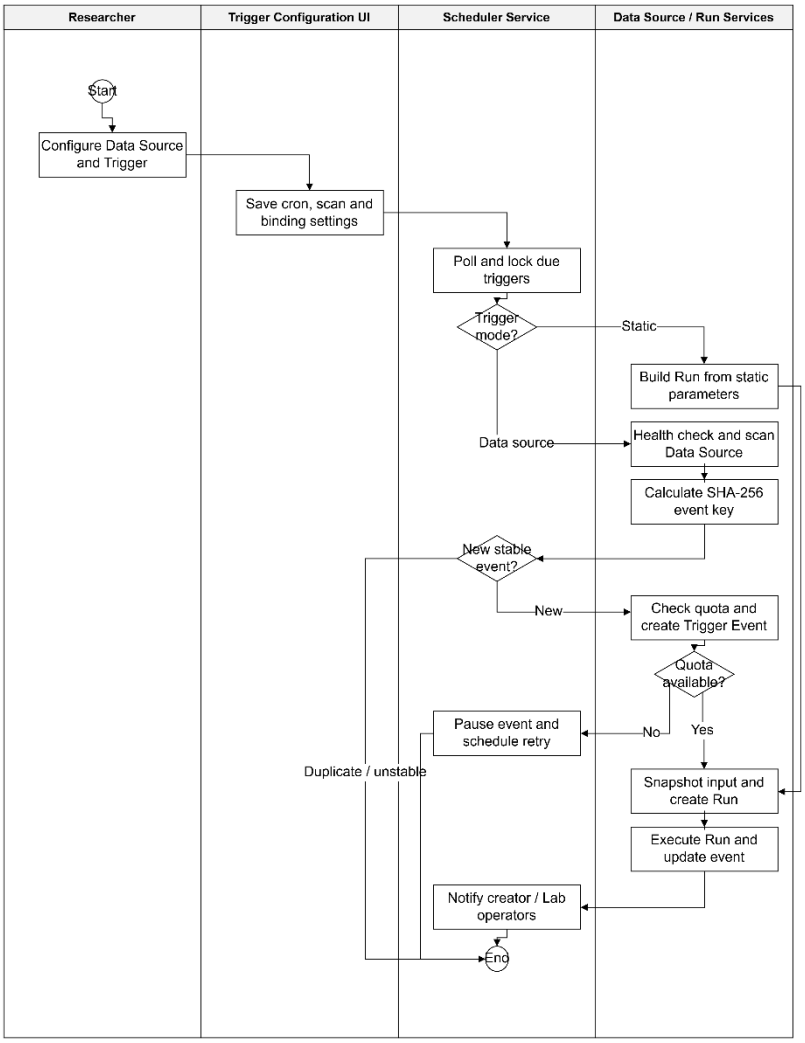


Figure 3.55. *Activity Diagram – Scheduled and data-triggered Pipeline execution*

CHAPTER 4. TESTING AND EVALUATION

4.1. Testing Objectives

Testing aims to verify that BioFlow-Scope satisfies the designed requirements in the local test environment. The focus includes correctness of Pipelines and DAGs, data management, authorization, Semantic Protein Search, the AI Agent, and reproducibility of results.

• Verify that core functions operate correctly in both success and error flows.

• Test security constraints, RBAC, quotas, and data isolation between users.

• Evaluate ESM-2, Qdrant, RAG/Tool Calling, and Reproducibility integration flows.

• Measure source-code coverage and record defects detected during test execution.

4.2. Test plan

4.2.1. Functions Under Test

The testing scope is grouped by module to avoid repeating the entire Use Case specification.

Table 4.*1. Functional groups under test*

Functional group	Main testing content
Authentication and accounts	Registration, login, profile, and personal AI configuration.
Pipeline and Run	Tool Registry, YAML/Smart Canvas, DAG, runtime parameter, Run, SSE, cancel, resume, provenance.
Protein Semantic Search	ESM-2, Qdrant, Redis cache, UniProt/PDB, and result export.
Dataset and Viewer	Upload, quota, binding, delete; BLAST/VCF/tree/MultiQC viewer.
AI Agent	Session, RAG citations, Tool Calling, and Pipeline draft generation.
Workspace/Project	Workspace/project creation, member invitations, RBAC, and History sharing.
Reproducibility	Bundle export, SHA-256, RO-Crate, and reproducibility verification.
Administration	User/quota administration, database inventory, and storage.
Frontend	TypeScript strict checking and Playwright E2E.

4.2.2. Approach

The thesis uses multi-layer testing: static frontend checks, unit tests for backend logic, integration/API tests for coordinated components, Playwright for E2E flows,

and live verification for AI/bioinformatics integrations. Security tests focus on tenant isolation, rate limiting, quotas, and access permissions.

4.2.3. Criteria for Test Success / Failure

Table 4.2. *Criteria for determining test pass/failure*

Criterion	Pass	Fail
Backend test	Executable tests PASS; environment-dependent tests may be SKIPPED only when the reason is clearly documented.	One or more tests FAIL, or a functional defect remains unresolved.
Frontend	TypeScript has no errors; E2E scenarios execute successfully.	The compiler reports errors or E2E execution does not complete.
Coverage	Instruction ≥ 50% and Class ≥ 55%.	Either threshold is not met.
Security	Unauthorized access is blocked; quotas/rate limits function correctly.	Metadata/resources are exposed or a guard fails to block access correctly.
Reproducibility	The bundle contains snapshots/checksums and the verifier detects mismatches.	Reproducibility data is missing or checksums are not verified.

4.2.4. Suspension Criteria and Resumption Requirements

Table 4.3. *Test suspension and resumption criteria*

Suspension condition	Resumption condition
PostgreSQL, Redis, Qdrant, or the embedding sidecar is unavailable.	The service becomes available again and health checks pass.
Build/migration fails or test data is invalid.	Correct the setup issue, normalize the data, and rerun from the affected step.
A test requires the Docker daemon but the environment does not provide a suitable socket.	Start the Docker daemon or mark the test as SKIPPED with a reason when the test is environment-dependent.
A High-severity defect is found that directly affects a core flow.	Fix the defect, rebuild, and run regression tests for the affected module.

4.2.5. Test Deliverables

Table 4.*4. Test deliverables*

Deliverable	Content
Test Case list	47 functional test cases across 9 module groups.
Automated test reports	Gradle/JUnit backend tests, TypeScript strict checks, and Playwright E2E.
Coverage and evidence	JaCoCo, API/live logs, checksums, and Semantic Search/Reproducibility evidence.
Defect log	Defect, severity, remediation, and status.

4.3. Test management

4.3.1. Test Activities and Process

Table 4.*5. Testing activities*

Stage	Main activities
Preparation	Start infrastructure; create test accounts/roles, data, and Pipelines.
Static and unit testing	Run TypeScript checks and backend unit tests.
Integration/API	Verify the database, cache, Qdrant, sidecar, APIs, and guards.
E2E and live verification	Run Playwright; verify Semantic Protein Search and AI/reproducibility flows.
Consolidation and regression	Record defects, fix issues, rerun related tests, and prepare the final test report.

4.3.2. Test Environment

Table 4.*6. Test environment*

Component	Main configuration
Workstation	Windows 11 Pro 64-bit; 8-core CPU; 16 GB RAM; NVMe SSD; JVM heap approximately 4 GB.
Application	OpenJDK 21; Spring Boot 3.3.4; Next.js 14.2.35; React 18; Node.js 20.17.0; TypeScript 5.6.3.
Data infrastructure	PostgreSQL 16; Redis 7.x; Qdrant 1.12.1.
AI and QA	Python 3.11/FastAPI; ESM-2; Ollama qwen3:8b; JUnit 5; JaCoCo 0.8.12; Playwright 1.61.1.

4.3.3. Communication Among Relevant Groups

The thesis did not establish multiple independent testing teams. Results are traced through Test Cases, Gradle/Playwright logs, JaCoCo reports, API logs, and the defect log. The AI Agent supports test execution and evidence consolidation.

4.3.4. Resources and Allocation

PostgreSQL, Redis, Qdrant, and Ollama services run in Docker; the backend and frontend use workstation resources. The embedding sidecar runs separately on Python/FastAPI. Tests were performed within the 16 GB RAM limit and do not require a multi-node production environment.

4.3.5. Training

The testing record does not document a separate training program. Testing tools were used directly within the thesis development environment.

4.3.6. Risks

Table 4.*7. Testing risks*

Risk	Impact	Mitigation
Supporting services unavailable	Integration testing is interrupted.	Perform health checks before execution and restart services when necessary.
Docker daemon dependency	Some runtime tests may be SKIPPED.	Document the reason clearly and do not treat an environment-related SKIP as a functional failure.
Environment limited to a local workstation	Does not fully represent production.	Limit conclusions to the current testbed.
Variability in external AI/UniProt/PDB services	Results or response times may vary.	Use mocks/stubs where appropriate and separate live verification from unit tests.
Low Branch Coverage	Some error branches remain uncovered.	Add tests for conditional branches in a future version.
No independent scientific benchmark is available	There is insufficient evidence to infer precision/recall or LLM quality.	Conclusions are limited to the executability of the tested flows.

4.4. Test scenarios

Table 4.*8. Test scenarios*

Scenario code	Scenario name	Number of Test Cases
KBT-01	Authentication and accounts	7
KBT-02	Pipeline and Run	15
KBT-03	Protein Semantic Search	4
KBT-04	Dataset	4
KBT-05	Viewer	4
KBT-06	AI Agent	4

Scenario code	Scenario name	Number of Test Cases
KBT-07	Workspace/Project	3
KBT-08	Reproducibility	3
KBT-09	Administration	3
	Total	47

4.5. Test cases

4.5.1. Functional Testing

Functional Test Cases are grouped by module to preserve traceability while avoiding repetition of the Use Case specifications. Each table below presents the main data or operation and the actual result of the final testing session.

4.5.1.1. Authentication and accounts

Table 4.9*. Functional testing – Authentication and accounts*

Test Case ID	Test content	Main data / operation	Result
TC_AUTH_001	Register a new account	Condition: The system is running. Data: email, password, fullName; Action: Send POST /api/auth/register with valid information	PASS - Registration succeeds and returns a JWT token and complete UserDto
TC_AUTH_002	Register a duplicate email	Condition: An account already exists. Data: duplicate email; Action: Send POST /api/auth/register with an existing email	PASS - The system rejects registration and returns EMAIL_ALREADY_EXISTS
TC_AUTH_003	Log in to the system	Condition: The account has been registered. Data: email, password; Action: Send POST /api/auth/login with correct credentials	PASS - Login succeeds and returns a valid accessToken
TC_AUTH_004	Log in with an incorrect password	Condition: The account has been registered. Data: incorrect password; Action: Send POST /api/auth/login with the wrong password	PASS - Returns 401 INVALID_CREDENTIALS
TC_AUTH_005	Retrieve current-user information	Condition: A valid accessToken is available. Data: Bearer JWT; Action: Send GET /api/auth/me with the Bearer token	PASS - Returns 200 OK with a complete profile
TC_AUTH_006	Update personal profile	Condition: Logged in. Data: new fullName; Action: Send PUT /api/settings/profile	PASS - The profile is updated correctly in the database

Test Case ID	Test content	Main data / operation	Result
TC_AUTH_007	Configure a personal AI Model	Condition: Logged in. Data: OpenAI API key / custom endpoint; Action: Send PUT /api/settings/ai-models	PASS - Configuration is stored securely

4.5.1.2. Pipeline and Run

Table 4.*10. Functional testing – Pipeline and Run*

Test Case ID	Test content	Main data / operation	Result
TC_PIPE_001	List the tool catalog (Tool Registry)	Condition: Logged in. Data: None; Action: Send GET /api/tools	PASS - Returns all 26 tools divided into 13 functional groups
TC_PIPE_002	View tool-parameter details	Condition: Logged in. Data: toolId=blast; Action: Send GET /api/tools/catalog/{toolId}	PASS - Metadata conforms to the bioflow.tool-definition.v1 schema
TC_PIPE_003	Create a Pipeline through the YAML Editor	Condition: Logged in. Data: two-step YAML (FastQC -> MUSCLE); Action: Send POST /api/pipelines with valid YAML	PASS - The Pipeline is created in the database with 201 Created
TC_PIPE_004	Create a Pipeline through Smart Canvas	Condition: Smart Canvas is open. Data: FastQC node connected to MultiQC; Action: Drag and drop tool nodes and connect compatible ports	PASS - Bidirectional Visual Graph <-> YAML synchronization works smoothly
TC_PIPE_005	Validate DAG syntax and cycles	Condition: YAML has been authored. Data: A -> B -> A; Action: Send POST /api/pipelines/lint with cyclic YAML	PASS - PipelineYamlException is raised correctly
TC_PIPE_006	Compile DAG execution order (Topological Sort)	Condition: Valid Pipeline. Data: Pipeline DAG; Action: Send POST /api/pipelines/{id}/compile	PASS - Topological ordering is fully deterministic
TC_PIPE_007	Automatically insert a Converter Node	Condition: Connect a SAM port to BAM. Data: The system warns or automatically inserts samtools_view; Action: Connect BWA-MEM to FreeBayes	PASS - Biological-format compatibility is resolved automatically
TC_PIPE_008	Configure runtime parameters for a Pipeline	Condition: Pipeline has parameters. Data: threads=8 min_qual=30; Action: Pass runtime parameters in the start-run request	PASS - CommandTemplating substitutes variables correctly
TC_PIPE_009	Start a Pipeline Run	Condition: Pipeline has been created. Data: Pipeline ID; Action: Send POST /api/pipelines/{id}/run	PASS - The Run is initialized and a snapshot log is recorded

Test Case ID	Test content	Main data / operation	Result
TC_PIPE_010	Stream execution logs (Live Log SSE)	Condition: Run is executing. Data: Run ID; Action: Connect to SSE /api/runs/{id}/logs	PASS - ServerSentEvents streams logs smoothly without blocking threads
TC_PIPE_011	Cancel an executing Pipeline process	Condition: Run is executing. Data: Run ID; Action: Send POST /api/runs/{id}/cancel	PASS - RunCancellationRegistry terminates the Docker container immediately
TC_PIPE_012	Resume from a failed step (Resume Run)	Condition: Run failed at step 2. Data: Run ID; Action: Send POST /api/runs/{id}/resume	PASS - Resume reuses valid artifacts from step 1
TC_PIPE_013	Execute Map-Over on a sample list (Batch Execution)	Condition: Pipeline has list input. Data: manifest of 10 samples; Action: Provide a manifest file containing 10 FASTQ samples	PASS - Map-over processes each sample independently and correctly
TC_PIPE_014	View Run lineage and Provenance	Condition: Run completed. Data: Run ID; Action: Send GET /api/runs/{id}/provenance	PASS - The lineage graph displays complete input/output hashes
TC_PIPE_015	Compare results between two Runs (Run Comparison)	Condition: Two Runs of the same Pipeline exist. Data: two Run IDs; Action: Send GET /api/runs/compare?runA=...&runB=...	PASS - Detailed parameter and metric differences are compared

4.5.1.3. Protein Semantic Search

Table 4.*11. Functional testing – Protein Semantic Search*

Test Case ID	Test content	Main data / operation	Result
TC_PROT_001	Search for a Protein by Sequence (ProtScope)	Condition: Valid amino-acid sequence. Data: PETase FASTA sequence (290 aa); Action: Send POST /api/protein/search	PASS - ESM-2 inference and Qdrant vector search return results in <1 s
TC_PROT_002	Search for a Protein by UniProt Accession	Condition: Valid accession. Data: P00533 (EGFR_HUMAN); Action: Send GET /api/protein/{accession}	PASS - UniProtClient enriches the information correctly
TC_PROT_003	Cache Protein-search results	Condition: The search has been performed once. Data: repeated sequence; Action: Resend POST /api/protein/search with the same sequence	PASS - Cache hit succeeds with response time <10 ms
TC_PROT_004	Export the Protein list to FASTA/CSV	Condition: Search results exist. Data: Top 50 hits; Action: Select Export FASTA/CSV	PASS - Exported file conforms to the biological format

4.5.1.4. Dataset

Table 4.*12. Functional testing – Dataset*

Test Case ID	Test content	Main data / operation	Result
TC_DATA_001	Upload research data files (Dataset Upload)	Condition: Valid FASTA/FASTQ file. Data: reads.fastq.gz (15 MB); Action: Send multipart POST /api/datasets/upload	PASS - The file is stored and assigned ownership
TC_DATA_002	Check and enforce storage quota	Condition: Standard Researcher account. Data: >20 GB; Action: Upload data exceeding the 20 GB quota	PASS - The 500 MB disk-floor protection works and excessive uploads are blocked
TC_DATA_003	Bind a Dataset to a Pipeline Input (Dataset Binding)	Condition: Dataset exists in the library. Data: Dataset ID; Action: Select the dataset from the picker when configuring the Run	PASS - The volume is mounted correctly at /data in the container
TC_DATA_004	Delete a Dataset and update Quota	Condition: Dataset is not locked by a Run. Data: Dataset ID; Action: Send DELETE /api/datasets/{id}	PASS - usedBytes decreases immediately

4.5.1.5. Viewer

Table 4.*13. Functional testing – Viewer*

Test Case ID	Test content	Main data / operation	Result
TC_VIEW_001	Interactive BLAST result viewer (Blast Viewer)	Condition: Run produces blast.out in format 6/7. Data: tabular BLAST output; Action: Send GET /api/viewers/blast/{runId}	PASS - The parser extracts all HSP columns and renders the visualization
TC_VIEW_002	VCF variant viewer (VCF Viewer)	Condition: Run produces a .vcf file. Data: Sample.vcf (SNPs/INDELs); Action: Send GET /api/viewers/vcf/{runId}	PASS - Virtual Table pagination handles 100K variants smoothly
TC_VIEW_003	Phylogenetic tree viewer (Phylo Tree)	Condition: Run produces a Newick .nwk file. Data: tree.nwk from IQ-TREE; Action: Send GET /api/viewers/tree/{runId}	PASS - The Newick tree is rendered correctly using D3/Phylotree
TC_VIEW_004	Aggregated quality report (MultiQC Viewer)	Condition: Run produces multiqc_report.html. Data: MultiQC HTML; Action: Send GET /api/viewers/html/{runId}	PASS - The iframe renders safely while isolating malicious JavaScript

4.5.1.6. AI Agent

***Table 4.**14. Functional testing – AI Agent*

Test Case ID	Test content	Main data / operation	Result
TC_AGENT_001	Create a scientific conversation session with the AI Agent	Condition: Logged in. Data: pipeline-design question; Action: Send POST /api/agent/sessions with the first question	PASS - The chat session is created successfully with an automatic title
TC_AGENT_002	Scientific Q&A with literature citations (RAG Citation)	Condition: Chat session is open. Data: bioinformatics question; Action: Ask about how the MUSCLE v5 algorithm works	PASS - The response includes scientific citations in APA/IEEE format
TC_AGENT_003	AI Agent automatically invokes a Tool (Tool Calling)	Condition: Chat session is open. Data: amino-acid sequence; Action: Request: 'Find proteins similar to the following sequence...'	PASS - The correct tool is invoked and results are formatted as interactive cards
TC_AGENT_004	AI Agent proposes and creates a Pipeline draft	Condition: Chat session is open. Data: workflow request; Action: Request: 'Create an RNA-seq analysis pipeline using Salmon'	PASS - Generates syntactically valid YAML with no errors

4.5.1.7. Workspace/Project

***Table 4.**15. Functional testing – Workspace/Project*

Test Case ID	Test content	Main data / operation	Result
TC_PROJ_001	Create a Research Project Workspace	Condition: Logged in. Data: name & description; Action: Send POST /api/projects	PASS - The Project is created with the default OWNER role
TC_PROJ_002	Invite members and assign Lab permissions (RBAC)	Condition: Workspace Owner. Data: email & role (MEMBER/VIEWER); Action: Send POST /api/workspaces/{id}/members	PASS - Permissions follow the RBAC matrix correctly
TC_PROJ_003	Share Analysis History (Shared History)	Condition: A History exists in the Workspace. Data: History ID; Action: Change visibility to WORKSPACE_SHARED	PASS - The History appears on the Lab's shared feed

4.5.1.8. Reproducibility

Table 4.*16. Functional testing – Reproducibility*

Test Case ID	Test content	Main data / operation	Result
TC_REPRO_001	Package a research object for publication (Export Research Object)	Condition: Run completed. Data: Run ID; Action: Send POST /api/reproducibility/export/{runId}	PASS - The RO-Crate-compliant ZIP bundle contains integrity checksums
TC_REPRO_002	Validate and verify a Reproduction bundle (Verify Reproducibility)	Condition: A .bioflow.zip file exists. Data: bundle ZIP file; Action: Send POST /api/reproducibility/verify	PASS - VERIFIED_REPRODUCIBLE status is confirmed
TC_REPRO_003	Freeze a Result version (Freeze Result Version)	Condition: A high-quality Run is to be archived. Data: Report ID; Action: Send POST /api/reports/{id}/freeze	PASS - The report changes to FROZEN status and can no longer be edited or deleted

4.5.1.9. Administration

Table 4.*17. Functional testing – Administration*

Test Case ID	Test content	Main data / operation	Result
TC_ADMIN_001	Manage Users & Quotas (Admin User Management)	Condition: Logged in with ADMIN privileges. Data: Admin token & User ID; Action: Send GET /api/admin/users and PUT quota	PASS - Quota administration succeeds
TC_ADMIN_002	Inventory and update Reference Databases	Condition: Logged in with ADMIN privileges. Data: Admin token; Action: Send GET /api/admin/databases	PASS - Lists the storage size and update date of all databases
TC_ADMIN_003	Monitor System Resources & Storage Breakdown	Condition: Logged in with ADMIN privileges. Data: Admin token; Action: Send GET /api/admin/storage	PASS - Reports detailed storage usage accurately to the byte

4.6. Test evaluation

Table 4.*18. Summary of test results*

Category	Result
Functional testing	47/47 PASS.
Backend	1.134 Test Case: 1.133 PASS, 1 SKIP do Docker daemon, 0 FAIL.
Coverage	Instruction 51,70%; Class 57,28%; Branch 42,24%.
Frontend/E2E	TypeScript strict check: 0 errors; 35 Playwright E2E scenarios: PASS.

Category	Result
Semantic/AI/Security	PETase: 982 ms total; Agent flow PASS; tenant isolation, rate limiting, quota, and RBAC met requirements.
Reproducibility/Defect	The bundle was confirmed by the verifier; all four defects discovered during testing were CLOSED after remediation.

Instruction Coverage and Class Coverage met the defined thresholds, while Branch Coverage reached 42.24%. This testing round did not include large-scale stress testing, independent penetration testing, precision/recall benchmarking for Semantic Protein Search, or an LLM-quality scoring framework. Therefore, the results apply only to the current testbed and are not a certification of production readiness or general scientific accuracy.

PART 3. CONCLUSION

1. Achieved Results

1.1. Knowledge and Skills

Gained a comprehensive understanding of the software development lifecycle, including requirements analysis, system design, implementation, testing, deployment, and maintenance. Acquired practical experience in developing a bioinformatics web platform using Spring Boot, Next.js, PostgreSQL, Redis, Qdrant, Docker, FastAPI, and related technologies.

Developed system analysis and design skills through the construction of Use Case diagrams, activity diagrams, C4 architecture diagrams, database models, API specifications, and workflow designs.

Gained experience in integrating bioinformatics tools into reproducible pipelines using containerized execution, while managing input data, parameters, logs, artifacts, checksums, and provenance. Developed practical knowledge of AI integration through Semantic Protein Search, protein embeddings, vector databases, Retrieval-Augmented Generation (RAG), tool calling, and Human-in-the-Loop approval.

Strengthened problem-solving, system design, self-learning, testing, and research skills while working with a multidisciplinary system combining software engineering, bioinformatics, and artificial intelligence.

1.2. Program and Practical Application Potential

BioFlow-Scope has completed its main functional groups, including user authentication and authorization, pipeline design, Run execution and monitoring, Dataset and Artifact management, Analyze History, Workspace collaboration, and administration.

The system provides a visual Smart Canvas for designing bioinformatics pipelines and executes scientific tools using Docker/BioContainers, helping improve workflow traceability and reproducibility.

Semantic Protein Search uses ESM-2 embeddings and Qdrant vector search to retrieve related proteins and enrich results with scientific metadata from resources such as UniProt and PDB.

The AI Agent combines RAG and tool calling to assist users in retrieving information and interacting with system functions. Actions with potential side effects

are controlled through authorization, AI policy, evidence tracking, and Human-in-the-Loop approval.

The final evaluation recorded 47/47 functional test cases passed, 1,133 backend tests passed, 35/35 Playwright end-to-end scenarios passed, and no TypeScript type-checking errors.

The system can be deployed locally and provides a practical foundation for supporting researchers and students in building, executing, managing, and reproducing bioinformatics analyses.

2. Limitations and Challenges

The system was mainly developed and evaluated in a local environment. Large-scale deployment, high concurrent workloads, and distributed infrastructure have not yet been fully evaluated.

Integrating multiple technologies, including Spring Boot, Next.js, Docker, Qdrant, Redis, FastAPI, protein language models, and LLM services, increased the complexity of system development, testing, and deployment.

REFERENCES

[1] G. Project, “Galaxy Project - an open web-based platform for accessible, reproducible and transparent computational biomedical research,” 2026. [Online]. Available: https://galaxyproject.org/

[2] P. Di Tommaso, M. Chatzou, E. W. Floden, P. P. Barja, E. Palumbo, and C. Notredame, “Nextflow enables reproducible computational workflows,” *Nat. Biotechnol.*, vol. 35, pp. 316–319, 2017, doi: 10.1038/nbt.3820.

[3] P. A. Ewels *et al.*, “The nf-core framework for community-curated bioinformatics pipelines,” *Nat. Biotechnol.*, vol. 38, no. 3, pp. 276–278, 2020, doi: 10.1038/s41587-020-0439-x.

[4] S. F. Altschul, W. Gish, W. Miller, E. W. Myers, and D. J. Lipman, “Basic local alignment search tool,” *J. Mol. Biol.*, vol. 215, pp. 403–410, 1990, doi: 10.1016/S0022-2836(05)80360-2.

[5] T. U. Consortium, “UniProt: the Universal Protein Knowledgebase in 2025,” *Nucleic Acids Res.*, vol. 53, no. D1, pp. D609–D617, 2025, doi: 10.1093/nar/gkae1010.

[6] F. da Veiga Leprevost, “BioContainers: an open-source and community-driven framework for software standardization,” *Bioinformatics*, vol. 33, no. 16, pp. 2580–2582, 2017, doi: 10.1093/bioinformatics/btx192.

[7] M. Griffith, J. R. Walker, N. C. Spies, B. J. Ainscough, and O. L. Griffith, “Informatics for RNA Sequencing: A Web Resource for Analysis on the Cloud,” *PLoS Comput. Biol.*, vol. 11, no. 8, p. e1004393, 2015, doi: 10.1371/journal.pcbi.1004393.

[8] B. Bioinformatics, “FastQC: A Quality Control Tool for High Throughput Sequence Data.” [Online]. Available: https://www.bioinformatics.babraham.ac.uk/projects/fastqc/

[9] P. Ewels, M. Magnusson, S. Lundin, and M. Käller, “MultiQC: summarize analysis results for multiple tools and samples in a single report,” *Bioinformatics*, vol. 32, pp. 3047–3048, 2016, doi: 10.1093/bioinformatics/btw354.

[10] S. Chen, Y. Zhou, Y. Chen, and J. Gu, “fastp: an ultra-fast all-in-one FASTQ preprocessor,” *Bioinformatics*, vol. 34, pp. i884–i890, 2018, doi: 10.1093/bioinformatics/bty560.

[11] P. Danecek, “Twelve years of SAMtools and BCFtools,” *Gigascience*, vol. 10, p. giab008, 2021, doi: 10.1093/gigascience/giab008.

[12] A. Bankevich, “SPAdes: a new genome assembly algorithm and its applications to single-cell sequencing,” *Journal of Computational Biology*, vol. 19, pp. 455–477, 2012, doi: 10.1089/cmb.2012.0021.

[13] A. Gurevich, V. Saveliev, N. Vyahhi, and G. Tesler, “QUAST: quality assessment tool for genome assemblies,” *Bioinformatics*, vol. 29, pp. 1072–1075, 2013, doi: 10.1093/bioinformatics/btt086.

[14] R. Patro, G. Duggal, M. I. Love, R. A. Irizarry, and C. Kingsford, “Salmon provides fast and bias-aware quantification of transcript expression,” *Nat. Methods*, vol. 14, pp. 417–419, 2017, doi: 10.1038/nmeth.4197.

[15] M. Steinegger, M. Meier, M. Mirdita, H. Vöhringer, S. J. Haunsberger, and J. Söding, “HH-suite3 for fast remote homology detection and deep protein annotation,” *BMC Bioinformatics*, vol. 20, p. 473, 2019, doi: 10.1186/s12859-019-3019-7.

[16] K. Katoh and D. M. Standley, “MAFFT multiple sequence alignment software version 7: improvements in performance and usability,” *Mol. Biol. Evol.*, vol. 30, pp. 772–780, 2013, doi: 10.1093/molbev/mst010.

[17] B. Q. Minh, “IQ-TREE 2: New Models and Efficient Methods for Phylogenetic Inference in the Genomic Era,” *Mol. Biol. Evol.*, vol. 37, pp. 1530–1534, 2020, doi: 10.1093/molbev/msaa015.

[18] T. Seemann, “Prokka: rapid prokaryotic genome annotation,” *Bioinformatics*, vol. 30, pp. 2068–2069, 2014, doi: 10.1093/bioinformatics/btu153.

[19] D. E. Wood, J. Lu, and B. Langmead, “Improved metagenomic analysis with Kraken 2,” *Genome Biol.*, vol. 20, p. 257, 2019, doi: 10.1186/s13059-019-1891-0.

[20] L. Wratten, A. Wilm, and J. Göke, “Reproducible, scalable, and shareable analysis pipelines with bioinformatics workflow managers,” *Nat. Methods*, vol. 18, pp. 1161–1168, 2021, doi: 10.1038/s41592-021-01254-9.

[21] W. W. W. Consortium, “PROV-O: The PROV Ontology,” 2013. [Online]. Available: https://www.w3.org/TR/prov-o/

[22] R.-C. Community, “RO-Crate Metadata Specification 1.1,” 2023. doi: 10.5281/zenodo.7867028.

[23] I. S. Association, “IEEE 2791-2020: IEEE Standard for Bioinformatics Computations and Analyses Generated by High-Throughput Sequencing to Facilitate Communication,” 2020. [Online]. Available: https://standards.ieee.org/standard/2791-2020.html

[24] M. D. Wilkinson, “The FAIR Guiding Principles for scientific data management and stewardship,” *Sci. Data*, vol. 3, p. 160018, 2016, doi: 10.1038/sdata.2016.18.

[25] S. Leo, “Recording provenance of workflow runs with RO-Crate,” *PLoS One*, vol. 19, p. e0309210, 2024, doi: 10.1371/journal.pone.0309210.

[26] W. Liu, “PLMSearch: Protein language model powers accurate and fast sequence search for remote homology,” *Nat. Commun.*, vol. 15, p. 2775, 2024, doi: 10.1038/s41467-024-46808-5.

[27] P. Lewis *et al.*, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Advances in Neural Information Processing Systems 33*, 2020. [Online]. Available: https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html

[28] S. Yao and Y. Cao, “ReAct: Synergizing Reasoning and Acting in Language Models,” 2022. [Online]. Available: https://research.google/blog/react-synergizing-reasoning-and-acting-in-language-models/

[29] G. C. A. Center, “Choose a design pattern for your agentic AI system: Human-in-the-loop pattern,” 2026. [Online]. Available: https://docs.cloud.google.com/architecture/choose-design-pattern-agentic-ai-system

APPENDIX A. BIOFLOW-SCOPE INSTALLATION AND USER GUIDE

A.1. Deployment Scope

BioFlow-Scope is currently validated as a locally deployable research and development platform. The deployment model consists of a Next.js frontend, Spring Boot backend, FastAPI embedding sidecar, and infrastructure services running through Docker Compose. The repository does not provide Kubernetes manifests, production reverse-proxy configuration, or a high-availability deployment process; therefore, this appendix does not describe the system as a complete production platform.

Table A.*1. BioFlow-Scope deployment components*

Component	Technology	Default port	Role
Frontend	Next.js 14.2.35, React 18, TypeScript	3000	Web interface for Pipelines, Datasets, AI Agent, administration, and reproducibility.
Backend	Java 21, Spring Boot 3.3.5	8080	REST API, authentication, authorization, Pipeline runner, AI/HITL, and business data.
Embedding sidecar	FastAPI, Transformers, Sentence Transformers	8090	Generates 480-dimensional protein vectors and 384-dimensional text vectors.
PostgreSQL	PostgreSQL 16	5432	Stores transactional data, history, provenance, approvals, and configuration.
Redis	Redis 7	6379	Cache, rate limiting, and selected locking/state mechanisms.
Qdrant	Qdrant 1.11.3	6333/6334	Stores protein vectors and RAG knowledge-base vectors.
Ollama	Ollama local	11434	Provides a local conversational model for the AI Agent.
Keycloak	Keycloak 24.0.5	8081	Optional OIDC/SSO.

Component	Technology	Default port	Role
Docker runner	docker-java 3.4.0	Docker daemon	Runs bioinformatics tools in containers.

A.2. Environment Requirements

The surveyed environment used Windows 10 with Java 21.0.9, Node.js 24.13.0, npm 11.6.2, Python 3.13.9, Docker 29.1.3, and Docker Compose 2.40.3. These versions demonstrate that the system can run in the surveyed environment but do not imply compatibility with every newer or older version.

- Java 21 to run the Spring Boot backend.
- Node.js and npm to run root scripts and the Next.js frontend.
- Python with the embedding-sidecar dependencies; real mode requires Transformers, PyTorch, and Sentence Transformers.
- Docker Desktop or Docker Engine with Compose to run PostgreSQL, Redis, Qdrant, Ollama, and Keycloak.
- Sufficient disk capacity for Docker images, AI models, scientific data, artifacts, and Qdrant volumes.
- Do not store real API keys or passwords in the repository; use environment variables or local configuration files excluded from Git.

A.3. Preparing Source Code and Configuration

1. Open a terminal in the root directory of the BioFlow-Scope repository.

2. Copy `.env.example` to `.env` and fill in appropriate values. Do not commit `.env` to Git.

3. Check variables related to PostgreSQL, Redis, Qdrant, JWT, CORS, Ollama, the embedding sidecar, SMTP, and storage.

4. Enable an external AI provider only when a valid API key is available and the policy governing data that may leave the system has been determined.

```
cp .env.example .env
npm install
cd frontend
npm install
cd ..
```

A.4. Starting the Infrastructure and Application

The recommended sequence is to start the infrastructure first, followed by the sidecar, backend, and frontend. The `npm run dev` script can orchestrate the

development components, but during diagnosis each component should be run separately to make faults easier to isolate.

```
npm run infra:up
npm run dev
```

To run individual components separately:

```
npm run dev:sidecar
npm run dev:backend
npm run dev:frontend
```

***Table A.**2. Post-startup service checks*

Service	Check URL	Expected result
Frontend	http://localhost:3000	The web page returns HTTP 200.
Backend health	http://localhost:8080/actuator/health	Status is `UP`.
Embedding health	http://localhost:8090/health	Status is `ok`, with `dim=480` and `text_dim=384`; real mode has `stub=false`.
Qdrant	http://localhost:6333/collections	Returns a valid collection list.
Qdrant dashboard	http://localhost:6333/dashboard	The Qdrant administration interface is displayed.
Ollama	http://localhost:11434	The service responds; the model must be pulled separately.
Keycloak	http://localhost:8081	The Keycloak page or realm endpoint responds.

A.5. Preparing the Local AI Model

The default conversational model in the backend configuration is `qwen3:8b`. The repository provides a script to download the model through Ollama. A healthy Ollama container does not prove that the model has been downloaded; the model list should be checked or a test invocation should be executed before using the AI Agent.

```
scripts/download_models.sh
```

Embedding models are loaded by the sidecar according to configuration. In real mode, the default protein model is `facebook/esm2_t12_35M_UR50D`, while the default text-embedding model is `sentence-transformers/all-MiniLM-L6-v2`. The first run may take longer because models are downloaded and caches are initialized.

A.6. Initial Usage Flow

1. Log in/register, open the Dashboard, and check system status.
2. Create or import a Pipeline, lint the DAG, select a Dataset/Collection, and start a Run with the required parameters.
3. Monitor the Run, logs, and Artifacts; use the viewer or raw download according to format and access permissions.
4. Use Protein Search, the AI Agent, and reproducibility functions when needed; actions with side effects must pass through policy/approval.

A.7. Testing and Build

```
npm run test:frontend
npm run test:backend
npm run test:e2e
npm run build:frontend
npm run build:backend
```

During the survey, frontend type checking passed. The backend test task was rerun using `gradlew test --rerun-tasks`, recording 950 tests, 0 failures, 0 errors, and 1 skipped test. Integration tests and Playwright E2E were not run in that session; therefore, they must not be reported as passed unless an additional run is available.

A.8. Stopping the System and Troubleshooting

```
npm run infra:down
```

Table A.*3. Common problems and troubleshooting*

Symptom	Check	Resolution
Frontend does not open	Check the frontend terminal, port 3000, and dependencies.	Rerun `npm install` in `frontend`; check the API base URL variable.
Backend health is not UP	Check PostgreSQL, Redis, Qdrant, and migrations.	Inspect backend logs; ensure containers are healthy and the database configuration is correct.
Sidecar reports stub=true	Check `EMBEDDING_STUB` and ML dependencies.	Install ML requirements, enable real mode, and restart the sidecar.
AI Agent does not respond	Check Ollama/model availability, backend timeout, and session state.	Pull the model; check the model name and Ollama base URL.

Symptom	Check	Resolution
Protein Search returns no results	Check the collection, point count, dimensions, and sidecar.	Confirm that `swiss_prot_v2025` contains 480-dimensional cosine vectors; run a diagnostic query.
Docker tool fails	Check the Docker daemon, image, mounts, command, and timeout.	Pull the image; check input files, directory permissions, and the Run Step log.

APPENDIX B. API, DATABASE, AND STATE MACHINES

B.1. API Architecture

The backend uses REST JSON and SSE for real-time flows; most `/api/**` endpoints require authentication, while public endpoints are restricted by token/snapshot scope.

Table J.*1. Main API endpoint groups*

Module	Endpoint group	Actor/permission	Purpose
Auth	`/api/auth/*`	Guest/User	Registration, login, and current-user information.
Dashboard	`/api/dashboard/*`	Authenticated user	Statistics and recent Runs.
Pipeline	`/api/pipelines/*`	Owner/Workspace permission	Pipeline CRUD, graph, versioning, and Run creation.
Run	`/api/runs/*`	Run permission	Details, execute, cancel, resume, rerun, log SSE, and artifacts.
Tool catalog	`/api/tools*`	Authenticated user	Runtime tool catalog.
Protein	`/api/protein/*`	Authenticated + rate limit	Semantic search and protein lookup.
Dataset	`/api/datasets/*`	Owner/Workspace permission	Dataset, file, collection, import job.
Viewer	`/api/artifacts/{id}/*`	Artifact/run permission	BLAST/VCF parsing, raw output, and HTML.
AI Agent	`/api/agent/*`	Authenticated user	Session, message SSE, tools, tasks, and notebook export.
Approval	`/api/agent/approvals/*`	Approval authorization	Approve, reject, revoke, execute, and ledger.
Evidence/Citation	`/api/agent/*/evidence`, `/api/citations/*`	User/Admin scope	Evidence, claims, and citations.
Workspace/Project	`/api/workspaces/*`, `/api/projects/*`	Membership/role	Member, policy, shared asset, task/note.
Reproducibility	`/api/reproducibility/*`, export/report/public	Run owner/public token	Readiness, export, import, verification, and sharing.
Admin	`/api/admin/*`	ADMIN	Users, databases, quotas, storage, knowledge, and audit.

B.2. Database Table Groups

Flyway migrations from V1 through V58 were observed. Tables are organized by business module; the list below is a condensed index and does not replace the per-column data dictionary.

***Table J.**2. Main database-table groups*

Module	Main tables
Auth/profile	`users`, `user_settings`.
Pipeline/run	`pipelines`, `pipeline_versions`, `runs`, `run_steps`, `run_logs`, `artifacts`, `scheduled_runs`.
Protein search	`protein_searches`.
Dataset	`datasets`, `dataset_files`, `dataset_tags`, `dataset_import_jobs`, `run_dataset_inputs`, `dataset_collections`, `dataset_collection_items`.
Admin database	`bio_databases`, `bio_database_versions`, `bio_database_update_jobs`.
Agent/chat/memory	`agent_sessions`, `agent_messages`, `agent_tool_calls`, `projects`, `project_memories`, `agent_attachments`.
HITL/orchestration	`agent_tasks`, `agent_steps`, `agent_scratchpad_entries`, `approval_requests`, `approval_ledger`.
Workspace/project	`workspaces`, `workspace_members`, `workspace_invitations`, `workspace_role_policies`, `project_members`, `project_assets`, `project_tasks`, `project_notes`, `project_activity`.
Publication/reproducibility	`public_run_tokens`, `reproduction_export_jobs`, `reproduction_import_sessions`, `reproduction_reports`, `reproducibility_audit_events`.
Evidence/policy/citation	`agent_evidence`, `agent_claims`, `agent_claim_evidence`, `citation_records`, `retraction_watch_cache`, `ai_policies`, `ai_policy_violations`.
Notification/quota	`notifications`, `broadcasts`, `quotas`, `admin_audit_log`.
Data source/scheduler	`data_sources`, `pipeline_triggers`, `trigger_events`, `data_source_roots`.

B.3. Main State Machines

***Table J.**3. Main state machines*

Object	State	Notes
Run	QUEUED, RUNNING, COMPLETED, FAILED, CANCELLED	Terminal states cannot transition arbitrarily.
RunStep	PENDING, RUNNING, COMPLETED, FAILED, SKIPPED	Orchestrated by DagExecutor.

Object	State	Notes
DatasetImportJob	QUEUED, RUNNING, COMPLETED, FAILED, CANCELLED, BLOCKED_LOCAL_STORAGE_CAP, AWAITING_COLAB	Reflects storage/local-processing constraints.
AgentTask	PLANNING, READY, RUNNING, WAITING_APPROVAL, WAITING_RUN, PAUSED, COMPLETED, FAILED, CANCELLED	Connects planning, approval, and Run execution.
ApprovalRequest	PENDING, APPROVED, REJECTED, EXPIRED, REVOKED, CONSUMED	Fingerprint, row lock, and one-time consumption.
TriggerEvent	DISCOVERED, READY, RUN_CREATED, RETRY_SCHEDULED, FAILED, IGNORED_*, PAUSED_QUOTA	Orchestrates data-source triggers.
ExportJob	QUEUED, COLLECTING, VALIDATING, READY, FAILED, CANCELLED, EXPIRED	Has a `canTransitionTo` guard.
ImportSession	UPLOADING, QUARANTINED, VALIDATING, PREVIEW_READY, REJECTED, RESOLVING, DRY_RUN_READY, BLOCKED, MATERIALIZED, EXPIRED	Has a `canTransitionTo` guard.
DatabaseUpdateJob	QUEUED, RUNNING, COMPLETED, FAILED, CANCELLED	Updates reference databases.

B.4. API Design and State Transition Rules

- Controllers only receive requests and pass them to services; authorization must reside in security/resource services and must not be delegated to the frontend.
- Endpoints with side effects must return clear errors when a state transition is invalid.
- SSE streams should provide reconnection, replay, or final-state mechanisms so clients do not lose information.
- Public endpoints must limit responses according to snapshot/token scope.
- Admin endpoints must audit important actions.
- Import/export jobs should be idempotent to an appropriate degree and must not materialize data before validation/dry-run.
- API errors must not expose stack traces, secrets, internal paths, or raw credentials.